

Python для Excel

Excel — это самый популярный в мире табличный редактор, но его язык автоматизации VBA давно перестал развиваться.

Python — самый востребованный язык программирования, он хорошо работает с данными и прекрасно подходит на роль языка сценариев Excel. Вот почему сочетание Excel и Python актуально и привлекательно.

В книге показано, как эффективно интегрировать эти два мира и начать работу по автоматизации Excel с помощью Python. При этом знание Python приветствуется, но не обязательно, так как в книге есть введение во все используемые инструменты, включая вводный курс по языку Python.

- Освойте работу с современными инструментами, включая блокноты Jupyter и Visual Studio Code.
- Используйте pandas для сбора, очистки и анализа данных и замены типичных вычислений в Excel.
- Автоматизируйте рутинные задачи, такие как объединение рабочих книг Excel и создание отчетов Excel.
- Используйте xlwings для создания интерактивных инструментов Excel, использующих Python в качестве механизма вычислений.
- Подключайте Excel к базам данных и файлам CSV и получайте данные из Интернета с помощью кода Python.
- Используйте Python как единый инструмент для замены VBA, Power Query и Power Pivot.



Архив с цветными иллюстрациями можно скачать по ссылке <https://zip.bhv.ru/9785977568821.zip>, а также со страницы книги на сайте bhv.ru.

ISBN 978-5-9775-6882-1



191036, Санкт-Петербург,
Гончарная ул., 20
Тел.: (812) 717-10-50,
339-54-17, 339-54-28
E-mail: mail@bhv.ru
Internet: www.bhv.ru

Python для Excel

Современная среда для автоматизации и анализа данных



Python for Excel

*A Modern Environment for Automation
and Data Analysis*

Felix Zumstein

Beijing • Boston • Farnham • Sebastopol • Tokyo

O'REILLY®

Феликс Зумштейн

Python для Excel

Современная среда для автоматизации
и анализа данных

Санкт-Петербург

«БХВ-Петербург»

2023

УДК 004.43
ББК 32.973.26-018.1
3-93

Зумштейн Ф.

3-93 Python для Excel: Пер. с англ. — СПб.: БХВ-Петербург, 2023. — 336 с.: ил.
ISBN 978-5-9775-6882-1

Книга посвящена автоматизации Excel с помощью языка программирования Python. Описаны дистрибутив Anaconda Python и современные средства разработки, такие как менеджеры пакетов Conda и pip, блокноты Jupyter и Visual Studio Code. Даны необходимые основы языка Python и введение в анализ данных с помощью библиотеки pandas. Приведены приемы чтения и записи файлов Excel без Excel. Рассмотрено программирование приложений Excel с помощью популярного пакета с открытым исходным кодом xlwings: автоматизация Excel, инструменты на основе технологии Python, трекер пакетов Python, а также функции, определяемые пользователем.

Электронный архив на сайте издательства содержит цветные иллюстрации к книге.

Для опытных пользователей Excel и программистов

УДК 004.43
ББК 32.973.26-018.1

Научный редактор:

Инженер-тестировщик компании КРОК *Дмитрий Колфилд*

Группа подготовки издания:

Руководитель проекта	<i>Евгений Рыбаков</i>
Зав. редакцией	<i>Людмила Гауль</i>
Редактор	<i>Наталья Смирнова</i>
Компьютерная верстка	<i>Натальи Смирновой</i>
Оформление обложки	<i>Зои Канторович</i>

© 2023 BHV

Authorized Russian translation of the English edition of Python for Excel ISBN 9781492081005 © 2021 Zoomer Analytics LLC.

This translation is published and sold by permission of O'Reilly Media, Inc., which owns or controls all rights to publish and sell the same.

Авторизованный перевод с английского языка на русский издания Python for Excel ISBN 9781492081005 © 2021 Zoomer Analytics LLC.

Перевод опубликован и продается с разрешения компании-правообладателя O'Reilly Media, Inc.

Подписано в печать 07.06.23.

Формат 70×100^{1/16}. Печать офсетная. Усл. печ. л. 27,09.

Тираж 1200 экз. Заказ № 6933.

"БХВ-Петербург", 191036, Санкт-Петербург, Гончарная ул., 20.

Отпечатано с готового оригинал-макета

ООО "Принт-М", 142300, М.О., г. Чехов, ул. Полиграфистов, д. 1

ISBN 978-1-492-08100-5 (англ.)

ISBN 978-5-9775-6882-1 (рус.)

© Zoomer Analytics LLC, 2021

© Перевод на русский язык, оформление.

ООО "БХВ-Петербург", ООО "БХВ", 2023

Содержание

Предисловие	11
Почему я написал эту книгу.....	12
Кому адресована эта книга.....	12
Структура книги.....	13
Версии Python и Excel.....	14
Условные обозначения, используемые в этой книге	15
Примеры использования кода.....	15
Иллюстрации	16
Онлайн-обучение O'Reilly	17
Как с нами связаться	17
Благодарности	17
 Часть I. Введение в Python	19
 Глава 1. Зачем нужен Python для Excel?.....	21
Excel как язык программирования	22
Excel в новостях	23
Передовые методы программирования.....	24
Современный Excel.....	29
Python для Excel	31
Читабельность и эксплуатационная пригодность.....	31
Стандартная библиотека и менеджер пакетов	32
Научные вычисления.....	34
Особенности современного языка.....	35
Кросс-платформенная совместимость	36
Заключение	36
 Глава 2. Среда разработки	38
Дистрибутив Anaconda Python.....	39
Установка.....	39
Anaconda Prompt	40
Python REPL: интерактивная сессия Python.....	43
Менеджеры пакетов: Conda и pip	44
Среды Conda.....	46

Jupyter Notebooks.....	47
Запуск блокнотов Jupyter	48
Ячейки блокнота	49
Режим редактирования и командный режим	51
Порядок выполнения имеет значение	52
Завершение работы блокнотов Jupyter	52
Visual Studio Code	53
Установка и настройка	55
Запуск скрипта на Python	57
Заключение	61
Глава 3. Приступая к работе с Python.....	63
Типы данных	63
Объекты	64
Числовые типы	65
Логический тип данных	67
Строки.....	69
Индексирование и нарезка	70
Индексирование	70
Нарезка (Slicing).....	71
Структуры данных	72
Списки.....	72
Словари	75
Кортежи	76
Множества	77
Управление потоком.....	78
Блоки кода и оператор <i>pass</i>	78
Оператор <i>if</i> и условные выражения	79
Циклы <i>for</i> и <i>while</i>	80
Анализ списков, словарей и множеств	83
Организация кода.....	84
Функции.....	84
Модули и инструкция по импорту	86
Класс <i>datetime</i>	88
PEP 8: Руководство по стилю для кода Python.....	90
PEP 8 и VS Code.....	92
Подсказки по типам	93
Заключение	94
Часть II. ВВЕДЕНИЕ В PANDAS	95
Глава 4. Основы NumPy	97
Начало работы с NumPy	97
Массив NumPy	97

Векторизация и транслирование	99
Универсальные функции (ufunc)	101
Создание и манипулирование массивами	102
Получение и установка элементов массива	102
Полезные конструкторы массива	103
Представления и копирование	103
Заключение	104
Глава 5. Анализ данных с помощью pandas	105
DataFrame и Series	105
Индекс	108
Столбцы	110
Манипулирование данными	111
Выбор данных	111
Изменение данных	117
Отсутствующие данные	120
Дубликаты данных	122
Арифметические операции	123
Работа с текстовой колонкой	125
Использование функции	126
Просмотр и копирование	127
Объединение DataFrames	127
Объединение	128
Объединение и слияние	129
Описательная статистика и агрегация данных	132
Описательная статистика	132
Группировка	133
Pivoting и Melting	134
Построение графиков	135
Matplotlib	135
Plotly	137
Импорт и экспорт DataFrames	140
Экспорт CSV файлов	141
Импорт CSV-файлов	142
Заключение	144
Глава 6. Анализ временных рядов с помощью pandas	145
DatetimeIndex	146
Создание DatetimeIndex	146
Фильтрация DatetimeIndex	148
Работа с часовыми поясами	150
Общие манипуляции с временными рядами	151
Смещение и процентные изменения	151
Пересчет и корреляция	153

Повторная выборка	156
Скользящее окно	157
Ограничения при работе с pandas	158
Заключение	159

Часть III. Чтение и запись файлов Excel без Excel 161

Глава 7. Манипулирование файлами Excel с помощью pandas 163

Тематическое исследование: отчетность в Excel	163
Чтение и запись файлов Excel с помощью pandas	167
Функция read_excel и класс ExcelFile	167
Метод to_excel и класс ExcelWriter	173
Ограничения при работе pandas с файлами Excel	174
Заключение	175

Глава 8. Манипулирование файлами Excel с помощью пакетов

reader и writer 176

Пакеты reader и writer	176
В каких случаях какой пакет используется	177
Модуль excel.py	178
OpenPyXL	180
XlsxWriter	184
pyxlsb	186
xlrd, xlwt, and xlutils	187
Работа с xlwt	189
Расширенный круг задач для reader и writer	190
Работа с большими файлами Excel	191
Форматирование данных в Excel	195
Тематическое исследование (повторное): отчетность в Excel	200
Заключение	201

Часть IV. Программирование приложения Excel

с помощью xlwings..... 205

Глава 9. Автоматизация Excel 205

Начало работы с xlwings	206
Использование Excel в качестве средства просмотра данных	206
Объектная модель Excel	208
Запуск кода VBA	215
Конвертеры, опции и коллекции	216
Работа с DataFrames	216
Конвертеры и опции	217

Диаграммы, рисунки и определенные имена	220
Случай из практики (повторный анализ): отчетность в Excel	223
Расширенные темы xlwings	225
Основы xlwings	225
Улучшение производительности	227
Как действовать при отсутствии недостающих функций	229
Заключение	230
Глава 10. Инструменты Excel на основе технологии Python	231
Использование Excel в качестве интерфейса xlwings	231
Настройка Excel	232
Команда Quickstart	233
Run Main	234
Функция RunPython	235
Развертывание	240
Зависимости Python	240
Автономные рабочие книги: избавление от настройки xlwings	241
Иерархия конфигурации	242
Настройки	243
Заключение	244
Глава 11. Трекер пакетов Python	245
Что мы будем создавать	245
Основной функционал	247
Web APIs	248
Базы данных	251
Исключения	260
Структура приложения	263
Внешний интерфейс	264
Внутренний интерфейс	268
Отладка	271
Заключение	273
Глава 12. Функции, определяемые пользователем (UDFs)	274
Начало работы с UDF	274
UDF Quickstart	275
Тематическое исследование: Google Trends	280
Введение в Google Trends	280
Работа с DataFrames и динамическими массивами	282
Получение данных из Google Trends	287
Построение графиков с помощью UDF	291
Отладка UDFs	293
Дополнительные вопросы по UDF	294
Базовая оптимизация производительности	295

Кэширование	297
Декоратор Sub	299
Заключение	301
 Часть V. Приложения.....	303
Приложение А. Среда Conda.....	305
Создание новой среды Conda.....	305
Отключение автоматической активации.....	307
Приложение В. Расширенные функциональные возможности VS Code.....	308
Отладчик	308
Блокноты Jupyter в VS Code.....	310
Запуск блокнотов Jupyter	310
Сценарии Python с ячейками кода.....	311
Приложение С. Дополнительные концепции Python	313
Классы и объекты.....	313
Работа с объектами datetime с учетом временной зоны	315
Изменяемые и неизменяемые объекты Python.....	316
Вызов функций с изменяемыми объектами в качестве аргументов	317
Функции с изменяемыми объектами в качестве аргументов по умолчанию.....	319
Об авторе	321
Обложка	323
Предметный указатель	325

Корпорация Microsoft создала на UserVoice форум по Excel, где каждый может представить новую идею, чтобы другие ознакомились и проголосовали за нее. И самым популярным запросом на этом форуме был «Python как язык сценариев для Excel», который набрал по приблизительным оценкам в два раза больше голосов, чем второй по популярности запрос о функциональных возможностях программы. И хотя с 2015 года когда эта идея была озвучена на форуме, ничего не произошло, в конце 2020 года у пользователей Excel появилась надежда: Гвидо ван Россум, создатель Python, написал в твиттере, что его «выход на пенсию был скучным», и он присоединится к Microsoft. Повлияет ли этот шаг на интеграцию Excel и Python, я не знаю. Однако мне известно как эту комбинацию сделать привлекательной, и в этой книге я расскажу, как можно начать совместно использовать Excel и Python. Далее, в двух словах, о чем эта книга.

Основная идея использовать Python совместно Excel основывается на том, что мы живем в мире данных. В настоящее время огромные массивы данных обо всем находятся в открытом доступе. Зачастую эти наборы данных настолько велики, что уже не помещаются в электронную таблицу. Несколько лет назад эти массивы можно было назвать *большими массивами данных (big data)*, но сегодня набор данных из нескольких миллионов строк не представляет собой ничего особенного. Excel эволюционировал, чтобы справиться с этой тенденцией: он представил Power Query для загрузки и очистки наборов данных, которые не вписываются в предварительную таблицу, и Power Pivot, надстройку для выполнения анализа этих массивов данных и представления результатов. Power Query основан на Power Query M formula language (M) (язык формул Power Query M), в то время как Power Pivot разрабатывает алгоритмы, используя формулы анализа данных (Data Analysis Expressions или DAX). Для автоматизации в своем файле Excel некоторых операций вы можете использовать встроенный в Excel язык автоматизации Visual Basic for Applications, или VBA. То есть для простых операций вам станут доступны для использования VBA, Power Query M и DAX. Но здесь возникает проблема — все эти языки, особенно для Excel и Power BI, работают только в среде Microsoft (о Power BI я кратко расскажу в *главе 1*).

С другой стороны, Python — это язык программирования общего назначения, ставший одним из самых популярных среди аналитиков и специалистов по обработке данных. Если вы используете Python совместно с Excel, вы получаете воз-

возможность использовать язык программирования, который хорошо справляется со всеми задачами, будь то автоматизация работы Excel, доступ и подготовка наборов данных или выполнение задач анализа и визуализации данных. Самое главное, вы можете использовать свои навыки работы с Python вне Excel: если вам нужно увеличить вычислительные мощности, вы можете легко перенести свою математическую модель, симуляцию или приложение машинного обучения в облако, где вас ждут практически неограниченные вычислительные ресурсы.

Почему я написал эту книгу

Благодаря моей работе над `xlwings`, пакетом автоматизации Excel, с которым мы познакомимся в *части IV* этой книги, я тесно общаюсь со многими пользователями, использующими Python для Excel. Общение происходит через трекер задач на GitHub, через сообщество на StackOverflow или непосредственно на мероприятиях, например на встречах или конференциях.

Меня регулярно просят порекомендовать ресурсы для знакомства с Python. Хотя нехватки ознакомительных материалов по Python, конечно, нет, они часто бывают либо недостаточно полными (нет информации об анализе данных), либо специализированными (полное научное описание). Тем не менее, пользователи Excel, как правило, находятся где-то посередине: они, конечно, работают с данными, но полное научное описание может оказаться для них слишком сложным. Кроме того, у них часто возникают специфические задачи и связанные с ними вопросы, на которые нет ответа ни в одном из существующих пособий. Вот некоторые из этих вопросов:

- ◆ Какой пакет Python-Excel мне нужен для решения той или иной задачи?
- ◆ Как подключиться к базе данных с помощью Power Query на Python?
- ◆ Что можно использовать вместо автофильтра Excel или сводной таблицы (pivot table) в Python?

Я написал эту книгу, чтобы читатель, ничего не зная о Python, смог автоматизировать свои задачи, ориентированные на Excel, и использовать инструменты Python для анализа данных и научных вычислений в Excel без каких-либо обходных путей.

Кому адресована эта книга

Если вы продвинутый пользователь Excel, который хочет с помощью современного языка программирования обойти ограничения Excel, эта книга для вас. Как правило, каждый месяц вы тратите время для загрузки, очистки и копирования/вставки больших объемов данных в критически важные электронные таблицы. Хотя существуют различные способы преодоления ограничений Excel, в этой книге мы рассмотрим, как использовать Python для решения этой задачи. Желательно, чтобы у вас было базовое представление о программировании (неважно, на каком языке

программирования), то есть вы уже работали с функциями, циклами и имеете представление о том, что такое целое число или строка. Вам также не помешает в освоении этой книги навык написания сложных формул для ячеек или опыт настройки записанных макросов VBA. От вас не требуется никакого опыта работы с Python, так как в курсе есть введение во все инструменты, которые мы будем использовать, включая вводный курс по самому Python. Если вы занимаетесь разработкой на VBA, вы найдете регулярные сравнения между Python и VBA, которые позволят вам преодолеть общие трудности и начать работу.

Эта книга также может быть полезна, если вы являетесь разработчиком Python и хотите узнать о различных способах работы Python как с приложением Excel, так и файлами Excel, чтобы выбрать правильный пакет с учетом требований ваших бизнес-пользователей.

Структура книги

В этой книге я покажу вам все аспекты истории Python для Excel. Книга разделена на четыре части:

Часть I: Введение в Python

В этой части, прежде чем познакомить вас с инструментами, которые мы будем использовать в данной книге, а именно: дистрибутив Anaconda Python, Visual Studio Code и блокноты Jupyter, мы рассмотрим причины, по которым Python стал таким привлекательным дополнением для Excel.

Часть II: Введение в pandas

pandas — это библиотека Python для анализа данных. Мы разберемся, как заменить рабочие книги Excel комбинацией Jupyter Notebook и pandas. Обычно код, использующий библиотеку pandas, проще в обслуживании и эффективнее, чем рабочая книга Excel, и вы можете работать с наборами данных, которые не могут быть помещены в электронную таблицу. В отличие от Excel, pandas позволяет запускать код в любом окружении, включая облако.

Часть III: Чтение и запись файлов Excel без Excel

Эта часть посвящена обработке файлов Excel с помощью одного из следующих пакетов Python: pandas, OpenPyXL, XlsxWriter, pyxlsb, xlrd и xlwt. Эти пакеты способны читать и записывать книги Excel непосредственно на диск и, таким образом, заменяют приложение Excel. Поскольку вам не требуется установка Excel, они работают на любой платформе, которая поддерживает Python, включая Windows, macOS и Linux. Типичным вариантом использования пакета reader является считывание данных из файлов Excel, которые вы получаете каждое утро от внешней компании или системы, и сохранение их содержимого в базе данных. Как правило, пакет writer используется для обеспечения функциональности известной функции **Экспорт в Excel**, которую можно найти почти в каждом приложении.

Часть IV: Программирование в приложении Excel с помощью xlwings

В этой части будет показано, как для автоматизации приложения Excel мы можем использовать Python с пакетом xlwings вместо чтения и записи файлов Excel на диск. Эта часть требует локальной установки Excel. Мы научимся открывать рабочие книги Excel и работать с ними. Помимо чтения и записи файлов через Excel мы создадим интерактивные инструменты Excel: после нажатия только одной кнопки Python выполнит то, что раньше делалось с помощью макросов VBA. Например, трудоемкие вычисления. Кроме того, мы узнаем, как вместо VBA написать *пользовательские функции*¹ (UDFs) на Python.

Важно понимать фундаментальную разницу между чтением и записью *файлов* Excel (*часть III*) и программированием *приложения* Excel (*часть IV*), как показано на рис. П.1.

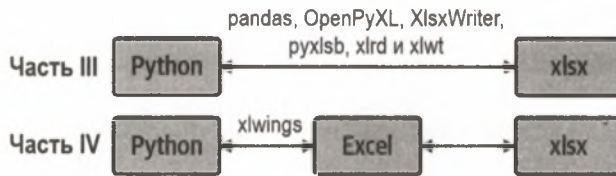


Рис. П.1. Чтение и запись файлов Excel (часть III) и программирование Excel (часть IV)

Для изучения *части III* не требуется установка Excel, так как все работает на всех платформах, которые поддерживают Python, в основном на Windows, macOS и Linux. Для *части IV* потребуется локальная установка Microsoft Excel, и поэтому она будет актуальной для тех платформ, которые поддерживают Microsoft Excel, то есть Windows и macOS.

Версии Python и Excel

При написании этой книги использовался Python 3.8 — версия, которая поставляется с последней версией дистрибутива Anaconda Python. Если вы хотите использовать более новую версию Python, следуйте инструкциям, размещенным на домашней странице книги: <https://www.xlwings.org/book>. При этом убедитесь, что вы не используете более старую версию. Внесенные в Python 3.9 изменения я буду комментировать.

Подразумевается, что вы используете современную версию Excel, то есть как минимум Excel 2007 в Windows и Excel 2016 в macOS. Локально установленная версия Excel, поставляемая с подпиской Microsoft 365, также прекрасно работает. Более того, я даже рекомендую ее, поскольку в ней содержатся новейшие функции, которые

¹ Компания Microsoft начала использовать термин пользовательские функции вместо UDF. В этой книге я буду продолжать называть их UDF.

вы не найдете в других версиях Excel. Кроме того, при написании книги я использовал именно эту версию, поэтому, если вы используете другую версию Excel, вы заметите небольшую разницу в названиях или расположении пунктов меню.

Условные обозначения, используемые в этой книге

В этой книге используются следующие типографские обозначения:

Курсив

Этим шрифтом выделяются новые термины.

Полужирный шрифт

Используется для выделения адресов электронной почты, элементов интерфейса.

Моноширинный шрифт

Выделены листинги программ. Внутри параграфов используется для обозначения элементов программы, таких как имена переменных, функции и типы данных.

Моноширинный шрифт (полужирный)

Обозначает команды или другой текст, который пользователь должен набрать самостоятельно.

Моноширинный шрифт (курсив)

Показывает текст, который следует заменить значениями, предоставленными пользователем, или значениями, определяемыми контекстом.



Этот элемент означает совет или предложение.



Данный элемент обозначает общее примечание.



Этот элемент обозначает предупреждение или предостережение.

Примеры использования кода

Я поддерживаю веб-страницу с дополнительной информацией, которая поможет вам в работе с этой книгой. Непременно ознакомьтесь с ней, особенно если вы столкнулись с проблемой.

Дополнительные материалы (примеры кода, упражнения и т. д.) доступны для скачивания по адресу <https://github.com/fzumstein/python-for-excel>. Чтобы загрузить вспомогательный репозиторий, нажмите на зеленую кнопку **Code**, а затем выберите

команду **Download ZIP**. В операционной системе Windows, чтобы разархивировать содержащиеся в архиве файлы в папку, после загрузки архива щелкните правой кнопкой мыши на скачанном файле и выберите команду **Extract All**. Для распаковки архива в macOS дважды щелкните мышью на скачанном файле. Если вы умеете работать с Git, вы можете использовать Git для копирования репозитория на локальный жесткий диск, поместив там, где пожелаете. Я же эту папку, на которую буду иногда ссылаться, разместил у себя на компьютере по следующему пути:

```
C:\Users\username\python-for-excel
```

Если вы загрузите и разархивируете ZIP-файл в операционной системе Windows, вы получите структуру папок, подобную этой (обратите внимание на повторяющиеся названия папок):

```
C:\...\Downloads\python-for-excel-1st-edition\python-for-excel-1st-edition
```

Копирование содержимого этой папки в папку `C:\Users\<username>\python-for-excel` значительно облегчит вам дальнейшую работу. Эти же рекомендации актуальны и для операционной системы macOS, т.е. скопируйте файлы в `/Users/<username>/python-for-excel`.

Если у вас возникли технические вопросы или проблемы с использованием примеров кода, пожалуйста, отправьте письмо по адресу **bookquestions@oreilly.com**.

Эта книга поможет вам в вашей работе. Если в ней предлагается пример кода, вы можете использовать его в своих программах и документации. Вам не нужно обращаться к нам за разрешением, если вы не воспроизводите значительную часть кода. Например, написание программы, использующей несколько фрагментов кода из этой книги, не требует разрешения. Однако на продажу или распространение примеров из книг O'Reilly необходимо разрешение. Если при ответе на вопрос требуется ссылка на эту книгу или на пример кода, разрешение на эти действия можно и не получать. Оно необходимо для включения большого количества примеров кода из этой книги в документацию вашего продукта.

Мы не требуем, чтобы при ссылке на материал указывалось авторство, но приветствуем это. В понятие авторства обычно включаются название книги, имя и фамилия автора, указание издательства и ISBN (международный стандартный книжный номер). Например: «Python для Excel», Феликс Зумштейн (изд-во O'Reilly). Copyright 2021 Zoomer Analytics LLC, 978-1-492-08100-5.

Если вы полагаете, что использование примеров кода выходит за рамки добросовестного использования или вышеуказанного разрешения, не стесняйтесь обращаться к нам по адресу **permissions@oreilly.com**.

Иллюстрации

PDF-файл с цветными скриншотами и рисунками, используемыми в этой книге, можно скачать по ссылке: **<https://zip.bhv.ru/9785977568821.zip>**, а также со страницы книги на сайте **bhv.ru**.

Онлайн-обучение O'Reilly

O'REILLY®

Более 40 лет компания O'Reilly Media обеспечивает обучение технологиям и бизнесу, давая необходимые знания, чтобы помочь компаниям добиться успеха.

Наша профессиональная сеть экспертов и новаторов делится своими знаниями и опытом через книги, статьи и нашу платформу онлайн-обучения. Платформа онлайн-обучения O'Reilly предоставляет вам доступ по запросу к интерактивным учебным курсам, путям углубленного обучения, интерактивным средам программирования и обширной коллекции текстов и видео от O'Reilly и более 200 других издателей. Для получения дополнительной информации посетите сайт <http://oreilly.com>.

Как с нами связаться

Комментарии и вопросы, касающиеся этой книги, направляйте издателю:

O'Reilly Media, Inc.

1005 Gravenstein Highway North

Sebastopol, CA 95472

800-998-9938 (in the United States or Canada)

707-829-0515 (international or local)

707-829-0104 (fax)

У этой книги есть веб-страница, где мы перечисляем ошибки, примеры и любую дополнительную информацию. Эта страница находится по адресу <https://oreil.ly/py4excel>.

Пишите по адресу bookquestions@oreilly.com, чтобы оставить комментарий или задать технические вопросы по этой книге.

Более подробную информацию о наших книгах, курсах, конференциях и новостях можно найти на нашем сайте <http://www.oreilly.com>.

Ищите нас на Facebook: <http://facebook.com/oreilly>.

Следите за нами в Twitter: <http://twitter.com/oreillymedia>.

Смотрите нас на YouTube: <http://www.youtube.com/oreillymedia>.

Благодарности

Как начинающий автор я невероятно благодарен за помощь, которую мне оказали многие люди на этом пути — они облегчили мне это исследование! В компании O'Reilly я хотел бы поблагодарить своего редактора Мелиссу Поттер (Melissa Potter), которая проделала огромную работу, поддерживая мою мотивацию и от-

слеживая график работы над книгой, а также помогла мне привести эту книгу в пригодный для чтения вид. Я также хотел бы поблагодарить Мишель Смит (Michelle Smith), которая работала со мной над первоначальным вариантом книги, и Даниэля Эльфанбаума (Daniel Elfanbaum), который не устал отвечать на мои технические вопросы.

Выражаю огромную благодарность всем моим коллегам, друзьям и клиентам, которые потратили много часов на чтение самых ранних вариантов моих черновиков. Их отзывы оказались крайне важными, чтобы сделать книгу более понятной, а некоторые практические примеры, которыми они поделились со мной, основаны на реальных ситуациях, возникающих в Excel. Я благодарю Адама Родригеса (Adam Rodriguez), Мано Бислара (Mano Beeslar), Саймона Шигга (Simon Schiegg), Руи Да Косту (Rui Da Costa), Юрга Нагера (Jürg Nager) и Кристофа де Монтришара (Christophe de Montrichard).

Я также получил ценные замечания от читателей версии Early Release, которая была опубликована на платформе онлайн-обучения O'Reilly. Благодарю Фелипе Мариона (Felipe Maion), Рэя Доу (Ray Doue), Колю Миневски (Kolyu Minevski), Скотта Драммонда (Scott Drummond), Фолкера Рота (Volker Roth) и Дэвида Раттлса (David Ruggles)!

Мне очень повезло, что книгу рецензировали высококвалифицированные технические рецензенты, и я очень ценю их тяжелую работу, которую они проделали в условиях большого дефицита времени.

Благодарю за помощь Джордана Голдмайера (Jordan Goldmeier), Джорджа Маунта (George Mount), Андреаса Кленова (Andreas Clenow), Вернера Бронниманна (Werner Brönnimann) и Эрика Морейру (Eric Moreira)!

Особая благодарность Бьорну Штилю (Björn Stiel), который был не только техническим рецензентом, так как у него я научился многим вещам, о которых и пишу в этой книге. Мне было приятно работать с Вами последние несколько лет!

И последнее, но не менее важное: я хотел бы выразить благодарность Эрику Рейнольдсу (Eric Reynolds), который в 2016 году объединил свой проект ExcelPython с базой кода xlwings. Он также переработал весь пакет с нуля, благодаря чему мой ужасный API (интерфейс между прикладными программами и операционной системой) первых дней остался в прошлом.

ВВЕДЕНИЕ В PYTHON

Зачем нужен Python для Excel?

Обычно у пользователей Excel начинают возникать сомнения в возможностях своих инструментов для работы с электронными таблицами, когда возникают ограничения. Классический пример, когда рабочие книги Excel настолько перегружены данными и формулами, что работа компьютера в лучшем случае замедляется, а, в худшем — случится аварийное завершение программы. Тем не менее, имеет смысл проверить настройки системы, пока все не пошло наперекосяк: если вы работаете с критически важными книгами, где ошибки могут привести к финансовому или репутационному ущербу, или если вы каждый день часами обновляете книги Excel вручную, вам следует с помощью языка программирования научиться автоматизировать свои процессы. Автоматизация снижает риск человеческой ошибки и позволяет вам тратить свое время на более продуктивные задачи, чем копирование/вставка данных в электронную таблицу Excel.

В этой главе я приведу несколько аргументов, почему Python в сочетании с Excel является отличным выбором, и чем это сочетание лучше по сравнению со встроенным в Excel языком автоматизации VBA. После представления Excel в виде языка программирования и понимания его особенностей я укажу на специфические особенности, из-за которых Python по сравнению с VBA намного сильнее. Для начала, однако, давайте рассмотрим историю происхождения двух наших главных героев!

Как компьютерные технологии Excel и Python используются уже очень давно: Excel был представлен в 1985 году компанией Microsoft — и это может стать сюрпризом для многих — он был доступен только для Apple Macintosh. Только в 1987 году Microsoft Windows получила свою первую версию в виде Excel 2.0. Однако Microsoft не была первым представителем на рынке электронных таблиц: в 1979 году VisiCorp выпустила программу VisiCalc, а в 1983 году компанией Lotus Software была представлена на рынке Lotus 1-2-3. Но Excel — это не первая разработка Microsoft: тремя годами ранее они выпустили Multiplan, программу для работы с электронными таблицами, которую можно было использовать в MS-DOS и некоторых других операционных системах. Но не в Windows.

Python появился на свет в 1991 году, всего через шесть лет после Excel. В 2005 году он становится серьезной альтернативой для научных расчетов, когда впервые был выпущен *NumPy*, пакет для вычислений на основе массивов и линейной алгебры. NumPy объединил два пакета-предшественника и, таким образом, оптимизировал

все усилия по развитию научных вычислений в единый проект. Сегодня он составляет основу бесчисленных научных пакетов, включая *pandas*, вышедший в 2008 году и в значительной степени отвечающий за широкое внедрение Python в мире науки о данных и финансов, которое началось после 2010 года. Благодаря *pandas* Python, наряду с R, стал одним из наиболее часто используемых языков для решения научных задач, связанных с данными, таких как анализ данных, статистика и машинное обучение.

И то, что Python и Excel были давным-давно разработаны, не единственное, что их объединяет: Excel и Python также являются языками программирования. То, что Python — это язык программирования, это вас не удивит, а вот что касается Excel я поясню далее.

Excel как язык программирования

Начнем с представления Excel как языка программирования. Этот раздел поможет вам понять, почему в новостях так часто появляются сообщения о проблемах с электронными таблицами. Затем мы рассмотрим несколько неплохих способов, которые сообщество разработчиков программного обеспечения разработало, и которые помогут избавиться от многих типичных ошибок Excel. В заключение мы кратко познакомимся с Power Query и Power Pivot, двумя современными инструментами Excel, которые обеспечивают ту функциональность, для которой мы будем использовать *pandas*. Если вы используете Excel не только для составления списка продуктов, то вы непременно используете такие функции, как например `=SUM(A1:A4)`, для суммирования диапазона ячеек. Если вы немного поразмышляете над тем, как это работает, то заметите, что значение ячейки обычно зависит от одной или нескольких других ячеек, которые могут использовать функции, зависящие от одной или нескольких других ячеек, и т. д. Выполнение таких вложенных вызовов функций ничем не отличается от того, как работают другие языки программирования, только код пишется не в текстовых файлах, а в ячейках. И если это вас еще не убедило: в конце 2020 года Microsoft объявила о введении *лямбда-функций*, которые позволяют писать многократно используемые функции на собственном языке формул Excel, то есть без необходимости использовать другой язык, например VBA. По словам Брайана Джонса, руководителя отдела продуктов Excel, это был недостающий компонент, который наконец-то сделал Excel «настоящим» языком программирования¹. Это также означает, что пользователей Excel смело можно называть программистами Excel!

Однако есть одна особенность в работе программистов Excel: большинство из них являются бизнес-пользователями или экспертами в своей области и у них нет даже формального образования в области компьютерных наук. Это коммерсанты, бух-

¹ Вы можете прочитать анонс лямбда-функций в блоге Excel.

галтеры или инженеры, и я привел лишь несколько примеров. Их инструментарий электронных таблиц предназначен для решения бизнес-задач и часто игнорирует лучшие практики разработки программного обеспечения. Как следствие, после вычислений данный инструментарий часто смешивает входные данные, промежуточные вычисления и выходные данные на одних и тех же листах. И для их правильной работы могут потребоваться не стандартные методы, а критические изменения данных могут быть выполнены в обход системы безопасности. Другими словами, инструментам для работы с электронными таблицами не хватает надежной архитектуры приложений, и они часто не документированы и не проверены. Иногда эти проблемы могут иметь катастрофические последствия: если вы забудете пересчитать свой брокерский счет перед заключением сделки, вы можете купить или продать неправильное количество акций, что может привести к потере денег. А если, как мы увидим дальше, вы управляете не только собственными финансами, то о вашей неудаче станет известно из новостей.

Excel в новостях

Excel — постоянный герой новостей, и за время написания этой статьи две новые истории попали в заголовки газет. Первая была посвящена Комитету по номенклатуре генов HUGO, который переименовал несколько человеческих генов, чтобы Excel больше не интерпретировал их как даты. Например, чтобы предотвратить переименование гена `MARCH1` в `1-Mar`, он был переименован в `MARCHF1`². Во второй истории Excel обвинили в задержке сообщений о результатах 16 000 тестов на COVID-19 в Англии. Проблема была вызвана тем, что результаты тестирования записывались в старый формат файла Excel (`.xls`), который был ограничен примерно 65 000 строками. Это означало, что более крупные наборы данных просто отсекались за этим ограничением³. Хотя эти две истории свидетельствуют о сохраняющейся важности и доминировании Excel в современном мире, вероятно, нет другого более известного «инцидента с Excel», чем «Лондонский кит».

Лондонский кит — это прозвище трейдера, чьи торговые ошибки в 2012 году заставили JP Morgan объявить об ошеломляющих убытках в размере 6 миллиардов долларов. Источником проблемы стала модель риска стоимости на основе Excel, которая существенно недооценивала истинный риск потери денег в одном из их портфелей. Отчет компании JPMorgan Chase & Co. Management Task Force Regarding 2012 CIO Losses⁴ (2013) упоминает, что «модель работала через серию электронных таблиц Excel, которые должны были быть заполнены вручную, путем копирования и вставки данных из одной таблицы в другую». В дополнение к этим

² James Vincent, «Ученые переименовывают человеческие гены, чтобы Microsoft Excel перестал ошибочно воспринимать их как даты», The Verge, 6 августа, 2020, <https://oreil.ly/0qo-n>.

³ Leo Kelion, «Excel: Почему использование инструмента Microsoft привело к потере результатов COVID-19», BBC News, 5 октября, 2020, <https://oreil.ly/vvB6o>.

⁴ Википедия в своей статье, рассказывая об этом случае, ссылается на данный документ.

операционным проблемам у них была логическая ошибка: в одном из расчетов они делили на сумму, а не на среднее значение.

Если вы хотите изучить больше таких примеров, загляните на Horror Stories, веб-страницу, которую ведет Европейская группа по рискам электронных таблиц (European Spreadsheet Risks Interest Group, EuSpRIG). Чтобы ваша компания не попала в новости с подобной историей, давайте рассмотрим несколько передовых методов, которые сделают вашу работу с Excel намного безопаснее.

Передовые методы программирования

В этом разделе вы познакомитесь с наиболее значимыми передовыми методами программирования, включая разделение задач, принцип DRY, тестирование и контроль версий. Как мы увидим далее, эти методы будет применять проще, когда вы вместе с Excel начнете использовать Python.

Разделение задач

Одним из наиболее важных принципов проектирования в программировании является *разделение задач*, иногда также называемое *модульностью*. Это означает, что о связанном наборе функциональных возможностей должна заботиться независимая часть программы, которую можно легко заменить без ущерба для остальной части приложения. На самом высоком уровне приложение часто делится на следующие уровни⁵:

- ◆ Слой представления (Presentation Layer);
- ◆ Слой бизнес-логики (Domain Layer)
- ◆ Слой данных (Data Layer)

Чтобы объяснить эти слои, рассмотрим простой конвертер валют, похожий на тот, что показан на рис. 1.1. Вы найдете файл Excel `currency_converter.xlsx` в папке xl сопутствующего репозитория.

Вот как работает приложение: введите сумму и валюту в ячейки A4 и B4 соответственно, и Excel пересчитает их в доллары США в ячейке D4. Многие приложения для работы с электронными таблицами используют такой дизайн и применяются на предприятиях каждый день. Позвольте мне разбить приложение на составляющие его слои:

Слой представления

Это то, что вы видите и с чем взаимодействуете, то есть пользовательский интерфейс: значения ячеек A4, B4 и D4 вместе с их метками составляют экранный слой конвертера валют.

⁵ Терминология взята из Руководства по архитектуре приложений Microsoft, 2-е издание, которое доступно онлайн.

Слой бизнес-логики

Этот уровень обеспечивает логику, специфичную для приложения: ячейка D4 определяет, как сумма конвертируется в доллары США. Формула `=A4 * VLOOKUP(B4, F4:G11, 2, FALSE)` переводится как сумма, умноженная на обменный курс.

Слой данных

Как следует из названия, этот слой обеспечивает доступ к данным: часть `VLOOKUP` ячейки D4 осуществляет эту работу.

Уровень данных получает доступ к данным из таблицы курсов валют, которая начинается с ячейки F3 и служит базой данных этого небольшого приложения. Если вы были внимательны, то, вероятно, заметили, что ячейка D4 отображается на всех трех уровнях: в этом простом приложении в одной ячейке смешаны слой представления, слой бизнес-логики и слой данных.

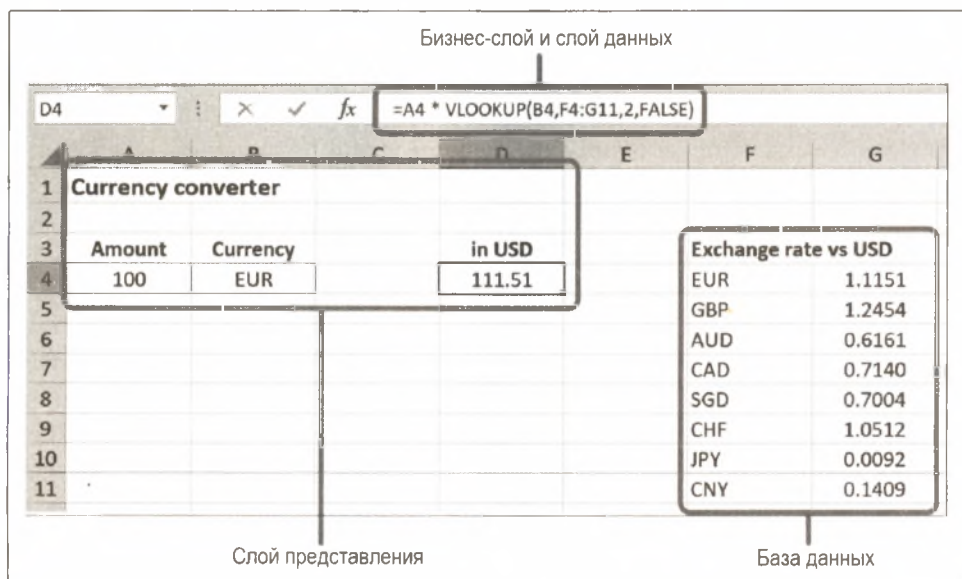


Рис. 1.1. currency_converter.xlsx

Смешение слоев не является проблемой для такого простого конвертера валют, но очень часто небольшой файл Excel вскоре превращается в гораздо более серьезное приложение. Как улучшить эту ситуацию? Большинство профессиональных ресурсов для разработчиков Excel советуют использовать отдельные листы для каждого слоя, в терминологии Excel обычно называемого *входами*, *расчетами* и *выходами*. Часто это сочетается с выбором определенного цветового фона для каждого слоя. Например, синий фон для всех входных ячеек. В *главе 11* мы построим реальное приложение на основе этих слоев: в Excel будет слой представления, а слой бизнес-логики и слой данных будут перенесены в Python, где гораздо проще правильно структурировать код.

Теперь, когда вы знаете, что такое разделение задач, давайте узнаем, что такое принцип DRY!

Принцип DRY⁶

Книга «Программист-прагматик» (авторы Э. Хант и Д. Томас, Pearson Education) популяризировала принцип DRY: *не повторяйтесь*. Отсутствие дублирования кода означает меньшее количество строк кода и меньшее количество ошибок, что облегчает сопровождение кода. Если ваша бизнес-логика сосредоточена в формулах ячеек, то применить принцип DRY практически невозможно, так как нет механизма, позволяющего повторно использовать ее в другой рабочей книге. К сожалению, это означает, чтобы запустить новый проект Excel, необходимо скопировать рабочую книгу из предыдущего проекта или из шаблона. Если вы пишете на VBA, то наиболее распространенным фрагментом кода, который используется многократно, является *функция*.

Функция дает вам доступ к одному и тому же блоку кода, например из нескольких макросов. Если у вас есть несколько функций, которые вы постоянно используете, вам может потребоваться разделить их между рабочими книгами. Стандартный инструмент VBA для обмена кодом между рабочими книгами — это дополнения. Но для дополнений VBA не хватает надежного способа их распространения и обновления. Хотя Microsoft представила внутренний архив дополнений Excel для решения этой проблемы, он работает только с дополнениями на основе JavaScript, так что это не вариант для разработчиков программ на VBA. Это означает, что в VBA все еще очень часто используется подход «копировать/вставить». Предположим, что вам нужна функция *кубического сплайна* в Excel. Функция кубического сплайна представляет собой способ интерполяции кривой по нескольким заданным точкам в системе координат и часто используется трейдерами с фиксированным доходом для построения кривой процентных ставок для всех сроков погашения на основе нескольких известных комбинаций сроков погашения и процентных ставок. Если вы введете в поисковике запрос «Кубический сплайн в Excel», потребуется немного времени, чтобы получить страницу кода VBA, который выполнит то, что вы задумали. Но дело в том, что чаще всего эти функции были написаны одним человеком, возможно, с благими намерениями, но без формального документирования и тестирования. Возможно, они работают для большинства входов. Но как быть, если возникает нестандартная ситуация? Если вы управляете многомиллионным портфелем с фиксированным доходом, вы хотите иметь то, чему вы можете доверять. По крайней мере, именно это вы услышите от своих внутренних аудиторов, когда они узнают, откуда берется код.

Python позволяет легко распространять код с помощью менеджера пакетов, и это будет показано в последнем разделе данной главы. Однако прежде чем перейти к этой теме, давайте поговорим о тестировании — одном из краеугольных камней надежной разработки программного обеспечения.

⁶ Принцип DRY, или Don't Repeat Yourself — Не повторяйся, заключается в уменьшении сложности кода и снижения количества повторяющейся информации.

Тестирование

Когда вы попросите разработчика Excel протестировать свою рабочую книгу, он, скорее всего, выполнит несколько случайных проверок: нажмет кнопку и посмотрит, делает ли макрос то, что должен делать, или изменит несколько входных данных и проверит, будет ли выход данных правильным. Однако это рискованная стратегия: в Excel при вводе формул или данных легко можно ошибиться, не заметив ошибки, и, впоследствии эту ошибку будет трудно обнаружить. Например, вы можете перезаписать формулу с жестко заданным значением. Или вы корректируете формулу в скрытом столбце.

Если вы попросите профессионального разработчика программного обеспечения протестировать свой код, он напишет *модульные тесты*. Как следует из названия, это механизм для тестирования отдельных модулей вашей программы. Например, модульные тесты проверяют, что отдельная функция программы работает правильно. Большинство языков программирования предлагают способ автоматического запуска модульных тестов. Выполнение автоматизированных тестов значительно повышает надежность вашей кодовой базы и позволяет быть уверенным в том, что при редактировании кода вы не сломаете ничего из того, что сейчас работает.

Если вы посмотрите на инструмент конвертации валют, показанный на рис. 1.1, вы можете написать тест, который проверяет, правильно ли формула в ячейке D4 возвращает 105 долларов США при следующих входных данных: 100 EUR в качестве суммы и 1.05 в качестве обменного курса EURUSD. Зачем это нужно? Предположим, что вы случайно удалили ячейку D4 с формулой пересчета, и вам приходится ее переписывать: вместо того, чтобы умножать сумму на обменный курс, вы делите на нее — в конце концов, работа с валютами может сбивать с толку. Когда вы запустите вышеприведенный тест, вы получите ошибку, так как при делении 100 EUR на 1.05 105 USD не получится, как это ожидалось при тестировании. Таким образом, вы можете обнаружить и исправить формулу до того, как передадите электронную таблицу своим пользователям.

Практически все традиционные языки программирования предлагают один или несколько тестовых механизмов для написания модульных тестов без особых усилий — но только не Excel. К счастью, концепция модульных тестов достаточно проста, и, соединив Excel с Python, вы получаете доступ к мощным механизмам модульного тестирования Python. Хотя более подробное представление модульных тестов выходит за рамки этой книги, я рекомендую вам ознакомиться с моей статьей в блоге, где я рассказываю об этой теме на практических примерах.

Модульные тесты часто настраиваются на автоматический запуск при фиксации кода в системе контроля версий. В следующем разделе объясняется, что такое системы контроля версий и почему их затруднительно использовать с файлами Excel.

Контроль версий

Еще одной характеристикой профессиональных программистов является то, что они используют систему для *контроля версий* или *управления версиями*. Система

контроля версий (VCS) все время отслеживает изменения в вашем исходном коде, позволяя вам видеть, кто, что, когда и по какой причине изменил, и позволяет вам вернуться к старым версиям в любой момент времени. Наиболее популярной системой контроля версий в настоящее время является Git. Изначально он был создан для управления исходным кодом Linux и с тех пор завоевал мир программирования — даже Microsoft в 2017 году приняла Git для управления исходным кодом Windows. В мире Excel, напротив, наиболее популярная система контроля версий представлена в виде папки, где файлы архивируются следующим образом:

```
currency_converter_v1.xlsx
currency_converter_v2_2020_04_21.xlsx
currency_converter_final_edits_Bob.xlsx
currency_converter_final_final.xlsx
```

Если, в отличие от данного примера, разработчик Excel придерживается определенного соглашения в наименовании файлов, это не страшно. Но хранение истории версий ваших файлов на компьютере лишает вас возможности пользоваться важными аспектами контроля исходных текстов, такими как облегчение совместной работы, рецензирование, процессы подписания и журналы аудита. И если вы собираетесь сделать свои рабочие книги более надежными и стабильными, вы не захотите упустить эти моменты. Чаще всего профессиональные программисты используют Git в таких веб-платформах, как GitHub, GitLab, Bitbucket или Azure DevOps. Эти платформы позволяют работать с так называемыми *pull requests* или *merge requests*. Pull request предоставляет следующую информацию:

- ◆ кто является автором изменений;
- ◆ когда были внесены изменения;
- ◆ какова цель изменений, описанных в коммите;
- ◆ каковы детали изменений, показанные при помощи команды *diff*, т.е. в формате файла для сравнения, где выделены изменения зеленым цветом для нового кода и красным для удаленного кода.

Это позволит коллеге или руководителю группы просмотреть изменения и выявить нарушения. Часто дополнительная пара глаз может заметить пару багов или дать иную ценную информацию программисту. Учитывая все эти преимущества, становится интересно, почему же разработчики Excel предпочитают использовать локальную файловую систему и собственные соглашения об именах вместо профессиональной системы, такой как Git?

- ◆ Многие пользователи Excel просто не знают о Git или отказываются от него в самом начале, поскольку Git имеет довольно сложную систему обучения.
- ◆ Git позволяет нескольким пользователям параллельно работать над локальными копиями одного и того же файла. После того как все они сохраняют внесенные в файл изменения, Git объединит все изменения автоматически, без какого-либо ручного вмешательства. Но для файлов Excel это не срабатывает: если файлы изменяются одновременно в различных копиях одного и того же файла, Git не знает, как объединить эти изменения обратно в один файл.

- ◆ Даже если вы разберетесь с предыдущими проблемами, Git просто не может обеспечить такой же эффективной работы с файлами Excel, как с текстовыми файлами: он не способен отображать изменения между файлами Excel, что затрудняет нормальный процесс экспертной оценки.

Из-за всех этих проблем моя компания разработала xltrail — систему контроля версий на основе Git, которая умеет работать с файлами Excel. xltrail маскирует сложность Git, чтобы пользователям было удобно с ним работать, а также, если вы уже отслеживаете свои файлы, например, с помощью GitHub, позволяет подключаться к внешним Git-системам. xltrail отслеживает различные компоненты рабочей книги, включая формулы ячеек, именованные диапазоны, Power Queries и код VBA, позволяя вам реализовать классические преимущества контроля версий, включая рецензирование.

Еще один вариант облегчить контроль версий в Excel — перенести логические операции из Excel в файлы Python, что мы сделаем в *главе 10*. Так как файлы Python легко отслеживать с помощью Git, самая важная часть вашего инструмента для работы с таблицами будет под контролем.

Хотя этот раздел называется «*Передовые методы программирования*», в нем в основном рассказывается, почему эти методы труднее применять в Excel, чем в традиционном языке программирования, таком как Python. И прежде чем мы обратимся к Python, я хотел бы кратко представить Power Query и Power Pivot — попытки Microsoft модернизировать Excel.

Современный Excel

Современная эра Excel началась с Excel 2007, когда появились ленточное меню и новые форматы файлов (например, *xlsx* вместо *xls*). Однако сообщество пользователей Excel использует *модернизированный* Excel для использования инструментов, которые были добавлены в Excel 2010: прежде всего, Power Query и Power Pivot. Они позволяют подключаться к внешним источникам данных и анализировать данные, размер которых не позволяет вставить их в электронную таблицу. Поскольку их функциональные возможности пересекаются с теми, что мы будем делать с pandas в *главе 5*, я кратко представлю набор функциональных возможностей в первой части этого раздела. Вторая часть посвящена Power BI, который можно описать как отдельное приложение для бизнес-аналитики, сочетающее в себе функциональность Power Query и Power Pivot с возможностями визуализации, а также встроенную поддержку Python!

Power Query и Power Pivot

В 2010 году компания Microsoft представила дополнение для Excel под названием *Power Query*. Power Query подключается к различным источникам данных, включая рабочие книги Excel, файлы CSV и базы данных SQL. Он также предлагает под-

ключение к таким платформам, как Salesforce, и даже может быть расширен для подключения к системам, не входящим в комплект поставки. Основная функция Power Query — это работа с наборами данных, которые слишком велики, чтобы поместиться в электронную таблицу. После загрузки данных можно выполнить дополнительные действия по их очистке и обработке, чтобы они попали в Excel в пригодном для использования виде. Например, можно разделить столбец на два, объединить две таблицы или отфильтровать и сгруппировать данные. Начиная с Excel 2016, Power Query больше не является надстройкой, а может быть доступен непосредственно на вкладке **Данные**, нажатием кнопки **Получить данные**. В Mac OS Power Query доступен только частично, однако он активно разрабатывается, поэтому он должен быть полностью поддержан в следующей версии Excel.

Power Pivot идет рука об руку с Power Query: фактически это второй шаг после получения и очистки данных с помощью Power Query. Power Pivot помогает вам анализировать и представлять данные в удобном виде непосредственно в Excel. Рассматривайте его как традиционную сводную таблицу, которая, как и Power Query, может работать с большими наборами данных. Power Pivot позволяет определять формальные модели данных с взаимосвязями и иерархиями, а также добавлять вычисляемые столбцы с помощью языка формул DAX. Power Pivot появился в Excel 2010, но остается надстройкой и пока недоступен для mac OS.

Если вам нравится работать с Power Query и Power Pivot и вы собираетесь создавать на их основе информационные панели, возможно, стоит обратить внимание на Power BI, и давайте посмотрим, почему!

Power BI

Power BI — это отдельное приложение, которое было выпущено в 2015 году. Это ответ Microsoft на такие инструменты бизнес-анализа, как Tableau или Qlik. Power BI Desktop распространяется бесплатно, поэтому если вы хотите попробовать, как он работает, перейдите на домашнюю страницу Power BI и загрузите приложение. Обратите внимание, Power BI Desktop доступен только для Windows. Power BI предназначен для анализа больших наборов данных путем их визуализации в интерактивных инструментальных панелях. По своей сути он использует те же функции Power Query и Power Pivot, что и Excel. Коммерческие программы позволяют вам сотрудничать и обмениваться панелями мониторинга онлайн, но они отличаются от настольной версии. Основной причиной, почему мы рассматриваем Power BI в этой книге является то, что с 2018 года он поддерживает сценарии на Python. Python можно использовать как для части запроса, так и для части визуализации, воспользовавшись его библиотеками для построения графиков. На мой взгляд, использовать Python в Power BI не очень удобно, но главное здесь то, что Microsoft признала важность Python для анализа данных. Соответственно, есть большая надежда, что однажды Python найдет официальный путь и в Excel.

Так что же такого особенного в Python, что он попал в Power BI от Microsoft? В следующем разделе мы дадим на это несколько ответов.

Python для Excel

Excel включает в себе все, что нужно для хранения, анализа и визуализации данных. А поскольку Python наиболее эффективен в области научных вычислений, он идеально сочетается с Excel. Python является одним из немногих языков, который подходит как для профессионального программиста, так и для начинающего пользователя, который пишет несколько строк кода раз в несколько недель. Профессиональные программисты, с одной стороны, любят работать с Python, потому что это язык программирования общего назначения и, следовательно, позволяет вам достичь практически всего, не «прыгая через обручи». С другой стороны, новичкам нравится Python, потому что его легче изучать, чем другие языки. Как следствие, этот язык используется не только для специального анализа данных, но и для выполнения небольших задач по автоматизации, например в огромных производственных кодовых базах, таких как серверная часть Instagram⁷. Это означает, что когда ваш инструмент Excel на базе Python станет действительно востребованным, к проекту можно подключить веб-разработчика, который превратит ваш прототип Excel-Python в полноценное веб-приложение. Уникальность Python заключается в том, что часть с бизнес-логикой, скорее всего, не нужно переписывать, а можно перенести как есть из прототипа Excel в производственную веб-среду.

В этом разделе я представляю основные принципы Python и сравню их с Excel и VBA. Я затрону вопросы читабельности кода, стандартной библиотеки и менеджера пакетов Python, стека научных вычислений, современных возможностей языка и кроссплатформенной совместимости.

Читабельность и эксплуатационная пригодность

Если ваш код читабелен, это означает, что его легко выполнять и понимать — особенно посторонним людям, которые сами этот код не писали. Это облегчает выявление ошибок и сопровождение кода в дальнейшем. Вот почему одна из строк в «The Zen of Python»⁸ звучит так: «Читабельность имеет значение». «The Zen of Python» — это краткое изложение основных принципов проектирования Python, и мы познакомимся с ними в следующей главе. Давайте рассмотрим следующий фрагмент кода в VBA:

```
If i < 5 Then
    Debug.Print "i is smaller than 5"
ElseIf i <= 10 Then
    Debug.Print "i is between 5 and 10"
Else
    Debug.Print "i is bigger than 10"
End If
```

⁷ Подробнее о том, как Instagram использует Python, вы можете узнать в их инженерном блоге.

⁸ Разработчики языка Python придерживаются определенной философии программирования, называемой «The Zen of Python» («Дзен Python»). — *Примеч. ред.*

В VBA вы можете переформатировать этот фрагмент кода в следующий, который полностью соответствует коду, показанному выше:

```
If i < 5 Then
    Debug.Print "i is smaller than 5"
ElseIf i <= 10 Then
    Debug.Print "i is between 5 and 10"
Else
    Debug.Print "i is bigger than 10"
End If
```

В первом варианте визуальный отступ выравнивается с логикой кода. Это облегчает чтение и понимание кода, что опять же облегчает выявление ошибок. Во втором варианте разработчик, который впервые знакомится с кодом, при первом взгляде на него может и не заметить условия `ElseIf` и `Else`. Вероятность такой ошибки увеличивается, если код является частью большой кодовой базы.

Python не принимает код, отформатированный так, как показано во втором примере: он вынудит вас выровнять визуальные отступы согласно с логикой кода, что предотвратит проблемы с читабельностью. Python может это сделать, потому что он использует отступы для определения блоков кода, когда вы используете их в операторах `if` или циклах `for`. Вместо отступов большинство других языков используют фигурные скобки, а VBA использует такие ключевые слова, как `End If`, как было показано в фрагментах кода. Отступы для блоков кода применяются, потому что в программировании большая часть времени тратится на поддержание кода, а не на его написание. Наличие хорошо читаемого кода помогает новым программистам (или вам самим через несколько месяцев после написания кода) вернуться назад и понять, что происходит.

Все о правилах отступов мы узнаем в Python в *главе 3*, а пока перейдем к стандартной библиотеке: функционалу, который поставляется вместе с Python.

Стандартная библиотека и менеджер пакетов

Python поставляется с обширным набором встроенных функций, предоставляемых *стандартной библиотекой*. Сообщество Python любит ссылаться на это, говоря, что Python поставляется с «батареями в комплекте». Если вам нужно распаковать ZIP-файл, прочитать значения из CSV-файла или получить данные из Интернета, это все можно сделать с помощью стандартной библиотеки Python, как правило, написав всего несколько строк кода. В VBA для выполнения этих функций потребовалось бы написать гораздо больше строк кода или установить надстройку. И зачастую решения, которые вы найдете в интернете, применимы только к Windows, но не к macOS.

В то время как стандартная библиотека Python предоставляет впечатляющий объем функциональных возможностей, все еще существуют задачи, трудоемкие для программирования или медленно работающие, если полагаться только на стандартную библиотеку. Здесь на помощь приходит PyPI. PyPI обозначает *Python Package Index* и является гигантским репозиторием, куда каждый (в том числе и вы!) может загрузить

зить пакеты Python с открытым исходным кодом, которые добавляют дополнительную функциональность в Python.



PyPI по сравнению с PyPy

PyPI произносится как «pie rea eye». Это делается, чтобы отличить PyPI от PyPy, который произносится как «pie pie» и является быстродействующей альтернативой реализации Python.

Например, чтобы упростить получение данных из источников в Интернете, вы, с целью получить доступ к набору команд, которые не только эффективны, но и просты в использовании, можете установить пакет Requests. Для его установки в Python следует использовать менеджер пакетов *pip*, который запускается в командной строке или терминале. *pip* — это рекурсивная аббревиатура для *pip installs packages* (пакеты установки *pip*). Не волнуйтесь, если сейчас это звучит немного абстрактно; я подробно объясню, как это работает, в следующей главе. Сейчас главное понять, почему менеджеры пакетов так важны. Одна из основных причин заключается в том, что любой надежный пакет будет зависеть не только от стандартной библиотеки Python, но и от других пакетов с открытым исходным кодом, которые также размещены в PyPI. Эти зависимости могут снова зависеть от подзависимостей и так далее. *pip* рекурсивно проверяет зависимости и подзависимости пакета, загружает и устанавливает их. Он также позволяет легко обновлять пакеты, чтобы вы могли поддерживать свои зависимости в актуальном состоянии. Это значительно упрощает соблюдение принципа DRY, поскольку вам не нужно изобретать или копировать/вставлять то, что уже доступно на PyPI. Благодаря *pip* и PyPI у вас также появится надежный механизм для распространения и установки этих зависимостей, чего не хватает Excel с его традиционными дополнениями.

Программное обеспечение с открытым исходным кодом (OSS)

Теперь я хотел бы сказать несколько слов об открытом исходном коде, поскольку в этом разделе я несколько раз использовал это слово. Если программное обеспечение распространяется под лицензией с открытым исходным кодом, это означает, что его исходный код находится в свободном бесплатном доступе, что позволяет каждому вносить новые функциональные возможности, исправлять ошибки или предоставлять документацию. Сам Python и почти все сторонние пакеты Python имеют открытый исходный код, который поддерживается разработчиками в свободное время. Нельзя сказать, что это оптимальное решение: если ваша компания использует определенные пакеты, то лучше, чтобы эти пакеты разрабатывались и поддерживались профессиональными программистами. К счастью, научное сообщество Python признало, что некоторые пакеты настолько важны, что не стоит доверять их разработку и поддержку нескольким добровольцам, которые работают по вечерам и в выходные дни.

Именно поэтому в 2012 году была создана некоммерческая организация NumFOCUS, которая занимается спонсированием различных пакетов и проектов на языке Python в области научных вычислений. Наиболее популярными пакетами Python, спонсируемыми NumFOCUS, являются pandas, NumPy, SciPy, Matplotlib и Project Jupyter, но в настоящее время NumFOCUS также поддерживает пакеты на других языках, включая R, Julia и JavaScript. Есть несколько крупных корпоративных спонсоров, но каждый может присоединиться к NumFOCUS в качестве бесплатного члена сообщества — при этом пожертвования подлежат налогообложению.

С помощью `pip` можно установить пакеты практически для всего, но для пользователей Excel одними из самых популярных считаются пакеты для научных вычислений. Давайте узнаем немного больше о научных вычислениях с использованием Python в следующем разделе!

Научные вычисления

Важной особенностью успеха Python является тот факт, что он был создан как язык программирования общего назначения. Возможности для научных вычислений были добавлены позже в виде пакетов от сторонних разработчиков. Особенность такого подхода заключается в том, что специалист по изучению данных может использовать тот же язык для экспериментов и исследований, что и веб-разработчик, который в конечном итоге может создать готовое к производству приложение на основе вычислительного ядра. Возможность создавать научные приложения на основе одного языка снижает трудозатраты и время внедрения. Научные пакеты, такие как NumPy, SciPy и pandas, обеспечивают нам доступ к очень простому способу формулирования математических задач. В качестве примера рассмотрим одну из наиболее известных финансовых формул, используемых для расчета дисперсии портфеля⁹ в соответствии с современной теорией портфеля (Modern Portfolio Theory/ portfolio variance):

$$\sigma^2 = w^T C w$$

Дисперсия портфеля обозначается σ^2 ; w — это вектор весов отдельных активов; C — ковариационная матрица портфеля. Если w и C — это диапазоны Excel, вы можете вычислить дисперсию портфеля в VBA следующим образом:

```
variance = Application.MMult(Application.MMult(Application.Transpose(w), C), w)
```

Сравните это с почти математической нотацией в Python, предполагая, что w и C являются фреймами данных pandas или массивами Numpy (я официально представляю их в части II):

```
variance = w.T @ C @ w
```

⁹ Дисперсия портфеля — это процесс, который определяет степень риска или волатильности, связанной с инвестиционным портфелем. — *Примеч. пер.*

Но дело не только в эстетике и удобочитаемости: NumPy и pandas используют скомпилированный код Fortran и C под оболочкой, что дает, по сравнению с VBA, прирост производительности при работе с большими матрицами.

Отсутствие поддержки научных вычислений является очевидным ограничением в VBA. Но, как я покажу в следующем разделе, даже если рассматривать основные возможности языка, VBA отстает.

Особенности современного языка

Начиная с Excel 97, язык VBA в плане языковых возможностей не претерпел существенных изменений. Это, однако, не означает, что VBA больше не поддерживается: Microsoft поставляет обновления с каждым новым выпуском Excel, чтобы иметь возможность автоматизировать новые функции, представленные в этом выпуске. Например, в Excel 2016 добавлена поддержка автоматизации Power Query. Язык, который перестал развиваться более двадцати лет назад, упускает современные языковые концепции, которые были введены во все основные языки программирования за эти годы. Например, обработка ошибок действительно свидетельствует о возрасте VBA. Если вы хотите аккуратно обработать ошибку в VBA, это делается примерно так:

```
Sub PrintReciprocal(number As Variant)
    ' There will be an error if the number is 0 or a string
    On Error GoTo ErrorHandler
        result = 1 / number
    On Error GoTo 0
    Debug.Print "There was no error!"
Finally:
    ' Runs whether or not an error occurs
    If result = "" Then
        result = "N/A"
    End If
    Debug.Print "The reciprocal is: " & result
    Exit Sub
ErrorHandler:
    ' Runs only in case of an error
    Debug.Print "There was an error: " & Err.Description
    Resume Finally
End Sub
```

Как показано в приведенном примере, обработка ошибок в VBA предполагает использование таких *меток*, как `Finally` и `ErrorHandler`. Вы инструктируете код переходить к этим меткам с помощью операторов `GoTo` или `Resume`. Уже на раннем этапе было признано, что метки ответственны за то, что многие программисты называют «спагетти-кодом»¹⁰: наглядный способ заявить, что поток кода трудно отслежи-

¹⁰ Плохо спроектированная, не структурированная, запутанная и трудная для понимания программа. — Примеч. ред.

вать и, следовательно, трудно поддерживать. Именно поэтому практически все интенсивно развивающиеся языки ввели механизм `try/catch` — в Python он называется `try/except` — который я представлю в *главе 11*. Если вы опытный разработчик VBA, вас может заинтересовать, что Python поддерживает наследование классов — свойство объектно-ориентированного программирования, отсутствующее в VBA.

Помимо современных языковых возможностей, существует еще одно требование к современному языку программирования: кросс-платформенная совместимость. Давайте разберемся, почему это так важно!

Кросс-платформенная совместимость

Даже если вы разрабатываете свой код на локальном компьютере, работающем под управлением Windows или Mac OS, очень вероятно, что в какой-то момент вы захотите запустить свою программу на сервере или в облаке. Серверы выполняют ваш код по расписанию и обеспечивают доступ к вашему приложению отовсюду, откуда вы захотите, с необходимой вам вычислительной мощностью. В следующей главе я на практике покажу вам, как запускать код Python на сервере, рассказав о расположенных на сервере Jupyter Notebook. Подавляющее большинство серверов работает под управлением Linux, так как это стабильная, безопасная и экономически эффективная операционная система. А поскольку программы на Python работают без изменений на всех основных операционных системах, это избавит вас от многих проблем при переходе с локальной машины на промышленную.

И несмотря на то, что Excel VBA работает в Windows и macOS, довольно легко внедрить функции, которые работают только в Windows. В официальной документации по VBA или на форумах часто можно встретить код, подобный этому:

```
Set fso = CreateObject("Scripting.FileSystemObject")
```

Если у вас есть запрос `CreateObject` или вам, чтобы добавить ссылку, в редакторе VBA предлагают перейти в меню **Tools > References**, вы скорее всего, имеете дело с кодом, который будет работать только в Windows. Еще одной важной особенностью, за которой нужно следить, если вы хотите, чтобы ваши файлы Excel работали в Windows и macOS, — это *элементы управления ActiveX*. Элементы управления ActiveX — это такие компоненты, как кнопки и открывающиеся окна, которые можно разместить на ваших листах. Но они работают только в Windows. Постарайтесь избегать их, если вы хотите, чтобы ваша рабочая книга работала и на операционной системе macOS!

Заключение

В этой главе мы познакомились с Python и Excel, двумя очень популярными технологиями, существующими уже несколько десятилетий, что очень много по сравнению со многими другими технологиями, которые мы используем сегодня. «Лон-

донский кит»¹¹ послужил примером того, как может многое пойти не по плану (в долларовом отношении), когда вы не используете Excel соответствующим образом с критически важными рабочими книгами. Это подвигло нас изучить минимальный набор передовых методов программирования: разделение задач, следование принципу DRY, использование автоматизированного тестирования и контроля версий. Далее мы рассмотрели Power Query и Power Pivot — подход Microsoft к обработке данных, размер которых больше, чем может обработать ваша электронная таблица. Однако я считаю, что это не всегда правильное решение, так как ограничивает вас в мире Microsoft и не позволяет воспользоваться гибкостью и мощностью современных облачных решений.

Python обладает неоспоримыми возможностями, которых нет в Excel: стандартная библиотека, менеджер пакетов, библиотеки для научных вычислений и кроссплатформенная совместимость. Изучив сочетание Excel и Python, вы сможете получить лучшее из обоих миров и сэкономить время за счет автоматизации; совершить меньше ошибок, поскольку легче следовать передовым методам программирования; сможете перенести свое приложение и масштабировать его за пределы Excel, если это когда-нибудь понадобится.

Теперь, когда вы знаете, почему Python является таким мощным помощником для Excel, пришло время настроить среду разработки, чтобы иметь возможность написать свои первые строки кода на Python!

¹¹ Прозвище трейдера JP Morgan Брюно Мишель Иксиль, потерявшего минимум \$6,2 млрд. — *Примеч. ред.*

Среда разработки

Вам, вероятно, не терпится изучить основы Python, но прежде чем мы приступим к этому, сначала необходимо правильно настроить свой компьютер. Чтобы написать код VBA или Power Queries, достаточно запустить Excel и открыть редактор VBA или Power Query. В случае с Python все немного сложнее.

Мы начнем эту главу с установки дистрибутива Anaconda Python. Помимо установки Python, Anaconda также предоставит нам доступ к Anaconda Prompt и Jupyter Notebook — двум важным инструментам, которые мы будем использовать на всех этапах этой книги. *Anaconda Prompt* — это для Windows специальная командная строка или, если у вас macOS, терминал. Anaconda Prompt позволяет запускать скрипты Python и другие инструменты командной строки, с которыми мы познакомимся в этой книге. *Блокноты*¹ Jupyter позволяют работать с данными, кодом и диаграммами в интерактивном режиме, что делает их серьезным конкурентом рабочим книгам Excel. После знакомства с блокнотами Jupyter мы установим *Visual Studio Code* (VS Code), мощный текстовый редактор. VS Code хорошо подходит для написания, запуска и отладки сценариев Python и поставляется со встроенным Терминалом. На рис. 2.1 показано, что входит в состав Anaconda и VS Code.

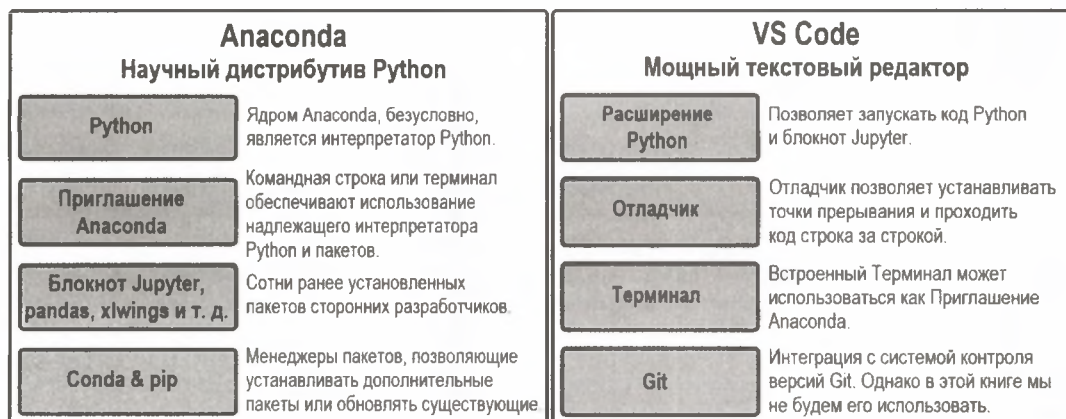


Рис. 2.1. Среда разработки

¹ Блокнот — проект, созданный в Jupyter. — Примеч. науч ред.

Поскольку эта книга посвящена Excel, в данной главе я уделю внимание Windows и macOS. Однако все приложения, которые мы упомянули, вплоть до *части III*, работают и в Linux. Для начала установим Anaconda!

Дистрибутив Anaconda Python

Anaconda, пожалуй, самый популярный дистрибутив Python, используемый для работы с данными, и поставляется с сотнями предустановленных пакетов сторонних разработчиков. В дистрибутив входят блокноты Jupyter и большинство других пакетов, которые широко используются в этой книге, включая pandas, OpenPyXL и xlwings. Anaconda Individual Edition предоставляется бесплатно для частного использования и гарантирует совместимость всех включенных пакетов друг с другом. Пакет приложений, входящих в дистрибутив, устанавливается в одну папку и может быть легко деинсталлирован снова. После его установки мы изучим несколько основных команд в Anaconda Prompt и запустим интерактивный Python. Затем мы познакомимся с менеджерами пакетов Conda и pip, после чего завершим этот раздел средами Conda. Давайте начнем с загрузки и установки Anaconda!

Установка

Перейдите на домашнюю страницу Anaconda² и загрузите последнюю версию программы Anaconda (Individual Edition). Убедитесь, что вы загрузили 64-битную графическую программу установки для версии Python 3.x³. После загрузки дистрибутива, чтобы начать процесс установки, дважды щелкните мышью на значке программы установки и выберите все настройки программы по умолчанию. Для получения более подробных инструкций по установке следуйте официальной документации⁴.



Другие дистрибутивы Python

Хотя в этой книге приводя инструкции, мы предполагали, что у вас установлена Anaconda Individual Edition, представленный код и основные понятия применимы и к любой другой установке Python. В этом случае вам придется установить требуемые зависимости, следуя инструкциям, представленным в файле *requirements.txt* в сопутствующем репозитории.

² Адрес домашней страницы Anaconda: <https://oreil.ly/QV7Na>.

³ 32-битные системы существуют только для операционной системы Windows и встречаются редко. Простой способ узнать, какая у вас версия Windows, — отобразить в проводнике содержимое диска C:. Если вы увидите папки Program Files и Program Files (x86), значит у вас установлена 64-разрядная версия Windows. Если на диске C: находится только папка Program Files, значит у вас установлена 32-разрядная версия операционной системы Windows.

⁴ Адрес официальной страницы с документацией: <https://oreil.ly/r01wn>.

Установив Anaconda, мы можем начать использовать Anaconda Prompt. Давайте посмотрим что он собой представляет и как работает!

Anaconda Prompt

Anaconda Prompt (Приглашение Anaconda) — это командная строка в Windows и терминал в macOS, которые настроены для работы с правильным интерпретатором Python и сторонними пакетами. Anaconda Prompt — это самый простой инструмент для запуска кода на Python, и в этой книге мы будем широко использовать его для запуска скриптов на Python и всевозможных инструментов командной строки, предлагаемых различными пакетами.



Anaconda Prompt без Anaconda

Если вы не устанавливали дистрибутив Anaconda Python, всякий раз, когда я буду предлагать вам вводить команды в Anaconda Prompt, вам придется использовать *командную строку* операционной системы Windows или *терминал* операционной системы macOS.

Если в операционной системе Windows вы никогда не пользовались командной строкой или терминалом в операционной системе macOS, не волнуйтесь: вам нужно знать лишь несколько полезных команд. Когда вы немного привыкните к Anaconda Prompt, то заметите, что выполнять команды с помощью этого приложения часто оказывается удобнее и быстрее, чем прохождение через графические меню пользователя. Приступим:

Windows

Нажмите кнопку меню **Пуск** и введите в поле ввода поиска ключевое слово **Anaconda Prompt**. В появившемся списке выберите строку **Anaconda Prompt**, но не **Anaconda Powershell Prompt**. Выбрать нужную строку можно с помощью клавиш со стрелками, после чего нажать клавишу <Enter> или щелкнуть на этой строке кнопкой мыши. Если вы предпочитаете запускать программу с помощью меню **Пуск**, вы найдете это приложение в папке Anaconda3. Так как это приложение будет использоваться на протяжении всей книги, желательно закрепить ярлык Anaconda Prompt на панели задач Windows. Строка ввода в Anaconda Prompt начинается с (base):

```
(base) C:\Users\felix>
```

macOS

В macOS вы не найдете приложения под названием Anaconda Prompt. Под Anaconda Prompt я подразумеваю терминал, который был настроен программой установки Anaconda для автоматической активации окружения Conda (об окружениях Conda я расскажу чуть позже): нажмите комбинацию клавиш <Com-

mand>-<Пробел> или откройте панель запуска, затем введите слово **Терминал** и нажмите клавишу <Ввод>. Или откройте **Finder** и перейдите в раздел **Приложения > Утилиты** (Applications > Utilities). Здесь вы увидите название приложения Терминал, для запуска которого дважды щелкните на нем мышью. В открытом терминале строка ввода должна начинаться с (base):

```
(base) felix@MacBook-Pro ~ %
```

Если у вас более старая версия macOS, строка ввода будет выглядеть примерно так:

```
(base) MacBook-Pro:~ felix$
```

В отличие от командной строки в Windows, терминал в macOS не показывает полный путь к текущему каталогу. Вместо этого домашний каталог, которым обычно является /Users/<имя пользователя>, будет обозначен тильдой. Чтобы посмотреть полный путь к текущему каталогу, введите команду **pwd** и нажмите клавишу <Enter>. С помощью команды **pwd** будет показано *имя текущей папки*, в которой мы находимся. Если после установки Anaconda строка ввода в вашем терминале не начинается с приглашения (base), это может быть вызвано следующими причинами: во время установки Anaconda у вас был запущен терминал, поэтому перезапустите его. Обратите внимание, если вы нажмете на красный крестик в верхней левой части окна терминала, вы не завершите работу программы, а только скроете окно приложения. Для завершения работы приложения щелкните правой кнопкой мыши на окне терминала и выберите из появившегося меню команду **Выход** (Exit) или щелкните кнопкой мыши на окне терминала, чтобы активировать его, и нажмите комбинацию клавиш <Command>-<Q>. Если после перезапуска терминала в начале строки вы увидите приветствие (base), значит, установка программы прошла успешно. Рекомендуется установить терминал на свой компьютер, так как это приложение будет использоваться на протяжении всей книги.

Запустив Anaconda Prompt, попробуйте выполнить команды, приведенные в табл. 2.1. Более подробные объяснения о каждой команде вы найдете после таблицы.

Таблица 2.1. Команды для Anaconda Prompt

Команда	Windows	macOS
Список файлов в текущем каталоге	dir	ls -la
Изменить каталог (относительный)	cd path\to\dir	cd path/to/dir
Изменить каталог (абсолютный)	cd C:\path\to\dir	cd /path/to/dir
Переход к диску D	D:	(не существует)
Переход в родительский каталог	cd ..	cd ..
Просмотр ранее введенных команд	↑ (стрелка вверх)	↑ (стрелка вверх)

Список файлов в текущем каталоге

В Windows, находясь в текущей папке, введите команду `dir` и нажмите клавишу `<Enter>`. В результате будет выведено содержимое каталога, в котором вы сейчас находитесь.

В macOS введите команду `ls -la` и нажмите клавишу `<Enter>`. `ls` — это сокращение от *list directory contents* (список содержимого каталога). `-la` выводит данные в формате длинного списка, отображая все файлы, включая скрытые.

Изменить каталог

Введите команду `cd Down` и нажмите клавишу `<Tab>`. `cd` — это сокращение *change directory* (сменить каталог). Если вы находитесь в своей домашней папке, то Anaconda Prompt, скорее всего, автоматически откроет папку `Downloads` (Загрузки). Если вы находитесь в другой папке или у вас нет папки с названием `Downloads`, то прежде чем нажать клавишу `<Tab>` для автозаполнения, введите начало названия папки, в которую вы желаете перейти, и которую видели, введя предыдущую команду (`dir` или `ls -la`), а затем нажмите клавишу `<Tab>`. Полное название папки автоматически появится в командной строке. Когда папка будет выбрана, нажмите клавишу `<Enter>`, чтобы перейти в требуемый каталог. Если у вас операционная система Windows и вам нужно перейти на диск, прежде чем вы сможете перейти в нужную директорию, введите сначала имя диска:

```
C:\Users\felix> D:
D:\> cd data
D:\data>
```

Обратите внимание, что, если путь начинается с имени каталога или файла, который находится в текущем каталоге, вы используете *относительный путь*, например `cd Downloads`. Если вы собираетесь выйти за пределы текущего каталога, укажите *абсолютный путь*, например `cd C:\Users` в Windows или `cd /Users` в macOS (обратите внимание на обратную косую черту в команде Windows и косую черту `/` в macOS).

Переход в родительский каталог

Чтобы перейти в ваш родительский каталог, т. е. в иерархии каталогов на один уровень выше, введите `cd ..`, затем нажмите клавишу `<Enter>` (убедитесь, что между `cd` и точками есть пробел). Вы можете данную команду комбинировать с именем каталога. Например если требуется перейти на один уровень вверх, а затем перейти на Рабочий стол, введите `cd ..\Desktop`. Если у вас операционная система macOS, замените в приведенной выше команде обратную косую черту на косую черту `/`.

Просмотр ранее введенных команд

Для просмотра предыдущих команд используйте клавишу со стрелкой вверх. Такой метод выбора повторяющихся команд сэкономит много времени и уменьшит количество нажатий на клавиши. Если вы проскочили нужную команду, чтобы вернуться по списку назад используйте клавишу со стрелкой вниз.

Расширения файлов

К сожалению, в операционных системах Windows и macOS расширения файлов в проводнике Windows Explorer или macOS Finder соответственно по умолчанию скрываются. Это может усложнить работу со сценариями Python и Anaconda Prompt, так как в этих приложениях вам придется ссылаться на файлы, включая их расширения. При работе с Excel отображение расширений файлов поможет вам понять, имеете ли вы дело по умолчанию с файлом `xlsx`, файлом `xlsm` с поддержкой макросов или любым другим форматом файлов Excel. Чтобы отобразить расширения файлов, выполните следующие действия:

Windows

Откройте **File Explorer** (Проводник файлов) и перейдите на вкладку **Вид** (View). В группе элементов управления **Показать/Скрыть**, установите флажок **Расширения имен файлов**.

macOS

Откройте **Finder** и перейдите в **Параметры**, нажав комбинацию клавиш `<Command>-<,>`. На вкладке **Дополнительно** установите флажок **Показывать все расширения файлов**.

Теперь все! Вы можете запустить Anaconda Prompt и выполнять команды в нужной директории. Вышеперечисленные команды вы будете использовать сразу же в следующем разделе, где я покажу вам, как начать интерактивный сеанс Python.

Python REPL: интерактивная сессия Python

Начать интерактивный сеанс Python можно с помощью Anaconda Prompt, выполнив команду `python`:

```
(base) C:\Users\felix>python
Python 3.8.5 (default, Sep 3 2020, 21:29:08) [...] :: Anaconda, Inc. on win32
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

Текст, который будет выведен в терминале операционной системы macOS, будет немного отличаться, но в основном никаких отличий не будет. Эта книга базируется на Python 3.8 – если вы хотите использовать более новую версию Python, обязательно ознакомьтесь с инструкциями на домашней странице книги⁵.

⁵ <https://xlwings.org/book>.



Anaconda Prompt Notation

В дальнейшем я буду начинать строки кода с `(base)>`, чтобы обозначить, что они вводятся в Anaconda Prompt. Например, чтобы запустить интерактивный интерпретатор Python, я напишу:

```
(base)> python
```

для Windows эта строка будет выглядеть примерно так:

```
(base) C:\Users\felix> python
```

в macOS строка будет похожей (помните, что терминал в macOS — это ваша Anaconda Prompt):

```
(base) felix@MacBook-Pro ~ % python
```

Давайте немного поэкспериментируем! Обратите внимание, три значка `>>>` в интерактивном сеансе означают, что Python ожидает вашего ввода; вам не нужно вводить эти три значка вводить не надо. Продолжайте вводить строки после трех значков `>>>` и для подтверждения введенных данных нажимайте клавишу `<Enter>`:

```
>>> 3 + 4
7
>>> "python " * 3
'python python python '
```

Этот интерактивный сеанс Python также называют Python *REPL*, что расшифровывается как *read-eval-print-loop*: Python считывает введенные данные, анализирует их, мгновенно выводит результат и ожидает следующего ввода. Помните «The Zen of Python», я упоминал о нем в предыдущей главе. Теперь вы можете прочитать полную версию, чтобы получить некоторое представление об основных принципах Python. Просто выполните команду, показанную ниже, нажав клавишу `<Enter>` после ее ввода:

```
>>> import this
```

Чтобы завершить сеанс Python, введите `quit()` и нажмите клавишу `<Enter>`. Или, если у вас установлена операционная система Windows, как альтернативный вариант, нажмите комбинацию клавиш `<Ctrl>+<Z>`, а далее клавишу `<Enter>`. В операционной системе macOS просто нажмите комбинацию клавиш `<Ctrl>+<D>`. Клавишу `<Enter>` при этом нажимать не нужно. После выхода из Python REPL самое время поэкспериментировать с Conda и pip, менеджерами пакетов, которые поставляются вместе с установкой Anaconda.

Менеджеры пакетов: Conda и pip

В предыдущей главе я уже сказал несколько слов о pip, менеджере пакетов Python: pip обеспечивает загрузку, установку, обновление и удаление пакетов Python, а также их зависимостей и подзависимостей. В то время как Anaconda работает с pip, у нее есть встроенный альтернативный менеджер пакетов под названием Conda. Преимущество Conda в том, что она способна не только устанавливать пакеты

Python, но и дополнительные версии интерпретатора Python. Если сказать кратко: пакеты добавляют в вашу установку Python дополнительную функциональность, которая не входит в стандартную библиотеку. Примером такого пакета является пакет `pandas`, о котором я подробно расскажу в *главе 5*. Поскольку он входит в комплект установки Python Anaconda, вам не придется устанавливать его вручную.



Conda по сравнению с pip

В Anaconda вы устанавливаете все, что может быть установлено с помощью Conda, и используете `pip` только для установки тех пакетов, которые Conda не может найти. Если ранее какие-то файлы были установлены с помощью `pip`, при переустановке Conda может эти файлы перезаписать.

В табл. 2.2 представлены команды, которые вы будете использовать чаще всего. Эти команды нужно ввести в Anaconda Prompt, и они помогут вам установить, обновить и удалить пакеты сторонних разработчиков.

Таблица 2.2. Команды Conda и `pip`

Действие	Conda	pip
Список всех установленных пакетов	<code>conda list</code>	<code>pip freeze</code>
Установка последней версии пакета	<code>conda install package</code>	<code>pip install package</code>
Установка определенной версии пакета	<code>conda install package=1.0.0</code>	<code>pip install package==1.0.0</code>
Обновление пакета	<code>conda update package</code>	<code>pip install --upgrade package</code>
Удаление пакета	<code>conda remove package</code>	<code>pip uninstall package</code>

Например, чтобы посмотреть, какие пакеты уже доступны в вашем дистрибутиве Anaconda, введите:

```
(base) > conda list
```

Всякий раз, когда в этой книге требуется пакет, не включенный в установку Anaconda, я буду указывать на это в явном виде и показывать, как его установить. Тем не менее, может быть целесообразным позаботиться об установке недостающих пакетов сейчас, чтобы не пришлось заниматься этим позже. Сначала установим `plotly` и `xlutils` — пакеты, устанавливаемые с помощью Conda:

```
(base) > conda install plotly xlutils
```

После выполнения этой команды Conda сообщит вам, что она собирается сделать, и потребует подтверждения выполнения операции, для чего вам потребуется на-

жать клавишу <Y>, а затем клавишу <Enter>. Далее следует установить пакеты `pixels` и `pytrends`. Так как эти пакеты недоступны через Conda, установим их с помощью `pip`:

```
(base)> pip install pyxlsb pytrends
```

В отличие от Conda, `pip` установит пакеты сразу после нажатия <Enter> и подтверждения выполняемых действий не потребует.



Версии пакетов

Большое количество пакетов Python часто обновляются и иногда вносят необратимые изменения. Это, вполне возможно, приведет к ошибкам в некоторых примерах, приведенных в этой книге. Я буду стараться следить за этими изменениями и публиковать исправления на домашней странице книги, но вы также можете создать среду Conda, которая использует те же версии пакетов, которые я использовал во время написания этой книги. Я познакомлю вас со средами Conda в следующем разделе, а подробные инструкции по созданию среды Conda с определенными пакетами вы найдете в *приложении А*.

Теперь вы знаете, как использовать Anaconda Prompt для запуска интерпретатора Python и устанавливать дополнительные пакеты.

Среды Conda

Возможно, вы задавались вопросом, почему в Anaconda Prompt в начале каждой строки ввода отображается слово `(base)`. Это имя активной *среды Conda*. Среда Conda — это отдельный «мир Python» с определенной версией Python и набором установленных пакетов с соответствующими версиями. Для чего это делается? Когда вы начнете параллельно работать над разными проектами, у таких проектов будут разные требования: в одном проекте может использоваться Python 3.8 с `pandas 0.25.0`, а в другом — Python 3.9 с `pandas 1.0.0`. Код, написанный для `pandas 0.25.0`, для работы с `pandas 1.0.0` во многих случаях нуждается в изменениях, поэтому вы не можете просто обновить версии Python и `pandas` без внесения изменений в ваш код. Использование отдельной среды Conda для каждого проекта гарантирует, что проект будет запускаться с правильными зависимостями. Хотя для дистрибутива Anaconda среды Conda специфичны, эта среда, называясь *виртуальным окружением*, существует в каждой версии Python. Среды Conda являются более эффективными, поскольку они облегчают работу не только с пакетами, а и с различными версиями Python.

Пока вы работаете над этой книгой, вам не придется менять окружение Conda, поскольку мы всегда будем использовать настроенное по умолчанию базовое окружение. Однако когда вы начинаете создавать реальные проекты, рекомендуется использовать одну среду Conda или виртуальную среду для каждого проекта, чтобы избежать любых потенциальных конфликтов между их зависимостями. Все, что вам нужно знать о работе с несколькими средами Conda, описано в *приложении А*.

Там же вы найдете инструкции по созданию среды Conda с точными версиями пакетов, которые я использовал для написания этой книги. Это позволит вам в течение многих лет использовать примеры, приведенные в этой книге, в исходном виде. Иначе придется следить за домашней страницей книги, отслеживая изменения, вносимые в новые версии Python и пакетов.

Раскрыв секреты, связанные со средами Conda, пришло время представить следующий инструмент, который мы будем интенсивно использовать в этой книге: Jupyter Notebooks!

Jupyter Notebooks

В предшествующем разделе я показал вам, как начать интерактивный сеанс Python с помощью Anaconda Prompt. Это удобно, если вам нужна «голая» среда для тестирования чего-то простого. Однако в основном вам потребуется более простая в использовании среда. Например, при использовании Python REPL, выполняемого в Anaconda Prompt, возврат к предыдущим командам и отображение графиков затруднены. К счастью, Anaconda поставляется не только с интерпретатором Python: в комплект также входит Jupyter, который стал одним из самых популярных способов исполнения кода Python в рамках науки о данных. Jupyter позволяет сохранять данные, объединяя исполняемый код Python с форматированным текстом, изображениями и диаграммами в интерактивном блокноте, который запускается в браузере. Они рассчитаны на новичков и поэтому особенно полезны на первых этапах вашего пути в Python. Однако Jupyter Notebook также пользуется огромной популярностью при обучении, создании прототипов и проведении исследований, поскольку облегчает получение результатов.

Jupyter Notebook стал настоящим конкурентом Excel, поскольку он выполняет те же операции, что и рабочая книга: вы можете быстро подготовить, проанализировать и наглядно представить данные. Разница с Excel состоит в том, что все действия происходят через написание кода Python вместо того, чтобы щелкать мышью в Excel. Еще одно преимущество заключается в том, что Jupyter не смешивает данные и бизнес-логику: в блокноте Jupyter хранятся ваш код и диаграммы, тогда как данные вы получаете из внешнего CSV-файла или базы данных. Наличие кода Python, отображаемого в блокноте, позволяет видеть, что происходит, в отличие от Excel, где формулы скрыты за значением ячейки. Блокноты Jupyter легко запускаются как локально, так и на удаленном сервере. Серверы обычно обладают большей мощностью, чем ваш локальный компьютер, и могут запускать ваш код самостоятельно, что трудно сделать с Excel. В этом разделе я покажу вам самые основы создания и навигации в блокноте Jupyter: мы узнаем о ячейках блокнота и увидим разницу между режимом редактирования и режимом команд. Далее мы разберемся, почему порядок запуска ячеек имеет значение, и завершим этот раздел изучением того, как правильно закрывать блокноты. Давайте начнем с нашего первого блокнота!

Запуск блокнотов Jupyter

В Anaconda Prompt перейдите в каталог репозитория вашего приложения, затем запустите сервер блокнота Jupyter:

```
(base) > cd C:\Users\username\python-for-excel
(base) > jupyter notebook
```

Эти команды автоматически запустят ваш браузер и покажут панель инструментов **Jupyter** и каталог с файлами, из которого вы выполняли команду. В правом верхнем углу панели инструментов **Jupyter** нажмите кнопку **New** и выберите из открывающегося списка строку **Python 3** (рис. 2.2).

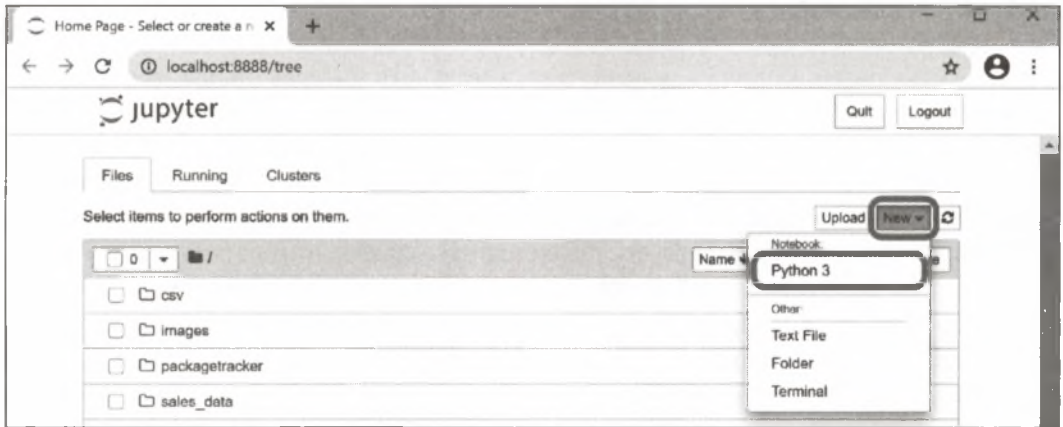


Рис. 2.2. Панель инструментов Jupyter

В результате откроется новая вкладка браузера с вашим первым пустым блокнотом Jupyter, как показано на рис. 2.3.

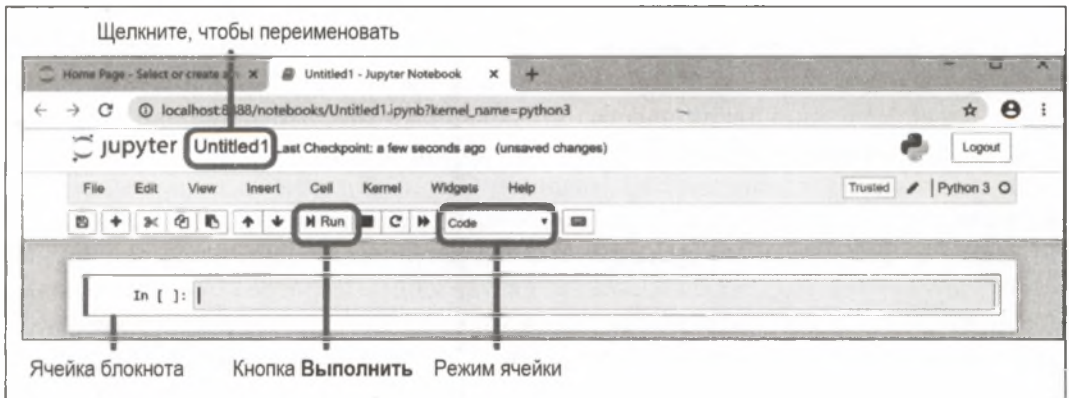


Рис. 2.3. Пустой блокнот Jupyter

Сразу после создания новой рабочей книги рекомендуется щелкнуть мышью на имени **Untitled1**, присвоенном по умолчанию, которое находится рядом с логоти-

пом Jupyter, и переименовать вашу рабочую книгу во что-то более осмысленное, например, `first_notebook`. В нижней части рис. 2.3 показана ячейка блокнота. Переходите к следующему разделу, чтобы узнать о ячейках больше!

Ячейки блокнота

На рис. 2.3 вы видите пустую ячейку с мигающим курсором. Если курсор не мигает, щелкните мышью в ячейке, правее от надписи `In [ ]`. Теперь повторите упражнение из предыдущего раздела: введите `3 + 4` и выполните действие в ячейке, нажав в строке меню сверху кнопку **Run**, или, что гораздо проще, нажав комбинацию клавиш `<Shift>+<Enter>`. Это позволит выполнить внесенный в ячейку код, напечатать результат под ячейкой и перейти к следующей ячейке. Немного подробнее: пока ячейка вычисляет, в ней будет присутствовать надпись `In [*]`, а когда вычисления будут выполнены, звездочка в квадратных скобках преобразуется в номер выполняемого действия, например `In [1]`. Это счетчик. Под ячейкой будет находиться соответствующий вывод, обозначенный тем же номером: `Out [1]`. Каждый раз, когда вы запускаете вычисление в ячейке, счетчик увеличивает свое значение на единицу, что позволяет увидеть, в каком порядке выполнялись действия в ячейках. В дальнейшем образцы кода я буду показывать в данном формате. Например, пример REPL выглядит следующим образом:

```
In [1]: 3 + 4
Out[1]: 7
```

Эта условность позволяет вам легко следить за ходом решения, набирая `3 + 4` в ячейке блокнота. Запустив вычисление, нажав комбинацию клавиш `<Shift>+<Enter>`, вы получите то, что я показываю в качестве вывода под именем `Out[1]`. Если вы читаете эту книгу в электронном формате, поддерживающем цвета, то наверняка обратите внимание, что входные ячейки форматируют строки, цифры и т. д. разными цветами, которые облегчают чтение. Это называется *подсветкой синтаксиса*.



Вывод ячейки

Когда последняя строка в ячейке возвращает значение, оно автоматически печатается в блокноте Jupyter в ячейке `Out [ ]`. Однако, когда используется функция печати или когда вы получаете исключение, значение печатается непосредственно под ячейкой `In` без метки `Out [ ]`. Примеры кода в этой книге отформатированы таким образом, чтобы отразить данный метод отображения данных.

Ячейки могут быть разных типов. Два из них представляют для нас интерес:

Code

Тип по умолчанию. Используйте его всякий раз, когда хотите запустить код Python.

Markdown

Синтаксис, который использует стандартные текстовые символы для форматирования и может быть использован для включения в блокнот красиво оформленных пояснений и инструкций.

Чтобы изменить тип ячейки на Markdown, выделите ячейку, затем выберите из открывающегося меню ячейки строку Markdown (см. рис. 2.3). В табл. 2.3 представлены комбинации клавиш для изменения режима ячейки. После замены пустой ячейки на ячейку Markdown введите следующий текст, который объясняет несколько правил Markdown:

```
# This is a first-level heading

## This is a second-level heading

You can make your text italic or bold or `monospaced`.

* This is a bullet point
* This is another bullet point
```

После нажатия комбинации клавиш <Shift>+<Enter> текст будет преобразован в хорошо отформатированный HTML. На этом этапе окно вашего блокнота должно выглядеть так, как показано на рис. 2.4. В ячейки Markdown также допускается включать изображения, видео или формулы; см. документацию по Jupyter Notebook⁶.

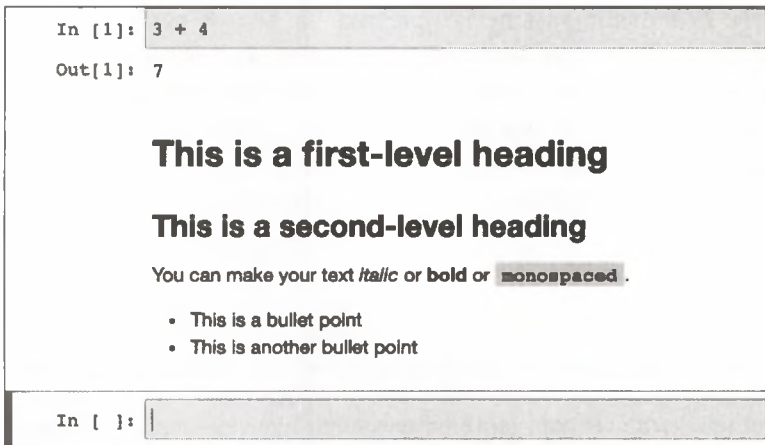


Рис. 2.4. Вверху блокнота ячейка после выполнения кода, внизу — ячейка типа Markdown

Теперь, когда вы узнали о типах ячеек Code и Markdown, пришло время познакомиться с более простым способом навигации между ячейками: в следующем разделе

⁶ <https://oreil.ly/eIGTF>. — Примеч. пер.

ле представлены режим редактирования и командный режим, а также несколько комбинаций клавиш.

Режим редактирования и командный режим

Когда вы работаете с ячейками в блокноте Jupyter, вы находитесь либо в режиме редактирования, либо в командном режиме:

Режим редактирования

Если щелкнуть мышью на ячейке, запустится режим редактирования: граница вокруг выбранной ячейки станет зеленой, а курсор в ячейке начнет мигать. Чтобы выбрать режим редактирования, можно мышью не щелкать. Вместо щелчка мышью на ячейке вы можете выделить ячейку и нажать клавишу <Enter>.

Командный режим

Для перехода в командный режим нажмите клавишу <Escape>; граница вокруг выбранной ячейки окрасится в синий цвет, а курсор мигать перестанет. Наиболее популярные комбинации клавиш, которые вы можете использовать, находясь в командном режиме, приведены в табл. 2.3.

Таблица 2.3. Сочетания клавиш (командный режим)

Комбинации клавиш	Действие
<Shift>+<Enter>	Выполнение операции в ячейке (работает также в режиме редактирования)
↑ (стрелка вверх)	Выделение вышестоящей ячейки
↓ (стрелка вниз)	Выделение нижестоящей ячейки
b	Вставка новой ячейки <i>под</i> выбранной ячейкой
a	Вставка новой ячейки <i>над</i> выбранной ячейкой
dd	Удалить текущую ячейку (введите два раза букву d)
m	Изменить тип ячейки на Markdown
y	Изменить тип ячейки на Code

Знание этих сочетаний клавиш позволит вам эффективно работать с блокнотом без необходимости постоянно переходить от клавиатуры к мыши. В следующем разделе я расскажу вам об одной распространенной ошибке, о которой необходимо помнить при использовании блокнотов Jupyter: о необходимости выполнять действия в ячейках по порядку.

Порядок выполнения имеет значение

Насколько просты и удобны блокноты для начинающих, настолько же легко в них и запутаться, если не выполнять вычисления в ячейках последовательно. Предположим, что у вас есть следующие ячейки блокнота, которые располагаются сверху вниз:

```
In [2]: a = 1
In [3]: a
Out[3]: 1
In [4]: a = 2
```

Ячейка `Out[3]` выводит значение 1, как и ожидалось. Однако, если вы вернетесь назад и выполните `In[3]` снова, вы окажетесь в такой ситуации:

```
In [2]: a = 1
In [5]: a
Out[5]: 2
In [4]: a = 2
```

В ячейке `Out[5]` теперь находится значение 2, что, скорее всего, не то, что вы ожидаете, выполняя действия в ячейках, перемещаясь сверху вниз. Особенно если ячейка `In[4]` находится гораздо ниже и, чтобы выполнить введенное в эту ячейку действие, потребовалась бы прокрутка вниз. Чтобы предотвратить подобные случаи, я бы рекомендовал вам повторно запустить не только одну ячейку, но и все ее предыдущие ячейки. Блокноты Jupyter предлагают вам простой способ сделать это, выбрав команду меню **Ячейка > Выполнить все вышеуказанное** (`Cell > Run all above`). После этих слов предостережения давайте посмотрим, как правильно завершить работу блокнота!

Завершение работы блокнотов Jupyter

Каждый блокнот работает в отдельном ядре Jupyter. Ядро — это «двигатель», запускающий код Python, который вы вводите в ячейку блокнота. Каждое ядро использует ресурсы вашей операционной системы в виде процессора и оперативной памяти. Поэтому, когда вы завершаете работу блокнота, вы также должны завершить работу его ядра, чтобы ресурсы снова могли быть использованы другими задачами — это предотвратит замедление работы системы. Самый простой способ завершить работу блокнота — выбрать команду меню **Файл > Заккрыть и остановить** (`File > Close and Halt`). Если вы просто закроете вкладку браузера, ядро автоматически не остановится. Как альтернативный вариант, вы можете закрыть запущенные записные книжки с помощью вкладки **Запущенные** (`Running`) на панели управления Jupyter.

Чтобы завершить работу всего сервера Jupyter, нажмите кнопку **Выход** (`Quit`), расположенную в правом верхнем углу панели Jupyter. Если вы уже завершили работу браузера, вы можете дважды нажать комбинацию клавиш `<Ctrl>+<C>` в `Anaconda Prompt`, где запущен сервер блокнота, или полностью завершить работу сервера `Anaconda Prompt`.

Блокноты Jupyter в облаке

Блокноты Jupyter стали настолько популярными, что различные облачные провайдеры предлагают их в качестве хостингового приложения. Здесь я расскажу о трех сервисах, которые можно использовать бесплатно. Преимущество этих служб в том, что они запускаются мгновенно и везде, где есть доступ к браузеру, без необходимости устанавливать что-либо локально. Вы могли бы, например, запустить образцы на планшете во время чтения первых трех частей. Но так как в *части IV* потребуется локальная установка Excel, облачный сервис работать не будет.

*Binder*⁷

Сервис, предоставляемый организацией Project Jupyter, которая занимается созданием блокнотов Jupyter. Binder предназначен для тестирования блокнотов Jupyter из публичных Git-репозиториях — вы ничего не храните в самом Binder и, следовательно, вам не нужно регистрироваться или входить в систему, чтобы использовать ее.

*Kaggle Notebooks*⁸

Платформа для исследования данных. Поскольку здесь проводятся научные конкурсы, вы получаете легкий доступ к огромной коллекции данных. Kaggle входит в состав Google с 2017 года.

*Google Colab*⁹ (сокращенно от Colaboratory)

Платформа для блокнотов Google. К сожалению, большинство сочетаний клавиш Jupyter здесь не работают, но вы можете получить доступ к файлам на диске Google Drive, включая Google Sheets.

Самый простой способ запустить записные книжки Jupyter из сопутствующего хранилища в облаке — это перейти по URL-адресу. Вы будете работать над копией сопутствующего репозитория, поэтому не бойтесь редактировать и переделывать то, что вам нравится!

Теперь, когда мы разобрались, как работать с Jupyter, давайте перейдем к изучению того, как писать и запускать стандартные сценарии Python. Для этого мы будем использовать Visual Studio Code, мощный текстовый редактор с хорошей поддержкой Python.

Visual Studio Code

В этом разделе мы установим и настроим *Visual Studio Code* (VS Code), бесплатный текстовый редактор с открытым исходным кодом от Microsoft. После знакомства с

⁷ <https://mybinder.org>. — Примеч. пер.

⁸ <https://kaggle.com>. — Примеч. пер.

⁹ <https://oreil.ly/4PLcS>. — Примеч. пер.

его важнейшими компонентами мы напишем первый скрипт на Python и запустим этот скрипт несколькими различными способами. Но сначала я объясню, почему в этой книге для работы с блокнотами Jupyter я не запускаю скрипт на Python, а использую VS Code.

Хотя блокноты Jupyter хорошо подходят для интерактивных рабочих процессов, таких как исследования, обучение и эксперименты, они не столь удобны, для написания скриптов Python, предназначенных для производственной среды и не нуждающихся в возможностях визуализации блокнотов. Кроме того, с помощью Jupyter трудно управлять более сложными проектами, в которых задействовано много файлов и разработчиков. В этом случае вы должны воспользоваться подходящим текстовым редактором для написания и запуска классических файлов Python. Теоретически вы могли бы использовать практически любой текстовый редактор (подойдет даже Блокнот), но на самом деле вам нужен тот, который «понимает» Python. То есть, текстовый редактор, который поддерживает, по крайней мере, следующие функции:

Подсветка синтаксиса

Редактор окрашивает слова в разные цвета в зависимости от того, представляют ли они функцию, строку, число и т. д.

Автозаполнение

Автозаполнение, или *IntelliSense*, как его называет Microsoft, разработано, чтобы вам было проще набирать текст, автоматически предлагает текстовые компоненты, что приводит к меньшему количеству ошибок.

И достаточно скоро у вас появятся другие потребности, к которым вы хотели бы получить доступ непосредственно из редактора:

Выполнить код

Переключение для выполнения кода между текстовым редактором и внешним Anaconda Prompt (т. е. Command Prompt или Terminal) может быть неудобным.

Отладчик

Отладчик позволяет просматривать код строка за строкой, чтобы понять, что происходит.

Управление версиями

Если вы используете Git для контроля версий ваших файлов, имеет смысл обрабатывать связанные с Git функции непосредственно в редакторе, чтобы не переключаться туда-сюда между двумя приложениями.

Существует широкий спектр инструментов, которые могут помочь вам во всем этом, и у каждого разработчика есть свои потребности и предпочтения. Некоторым действительно удобнее использовать простой текстовый редактор вместе с командной строкой. Другие могут предпочесть *интегрированную среду разработки* (integrated development environment или IDE): в IDE пытаются поместить все, что вам когда-либо понадобится, в один инструмент, что может сделать интегрированную среду разработки громоздкой.

Для этой книги я выбрал VS Code, потому что после первоначального выпуска в 2015 году, по данным опроса среди разработчиков StackOverflow Developer Survey 2019¹⁰, он стал одним из самых популярных редакторов кода. Что делает VS Code таким популярным инструментом? По сути, это оптимальная комбинация между голым текстовым редактором и полноценной IDE: VS Code — это мини IDE, которая поставляется «из коробки» со всем необходимым для программирования, но не более того:

Кросс-платформенная

VS Code работает с операционными системами Windows, macOS и Linux. Существуют также облачные версии, например GitHub Codespaces¹¹.

Встроенные инструменты

VS Code поставляется с отладчиком, поддерживает контроль версий Git и имеет встроенный Терминал, который можно использовать так же, как и Anaconda Prompt.

Расширения

Все остальное, например поддержка Python, добавляется с помощью расширений, которые можно установить одним щелчком мыши.

Легкая

В зависимости от вашей операционной системы размер программы установки VS Code занимает всего 50–100 МБ.



Visual Studio Code в сравнении с Visual Studio

Не путайте Visual Studio Code с Visual Studio, IDE! Хотя вы можете использовать Visual Studio для разработки на Python (она поставляется с PTVS — *Python Tools for Visual Studio*), это довольно сложная установка и традиционно используется для работы с языками .NET, такими как C#.

Чтобы узнать, согласны ли вы с моей оценкой VS Code, нет лучшего способа, чем установить его и попробовать самому. Следующий раздел поможет вам стартовать!

Установка и настройка

Загрузите инсталлятор с домашней страницы VS Code¹². Для получения актуальных инструкций по установке, пожалуйста, всегда обращайтесь к официальным документам.

¹⁰ <https://oreil.ly/savHe>. — Примеч. пер.

¹¹ <https://oreil.ly/bDGWE>. — Примеч. пер.

¹² <https://oreil.ly/26Jfa>. — Примеч. пер.

Windows

Дважды щелкните мышью на инсталляторе и выберите все настройки по умолчанию. Далее откройте меню **Пуск Windows** и в разделе **Visual Studio Code** выберите строку **VS Code**.

macOS

Дважды щелкните мышью на ZIP-файле, чтобы распаковать архив. Затем переместите Visual Studio Code.app в папку Applications: теперь вы можете запустить приложение с панели запуска. Если приложение не запускается, перейдите в **Системные настройки > Безопасность и конфиденциальность > Общие** (System Preferences > Security & Privacy > General) и выберите **Открыть в любом случае** (Open Anyway).

Когда вы открываете VS Code впервые, окно программы будет выглядеть так, как показано на рис. 2.5. Обратите внимание, что я переключился с темной темы, устанавливаемой по умолчанию, на светлую тему, чтобы облегчить чтение снимков с экрана.

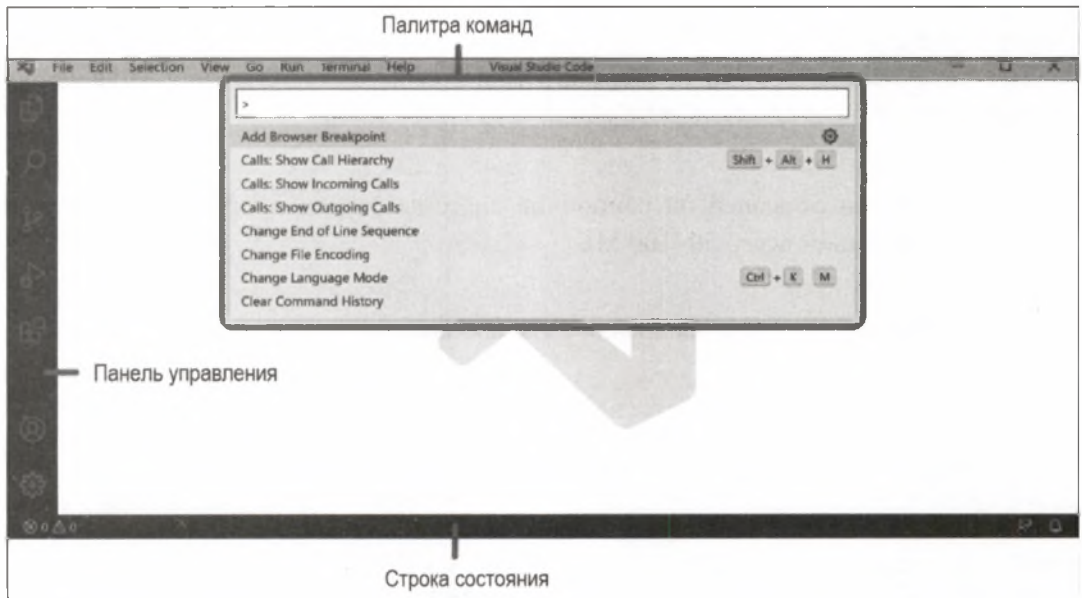


Рис. 2.5. Visual Studio Code

Activity Bar (Панель управления)

С левой стороны окна редактора находится панель управления, на которой сверху вниз расположены следующие кнопки:

- **Explorer** (Проводник);
- **Search** (Поиск);
- **Source Control** (Контроль источников);
- **Run** (Выполнить);
- **Extensions** (Расширения).

Status Bar (Строка состояния)

В нижней части окна редактора размещена строка состояния. Как только вы завершите настройку и отредактируете файл Python, вы увидите интерпретатор Python.

Command palette (Палитра команд)

Вы можете отобразить палитру команд с помощью клавиши <F1> или нажав комбинацию клавиш <Ctrl>+<Shift>+<P> (для Windows) или <Command>-<Shift>-<P> (для macOS). Если вы в чем-то не уверены, первым пунктом всегда должна быть палитра команд, поскольку она дает вам легкий доступ практически ко всему, что можно сделать с помощью VS Code. Например, если вам нужно узнать, какое действие выполняет та или иная комбинация клавиш, сначала введите интересующую вас комбинацию клавиш, выберите строку **Справка: Справочник сочетаний клавиш** (Help: Keyboard Shortcuts Reference) и нажмите клавишу <Enter>.

VS Code — очень хороший текстовый редактор даже в базовой комплектации, но для того, чтобы он хорошо работал с Python, необходимо настроить еще несколько параметров: нажмите на кнопку **Extensions** (Extensions) на панели управления и выберите Python. Установите официальное расширение Python, в котором Microsoft указывается в качестве его разработчика. Установка займет некоторое время, и после ее завершения вам, возможно, для завершения работы придется нажать кнопку перезагрузки. Или перезапустить VS Code полностью. Настройте конфигурацию в соответствии с вашей платформой:

Windows

Откройте палитру команд и введите **default shell** (оболочка по умолчанию). Выберите строку **Terminal: Select Default Shell** (Терминал: Выберите оболочку по умолчанию) и нажмите клавишу Enter. В открывшемся меню выберите строку **Командная строка** и подтвердите выбор нажатием клавиши <Enter>. Это необходимо, поскольку в противном случае VS Code не сможет правильно активировать среду Conda.

macOS

Откройте **Command Palette** (Палитру команд) и введите **shell command** (командная оболочка). Выберите следующую строку **Shell Command: Install 'code' command in PATH** (Командная оболочка: Установите команду 'code' в PATH) и нажмите клавишу <Enter>. Это делается, чтобы вы могли запускать VS Code из Anaconda Prompt (т. е. терминала).

Теперь, когда VS Code установлен и настроен, давайте используем его для написания и запуска нашего первого скрипта на Python!

Запуск скрипта на Python

Хотя VS Code можно запустить с помощью меню **Пуск** в Windows или из панели запуска в macOS, быстрее запустить VS Code получается из командной строки

Anaconda, где для запуска необходимо ввести команду `code`. Для этого откройте новый Anaconda Prompt, с помощью команды `cd` перейдите в каталог, в котором вы собираетесь работать, и для открытия текущего каталога ведите `vs Code` (он обозначен точкой):

```
(base)> cd C:\Users\username\python-for-excel
(base)> code .
```

Таким образом, запуск VS Code приведет к тому, что проводник на панели управления автоматически покажет содержимое каталога, в котором вы находились, когда выполняли команду кода.

Как альтернативный вариант, можно открыть каталог с помощью команд меню **Файл > Открыть папку** (File > Open Folder) (на macOS: **Файл > Открыть** (File > Open)), но такие действия могут вызвать ошибки разрешения на macOS, когда мы начнем использовать `xlwings` в *части IV*. Когда вы наведете курсор на список файлов в Проводнике на панели управления, на панели появится кнопка **Новый файл** (New File), как показано на рис. 2.6. Нажмите кнопку **Новый файл** (New File) и назовите свой файл `hello_world.py`, затем нажмите клавишу <Enter>. Когда этот файл откроется в редакторе, введите следующую строку кода:

```
print("hello world!")
```

Вы не забыли, что в блокнотах Jupyter удобно выводить возвращаемое значение последней строки автоматически? Когда вы запускаете традиционный сценарий Python, вам нужно явно указать Python, что печатать, поэтому здесь используется функция `print`. В строке состояния должна отобразиться версия Python, например «Python 3.8.5 64-bit (conda)». Щелчком мыши на этой строке, если у вас несколько интерпретаторов (сюда входят среды Conda), откроется палитра команд, из которой вы сможете выбрать другой интерпретатор Python. Теперь ваша конфигурация должна выглядеть так, как показано на рис. 2.6.

Перед запуском сценария обязательно сохраните его. Для этого в операционной системе Windows нажмите комбинацию клавиш <Ctrl>+<S> или <Command>-<S> в macOS. В блокнотах Jupyter мы могли просто выделить ячейку и для запуска введенного в эту ячейку кода, нажать комбинацию клавиш <Shift>+<Enter>. С помощью VS Code вы можете запустить свой код или из Anaconda Prompt, или нажав кнопку **Выполнить** (Run). Процесс запуска кода Python из Anaconda Prompt очень похож на то, как вы запускаете сценарии, размещенные на сервере, поэтому важно знать, как это работает.

Anaconda Prompt

Откройте Anaconda Prompt, выполните команду `cd`, с помощью которой вы перейдете в каталог со сценарием, затем запустите сценарий следующим образом:

```
(base)> cd C:\Users\username\python-for-excel
(base)> python hello_world.py
hello world!
```


Последняя строка — это вывод, который будет напечатан сценарием. Обратите внимание, что если вы находитесь не в том же каталоге, что и ваш файл Python, вам нужно использовать полный путь к вашему файлу Python:

```
(base)> python C:\Users\username\python-for-excel\hello_world.py
hello world!
```

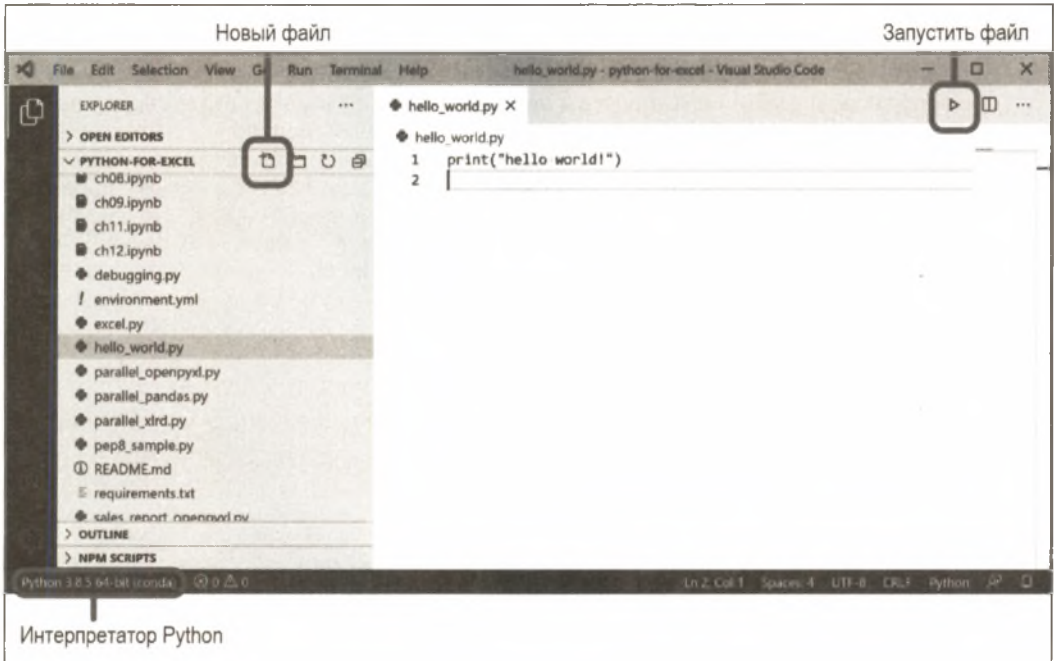


Рис. 2.6. VS Code с открытым файлом hello_world.py



Длинные пути к файлам в Anaconda Prompt

Для работы в Anaconda Prompt с длинными путями к файлам удобно использовать метод перетаскивания файлов. При этом будет записан полный путь, где бы ни находился курсор.

Anaconda Prompt в VS Code

Для работы с Anaconda Prompt в VS Code переключаться не нужно: VS Code имеет встроенный терминал, который можно открыть с помощью комбинации клавиш `<Ctrl>+<`>` или выбрав команду меню **Вид > Терминал** (View > Terminal). Так как терминал открывается в папке проекта, вам не нужно предварительно менять каталог:

```
(base)> python hello_world.py
hello world!
```

Кнопка запуска в VS Code

В VS code есть простой способ запустить ваш код без использования Anaconda Prompt: при редактировании файла Python в правом верхнем углу окна появится

зеленый значок **Play** — это кнопка **Запустить файл** (Run File), как показано на рис. 2.6. Если нажать на эту кнопку, внизу окна автоматически откроется терминал, в котором ваш код и будет запущен.



Открытие файлов в VS Code

Если один раз щелкнуть мышью на отображаемом в проводнике (Activity Bar) файле, реакция VS Code будет нестандартной: файл откроется в режиме предварительного просмотра, и это значит, если в этот файл вы не внесли изменения, следующий файл, на котором вы щелкните мышью, заменит его на вкладке. Чтобы изменить способ открытия файла на стандартный (одинарный щелчок мышью выделяет файл, двойной щелчок мышью его открывает), выберите команду меню **Установки > Настройки** (Preferences > Settings) или нажмите комбинацию клавиш `<Ctrl>+<+>` в Windows или `<Command>-<->` в macOS и выберите из открывающегося списка **Инструментальные средства > Список > «Список: Метод открывания»** строку «двойной щелчок» (Workbench > «List: Open Mode» на «doubleClick»).

Теперь вы узнали, как создавать, редактировать и запускать сценарии Python в VS Code. Но VS Code умеет делать гораздо больше: в *приложении В* я объясняю, как использовать отладчик и как можно запускать блокноты Jupyter с помощью VS Code.

Альтернативные текстовые редакторы и IDE

Инструменты — это нечто индивидуальное, и если в этой книге используются Jupyter и VS Code, это не значит, что вам не стоит присмотреться к другим приложениям.

Некоторые популярные текстовые редакторы:

*Sublime Text*¹³

Быстрый коммерческий текстовый редактор.

*Notepad++*¹⁴

Является бесплатной программой и существует уже очень долгое время, но работает только под Windows.

Vim or Emacs

Возможно, Vim¹⁵ или Emacs¹⁶ не самые лучшие варианты для начинающих программистов из-за их сложной программы обучения, но они очень популярны среди профессионалов. Соперничество между двумя этими редакторами настолько велико, что Википедия называет его «войной редакторов».

¹³ <https://oreil.ly/9FVLD>. — Примеч. пер.

¹⁴ <https://oreil.ly/7Ksk9>. — Примеч. пер.

¹⁵ <https://vim.org>. — Примеч. пер.

¹⁶ https://oreil.ly/z_Kz. — Примеч. пер.

К популярным IDE относятся:

*PyCharm*¹⁷

Версия PyCharm для пользователей сообщества распространяется бесплатно и является очень мощной, в то время как профессиональная версия распространяется на коммерческой основе и содержит поддержку научных инструментов и инструментов веб-разработки.

*Spyder*¹⁸

Похож на IDE MATLAB и поставляется с проводником для работы с переменными. Так как он включен в дистрибутив Anaconda, вы можете попробовать его, выполнив следующую команду в Anaconda Prompt: `(base) > spyder`.

*JupyterLab*¹⁹

Веб-интерфейс IDE, разработанный командой, создавшей Jupyter, который может запускать блокноты Jupyter. Кроме того, его разработчики попробовали объединить в одном инструменте все инструменты, необходимые для решения задач анализа и обработки данных.

*Wing Python IDE*²⁰

Давно созданный IDE. Существуют бесплатные упрощенные версии приложения и коммерческая версия, которая называется Wing Pro.

*Komodo IDE*²¹

Komodo IDE — это коммерческая IDE, разработанная компанией ActiveState и, помимо Python, поддерживающая множество других языков.

*PyDev*²²

Это Python IDE, основанный на популярном приложении Eclipse IDE.

Заключение

В этой главе я показал вам, как установить и использовать инструменты, с которыми мы будем работать: Anaconda Prompt, блокноты Jupyter и VS Code. Мы также запустили небольшую часть кода Python в Python REPL, в блокноте Jupyter и в виде сценария в VS Code.

¹⁷ <https://oreil.ly/Orlj->. — Примеч. пер.

¹⁸ <https://spyder-ide.org/>. — Примеч. пер.

¹⁹ <https://jupyter.org/>. — Примеч. пер.

²⁰ <https://wingware.com/>. — Примеч. пер.

²¹ <https://oreil.ly/Cdtab>. — Примеч. пер.

²² <https://pydev.org/>. — Примеч. пер.

Я рекомендую вам разобраться с Anaconda Prompt, так как, когда вы привыкнете к этой программе, вы получите дополнительные возможности по работе с кодом. Возможность работы в облаке с блокнотами Jupyter также очень удобна, поскольку позволяет запускать в браузере примеры кода, приведенные в первых трех частях этой книги.

Получив рабочую среду разработки, теперь вы готовы приступить к следующей главе, в которой мы подробно изучим Python, что позволит вам освоить остальную часть книги.

Приступая к работе с Python

После установки Anaconda и запуска блокнотов Jupyter у вас появилось все необходимое программное обеспечение для начала работы с Python. Хотя в этой главе мы рассмотрим только базовые сведения, все же и здесь вы найдете много полезного и интересного. Если вы только начинающий программист, вам, возможно, придется многое изучить. Однако большинство концепций станут более понятными, когда вы начнете использовать их в последующих главах в качестве элементов практических примеров, поэтому не стоит беспокоиться, если вы не до конца поняли что-то с первого раза. Всякий раз, когда между Python и VBA будут обнаруживаться существенные различия, я буду указывать на них, чтобы вы, зная об очевидных ловушках, могли плавно перейти от VBA к Python. Если же у вас VBA не установлен, эти разделы можете пропустить.

Я познакомлю вас с целыми числами и строками, которые являются основными типами данных в Python. Далее будут представлены структуры данных, такие как списки и словари, которые могут содержать несколько объектов. И, прежде чем перейти к рассмотрению функций и модулей, которые позволяют организовать и структурировать код, я продолжу ваше знакомство с операторами `if` и циклами `for` и `while`. В завершение этой главы я покажу вам, как правильно форматировать код Python. Как вы уже, наверное, поняли, эта глава будет насыщена в техническом плане, насколько это вообще возможно. Запуск примеров в блокноте Jupyter — хорошая возможность сделать наши занятия более интересными и увлекательными. Вы можете самостоятельно выполнить решение примеров с помощью блокнотов в сопутствующем репозитории.

Типы данных

Python, как и любой другой язык программирования, обрабатывает числа, текст, и логические значения, присваивая им разные *типы данных*. Типы данных, которые мы будем использовать наиболее часто, — это целые числа, числа с плавающей запятой, логические данные (их еще называют булевым типом данных) и строки. В этом разделе я расскажу о них по порядку и приведу несколько примеров.

Объекты

В Python все, из чего состоит код, является объектами. То есть объекты — это числа, строки, функции и все остальное. С ними мы и познакомимся в этой главе. Объекты, предоставляя доступ к набору переменных и функций, могут упростить сложные задачи и сделать их интуитивно понятными. Поэтому, прежде чем продолжить повествование, позвольте мне сказать несколько слов о переменных и функциях!

Переменные

Переменная в Python — это имя, которое присваивается объекту с помощью знака равенства. В первой строке следующего примера объекту 3 назначено имя *a*:

```
In [1]: a = 3
        b = 4
        a + b
Out[1]: 7
```

По сравнению с VBA, где для таких типов данных, как числа и строки, используется знак равенства, а для рабочих книг или листов — оператор *Set*, такой метод кажется более простым. В Python можно изменить тип переменной, присвоив ее новому объекту. Это называется *динамической типизацией*:

```
In [2]: a = 3
        print(a)
        a = "three"
        print(a)

3
three
```

В отличие от VBA, в Python важное значение имеет регистр, поэтому переменная с нижнего регистра *a* и переменная с верхнего регистра *A* — это две разные переменные. Имена переменных должны соответствовать определенным правилам:

- ◆ имена должны начинаться либо с буквы, либо с символа подчеркивания;
- ◆ имена должны состоять из букв, цифр и знаков подчеркивания.

После этого краткого знакомства с переменными, давайте посмотрим, как мы будем вызывать функции!

Функции

Более подробно с функциями я познакомлю вас в этой главе, но немного позже. На данный момент вы должны просто понимать, как вызывать такие встроенные функции, как `print`, которые мы использовали в предыдущем примере кода. Чтобы вызвать функцию, следует добавить круглые скобки к имени функции и указать аргументы внутри круглых скобок, что практически эквивалентно математической форме обозначения:

```
function_name(argument1, argument2, ...)
```

Давайте теперь рассмотрим, как переменные и функции работают с объектами!

Атрибуты и методы

В отношении объектов переменные называются *атрибутами*, а функции — *методами*: атрибуты предоставляют доступ к данным объекта, а методы позволяют выполнить действие. Для доступа к атрибутам и методам используется точечная нотация, например такая: `myobject.attribute` и `myobject.method()`.

Давайте представим это более наглядно: если вы пишете игру для автогонок, вы, скорее всего, будете использовать объект, представляющий автомобиль. Объекту `car` может быть присвоен атрибут `speed`, который позволяет получить значение текущей скорости через `car.speed`, и вы можете ускорить автомобиль, вызвав метод `car.accelerate(10)`, который увеличит скорость на десять миль в час.

Тип объекта, а вместе с ним и его поведение определяется *классом*, поэтому в предыдущем примере вам пришлось бы написать класс `Car`. Процесс получения объекта `car` из класса `Car` называется *инстанцированием*¹, и вы инстанцируете объект, вызывая класс так же, как и функцию: `car = Car()`.

В этой книге мы не будем писать собственные классы, но если вам интересно, как это делается, внимательно почитайте *приложение C*.

В следующем разделе, чтобы сделать текстовую строку прописной, мы для первого объекта применим метод, а ближе к концу главы, когда будем говорить об объектах `datetime`, вернемся к теме объектов и классов. Однако давайте перейдем к объектам, имеющим числовой тип данных!

Числовые типы

Тип данных `int` соответствует целым числам, а `float` — числам с плавающей запятой. Чтобы узнать, какой тип данных присвоен конкретному объекту, используйте встроенную функцию `type`:

```
In [3]: type(4)
Out[3]: int
In [4]: type(4.4)
Out[4]: float
```

Если вы хотите преобразовать число из типа `int` в тип `float`, достаточно добавить завершающую десятичную точку или конструктор `float`:

```
In [5]: type(4.)
Out[5]: float
In [6]: float(4)
Out[6]: 4.0
```

Тип числовых данных из последнего примера тоже можно изменить: используя конструктор `int`, вы можете преобразовать тип `float` в тип `int`.

```
In [7]: int(4.9)
Out[7]: 4
```

¹ Инстанцирование — это создание экземпляра класса и применяется к классу, а не к объекту.



Ячейки Excel всегда хранят значения с плавающей запятой

Вам, когда вы считываете число из ячейки Excel и предоставляете его функции Python, ожидающей в качестве аргумента целое число, потребуется преобразовать число типа `float` в число типа `int`. Дело в том, что числа в ячейках Excel всегда хранятся в виде значений с плавающей запятой, даже если Excel отображает значение в виде целого числа.

В Python имеется еще несколько числовых типов, которые я не буду использовать или обсуждать в этой книге: это десятичный (`decimal`), дробный (`fraction`) и комплексный (`complex`) тип данных. Если для получения точных результатов из-за погрешности в числах с плавающей запятой могут возникнуть проблемы (см. врезку), используйте десятичный тип данных. Но это единичные случаи. Как правило: если в Excel выполняются вычисления, используйте числа с плавающей запятой.

Погрешности чисел с плавающей запятой

По умолчанию Excel обычно показывает округленные числа: введите в ячейку `=1.125-1.1`, и получите `0.025`. Хотя этот результат вы и ожидаете, это не то значение, что Excel хранит внутри себя. Если вы измените формат отображения так, чтобы в ячейке отображалось не менее 16 десятичных знаков, отображаемое значение изменится на `0.0249999999999999`. Этот эффект вызван погрешностью значений с плавающей запятой: компьютеры «живут» в двоичном мире, где все вычисления производятся только с помощью нуля и единицы. Некоторые десятичные дроби, например `0,1`, не могут быть сохранены как конечное двоичное число с плавающей запятой, что и обуславливает результат вычитания. В Python вы получите тот же эффект, но Python не скрывает от вас десятичные дроби:

```
In [8]: 1.125 - 1.1
Out[8]: 0.0249999999999999
```

Математические операторы

Для операций с числами требуется использование математических операторов, таких как знак плюс или знак минус. Если вы работаете в Excel, здесь, за исключением оператора `power`, нет ничего примечательного:

```
In [9]: 3 + 4 # Sum (Сумма)
Out[9]: 7
In [10]: 3 - 4 # Subtraction (Вычитание)
Out[10]: -1
In [11]: 3 / 4 # Division (Деление)
Out[11]: 0.75
In [12]: 3 * 4 # Multiplication (Умножение)
Out[12]: 12
In [13]: 3**4 # The power operator (Excel uses 3^4) (Возведение в степень,
Excel использует знак 3^4)
```

```
Out[13]: 81
In [14]: 3 * (3 + 4) # Use of parentheses (Использование круглых скобок)
Out[14]: 21
```

Комментарий

В предыдущих примерах я описывал работу с использованием *комментариев* (например, `# Sum`). Комментарии помогают другим людям (и вам самим через несколько недель после написания кода) понять, что происходит в вашей программе. Рекомендуется комментировать только те моменты, которые могут вызвать затруднения при чтении кода: когда есть сомнения в объяснении, лучше не давать комментарий, чем использовать устаревшее объяснение, которое не согласуется с кодом. Все, что начинается с хэш-символа `#`, в Python является комментарием и при выполнении кода игнорируется:

```
In [15]: # This is a sample we've seen before (Этот образец мы уже видели раньше).

        # Every comment line has to start with a # (Каждая строка
комментария должна начинаться с символа #)
        3 + 4
Out[15]: 7
In [16]: 3 + 4 # This is an inline comment (Это встроенный комментарий)
Out[16]: 7
```

В большинстве редакторов предусмотрены сочетания клавиш для включения или отмены комментирования строк. В блокнотах Jupyter и VS Code это комбинация клавиш `<Ctrl>+</>` (Windows) или `<Command>+</>` (macOS). Обратите внимание, что ячейки Markdown в блокнотах Jupyter не принимают комментарии, если строка начинается со знака `#`. Markdown интерпретирует это как заголовок.

Теперь, когда мы познакомились с целыми числами и числами с плавающей запятой, перейдем к следующему разделу, в котором рассмотрим логический тип данных!

Логический тип данных

Логический тип данных в Python, так же, как в VBA, может принимать только два значения: `True` (Истина) или `False` (Ложь). Логические операторы `and`, `or` и `not` в Python начинаются с нижнего регистра (прописной буквы), хотя в VBA они начинаются с верхнего регистра (заглавной буквы).

```
In [17]: 3 == 4 # Equality (Excel uses 3 = 4) (Равенство (Excel использует 3 = 4))
Out[17]: False
In [18]: 3 != 4 # Inequality (Excel uses 3 <> 4) (Неравенство (Excel использует 3 <> 4))
Out[18]: True
In [19]: 3 < 4 # Smaller than. Use > for bigger than. (Меньше, чем. Используйте > для обозначения больше, чем.)
```

```

Out[19]: True
In [20]: 3 <= 4 # Smaller or equal. Use >= for bigger or equal. (Меньше или
равно. Используйте >= для больше или равно.)
Out[20]: True
In [21]: # You can chain logical expressions (Вы можете составлять цепочки
логических выражений)
# In VBA, this would be: 10 < 12 And 12 < 17 (В VBA это будет
выглядеть так: 10 < 12 и 12 < 17)
# In Excel formulas, this would be: =AND(10 < 12, 12 < 17) (В
формулах Excel это будет выглядеть так: =AND(10 < 12, 12 < 17))
10 < 12 < 17
Out[21]: True
In [22]: not True # "not" operator (оператор "нет")
Out[22]: False
In [23]: False and True # "and" operator (оператор "и")
Out[23]: False
In [24]: False or True # "or" operator (оператор "или")
Out[24]: True

```

Каждый объект Python оценивается или как `True` (Истина), или как `False` (Ложь). Большинство объектов имеют значение `True`, но есть и такие, которые оцениваются как `False`, включая `None` (см. врезку); `False`, 0 или незаполненные типы данных, например пустая строка (я представлю строки в следующем разделе).

None

`None` — это встроенная константа и, согласно официальной документации, представляет собой «отсутствие значения». Например, если функция однозначно ничего не возвращает, мы получим `None`. Это, как мы увидим в *Части III* и *Части IV*, хороший вариант для представления пустых ячеек в Excel.

Чтобы узнать, какое значение принял объект — `True` или `False`, используйте конструктор `bool`:

```

In [25]: bool(2)
Out[25]: True
In [26]: bool(0)
Out[26]: False
In [27]: bool("some text") # We'll get to strings in a moment (Мы поговорим о
строках в ближайшее время.)
Out[27]: True
In [28]: bool("")
Out[28]: False
In [29]: bool(None)
Out[29]: False

```

Теперь, когда мы познакомились с логическим типом данных, остается еще один базовый тип данных: текстовые данные, более известные как строки.

Строки

Если вы когда-либо работали со строками в VBA, длина которых превышает одну строку, и содержат переменные и литеральные кавычки, вы, вероятно, хотели бы, чтобы это было проще. К счастью, это та область, в которой Python особенно силен. Строки могут быть представлены с помощью двойных кавычек (") или одинарных кавычек ('). Единственное условие — вы должны начинать и заканчивать строку кавычками одного и того же типа. Вы можете использовать плюс (+) для объединения строк или звездочку (*) для повторения строк. Так как в предыдущей главе при тестировании Python REPL я уже показывал вам случай с повторением строк, ниже приведен пример объединения двух строк с использованием знака плюс:

```
In [30]: "A double quote string." + 'A single quote string.'
Out[30]: 'A double quote string. A single quote string.'
```

В зависимости от того, что вы хотите написать, использование одинарных или двойных кавычек может помочь вам легко вывести вложенные кавычки без необходимости их экранирования. Если вам все же необходимо экранировать символ, то перед ним ставится обратная косая черта:

```
In [31]: print("Don't wait! " + 'Learn how to "speak" Python.')
Don't wait! Learn how to "speak" Python.
In [32]: print("It's easy to \"escape\" characters with a leading \\.")
It's easy to "escape" characters with a leading \.
```

Когда вы смешиваете строки с переменными, вы обычно работаете с *f-строками*, что сокращенно обозначает *форматированный строковый литерал*. Просто поставьте символ *f* перед строкой и используйте переменные между фигурными скобками:

```
In [33]: # Note how Python allows you to conveniently assign multiple (Обратите
# values to multiple variables in a single line (значений для
# нескольких переменных в одной строке)
first_adjective, second_adjective = "free", "open source"
f"Python is {first_adjective} and {second_adjective}."
Out[33]: 'Python is free and open source.'
```

Как я уже в начале этого раздела упоминал, строки, как и все остальное, являются объектами, для выполнения действия над строкой предлагается несколько методов (т. е. функций). Например, так происходит преобразование между прописными и строчными буквами:

```
In [34]: "PYTHON".lower()
Out[34]: 'python'
In [35]: "python".upper()
Out[35]: 'PYTHON'
```

Получение помощи

Как узнать, какие атрибуты предлагают определенные объекты, например строки, и какие аргументы принимают их методы?

Ответ во многом зависит от используемого вами инструмента: в блокнотах Jupyter после ввода точки, следующей за объектом, нажмите клавишу <Tab>. Например, "python", далее клавиша <Tab>. На экране появится открывающийся список со всеми атрибутами и методами, которые предлагает данный объект. Если курсор находится в интересующем вас методе, например в круглых скобках "python".upper(), чтобы получить описание этой функции нажмите комбинацию клавиш <Shift>+<Tab>. VS Code будет автоматически отображать эту информацию в виде всплывающей подсказки. Если вы запускаете Python REPL в Anaconda Prompt, для получения доступных атрибутов используйте команду `dir("python")`, а для вывода описания метода upper — команду `help("python".upper)`. В остальном всегда полезно обратиться к онлайн-документации Python².

Если вы ищете документацию сторонних пакетов, таких как pandas, полезно поискать их на PyPI³, индексе пакетов Python, где вы найдете ссылки на соответствующие домашние страницы и документацию.

При работе со строками постоянно возникает необходимость выделить часть строки: например, вам может потребоваться извлечь часть USD из обозначения курса EURUSD. В следующем разделе будет показан мощный механизм индексирования и нарезки Python, который позволяет вам выполнять именно эти операции.

Индексирование и нарезка

Индексирование и нарезка дают доступ к определенным элементам последовательности. Поскольку строка — это и есть последовательность символов, мы можем использовать строку, чтобы изучить, как выполнить нарезку и индексирование. В следующем разделе мы познакомимся с дополнительными последовательностями, такими как списки и кортежи, которые поддерживают индексирование и нарезку.

Индексирование

На рис. 3.1 представлена концепция *индексирования*. Стандартный отсчет последовательности в Python начинается с нуля, и это значит, что первый элемент в последовательности обозначается индексом 0. Отрицательные индексы от -1 позволяют ссылаться на элементы с конца последовательности.



Распространенные ошибки у разработчиков VBA

Если вы знакомы с VBA, наверное, знаете, что индексирование — это стандартный источник ошибок. VBA для большинства последовательностей, таких как листы (Sheets(1)), использует индексацию, начиная отсчет с единицы, а индекса-

² <https://docs.python.org/>. — Примеч. пер.

³ <https://pypi.org/>. — Примеч. пер.

ция, в которой отсчет начинается с нуля, применяется для массивов (`MyArray(0)`), хотя это значение можно изменить. Еще одно различие между VBA и Python заключается в том, что в VBA для индексации используются круглые скобки, а в Python — квадратные.

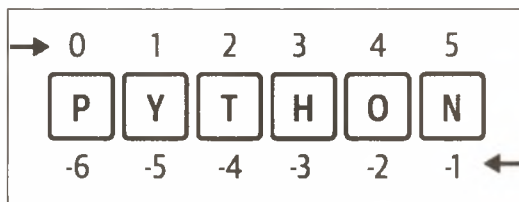


Рис. 3.1. Индексирование с начала и конца последовательности

Синтаксис для индексации следующий:

`sequence[index]` (последовательность[индекс])

Таким образом, вы получаете доступ к определенным элементам из строки:

```
In [36]: language = "PYTHON"
In [37]: language[0]
Out[37]: 'P'
In [38]: language[1]
Out[38]: 'Y'
In [39]: language[-1]
Out[39]: 'N'
In [40]: language[-2]
Out[40]: 'O'
```

Часто требуется извлечь сразу несколько символов, и именно в этом случае используется нарезка.

Нарезка (Slicing)

Если вам требуется получить из последовательности более одного элемента, используйте нарезку, синтаксис которой заключается в следующем:

`sequence[start:stop:step]` (последовательность[старт:стоп:шаг])

Python использует полуоткрытые интервалы: начальный индекс учитывается, а стоп-индекс — нет. Если аргументы `start` или `stop` введены не будут, в нарезку войдет вся последовательность, с начала или до конца соответственно. Аргумент `step` определяет направление и размер шага: например, при значении 2 будет возвращаться каждый второй элемент при чтении последовательности слева направо, а значение `-3` (минус три) — вернет каждый третий элемент последовательности, и читаться последовательность будет справа налево. Размер шага по умолчанию равен единице:

```
In [41]: language[:3] # Same as language[0:3] (От нулевого до третьего
Out[41]: 'PYT'
```

```

In [42]: language[1:3]
Out[42]: 'YT'
In [43]: language[-3:] # Same as language[-3:6] (От третьего элемента справа и
до шестого элемента слева)
Out[43]: 'HON'
In [44]: language[-3:-1]
Out[44]: 'HO'
In [45]: language[::2] # Every second element (Каждый второй элемент)
Out[45]: 'PTO'
In [46]: language[-1:-4:-1] # Negative step goes from right to left
(Отрицательный шаг отсчитывается справа налево)
Out[46]: 'NOH'

```

До сих пор мы рассматривали только одну операцию индексации или среза, но Python также позволяет объединять в *цепочку* несколько операций индексации и среза. Например, если вам необходимо получить второй символ из последних трех символов, вы можете сделать это следующим образом:

```

In [47]: language[-3:][1]
Out[47]: 'O'

```

Это то же самое, что и `language[-2]`, и в данном случае нет особого смысла использовать цепочку. Но цепочка будет более полезна, когда мы применим индексирование и нарезку к спискам. Одну из таких структур данных я собираюсь рассмотреть в следующем разделе.

Структуры данных

В состав Python входят мощные и эффективные структуры данных, которые значительно упрощают работу с коллекцией объектов. В этом разделе я собираюсь представить следующие структуры данных: списки, словари, кортежи и множества. Хотя каждая из этих структур данных отличается друг от друга своими характеристиками, все они способны хранить несколько объектов. Те, кто работал в VBA, возможно, уже использовали коллекции или массивы для хранения нескольких значений. VBA предоставляет такую структуру данных как словарь, которая работает практически так же, как словарь Python. Однако этот словарь доступен только в установленной «из коробки» версии Excel, и только для операционной системы Windows.

Для начала я предлагаю познакомиться со списками — структурой данных, которую вы, вероятно, будете использовать чаще всего.

Списки

Списки позволяют хранить объекты с различными типами данных. Из-за универсальности и многофункциональности этого инструмента скорее всего вы будете пользоваться списками постоянно. Список создается следующим образом:

```
[element1, element2, ...]
```

Ниже представлены два списка, один из которых содержит имена файлов Excel, а другой — несколько цифр:

```
In [48]: file_names = ["one.xlsx", "two.xlsx", "three.xlsx"]
        numbers = [1, 2, 3]
```

Как и строки, списки можно легко объединять с помощью знака плюс. Здесь показано, что списки могут содержать различные типы объектов:

```
In [49]: file_names + numbers
Out[49]: ['one.xlsx', 'two.xlsx', 'three.xlsx', 1, 2, 3]
```

Поскольку списки, как и все другие элементы, являются объектами, они также могут в виде своих элементов содержать другие списки. Я буду называть их вложенными списками:

```
In [50]: nested_list = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

Если вы отформатируете его так, чтобы он расположился в нескольких строках, то увидите, что это очень хорошая форма отображения матрицы или диапазона ячеек электронной таблицы. Обратите внимание: квадратные скобки позволяют неявно разрывать строки (см. врезку). С помощью индексации и нарезки вы получаете нужные вам элементы:

```
In [51]: cells = [[1, 2, 3],
                  [4, 5, 6],
                  [7, 8, 9]]
In [52]: cells[1] # Second row (Вторая строка)
Out[52]: [4, 5, 6]
In [53]: cells[1][1:] # Second row, second and third column (Вторая строка,
                    второй и третий столбцы)
Out[53]: [5, 6]
```

Продолжение о строках

Иногда строка кода может стать настолько длинной, что вам придется разбить ее на две строки или более строк, чтобы сохранить читабельность кода. С технической точки зрения, для разделения строки вы можете использовать или круглые скобки, или обратную косую черту:

```
In [54]: a = (1 + 2
              + 3)
In [55]: a = 1 + 2 \
          + 3
```

Однако руководство по стилистике Python, если это возможно, рекомендует использовать *неявные переносы строк*: в выражении, содержащем круглые, квадратные или фигурные скобки, используйте эти скобки для перевода строки без введения дополнительного символа. Подробнее о руководстве по стилистике Python я расскажу в конце этой главы.

Вы можете менять элементы в списках:

```
In [56]: users = ["Linda", "Brian"]
In [57]: users.append("Jennifer") # Most commonly you add to the end (Чаще
    всего добавляется в конце)
    users
Out[57]: ['Linda', 'Brian', 'Jennifer']
In [58]: users.insert(0, "Kim") # Insert "Kim" at index 0 (Вставьте "Kim" под
    индексом 0)
    users
Out[58]: ['Kim', 'Linda', 'Brian', 'Jennifer']
```

Чтобы удалить элемент, используйте метод `pop` или оператор `del`. Если `pop` — это метод, то `del` в Python реализован как оператор:

```
In [59]: users.pop() # Removes and returns the last element by default (По
    умолчанию удаляет и выводит последний элемент)
Out[59]: 'Jennifer'
In [60]: users
Out[60]: ['Kim', 'Linda', 'Brian']
In [61]: del users[0] # del removes an element at the given index (del удаляет
    элемент по заданному индексу)
```

Ниже список некоторых других полезных операций, которые можно выполнять с помощью списков:

```
In [62]: len(users) # Length (Длина)
Out[62]: 2
In [63]: "Linda" in users # Check if users contains "Linda" (Проверка, есть ли
    в списке пользователь "Linda")
Out[63]: True
In [64]: print(sorted(users)) # Returns a new sorted list (Возвращает новый
    отсортированный список)
    print(users) # The original list is unchanged (Первоначальный список
    остается неизменным)
['Brian', 'Linda']
['Linda', 'Brian']
In [65]: users.sort() # Sorts the original list (Сортировка исходного списка)
    users
Out[65]: ['Brian', 'Linda']
```

Обратите внимание, что вы также можете применять операторы `len` и `in` и к строкам:

```
In [66]: len("Python")
Out[66]: 6
In [67]: "free" in "Python is free and open source."
Out[67]: True
```

Чтобы получить доступ к элементам списка, вы обращаетесь к ним по их позиции или индексу, что не всегда удобно. Словари, о которых пойдет речь в следующем разделе, позволяют получить доступ к элементам по ключу (часто по имени).

Словари

В *словарях* ключ сопоставляется со значением. Вы в своей работе постоянно сталкиваетесь с комбинациями ключ/значение. Самый простой способ создания словаря заключается в следующем:

```
{key1: value1, key2: value2, ...}
```

В то время как списки позволяют обращаться к элементам последовательности по индексу, то есть по позиции элемента, словари позволяют обращаться к элементам по ключу. Как и в случае с индексами, доступ к ключам осуществляется с помощью квадратных скобок. В следующих примерах кода будет использоваться валютная пара (ключ), сопоставленная с обменным курсом (значение):

```
In [68]: exchange_rates = {"EURUSD": 1.1152,
    "GBPUSD": 1.2454,
    "AUDUSD": 0.6161}

In [69]: exchange_rates["EURUSD"] # Access the EURUSD exchange rate (Доступ к
обменному курсу EURUSD)

Out[69]: 1.1152
```

Следующие примеры показывают, как изменять существующие значения и добавлять новые пары ключ/значение:

```
In [70]: exchange_rates["EURUSD"] = 1.2 # Change an existing value (Изменить
существующее значение)

exchange_rates

Out[70]: {'EURUSD': 1.2, 'GBPUSD': 1.2454, 'AUDUSD': 0.6161}

In [71]: exchange_rates["CADUSD"] = 0.714 # Add a new key/value pair (Добавить
новую пару ключ/значение)

exchange_rates

Out[71]: {'EURUSD': 1.2, 'GBPUSD': 1.2454, 'AUDUSD': 0.6161, 'CADUSD': 0.714}
```

Самый простой способ объединить два или более словаря — это распаковать их в один новый словарь. Для распаковки словаря необходимо ввести две звездочки. Если второй словарь содержит ключи из первого, то значения в первом будут перепределены. Вы можете увидеть, как это происходит, взглянув на обменный курс GBPUSD:

```
In [72]: {**exchange_rates, **{"SGDUSD": 0.7004, "GBPUSD": 1.2222}}

Out[72]: {'EURUSD': 1.2,
    'GBPUSD': 1.2222,
    'AUDUSD': 0.6161,
    'CADUSD': 0.714,
    'SGDUSD': 0.7004}
```

Python 3.9 в качестве специального оператора слияния для словарей ввел символ | (вертикальная черта), что позволяет упростить предыдущее выражение до показанного ниже:

```
exchange_rates | {"SGDUSD": 0.7004, "GBPUSD": 1.2222}
```

Ключами могут быть разные объекты. Ниже приведен пример с целыми числами:

```
In [73]: currencies = {1: "EUR", 2: "USD", 3: "AUD"}
In [74]: currencies[1]
Out[74]: 'EUR'
```

Словари, используя метод `get`, позволяют вам использовать значение по умолчанию в случае, если ключ не существует:

```
In [75]: # currencies[100] would raise an exception. Instead of 100,
(currencies[100] вызовет исключение. Вместо 100)
        # you could use any other non-existing key, too. (можно использовать
любой другой ключ, который ранее не применялся.)
        currencies.get(100, "N/A")
Out[75]: 'N/A'
```

Словари можно использовать в VBA с применением оператора `Case`. Предыдущий пример можно записать в VBA следующим образом:

```
Select Case x
Case 1
    Debug.Print "EUR"
Case 2
    Debug.Print "USD"
Case 3
    Debug.Print "AUD"
Case Else
    Debug.Print "N/A"
End Select
```

Теперь, когда вы знаете, как работать со словарями, давайте перейдем к следующей структуре данных — кортежам.

Кортежи

Кортежи похожи на списки. Разница лишь в том, что после создания кортежа его элементы не могут быть изменены. Хотя кортежи и списки во многих случаях взаимозаменяемы, кортеж является оптимальным выбором для набора элементов, который на всех этапах работы программы никогда не меняется. Кортежи создаются посредством разделения значений запятыми:

```
mytuple = element1, element2, ...
```

Использование круглых скобок часто облегчает чтение:

```
In [76]: currencies = ("EUR", "GBP", "AUD")
```

Кортежи, так же как и списки, позволяют получать доступ к элементам, но они не допускают изменения элементов. Вместо этого конкатенация⁴ кортежей соз-

⁴ Конкатенация — операция, позволяющая «склеить» объекты линейной структуры, обычно строк. — *Примеч. пер.*

даст новый кортеж за кадром, а затем свяжет вашу переменную с этим новым кортежем:

```
In [77]: currencies[0] # Accessing the first element (Обращение к первому
элементу)
Out[77]: 'EUR'
In [78]: # Concatenating tuples will return a new tuple.
currencies + ("SGD",)
Out[78]: ('EUR', 'GBP', 'AUD', 'SGD')
```

Разницу между изменяемыми и неизменяемыми объектами я подробно объясню в *приложении С*, а пока давайте рассмотрим последнюю в этом разделе структуру данных — множества.

Множества

Множества — это коллекции, в которых нет повторяющихся элементов. Хотя вы можете использовать их для операций с теорией множеств, на практике они часто помогают вам получить уникальные значения списка или кортежа. Вы создаете наборы с помощью фигурных скобок:

```
{element1, element2, ...}
```

Чтобы в списке или кортеже выявить уникальные объекты, используйте конструктор `set` следующим образом:

```
In [79]: set(["USD", "USD", "SGD", "EUR", "USD", "EUR"])
Out[79]: {'EUR', 'SGD', 'USD'}
```

Кроме того, вы можете применять операции теории множеств, такие как пересечение и объединение:

```
In [80]: portfolio1 = {"USD", "EUR", "SGD", "CHF"}
portfolio2 = {"EUR", "SGD", "CAD"}
In [81]: # Same as portfolio2.union(portfolio1) (То же самое, что
portfolio2.union(portfolio1))
portfolio1.union(portfolio2)
Out[81]: {'CAD', 'CHF', 'EUR', 'SGD', 'USD'}
In [82]: # Same as portfolio2.intersection(portfolio1) (То же самое, что
portfolio2.intersection(portfolio1))
portfolio1.intersection(portfolio2)
Out[82]: {'EUR', 'SGD'}
```

Полный обзор операций с наборами см. в официальных документах⁵. Прежде чем двигаться дальше, давайте воспользуемся табл. 3.1 для анализа четырех структур данных, с которыми мы только что познакомились. Здесь для каждой структуры данных показаны образцы в обозначениях, которые я использовал в предыдущих параграфах, так называемых *литералах*. Кроме того, я перечисляю их конструкторы, которые предлагают альтернативу использования литералов и часто использу-

⁵ <https://oriel.ly/ju4ed>. — Примеч. пер.

ются для преобразования одной структуры данных в другую. Например, чтобы преобразовать кортеж в список, выполните следующие действия:

```
In [83]: currencies = "USD", "EUR", "CHF"
          currencies
Out[83]: ('USD', 'EUR', 'CHF')
In [84]: list(currencies)
Out[84]: ['USD', 'EUR', 'CHF']
```

Таблица 3.1. Структуры данных

Структура данных	Литералы	Конструктор
Список	[1, 2, 3]	list((1, 2, 3))
Словарь	{"a": 1, "b": 2}	dict(a=1, b=2)
Кортеж	(1, 2, 3)	tuple([1, 2, 3])
Множество	{1, 2, 3}	set((1, 2, 3))

На данном этапе вы уже познакомились со всеми наиболее важными типами данных. Это базовые данные, такие как числа с плавающей запятой, строки, и структуры данных: списки и словари. В следующем разделе мы перейдем к управлению потоком.

Управление потоком

В этом разделе представлен оператор `if`, а также циклы `for` и `while`. Оператор `if` позволяет выполнять определенные строки кода только при выполнении заданного условия, а циклы `for` и `while` выполняют блок кода многократно. В конце раздела я также расскажу о способе построения списков, которые могут заменить циклы `for`. Я начну этот раздел с определения блоков кода, для чего мне также необходимо представить одну из наиболее примечательных особенностей Python: значимое белое пространство.

Блоки кода и оператор `pass`

Блок кода определяет раздел в вашем исходном коде, который используется для чего-то особенного. Например, вы используете блок кода для определения строк, по которым выполняется цикл вашей программы, или он составляет определение функции. В Python блоки кода определяются с помощью отступов. И это существенное отличие Python от VBA, где блок определяется с помощью ключевых слов, или от большинства других языков, где блоки определяются фигурными скобками. Это называется *значимым белым пространством*. Сообщество Python в качестве отступа выбрало четыре пробела, которые обычно вводятся однократным нажатием

клавиши <Tab>. И в блокнотах Jupyter, и в VS Code одиночное нажатие на клавишу <Tab> автоматически преобразуется в четыре пробела. Позвольте мне показать вам, как формально определяются блоки кода с помощью оператора `if`:

```
if condition:
    pass # Do nothing (Ничего не делать)
```

Строка, предшествующая блоку кода, всегда заканчивается двоеточием. Когда конец блока кода достигнут и вам больше отступов не требуется, чтобы создать фиктивный блок кода, который ничего не выполняет, необходимо использовать оператор `pass`. В VBA оператор `pass` не используется, и код выглядит следующим образом:

```
If condition Then
    ' Do nothing
End If
```

Теперь, когда вы знаете, как определять блоки кода, давайте в следующем разделе и начнем их использовать. И здесь я должным образом представлю оператор `if`.

Оператор `if` и условные выражения

Чтобы представить оператор `if`, позвольте мне воспроизвести пример из *разд. «Читабельность и эксплуатационная пригодность» главы 1*, но на этот раз на языке Python:

```
In [85]: i = 20
        if i < 5:
            print("i is smaller than 5")
        elif i <= 10:
            print("i is between 5 and 10")
        else:
            print("i is bigger than 10")
i is bigger than 10
```

Если бы вы выполнили те же действия, что мы делали в *главе 1*, то есть сделали отступы от операторов `elif` и `else`, то получили бы `SyntaxError` (ошибка синтакса). Python не позволит вам делать отступы в коде, не соответствующие логике. По сравнению с VBA, в Python ключевые слова пишутся в нижнем регистре, а вместо оператора `ElseIf`, применяемого в VBA, в Python используется оператор `elif`. Операторы `if` — это простой способ определить, является ли программист новичком в Python или он уже принял этот стиль: в Python простой оператор `if` не заключается в скобки, и чтобы проверить, является ли значение `True` (истинным), вам не придется делать это явно. Вот что я имею в виду:

```
In [86]: is_important = True
        if is_important:
            print("This is important.")
        else:
            print("This is not important.")
This is important.
```

Таким же способом вы можете проверить, пуста ли последовательность, например список, или нет:

```
In [87]: values = []
        if values:
            print(f"The following values were provided: {values}")
        else:
            print("There were no values provided.")
There were no values provided.
```

Программисты, работающие с другими языками, часто используют `if` (`is_important == True`) or `if len(values) > 0` instead.

Условные выражения, также называемые *тернарными операторами*, позволяют использовать более компактный стиль или простые операторы `if/else`:

```
In [88]: is_important = False
        print("important") if is_important else print("not important")
not important
```

Познакомившись с операторами `if` и условными выражениями, давайте в следующем разделе рассмотрим циклы `for` и `while`.

Циклы *for* и *while*

Если вам нужно повторить какое-то действие, например вывести на печать значение десяти разных переменных, вы можете не копировать/вставлять оператор `print` десять раз. Вместо этого используйте цикл `for`, который выполнит эту работу за вас. Циклы `for` перебирают элементы последовательности, например списка, кортежа или строки (напоминаем, строки — это последовательности символов). Для начала давайте создадим цикл `for`, который принимает каждый элемент списка валют, присваивает его переменной `currency` и выводит полученный таким образом элемент на печать — один за другим, пока элементов в списке больше не останется:

```
In [89]: currencies = ["USD", "HKD", "AUD"]

        for currency in currencies:
            print(currency)

USD
HKD
AUD
```

В качестве примечания отмечу, что работа оператора `For Each` в VBA похожа на работу цикла `for` в Python. В VBA предыдущий пример можно записать следующим образом:

```
Dim currencies As Variant
Dim curr As Variant 'currency is a reserved word in VBA (валюта является
резервированным словом в VBA)

currencies = Array("USD", "HKD", "AUD")
For Each curr In currencies
    Debug.Print curr
Next
```

В Python, если в цикле `for` потребуется переменная-счетчик, в этом вам помогут встроенные модули `range` или `enumerate`. Сначала рассмотрим модуль `range`, который предоставляет последовательность чисел: при вызове этого модуля вы предоставляете или один аргумент `stop`, или два аргумента `start` и `stop` и необязательный аргумент `step`. Как и в случае с нарезкой, `start` является начальной точкой для генерации последовательности чисел, `stop` – точка перед завершением `range`, а `step` определяет размер шага генерации, при этом значение шага по умолчанию равно 1:

```
range(stop)
range(start, stop, step)
```

`range` вычисляется скрытно, что означает, что без прямого запроса последовательность, которую он генерирует, вы не увидите:

```
In [90]: range(5)
Out[90]: range(0, 5)
```

Преобразование диапазона в список решает эту проблему:

```
In [91]: list(range(5)) # stop argument (аргумент stop)
Out[91]: [0, 1, 2, 3, 4]
In [92]: list(range(2, 5, 2)) # start, stop, step arguments (аргументы start,
stop и step)
Out[92]: [2, 4]
```

Но в большинстве случаев нет необходимости преобразовывать диапазон в список:

```
In [93]: for i in range(3):
          print(i)

0
1
2
```

Если вам при циклическом просмотре последовательности нужна переменная-счетчик, используйте оператор `enumerate`. Этот оператор возвращает последовательность кортежей (индекс, элемент). По умолчанию индекс начинается с нуля и увеличивается с шагом на единицу. Вы можете использовать `enumerate` в следующем цикле:

```
In [94]: for i, currency in enumerate(currencies):
          print(i, currency)

0 USD
1 HKD
2 AUD
```

Перебор кортежей и множеств выполняется так же, как и списков. Когда вы примените цикл к словарям, Python будет перебирать ключи:

```
In [95]: exchange_rates = {"EURUSD": 1.1152,
                           "GBPUSD": 1.2454,
                           "AUDUSD": 0.6161}
          for currency_pair in exchange_rates:
              print(currency_pair)

EURUSD
GBPUSD
AUDUSD
```

Используя метод `items`, вы одновременно в виде кортежа получаете ключ и значение:

```
In [96]: for currency_pair, exchange_rate in exchange_rates.items():
          print(currency_pair, exchange_rate)
EURUSD 1.1152
GBPUSD 1.2454
AUDUSD 0.6161
```

Чтобы выйти из цикла, используйте оператор `break`:

```
In [97]: for i in range(15):
          if i == 2:
              break
          else:
              print(i)

0
1
```

Оставшуюся часть цикла можно пропустить с помощью оператора `continue`. Это означает, что выполнение продолжается в новом цикле, со следующего элемента:

```
In [98]: for i in range(4):
          if i == 2:
              continue
          else:
              print(i)

0
1
3
```

При сравнении циклов `for` для VBA и Python обратите внимание на небольшое отличие: в VBA после завершения цикла переменная счетчика увеличивается, переходя верхний предел:

```
For i = 1 To 3
    Debug.Print i
Next i
Debug.Print i
```

Вывод на печать:

```
1
2
3
4
```

В Python цикл ведет себя так, как вы, вероятно, и ожидаете:

```
In [99]: for i in range(1, 4):
          print(i)
          print(i)

1
2
3
3
```

Вместо перебора последовательности для выполнения цикла можно воспользоваться циклом `while`, пока истинно определенное условие:

```
In [100]: n = 0
          while n <= 2:
              print(n)
              n += 1

0
1
2
```



Расширенная нотация

В последнем примере я использовал расширенную нотацию присваивания: `n += 1`. Смысл этого выражения такой же, как и `n = n + 1`. Расширенная нотация также работает со всеми другими математическими операторами, которые я представил ранее; например, для знака минус вы могли бы ввести `n -= 1`.

Для дальнейшей обработки довольно часто возникает необходимость собрать определенные элементы в список. В этом случае Python предлагает альтернативу написанию циклов: списки, словари и множества.

Анализ списков, словарей и множеств

Анализ списков, словарей и множеств с технической точки зрения — это создание соответствующей структуры данных, что зачастую заменяет цикл `for`. Поэтому я здесь такой анализ и представлю. Предположим, что в следующем списке валютных пар USD вы хотите отобрать те валюты, где USD котируется как вторая валюта. Вы можете написать следующий цикл `for`:

```
In [101]: currency_pairs = ["USDJPY", "USDGBP", "USDCHF",
                             "USDCAD", "AUDUSD", "NZDUSD"]

In [102]: usd_quote = []
          for pair in currency_pairs:
              if pair[3:] == "USD":
                  usd_quote.append(pair[:3])
          usd_quote

Out[102]: ['AUD', 'NZD']
```

Часто это легче написать с помощью *анализа списка*. Анализ — это быстрый способ создания списка. Вы можете взять синтаксис этого цикла из следующего примера, который делает то же самое, что и предыдущий цикл `for`:

```
In [103]: [pair[:3] for pair in currency_pairs if pair[3:] == "USD"]
Out[103]: ['AUD', 'NZD']
```

Если у вас нет условия, которое нужно выполнить, достаточно исключить часть `if`. Например, чтобы инвертировать все валютные пары так, чтобы первая валюта шла второй и наоборот, вы должны сделать следующее:

```
In [104]: [pair[3:] + pair[:3] for pair in currency_pairs]
Out[104]: ['JPYUSD', 'GBPUSD', 'CHFUSD', 'CADUSD', 'USDAUD', 'USDNZD']
```

Для словарей предусмотрен словарный анализ:

```
In [105]: exchange_rates = {"EURUSD": 1.1152,
                             "GBPUSD": 1.2454,
                             "AUDUSD": 0.6161}
          {k: v * 100 for (k, v) in exchange_rates.items()}
Out[105]: {'EURUSD': 111.52, 'GBPUSD': 124.54, 'AUDUSD': 61.61}
```

А для множеств предусмотрен анализ множеств:

```
In [106]: {s + "USD" for s in ["EUR", "GBP", "EUR", "HKD", "HKD"]}
Out[106]: {'EURUSD', 'GBPUSD', 'HKDUSD'}
```

На данном этапе вы уже можете писать простые сценарии, так как знаете основную часть строительных блоков Python. В следующем разделе вы узнаете, как организовать свой код, чтобы он оставался удобным для обслуживания, когда ваши сценарии начнут разрастаться.

Организация кода

В этом разделе мы рассмотрим, как привести код в удобочитаемую структуру: я начну с представления функций, которые вам часто понадобятся, со всеми подробностями, а затем покажу, как разделить ваш код на различные модули Python. Полученные знания о модулях позволят нам закончить этот раздел рассмотрением модуля `datetime`, который является частью стандартной библиотеки.

Функции

Даже если вы будете использовать Python только для написания простых скриптов, вам все равно придется постоянно использовать функции: они являются одной из самых важных конструкций каждого языка программирования и позволяют повторно использовать одни и те же строки кода из любой точки вашей программы. Мы начнем этот раздел с определения функции, а затем посмотрим, как ее вызывать!

Определение функций

Чтобы написать собственную функцию в Python, необходимо использовать ключевое слово `def`, которое обозначает *определение* функции. В отличие от VBA, в Python нет различий между функцией и вспомогательной процедурой. В Python эквивалентом вспомогательной процедуры является обычная функция, которая ничего не возвращает. Функции в Python следуют синтаксису для блоков кода, т. е. вы заканчиваете первую строку двоеточием и делаете отступ в теле функции:

```
def function_name(required_argument, optional_argument=default_value, ...):
    return value1, value2, ...
```

Обязательные аргументы

Не имеют значения по умолчанию. Несколько аргументов разделяются запятыми.

Необязательные аргументы

Вы объявляете аргумент необязательным, задавая значение по умолчанию. Чтобы сделать аргумент необязательным, если нет значения по умолчанию, используется класс `None`.

Возвращаемое значение

Оператор `return` определяет значение, которое возвращает функция. Если этот оператор не использовать, функция автоматически возвращает `None`. Python позволяет вам возвращать несколько значений, разделенных запятыми.

Чтобы поэкспериментировать с функциями, давайте определим функцию, которая способна преобразовывать температуру из градусов Фаренгейта или Кельвина в градусы Цельсия:

```
In [107]: def convert_to_celsius(degrees, source="fahrenheit"):
           if source.lower() == "fahrenheit":
               return (degrees-32) * (5/9)
           elif source.lower() == "kelvin":
               return degrees - 273.15
           else:
               return f"Don't know how to convert from {source}"
```

Я использую построчный метод `lower`, который преобразует предоставленные строки в нижний регистр. Это позволяет нам принимать исходную строку (`source string`) с любого регистра — верхнего или нижнего, при этом преобразование все равно произойдет. Когда функция `convert_to_celsius` определена, давайте посмотрим, как мы можем ее вызвать!

Вызов функции

Как уже кратко упоминалось в начале этой главы, для вызова функции следует к ее имени добавить круглые скобки, в которых заключаются аргументы функции:

```
value1, value2, ... = function_name(positional_arg, arg_name=value, ...)
```

Позиционные аргументы

Если вы в качестве позиционного аргумента предоставляете значение (`positional_arg`), значения подбираются к аргументам в соответствии с их позицией в определении функции.

Именованные аргументы

Предоставляя аргумент в форме `arg_name=value`, вы предоставляете аргумент в виде ключевого слова. Преимущество этого способа заключается в том, что вы можете предоставлять аргументы в любом порядке. Кроме того, такой способ предоставления аргумента более понятен для чтения и может облегчить его понимание. Например, если функция определена как `f(a, b)`, вы можете вызвать ее следующим образом: `f(b=1, a=2)`. Такая концепция существует и в VBA, где вы можете использовать аргументы ключевых слов, вызывая функцию следующим образом: `f(b:=1, a:=1)`.

Давайте поэкспериментируем с функцией `convert_to_celsius`, чтобы увидеть, как все это происходит на практике:

```
In [108]: convert_to_celsius(100, "fahrenheit") # Positional arguments
(Позиционные аргументы)
Out[108]: 37.77777777777778
In [109]: convert_to_celsius(50) # Will use the default source (fahrenheit) (
Используется источник по умолчанию (фаренгейт))
Out[109]: 10.0
In [110]: convert_to_celsius(source="kelvin", degrees=0) # Keyword arguments
(Именованные аргументы)
Out[110]: -273.15
```

Теперь, когда вы научились определять и вызывать функции, давайте посмотрим, как организовать их с помощью модулей.

Модули и инструкция по импорту

Когда вы пишете код для больших проектов, вам придется в какой-то момент разделить его на разные файлы, чтобы организовать в простую для понимания структуру. Как мы уже видели в предыдущей главе, файлы Python имеют расширение `.py`, и основной файл обычно называется *скрипт*. Если вы хотите, чтобы ваш основной скрипт получил доступ к функциям из других файлов, вам нужно сначала импортировать эти функции. В этом контексте исходные файлы Python называются *модулями*. Чтобы лучше понять, как это работает и каковы различные варианты импорта, посмотрите на файл `temperature.py` в сопутствующем репозитории, открыв его с помощью VS Code (пример 3.1). Если вам необходимо вспомнить, как открывать файлы в VS Code, еще раз просмотрите главу 2.

Пример 3.1. `temperature.py`

```
TEMPERATURE_SCALES = ("fahrenheit", "kelvin", "celsius")

def convert_to_celsius(degrees, source="fahrenheit"):
    if source.lower() == "fahrenheit":
        return (degrees-32) * (5/9)
    elif source.lower() == "kelvin":
        return degrees - 273.15
    else:
        return f"Don't know how to convert from {source}"

print("This is the temperature module.")
```

Чтобы иметь возможность импортировать модуль температуры из блокнота Jupyter, необходимо, чтобы блокнот Jupyter и модуль температуры находились в одном ката-

логе — как и в случае с партнерским репозиторием. Для импорта используется только имя модуля, без расширения `.py`. После выполнения оператора `import` вы через нотацию `dot` получите доступ ко всем объектам этого модуля Python. Например, для выполнения преобразования используйте `temperature.convert_to_celsius()`:

```
In [111]: import temperature
This is the temperature module (Это температурный модуль).
In [112]: temperature.TEMPERATURE_SCALES
Out[112]: ('fahrenheit', 'kelvin', 'celsius')
In [113]: temperature.convert_to_celsius(120, "fahrenheit")
Out[113]: 48.88888888888889
```

Обратите внимание, что для `TEMPERATURE_SCALES` я использовал заглавные буквы. Этим я показал, что это константа — я расскажу о константе подробнее в конце этой главы. Когда начнется выполнение ячейки с `import temperature`, Python запустит файл `temperature.py`, который будет выполняться, начиная с верхней строки. Вы можете легко проследить, как это происходит, поскольку импорт модуля вызывает функцию `print`, расположенную в нижней части файла `temperature.py`.



Модули импортируются только один раз

Если вы снова запустите ячейку импорта температуры, вы обнаружите, что в ней больше ничего не выводится. Это происходит потому, что модули Python импортируются только один раз за сессию. Если в импортируемом модуле вы измените код, вам придется перезапустить интерпретатор Python, чтобы все изменения были приняты, для чего в блокноте Jupyter следует выбрать команду **Kernel > Restart**.

В действительности в модулях обычно ничего не выводится. Это было сделано только для того, чтобы показать вам, что происходит при повторном импорте модуля. Чаще всего вы в свои модули помещаете функции и классы (подробнее о классах см. приложение C). Если вы не хотите вводить `temperature` при каждом использовании объекта из модуля `temperature`, измените инструкцию `import` следующим образом:

```
In [114]: import temperature as tp
In [115]: tp.TEMPERATURE_SCALES
Out[115]: ('fahrenheit', 'kelvin', 'celsius')
```

Присвоение вашему модулю короткого псевдонима `tp` может облегчить его использование, при этом всегда будет понятно, откуда взялся объект. При использовании псевдонима многие сторонние пакеты предлагают определенное соглашение. Например, для `pandas` используется `import pandas as pd`. Есть еще один вариант импорта объектов из другого модуля:

```
In [116]: from temperature import TEMPERATURE_SCALES, convert_to_celsius
In [117]: TEMPERATURE_SCALES
Out[117]: ('fahrenheit', 'kelvin', 'celsius')
```



Папка `__pycache__`

При импорте модуля температуры вы увидите, что Python создаст папку `__pycache__` для файлов с расширением `.pyc`. Это байткод — скомпилированные файлы, которые интерпретатор Python создает, когда вы импортируете модуль. Мы можем пока просто игнорировать эту папку, поскольку это техническая деталь того, как Python исполняет ваш код.

При использовании синтаксиса `from x import` вы импортируете только определенные объекты. Импорт происходит непосредственно в пространство имен вашего основного скрипта: то есть, не просматривая инструкции импорта, вы не сможете определить, были ли импортированные объекты определены в текущем скрипте Python, записной книжке Jupyter или они получены из другого модуля. Это может привести к конфликтам: если в вашем основном скрипте есть функция `convert_to_celsius`, она переопределит объект, который импортируется из модуля `temperature`. Однако, если вы используете один из двух предыдущих методов, ваша локальная функция и функция из импортированного модуля могут располагаться совместно, например как `convert_to_celsius` и `temperature.convert_to_celsius`.



Не называйте свои скрипты по аналогии с существующими пакетами

Распространенной ошибкой является присвоение вашему файлу Python того же имени, что и существующему пакету или модулю Python. Например, если вы создаете файл для тестирования некоторых функций `pandas`, не называйте этот файл `pandas.py`, так как из-за этого могут возникнуть конфликты.

Теперь, узнав, как работает механизм импорта, давайте сразу же воспользуемся им для импорта модуля `datetime`! Это также позволит вам узнать еще несколько деталей об объектах и классах.

Класс `datetime`

Работа с датой и временем — обычная операция в Excel, но она имеет свои ограничения: например, временной формат ячеек Excel не поддерживает единицы измерения меньше, чем миллисекунды, а часовые пояса вообще не поддерживаются. В Excel дата и время хранятся в виде простого числа с плавающей запятой, называемого *порядковым номером даты*. Затем ячейка Excel форматируется для отображения значения в виде даты и/или времени. Например, 1 января 1900 года имеет порядковый номер даты 1, что означает, что это самая ранняя дата, с которой может работать Excel. Если значение даты находится в целой части числа, то значение времени переводится в десятичную часть числа с плавающей запятой, например 01/01/1900 10:10:00 представляется как 1.4236111111.

Для работы с датой и временем в Python импортируется модуль `datetime`, который является частью стандартной библиотеки. Модуль `datetime` содержит одноименный класс, который позволяет нам создавать объекты `datetime`. Поскольку наличие одинаковых имен у модуля и класса может привести к конфликту, в этой книге я буду использовать следующее соглашение об импорте: `import datetime as dt`. Это позволяет легко различать модуль (`dt`) и класс (`datetime`).

До этого момента мы для создания таких объектов, как списки или словари, в основном использовали литералы. Литералы относятся к синтаксису, который Python распознает как определенный тип объекта — для списка это будет что-то вроде `[1, 2, 3]`. Однако большинство объектов должны быть созданы путем вызова их класса: этот процесс называется *созданием экземпляра*, и поэтому объекты называются *экземплярами класса*. Вызов класса происходит так же, как и вызов функции, то есть добавляются круглые скобки к имени класса и предоставляются аргументы так же, как мы это делали с функциями. Чтобы создать объект `datetime`, необходимо класс вызвать следующим образом:

```
import datetime as dt
dt.datetime(year, month, day, hour, minute, second, microsecond, timezone)
```

Давайте на нескольких примерах рассмотрим, как работать с объектами времени даты в Python. В рамках этого введения проигнорируем часовые пояса и будем работать с объектами `datetime`, не зависящими от них:

```
In [118]: # Import the datetime module as "dt" (Импортируйте модуль datetime
под именем "dt")

import datetime as dt

In [119]: # Instantiate a datetime object called "timestamp" (Создайте объект
datetime с именем timestamp.)

timestamp = dt.datetime(2020, 1, 31, 14, 30)
timestamp

Out[119]: datetime.datetime(2020, 1, 31, 14, 30)

In [120]: # Datetime objects offer various attributes, e.g., to get the day
(Объекты Datetime предлагают различные атрибуты, например для получения даты)

timestamp.day

Out[120]: 31

In [121]: # The difference of two datetime objects returns a timedelta object
(Разница между двумя объектами datetime возвращает объект timedelta)

timestamp - dt.datetime(2020, 1, 14, 12, 0)

Out[121]: datetime.timedelta(days=17, seconds=9000)

In [122]: # Accordingly, you can also work with timedelta objects
(Соответственно вы также можете работать с объектами timedelta)

timestamp + dt.timedelta(days=1, hours=4, minutes=11)

Out[122]: datetime.datetime(2020, 2, 1, 18, 41)
```

Для *форматирования* объектов `datetime` в строки используйте метод `strftime`, а для *разбора* строки и преобразования ее в объект `datetime` используйте функцию

`strftime` (обзор принятых кодов формата можно найти в документации по `datetime`⁶):

```
In [123]: # Format a datetime object in a specific way (Отформатируйте объект
datetime определенным образом)
        # You could also use an f-string: f"{timestamp:%d/%m/%Y %H:%M}" (Вы
        также можете использовать f-строку: f"{временная метка: d/%m/%Y
        %H:%M}")
        timestamp.strftime("%d/%m/%Y %H:%M")
Out[123]: '31/01/2020 14:30'
In [124]: # Parse a string into a datetime object (Разбор строки в объект
datetime)
        dt.datetime.strptime("12.1.2020", "%d.%m.%Y")
Out[124]: datetime.datetime(2020, 1, 12, 0, 0)
```

После этого краткого введения в модуль `datetime` давайте перейдем к последней теме этой главы, которая касается правильного форматирования вашего кода.

PEP 8: Руководство по стилю для кода Python

Возможно, вам было интересно, почему я иногда использую имена переменных с подчеркиванием или заглавными буквами. В этом разделе я объясню свой вариант форматирования, познакомив вас с официальным руководством по стилю Python. В Python для обсуждения введения новых возможностей языка используются так называемые Python Enhancement Proposals (PEP) или Предложения по усовершенствованию Python. Номер одного из таких предложений, на который ссылается Руководство по стилю для кода Python, следующий: PEP 8. PEP 8 — это набор рекомендаций по стилю для сообщества Python; если все, кто работает с одним и тем же кодом, придерживаются одного и того же руководства по стилю, код становится намного более читабельным. Это особенно важно в мире с открытым исходным кодом, где многие программисты работают над одним и тем же проектом, часто не зная друг друга лично. В примере 3.2 показан короткий файл Python, где представлены наиболее важные соглашения.

Пример 3.2. `pep8_sample.py`

```
"""This script shows a few PEP 8 rules (Этот сценарий демонстрирует несколько
правил PEP 8).
"""
import datetime as dt
TEMPERATURE_SCALES = ("fahrenheit", "kelvin", "celsius")

class TemperatureConverter:
```

⁶ <https://oriel.ly/gXOts>. — Примеч. пер.


```

    pass # Doesn't do anything at the moment (В данный момент      6
           ничего не происходит)
def convert_to_celsius(degrees, source="fahrenheit"):              7
    """This function converts degrees Fahrenheit or Kelvin        8
    into degrees Celsius (Эта функция преобразует градусы
    Фаренгейта или Кельвина в градусы Цельсия).
    """
    if source.lower() == "fahrenheit":                              9
        return (degrees-32) * (5/9)                                10
    elif source.lower() == "kelvin":
        return degrees - 273.15
    else:
        return f"Don't know how to convert from {source} (Не знаю,
        как конвертировать из {источника})"

celsius = convert_to_celsius(44, source="fahrenheit")              11
non_celsius_scales = TEMPERATURE_SCALES[:-1]                      12
print("Current time: " + dt.datetime.now().isoformat())
print(f"The temperature in Celsius is: {celsius} (Температура в градусах Цельсия
составляет: {celsius})")

```

- 1 В самом верху находится строка документа `docstring`. В ней объясняется, что делает скрипт/модуль. Строка документа — это особый тип строки, заключенный в тройные кавычки. Помимо того, что `docstring` служит в качестве строки для документирования вашего кода, он также позволяет легко записывать строки в несколько строк, и полезен, если ваш текст содержит много двойных или одинарных кавычек, поскольку их не нужно экранировать. Он также полезен для написания многострочных SQL-запросов, как это будет показано в *главе 11*.
- 2 Все импортируемые данные находятся в верхней части файла, по одному в строке. Сначала перечислите импортируемые файлы из стандартной библиотеки, затем из сторонних пакетов и, наконец, из ваших собственных модулей. В этом примере используется только стандартная библиотека.
- 3 Для констант используйте заглавные буквы с подчеркиванием. Рекомендуется не превышать максимальную длину строки более 79 символов. Для неявного переноса строки по возможности используйте круглые скобки, квадратные скобки или фигурные скобки.
- 4 Отделите от остального кода классы и функции двумя пустыми строками.
- 5 Несмотря на то, что многие классы, такие как `datetime`, пишутся в нижнем регистре, в именах ваших собственных классов должны использоваться слова с заглавной буквы. Подробнее о классах см. в *приложении С*.
- 6 Встроенные комментарии должны быть отделены от кода не менее чем двумя пробелами. Блок кода следует отделять четырьмя пробелами.

- 7 Функции и аргументы функций, если это улучшает читабельность, должны использовать имена в нижнем регистре с подчеркиванием. Не используйте пробелы между именем аргумента и его значением по умолчанию.
- 8 В docstring также должны быть перечислены и объяснены аргументы функции. Я не стал делать этого в данном примере, чтобы сделать его коротким. Полную информацию вы найдете в файле `excer.py`, который включен в сопровождающий репозиторий, с которым мы познакомимся в *главе 8*.
- 9 Не используйте пробелы перед двоеточием.
- 10 Ставьте пробелы вокруг математических операторов. Если используются операторы с разными приоритетами, пробелы добавляются только вокруг операторов с наименьшим приоритетом. Поскольку умножение в этом примере имеет самый низкий приоритет, я добавил пробелы вокруг этого оператора.
- 11 Для написания имен переменных используется нижний регистр. Используйте подчеркивание, если это улучшает читабельность кода. При присвоении имени переменной вокруг знака равенства ставьте пробелы. Однако, если знак равенства применяется для вызова функции, и используются аргументы ключевых слов, пробелы вокруг него не ставятся.
- 12 При индексировании и нарезке не используйте пробелы вокруг квадратных скобок.

Это упрощенное изложение PEP 8. Поэтому, прежде чем вы начнете более глубоко изучать Python, вам будет полезно ознакомиться с оригинальным PEP 8⁷. Там четко указано, что это только рекомендации, и что приоритетными являются ваши правила стиля. Если вас интересуют другие общедоступные руководства, вы можете взглянуть на руководство по стилю Google для Python, которое очень похоже на рекомендации PEP 8. На практике большинство программистов, использующих Python, не слишком строго придерживаются рекомендаций PEP 8, и игнорирование максимальной длины строки в 79 символов, вероятно, является самым распространенным нарушением. Поскольку во время написания кода могут возникнуть сложности с правильным его форматированием, стиль можете проверить автоматически. В следующем разделе показано, как форматировать код в VS Code.

PEP 8 и VS Code

При работе с VS Code есть простой способ убедиться, что ваш код соответствует рекомендациям PEP 8: для этого используется *линтер*. Линтер проверяет ваш исходный код на наличие синтаксических и стилевых ошибок. Откройте палитру команд (в Windows для этого нажмите комбинацию клавиш `<Ctrl>+<Shift>+<P>`, а в Mac OS — `<Command>-<Shift>-<P>` в), найдите Python и выберите **Linter**. Одним

⁷ <https://pypi.org/project/pep8/>. — Примеч. пер.

из самых востребованных вариантов является flake8, пакет, который поставляется с предустановленной Anaconda. Если эта функция активирована, каждый раз при сохранении файла VS Code будет подчеркивать волнистыми линиями найденные ошибки. Если на эту волнистую линию навести курсор, на экране появится всплывающая подсказка с пояснениями. Чтобы подчеркивание отключить, найдите в палитре команд Python команду **Python: Enable Linting** (Python: Включить отрисовку) и выберите команду **Disable Linting** (Отключить отрисовку). Если вы для получения отчета в Anaconda Prompt запустите flake8, вы получите отчет только в случае, если будут найдены какие-то нарушения. Если нарушений PEP 8 выявлено не будет, после запуска этой команды на `pep8_sample.py` вы отчет не получите:

```
(base)> cd C:\Users\username\python-for-excel
(base)> flake8 pep8_sample.py
```

Недавно Python сделал еще один шаг вперед в статическом анализе кода, добавив поддержку *подсказок по типам*. В следующем разделе объясняется, как это работает.

Подсказки по типам

В VBA часто встречается код, в котором каждая переменная сопровождается аббревиатурой типа данных, например `strEmployeeName` или `wbWorkbookName`. Несмотря на то, что в Python переменные можно обозначать таким же образом, так делать не принято. Также в VBA вы не найдете эквивалента операторам `Option Explicit` или `Dim`, служащим для объявления типа переменной. В Python 3.5 вместо этого появилась функция, которая называется *аннотацией* типа и позволяет объявить тип данных переменной. Аннотацию можно и не использовать. Это не влияет на выполнение кода интерпретатором Python (однако существуют сторонние пакеты, такие как `pydantic`⁸, которые могут использовать аннотации во время выполнения кода). Основная цель аннотаций — позволить текстовым редакторам, таким как VS Code, выявлять больше ошибок до начала выполнения кода, но они также могут повысить качество автозаполнения кода в VS Code и других редакторах. Наиболее популярным средством проверки типов для аннотированного кода является *муру*, который VS Code предлагает в качестве линтера. Чтобы получить представление о том, как работают аннотации типов в Python, вот небольшой пример без аннотаций:

```
x = 1

def hello(name):
    return f"Hello {name}!"
```

И снова с аннотацией:

```
x: int = 1

def hello(name: str) -> str:
    return f"Hello {name}!"
```

⁸ <https://oriel.ly/J9W8h>. — Примеч. пер.

Поскольку аннотации обычно имеют больше смысла в больших кодовых базах, я не буду использовать их в оставшейся части этой книги.

Заключение

Эта глава была подробным введением в Python. Мы познакомились с наиболее важными составными блоками языка, включая структуры данных, функции и модули. Мы также коснулись некоторых особенностей Python, таких как значимое белое пространство и рекомендации по форматированию кода, более известные как PEP 8. Чтобы продолжить изучение этой книги, вам не нужно знать все подробно: для новичка достаточно сведений о списках и словарях, индексировании и нарезке, а также о работе с функциями, модулями, циклами `for` и операторами `if`.

По сравнению с VBA, я считаю Python более логичным и функционально эффективным и в то же время более простым в изучении. Если вы являетесь ярим поклонником VBA и эта глава вас еще не убедила, я уверен, что следующая часть вас окончательно убедит, и прежде чем начать наше знакомство с анализом данных при помощи библиотеки `pandas`, я познакомлю вас с вычислениями на основе массивов. Давайте начнем *часть II* с изучения нескольких основ NumPy!

ВВЕДЕНИЕ В PANDAS

Как уже говорилось в *главе 1*, NumPy — это базовый пакет, предназначенный для научных вычислений в Python и обеспечивающий поддержку вычислений на основе массивов и линейной алгебры. Поскольку NumPy является фундаментом для pandas, в этой главе я познакомлю вас с его базовыми принципами. После объяснения, что такое массив NumPy, мы рассмотрим векторизацию и транслирование, две важных концепции, которые позволяют писать лаконичный математический код и с которыми вы впоследствии встретитесь в pandas. Далее мы рассмотрим, почему NumPy предлагает специальные функции, называемые универсальными, и под конец этой главы узнаем, как устанавливать значения массива и в чем разница между представлением и копией массива NumPy. Даже если мы в этой книге практически не будем использовать NumPy, знание его базовых принципов облегчит изучение pandas в следующей главе.

Начало работы с NumPy

В этом разделе мы узнаем об одномерных и двумерных массивах NumPy и о том, что скрывается за техническими терминами *векторизация*, *транслирование* и *универсальная функция*.

Массив NumPy

Для выполнения вычислений на основе массива с вложенными списками, с которыми мы познакомились ранее, вам придется написать какой-то цикл. Например, чтобы добавить число к каждому элементу вложенного списка, вы можете использовать следующую трактовку вложенного списка:

```
In [1]: matrix = [[1, 2, 3],
                  [4, 5, 6],
                  [7, 8, 9]]

In [2]: [[i + 1 for i in row] for row in matrix]
Out[2]: [[2, 3, 4], [5, 6, 7], [8, 9, 10]]
```

Данная трактовка не совсем удобна для чтения и, что особенно важно, если вы применяете данный метод к большим массивам, перебор каждого элемента очень

замедляет вычисления. В зависимости от условий задачи и размера массивов вычисления с использованием массивов NumPy вместо задействованных списков Python могут ускорить вычисления от нескольких до сотен раз. NumPy достигает такой производительности за счет использования кода, написанного на C или Fortran, — это компилируемые языки программирования, которые намного быстрее Python. Массив NumPy — это N-мерный массив для *однородных данных*. Однородность означает, что все элементы в массиве должны иметь один и тот же тип данных. Чаще всего мы имеем дело с одномерными и двумерными массивами чисел с плавающей запятой, как схематично показано на рис. 4.1.

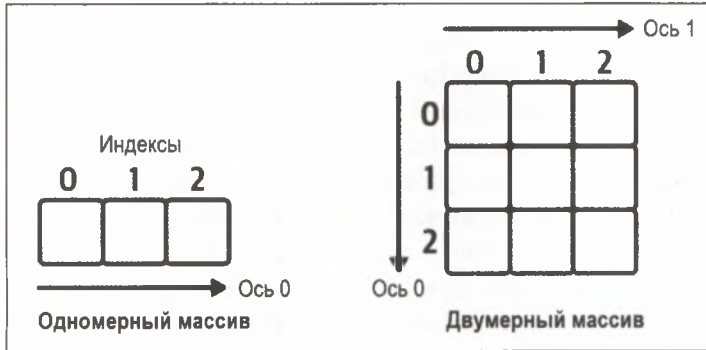


Рис. 4.1. Одномерный и двумерный массивы NumPy

Давайте создадим одномерный и двумерный массивы, с которыми будем работать на протяжении всей этой главы:

```
In [3]: # Сначала импортируем NumPy
import numpy as np

In [4]: # При построении массива с простым списком
# получается одномерный массив
array1 = np.array([10, 100, 1000.])

In [5]: # При построении массива с вложенным списком
# получается двумерный массив
array2 = np.array([[1., 2., 3.],
                   [4., 5., 6.]])
```



Размерность массива

Важно отметить разницу между одномерным и двумерным массивом: одномерный массив имеет только одну ось и, следовательно, у него отсутствует явная ориентация по столбцам и строкам. Хотя такое поведение похоже на поведение массива в VBA, вам, если вы работали ранее в таком языке программирования как, например, MATLAB, где одномерные массивы всегда имеют ориентацию по столбцам и строкам, придется к этому привыкнуть.

Даже если `array1` (массив1) состоит из целых чисел, за исключением последнего элемента (это элемент с плавающей запятой), однородность массивов NumPy за-

ставляет принять тип данных массива `float64`, которая способна хранить все элементы. Чтобы узнать о типе данных массива, обратитесь к его атрибуту `dtype`:

```
In [6]: array1.dtype
Out[6]: dtype('float64')
```

Поскольку атрибут `dtype` возвращает вместо типа `float`, с которым мы встречались в прошлой главе, тип `float64`, это говорит о том, что NumPy использует свои собственные числовые типы данных, которые более детализированы, чем типы данных Python. Обычно это не вызывает затруднений, так как в большинстве случаев преобразование между различными типами данных в Python и NumPy происходит автоматически. Если вам когда-нибудь понадобится преобразовать тип данных NumPy в один из основных типов данных Python, просто используйте соответствующий конструктор (о доступе к элементу из массива я расскажу в ближайшее время):

```
In [7]: float(array1[0])
Out[7]: 10.0
```

Полный список типов данных NumPy можно найти в документации NumPy¹. С помощью массивов NumPy, как мы увидим далее, можно написать простой код для выполнения вычислений на основе массивов.

Векторизация и транслирование

Если вы суммируете скаляр и массив NumPy, NumPy выполнит поэлементную операцию, что означает, что вам не придется самостоятельно перебирать элементы. Сообщество NumPy называет такую операцию *векторизацией*. Это позволяет писать компактный код, практически представляющий математическую нотацию:

```
In [8]: array2 + 1
Out[8]: array([[2., 3., 4.],
               [5., 6., 7.]])
```



Скаляр

Scalar (скаляр) относится к базовому типу данных Python, такому как `float` (число с плавающей запятой) или `string` (строка). Это позволяет отличить данные базового типа от структур данных с несколькими элементами, таких как списки и словари или одно- и двумерные массивы NumPy.

Тот же принцип применяется при работе с двумя массивами: NumPy выполняет операцию поэлементно:

```
In [9]: array2 * array2
Out[9]: array([[ 1.,  4.,  9.],
               [16., 25., 36.]])
```

¹ <https://oriel.ly/irDyH>. — Примеч. пер.

Если вы в арифметической операции используете два массива разного размера, NumPy по возможности автоматически подгонит размер меньшего массива по размеру большего массива, чтобы их размеры оказались совместимыми. Это называется транслированием:

```
In [10]: array2 * array1
Out[10]: array([[ 10., 200., 3000.],
               [ 40., 500., 6000.]])
```

Для выполнения матричных умножений или точечных произведений используется оператор `@`²:

```
In [11]: array2 @ array2.T # array2.T - это сокращение для
                           # array2.transpose()
Out[11]: array([[14., 32.],
               [32., 77.]])
```

Пусть вас не пугает терминология, такая как скаляр, векторизация или транслирование, которую я ввел в этом разделе! Если вы когда-либо работали с массивами в Excel, все это должно выглядеть очень просто, как на рис. 4.2. На снимке экрана показан файл `array_calculations.xlsx`, который находится в каталоге `x1` репозитория-компаньона.

	A	B	C	D	E	F	G	H	I	J
1	array1				array2 + 1					
2		10	100	1000	2	3	4			
3					5	6	7	=A5:C6 + 1		
4	array2									
5		1	2	3	array2 * array2					
6		4	5	6	1	4	9			
7					16	25	36	=A5:C6 * A5:C6		
8										
9					array2 * array1					
10					10	200	3000			
11					40	500	6000	=A5:C6 * A2:C2		
12										
13					array2 @ array2.T					
14					14	32				
15					32	77		=MMULT(A5:C6, TRANSPOSE(A5:C6))		

Рис. 4.2. Вычисления в Excel на основе массивов

Теперь вы знаете, что в массивах арифметические операции выполняются последовательно (то есть, каждое арифметическое действие выполняется последовательно, шаг за шагом — *step-by-step*), но как можно применить функцию к каждому элементу массива? Для этого и существуют универсальные функции.

² Если вы подзабыли последний курс линейной алгебры, можете пропустить этот пример — умножение матриц не является тем, на чем строится эта книга.

Универсальные функции (ufunc)

Универсальные функции (ufunc) работают с каждым элементом массива NumPy. Например, если вы для массива NumPy используете стандартную функцию квадратного корня Python из модуля `math`, вы получите ошибку:

```
In [12]: import math
In [13]: math.sqrt(array2) # Это вызовет ошибку
-----
TypeError                                Traceback (most recent call last)
<ipython-input-13-5c37e8f41094> in <module>
----> 1 math.sqrt(array2) # This will raise an Error
```

```
TypeError: only size-1 arrays can be converted to Python scalars
```

Конечно, вы можете написать вложенный цикл для получения квадратного корня из каждого элемента, а затем снова построить массив NumPy из полученного результата:

```
In [14]: np.array([math.sqrt(i) for i in row] for row in array2))
Out[14]: array([[1.          , 1.41421356, 1.73205081],
                [2.          , 2.23606798, 2.44948974]])
```

Это будет работать в тех случаях, когда NumPy не предлагает универсальную функцию и массив достаточно мал. Однако если в NumPy есть универсальная функция, используйте ее, так как она намного быстрее работает с большими массивами. Кроме того, ее легче вводить и читать:

```
In [15]: np.sqrt(array2)
Out[15]: array([[1.          , 1.41421356, 1.73205081],
                [2.          , 2.23606798, 2.44948974]])
```

Некоторые из универсальных функций NumPy, такие как `sum`, доступны и в виде методов массива: если вам нужна сумма каждого столбца, выполните следующие действия:

```
In [16]: array2.sum(axis=0) # Возвращает одномерный массив
Out[16]: array([5., 7., 9.])
```

Аргумент `axis=0` соответствует оси вдоль строк, а `axis=1` указывает на ось вдоль столбцов, как показано на рис. 4.1. Если убрать аргумент `axis`, то весь массив суммируется:

```
In [17]: array2.sum()
Out[17]: 21.0
```

В этой книге вы неоднократно будете использовать универсальные функции NumPy, поскольку они используются со структурами данных `pandas`.

До сих пор мы работали с целым массивом. В следующем разделе вы узнаете, как работать с отдельными частями массива, и познакомитесь с несколькими полезными конструкторами массивов.

Создание и манипулирование массивами

Я начну этот раздел с получения и установки определенных элементов массива, а затем представлю несколько полезных конструкторов массивов, включая один для создания псевдослучайных чисел. Эти конструкторы можно использовать для метода Монте-Карло. И в завершение я объясню разницу между представлением и копией массива.

Получение и установка элементов массива

В прошлой главе я показал вам, как, чтобы получить доступ к определенным элементам, индексировать и нарезать списки. Для работы с вложенными списками, например с матрицей, показанной в первом примере, в этой главе вы можете использовать *цепную индексацию* (*chained indexing*): `matrix[0][0]` даст вам первый элемент первой строки. Однако в массивах NumPy аргументы `index` и `slice` для обоих измерений предоставляются в одной паре квадратных скобок:

```
numpy_array[row_selection, column_selection]
```

Для одномерных массивов это упрощается до `numpy_array[selection]`. При выборе одного элемента вы получите скаляр; в противном случае — одномерный или двумерный массив. Помните, что в обозначении срезов используется начальный индекс (включить) и конечный индекс (исключить) с двоеточием между ними, как показано в образце `start:end`. Если убрать начальный и конечный индексы, останется двоеточие, которое обозначает все строки или все столбцы двумерного массива. На рис. 4.3 приведено несколько примеров, но я предлагаю еще раз взглянуть на рис. 4.1, так как там обозначены индексы и оси. Учтите, что при нарезке столбца или строки двумерного массива вы получаете одномерный массив, а не двумерный вектор столбцов или строк!

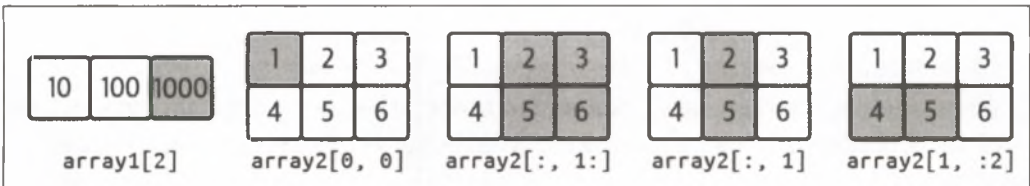


Рис. 4.3. Выбор элементов массива NumPy

Позэкспериментируйте с примерами, показанными на рис. 4.3, выполнив следующий код:

```
In [18]: array1[2] # Возвращает скаляр
Out[18]: 1000.0
In [19]: array2[0, 0] # Возвращает скаляр
Out[19]: 1.0
```

```

In [20]: array2[:, 1:] # Возвращает двумерный массив
Out[20]: array([[2., 3.],
               [5., 6.]])
In [21]: array2[:, 1] # Возвращает одномерный массив
Out[21]: array([2., 5.])
In [22]: array2[1, :2] # Возвращает одномерный массив
Out[22]: array([4., 5.])

```

До сих пор я создавал массивы образцов вручную, т. е. задавая числа в списке. Но NumPy предоставляет несколько полезных функций для построения массивов.

Полезные конструкторы массива

NumPy предлагает несколько способов построения массивов, которые, как мы увидим в *главе 5*, также будут полезны для создания массивов в pandas DataFrames. Одним из способов простого создания массивов является использование функции `arange`. Данная функция обеспечивает *диапазон массива*, что похоже на встроенный диапазон, с которым мы познакомились в предыдущей главе, с той разницей, что `arange` возвращает массив NumPy. Комбинируя данную функцию с `reshape`, мы можем быстро сгенерировать массив с нужными размерами:

```

In [23]: np.arange(2 * 5).reshape(2, 5) # 2 строки, 5 столбцов
Out[23]: array([[0, 1, 2, 3, 4],
               [5, 6, 7, 8, 9]])

```

Другой распространенной потребностью, например для метода Монте-Карло, является генерация массивов нормально распределенных псевдослучайных чисел. NumPy упрощает эту задачу:

```

In [24]: np.random.randn(2, 3) # 2 строки, 3 столбца
Out[24]: array([[ -0.30047275, -1.19614685, -0.13652283],
               [ 1.05769357,  0.03347978, -1.2153504 ]])

```

Другие полезные конструкторы, которые стоит изучить, это `np.ones` и `np.zeros`, предназначенные для создания массивов с нулями и единицами, и `np.eye` для создания единичной матрицы. С некоторыми из этих конструкторов мы еще встретимся в следующей главе, а пока давайте узнаем о разнице между представлением и копией массива NumPy.

Представления и копирование

Массивы NumPy при их нарезке возвращают *представления*. Это означает, что вы работаете без копирования данных с подмножеством исходного массива. Поэтому установка значения в представлении также изменит исходный массив:

```

In [25]: array2
Out[25]: array([[1., 2., 3.],
               [4., 5., 6.]])

```



```

In [26]: subset = array2[:, :2]
         subset
Out[26]: array([[1., 2.],
               [4., 5.]])
In [27]: subset[0, 0] = 1000
In [28]: subset
Out[28]: array([[1000.,  2.],
               [  4.,  5.]])
In [29]: array2
Out[29]: array([[1000.,  2.,  3.],
               [  4.,  5.,  6.]])

```

Если это не тот результат, что вы ожидаете, вам придется изменить In [26] следующим образом:

```
subset = array2[:, :2].copy()
```

При работе с копией исходный массив остается неизменным.

Заключение

В этой главе я показал вам, как работать с массивами NumPy и что стоит за такими выражениями, как векторизация и транслирование. Если отбросить эти технические термины, работа с массивами должна быть интуитивно понятной, поскольку они очень точно следуют математической нотации. Хотя NumPy — невероятно мощная библиотека, когда вы хотите использовать ее для анализа данных, могут проявиться две основные проблемы:

- ◆ весь массив NumPy должен состоять из данных одного типа. Это, означает, что вы не сможете выполнить ни одну из арифметических операций, которые мы делали в этой главе, если ваш массив содержит, например, смесь текста и чисел. Как только речь заходит о тексте, массив приобретает тип данных `object`, что не позволит выполнять математические операции;
- ◆ при анализе данных с помощью массивов NumPy трудно понять, к чему относится каждый столбец или строка, потому что обычно столбцы выбираются по их позиции, как, например, в массиве `array2[:, 1]`.

pandas решил эти проблемы, предоставив более совершенные структуры данных, базирующиеся на основе массивов NumPy. Что это такое и как это работает — тема следующей главы.

Анализ данных с помощью pandas

В этой главе вы познакомитесь с *pandas*, библиотекой анализа данных на Python или, как я люблю называть, электронной таблицей со сверхспособностями на основе Python. *pandas* настолько мощный, что некоторые компании, с которыми я работал, смогли полностью избавиться от Excel, заменив его комбинацией блокнотов Jupyter и *pandas*. Однако я, с точки зрения читателя этой книги, предполагаю, что вы будете использовать Excel. И в этом случае *pandas* будет служить интерфейсом для получения данных из электронных таблиц. *pandas* не только ускоряет решение, но и снижает вероятность ошибок при выполнении особенно сложных задач в Excel. Некоторые из этих задач включают получение больших наборов данных из внешних источников и работу со статистикой, временными рядами и интерактивными диаграммами. Наиболее важными сверхспособностями *pandas* считаются векторизация и выравнивание данных. Как мы уже видели в предыдущей главе, в случае с массивами NumPy векторизация позволяет писать сжатый код на основе массивов, в то время как выравнивание данных гарантирует отсутствие несоответствия данных при работе с несколькими наборами данных.

Эта глава посвящена всему пути анализа данных: глава начинается с очистки и подготовки данных, а затем будет показано, как извлечь информацию из больших наборов данных с помощью агрегации, описательной статистики и визуализации. В конце главы мы увидим, как можно импортировать и экспортировать данные с помощью *pandas*. Но сначала о главном — давайте начнем со знакомства с основными структурами данных *pandas*: *DataFrame* и *Series*!

DataFrame и Series

DataFrame и *Series* являются основными структурами данных в *pandas*. В этом разделе я представлю их с акцентом на основные компоненты *DataFrame*: индекс, столбцы и данные. *DataFrame* похож на двумерный массив NumPy, но он имеет метки столбцов и строк, и каждый столбец может содержать различные типы данных. Извлекая один столбец или строку из *DataFrame*, вы получаете одномерную серию. Опять же, серия похожа на одномерный массив NumPy с метками. Когда вы посмотрите на структуру *DataFrame* на рис. 5.1, не потребуется большого вообра-

жения, чтобы увидеть, что DataFrames могут превратиться в ваши электронные таблицы на основе Python.

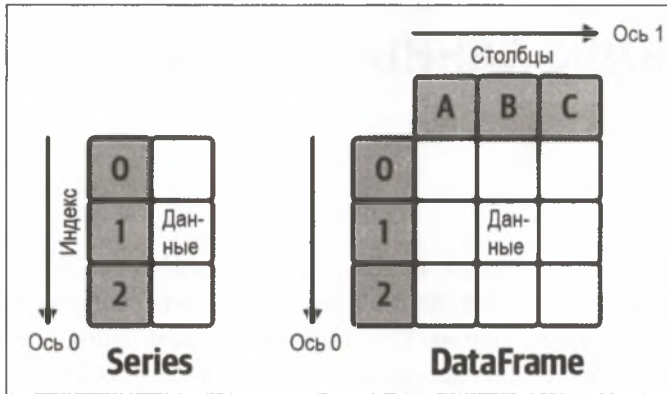


Рис. 5.1. pandas Series и DataFrame

Чтобы показать вам, как легко перейти от электронной таблицы к DataFrame, рассмотрим следующую таблицу Excel, представленную на рис. 5.2, в которой показаны участники онлайн-курса с их оценками. Вы найдете соответствующий файл `course_participants.xlsx` в папке `x1` партнерского репозитория.

	A	B	C	D	E	F
1	user_id	name	age	country	score	continent
2	1001	Mark	55	Italy	4.5	Europe
3	1000	John	33	USA	6.7	America
4	1002	Tim	41	USA	3.9	America
5	1003	Jenny	12	Germany	9	Europe

Рис. 5.2. `course_participants_race.xlsx`

Чтобы сделать данную таблицу Excel доступной в Python, начните с импорта `pandas`, затем используйте функцию `pandas.read_excel`, которая вернет DataFrame:

```
In [1]: import pandas as pd
In [2]: pd.read_excel("x1/course_participants.xlsx")
Out[2]:
```

	user_id	name	age	country	score	continent
0	1001	Mark	55	Italy	4.5	Europe
1	1000	John	33	USA	6.7	America
2	1002	Tim	41	USA	3.9	America
3	1003	Jenny	12	Germany	9.0	Europe



Функция `read_excel` в Python 3.9

Если вы запускаете `pd.read_excel` с Python 3.9 или выше, убедитесь, что используете как минимум `pandas 1.2`, иначе вы получите ошибку при чтении `xlsx`-файлов.

Если вы запустите этот процесс в блокноте Jupyter, DataFrame будет красиво отформатирован в виде HTML-таблицы, что делает его еще более похожим на то, как таблица выглядит в Excel. Я посвящу чтению и записи файлов Excel с помощью pandas всю главу 7, поэтому приведенный ранее пример, который показывает, что электронные таблицы и DataFrames действительно очень похожи, был вводным. Давайте теперь создадим этот DataFrame с нуля, не считывая его из файла Excel: одним из способов создания DataFrame является предоставление данных в виде вложенного списка, а также значений для столбцов и индекса:

```
In [3]: data=[["Mark", 55, "Italy", 4.5, "Europe"],
              ["John", 33, "USA", 6.7, "America"],
              ["Tim", 41, "USA", 3.9, "America"],
              ["Jenny", 12, "Germany", 9.0, "Europe"]]

df = pd.DataFrame(data=data,
                  columns=["name", "age", "country",
                          "score", "continent"],
                  index=[1001, 1000, 1002, 1003])

df
```

```
Out[3]:
```

	name	age	country	score	continent
1001	Mark	55	Italy	4.5	Europe
1000	John	33	USA	6.7	America
1002	Tim	41	USA	3.9	America
1003	Jenny	12	Germany	9.0	Europe

Вызвав метод `info`, вы получите некоторую основную информацию, прежде всего количество строк данных и типы данных для каждого столбца:

```
In [4]: df.info()
<class 'pandas.core.frame.DataFrame'>
Int64Index: 4 entries, 1001 to 1003
Data columns (total 5 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   name        4 non-null     object
1   age         4 non-null     int64
2   country     4 non-null     object
3   score       4 non-null     float64
4   continent   4 non-null     object
dtypes: float64(1), int64(1), object(3)
memory usage: 192.0+ bytes
```

Если вас интересует только тип данных ваших столбцов, запустите `df.dtypes`. Колонки со строками или смешанными типами данных будут иметь тип данных `object`¹. Давайте теперь подробнее рассмотрим индекс и столбцы DataFrame.

¹ В pandas 1.0.0, чтобы упростить некоторые операции и сделать их более согласованными с текстом, появился специальный тип данных `string`. Поскольку этот тип данных все еще является экспериментальным, я не буду использовать его в этой книге.

Индекс

Метки строк DataFrame называются *индексом*. Если у вас нет значимого индекса, при формировании DataFrame не вносите в него изменений. pandas автоматически создаст целочисленный индекс, начинающийся с нуля. Мы видели это в первом примере, когда считывали DataFrame из файла Excel. Индекс позволяет pandas быстрее искать данные и необходим для многих распространенных операций, например для объединения двух DataFrames. Вы обращаетесь к объекту index следующим образом:

```
In [5]: df.index
Out[5]: Int64Index([1001, 1000, 1002, 1003], dtype='int64')
```

Если вы считаете это необходимым, присвойте индексу имя. Давайте создадим таблицу в Excel и назовем ее `user_id`:

```
In [6]: df.index.name = "user_id"
df
Out[6]:
```

	name	age	country	score	continent
user_id					
1001	Mark	55	Italy	4.5	Europe
1000	John	33	USA	6.7	America
1002	Tim	41	USA	3.9	America
1003	Jenny	12	Germany	9.0	Europe

В отличие от первичного ключа базы данных индекс DataFrame, возможно, будет дублирован, но в этом случае поиск значений, вероятнее всего замедлится. Чтобы преобразовать индекс в обычный столбец, используйте `reset_index`, а чтобы установить новый индекс, используйте `set_index`. Если вы не хотите потерять существующий индекс при установке нового, не забудьте сначала сбросить его:

```
In [7]: # "reset_index" преобразует индекс в столбец, заменяя
# индекс на индекс по умолчанию.
# Это соответствует DataFrame с самого начала, который мы
# загрузили из Excel.
df.reset_index()
Out[7]:
```

	user_id	name	age	country	score	continent
0	1001	Mark	55	Italy	4.5	Europe
1	1000	John	33	USA	6.7	America
2	1002	Tim	41	USA	3.9	America
3	1003	Jenny	12	Germany	9.0	Europe

```
In [8]: # "reset_index" превращает "user_id" в обычный столбец,
# а "set_index" превращает столбец "name" в индекс
df.reset_index().set_index("name")
Out[8]:
```

	user_id	age	country	score	continent
name					
Mark	1001	55	Italy	4.5	Europe
John	1000	33	USA	6.7	America
Tim	1002	41	USA	3.9	America
Jenny	1003	12	Germany	9.0	Europe

Выполняя `df.reset_index().set_index("name")`, вы используете *цепочку методов*: поскольку `reset_index()` возвращает `DataFrame`, вы можете напрямую вызвать другой метод `DataFrame` без необходимости предварительно записывать промежуточный результат.



Методы `DataFrame` возвращают копии

Каждый раз, когда вы вызываете метод для `DataFrame` в формате `df.method_name()`, то получаете копию `DataFrame`, к которой выбранный метод был применен. А в исходный `DataFrame` изменения не вносятся. В примере выше мы только что так и поступили, введя `df.reset_index()`. Если бы вы хотели изменить исходный `DataFrame`, вам бы пришлось обратно присвоить возвращаемое значение исходной переменной, как показано ниже:

```
df = df.reset_index()
```

Поскольку мы этого не делаем, значит наша переменная `df` по-прежнему хранит свои исходные данные. В следующих примерах также вызываются методы `DataFrame`, которые не вносят изменения в исходный `DataFrame`.

Чтобы изменить индекс, используйте метод переиндексации:

```
In [9]: df.reindex([999, 1000, 1001, 1004])
Out[9]:
```

	name	age	country	score	continent
user_id					
999	NaN	NaN	NaN	NaN	NaN
1000	John	33.0	USA	6.7	America
1001	Mark	55.0	Italy	4.5	Europe
1004	NaN	NaN	NaN	NaN	NaN

Это первый пример совмещения данных в работе: при переиндексации все строки, соответствующие новому индексу, будут заполнены, а строки с отсутствующими значениями (NaN) будут добавлены вместо отсутствующей информации. Неиспользуемые элементы индекса будут сброшены. Введение NaN будет представлено в этой главе, но немного позже. Ну а для сортировки индекса используйте метод `sort_index`:

```
In [10]: df.sort_index()
Out[10]:
```

	name	age	country	score	continent
user_id					
1000	John	33	USA	6.7	America
1001	Mark	55	Italy	4.5	Europe
1002	Tim	41	USA	3.9	America
1003	Jenny	12	Germany	9.0	Europe

Если вместо этого вы хотите отсортировать строки по одному или нескольким столбцам, используйте `sort_values`:

```
In [11]: df.sort_values(["continent", "age"])
Out[11]:
```

	name	age	country	score	continent
user_id					
1000	John	33	USA	6.7	America
1002	Tim	41	USA	3.9	America

1003	Jenny	12	Germany	9.0	Europe
1001	Mark	55	Italy	4.5	Europe

В примере показано, произвести сортировку сначала по `continent`, затем по `age`. Если вы хотите сортировать только по одному столбцу, вы также можете указать имя столбца в виде строки:

```
df.sort_values("continent")
```

Мы рассмотрели базовые принципы работы с индексом. Теперь я предлагаю обратить наше внимание на горизонтальный эквивалент индекса — столбцы `DataFrame`!

Столбцы

Чтобы получить информацию о столбцах `DataFrame`, выполните следующий код:

```
In [12]: df.columns
Out[12]: Index(['name', 'age', 'country', 'score', 'continent'],
              dtype='object')
```

Если при создании `DataFrame` вы не указали имена столбцов, `pandas` пронумерует столбцы с использованием целых чисел, начиная с нуля. Однако для столбцов такое переименование в большинстве случаев не целесообразно, так как в них содержатся переменные, и, поэтому столбцы легко именуются. Вы присваиваете имя заголовкам столбцов так же, как мы поступали при индексации строк:

```
In [13]: df.columns.name = "properties"
df
Out[13]: properties  name  age  country  score  continent
user_id
1001             Mark   55    Italy    4.5    Europe
1000             John   33     USA    6.7    America
1002             Tim   41     USA    3.9    America
1003             Jenny  12   Germany   9.0    Europe
```

Если вам не нравятся названия столбцов, переименуйте их:

```
In [14]: df.rename(columns={"name": "First Name", "age": "Age"})
Out[14]: properties First Name  Age  country  score  continent
user_id
1001             Mark   55    Italy    4.5    Europe
1000             John   33     USA    6.7    America
1002             Tim   41     USA    3.9    America
1003             Jenny  12   Germany   9.0    Europe
```

Если вы желаете удалить столбцы, используйте следующий синтаксис (в примере показано, как одновременно удалять столбцы и индексы):

```
In [15]: df.drop(columns=["name", "country"],
                  index=[1000, 1003])
Out[15]: properties  age  score  continent
user_id
1001             55    4.5    Europe
1002             41    3.9    America
```

Столбцы и индекс `DataFrame` представлены объектом `Index`, поэтому вы можете менять столбцы на строки и наоборот, транспонируя `DataFrame`:

```
In [16]: df.T # Сокращенно df.transpose()
Out[16]: user_id      1001      1000      1002      1003
         properties
         name      Mark      John      Tim      Jenny
         age       55       33       41       12
         country   Italy      USA      USA      Germany
         score     4.5      6.7      3.9       9
         continent Europe America America Europe
```

Здесь стоит помнить, что в наш `DataFrame` `df` изменения не вносятся, так как мы после вызова метода никогда не переименовывали возвращаемый `DataFrame` в исходную переменную `df`. Если вы хотите изменить порядок столбцов `DataFrame`, то можете применить метод `reindex`, который мы использовали при индексировании, но выбор столбцов в нужном порядке обычно становится интуитивно понятен:

```
In [17]: df.loc[:, ["continent", "country", "name", "age", "score"]]
Out[17]: properties continent country name age score
         user_id
         1001      Europe      Italy Mark 55 4.5
         1000      America      USA John 33 6.7
         1002      America      USA Tim 41 3.9
         1003      Europe Germany Jenny 12 9.0
```

Этот пример необходимо объяснить: все, что касается `loc` и того, как происходит выбор данных, является темой следующего раздела.

Манипулирование данными

Данные реального мира вряд ли можно получить на блюдецке с голубой каемочкой. Поэтому прежде чем работать с ними, эти данные необходимо очистить и привести в подходящий формат. В начале этого раздела мы рассмотрим, как отбирать данные из `DataFrame`, как изменять их, а также как работать с отсутствующими и дублирующимися данными. Затем мы выполним несколько вычислений с помощью `DataFrames` и посмотрим, как работать с текстовыми данными. В завершение этого раздела мы выясним, в каких случаях `pandas` возвращает представление, а в каких — копию данных. Довольно много понятий в этом разделе связаны с понятиями, которые мы уже рассматривали в прошлой главе по отношению к массивам `NumPy`.

Выбор данных

Начнем с доступа к данным по метке и позиции, а затем рассмотрим другие методы, включая булеву индексацию и выборку данных с помощью `MultiIndex`.

Выбор по метке

Наиболее распространенный способ доступа к данным `DataFrame`, — это обращение к их меткам. Используйте атрибут `loc`, который обозначает *местоположение*, чтобы указать, какие строки и столбцы вы хотите извлечь:

```
df.loc[row_selection, column_selection]
```

`loc` поддерживает нотацию `slice` и соответственно для выбора всех строк или столбцов соответственно принимает двоеточие. В табл. 5.1 приведено несколько примеров выделения различных частей из нашего образца `DataFrame` `df`.

Таблица 5.1. Выбор данных по метке

Выбор	Тип возвращаемых данных	Пример
Одиночное значение	Скаляр	<code>df.loc[1000, "country"]</code>
Один столбец (1d)	<code>Series</code>	<code>df.loc[:, "country"]</code>
Один столбец (2d)	<code>DataFrame</code>	<code>df.loc[:, ["country"]]</code>
Несколько столбцов	<code>DataFrame</code>	<code>df.loc[:, ["country", "age"]]</code>
Диапазон столбцов	<code>DataFrame</code>	<code>df.loc[:, "name":"country"]</code>
Один ряд (1d)	<code>Series</code>	<code>df.loc[1000, :]</code>
Один ряд (2d)	<code>DataFrame</code>	<code>df.loc[[1000], :]</code>
Несколько строк	<code>DataFrame</code>	<code>f.loc[[1003, 1000], :]</code>
Диапазон строк	<code>DataFrame</code>	<code>df.loc[1000:1002, :]</code>



Нарезка меток с замкнутыми интервалами

Использование нотации `slice` с метками не согласуется с тем, как работает все остальное в Python и pandas, где *используется* верхний предел.

Применяя сведения из табл. 5.1, воспользуемся `loc` для выбора скаляров, `Series` и `DataFrames`:

```
In [18]: # Использование скаляра для выбора строк и
         # столбцов возвращает скаляр
         df.loc[1001, "name"]

Out[18]: 'Mark'

In [19]: # Использование скаляра для выбора строки или
         # столбца возвращает Series
         df.loc[[1001, 1002], "age"]

Out[19]: user_id
         1001    55
         1002    41
         Name: age, dtype: int64
```

```
In [20]: # Выбор нескольких строк и столбцов возвращает DataFrame
         df.loc[:1002, ["name", "country"]]

Out[20]: properties  name country
         user_id
         1001      Mark   Italy
         1000      John    USA
         1002       Tim    USA
```

Для вас важно понимать разницу между DataFrame с одним или несколькими столбцами и серией: даже с одним столбцом DataFrames является двумерным, а Series — одномерным. И DataFrame, и Series имеют индекс, но только DataFrame имеет заголовки столбцов. Когда вы выбираете столбец в качестве Series, заголовок столбца становится названием Series. Многие функции или методы будут работать как с Series, так и с DataFrame, но в случае арифметических вычислений их поведение будет другим: в DataFrames pandas соотносит данные в соответствии с заголовками столбцов — подробнее об этом чуть позже в этой главе.



Ярлык для выбора столбца

Поскольку выбор столбцов — это довольно распространенная операция, pandas предлагает быстрый способ. Вместо:

```
df.loc[:, column_selection]
```

вы можете написать:

```
df[column_selection]
```

Например, `df["country"]` возвращает Series из нашего образца DataFrame, а `df[["name", "country"]]` возвращает DataFrame с двумя столбцами.

Выбор по позиции

Выбор подмножества DataFrame по позиции соответствует тому, что мы делали в начале этой главы с массивами NumPy. Однако в DataFrames необходимо использовать атрибут `iloc`, который обозначает целочисленное расположение:

```
df.iloc[row_selection, column_selection]
```

При работе со срезами вы имеете дело со стандартными полуоткрытыми интервалами. В табл. 5.2 рассмотрены те же примеры, которые мы рассматривали ранее в табл. 5.1.

Таблица 5.2. Выбор данных по позициям

Выбор	Тип возвращаемых данных	Пример
Одиночное значение	Скаляр	<code>df.iloc[1, 2]</code>
Один столбец (1d)	Series	<code>df.iloc[:, 2]</code>
Один столбец (2d)	DataFrame	<code>df.iloc[:, [2]]</code>
Несколько столбцов	DataFrame	<code>df.iloc[:, [2, 1]]</code>

Таблица 5.2 (окончание)

Выбор	Тип возвращаемых данных	Пример
Диапазон столбцов	DataFrame	<code>df.iloc[:, :3]</code>
Один ряд (1d)	Series	<code>df.iloc[1, :]</code>
Один ряд (2d)	DataFrame	<code>df.iloc[[1], :]</code>
Несколько строк	DataFrame	<code>df.iloc[[3, 1], :]</code>
Диапазон строк	DataFrame	<code>df.iloc[1:3, :]</code>

Вот как следует применять `iloc` — с использованием тех же примеров, которые мы уже разбирали с `loc`:

```
In [21]: df.iloc[0, 0] # Возвращает скаляр
Out[21]: 'Mark'
In [22]: df.iloc[[0, 2], 1] # Возвращает Series
Out[22]: user_id
         1001    55
         1002    41
         Name: age, dtype: int64
In [23]: df.iloc[:3, [0, 2]] # Возвращает DataFrame
Out[23]: properties  name country
         user_id
         1001      Mark   Italy
         1000     John    USA
         1002      Tim    USA
```

Выбор данных по метке или позиции — не единственное средство доступа к подмножеству вашего `DataFrame`. Другим важным способом является использование булевой индексации; давайте посмотрим, как это работает!

Выбор с помощью булевой индексации

Булево индексирование² относится к выбору подмножеств `DataFrame` с помощью `Series` или `DataFrame`, данные которых состоят только из логических значений `True` или `False`. `Boolean Series` используется для выбора определенных столбцов и строк `DataFrame`, в то время как `boolean DataFrames` используются для выбора определенных значений во всем `DataFrame`. Чаще всего для фильтрации строк `DataFrame` используется булева индексация — аналогично функции автофильтра в Excel. Например, вот как можно отфильтровать `DataFrame`, чтобы произошла выборка людей старше 40 лет, живущих в США:

```
In [24]: tf = (df["age"] > 40) & (df["country"] == "USA")
         tf # Series, в котором только True/False
```

² Логическое индексирование — когда есть только два вида логических данных `True` (Истина) или `False` (Ложь).

```

Out[24]: user_id
         1001    False
         1000    False
         1002     True
         1003    False
         dtype: bool
In [25]: df.loc[tf, :]
Out[25]: properties name  age country  score continent
         user_id
         1002      Tim   41     USA    3.9  America

```

Здесь я должен объяснить две особенности. Во-первых, из-за технических ограничений вы не можете использовать булевы операторы Python из *главы 3* для операций с DataFrames. Вместо булевых операторов следует использовать символы, показанные в табл. 5.3.

Таблица 5.3. Булевы операторы

Основные типы данных Python	DataFrames и Series
and	&
or	
not	~

Во-вторых, если у вас более одного условия, убедитесь, что каждое булево выражение заключено в круглые скобки, чтобы приоритет операторов не мешал вам: например, & имеет более высокий приоритет оператора, чем ==. Таким образом, без скобок выражение из выборки будет интерпретироваться как:

```
df["age"] > (40 & df["country"]) == "USA"
```

Если вы хотите отфильтровать индекс, вы можете обратиться к нему как к `df.index`:

```

In [26]: df.loc[df.index > 1001, :]
Out[26]: properties name  age country  score continent
         user_id
         1002      Tim   41     USA    3.9  America
         1003     Jenny  12  Germany    9.0   Europe

```

Для этого следует воспользоваться оператором `in` с базовыми структурами данных Python, такими как списки. С Series используйте `isin`. Так вы отфильтруете DataFrame на участников из Италии и Германии:

```

In [27]: df.loc[df["country"].isin(["Italy", "Germany"]), :]
Out[27]: properties name  age country  score continent
         user_id
         1001      Mark   55     Italy    4.5  Europe
         1003     Jenny  12  Germany    9.0  Europe

```


В то время как для задания булевых серий вы применяете `loc`, `DataFrames` для выбора значений булевых функций, заданных целым `DataFrame`, предлагают специальный синтаксис без `loc`:

```
df[boolean_df]
```

Этот синтаксис может стать особенно полезным, если ваш `DataFrame` состоит только из чисел. Предоставление `DataFrame` логических значений возвращает `DataFrame` со значением `NaN` во всех случаях, когда `Boolean DataFrame` имеет значение `False`. Опять же, более подробное обсуждение `NaN` последует в ближайшее время. Давайте начнем с создания нового образца `DataFrame` под названием `rainfall`, который состоит только из чисел:

```
In [28]: # Это количество осадков за год в миллиметрах
rainfall = pd.DataFrame(data={"City 1": [300.1, 100.2],
                              "City 2": [400.3, 300.4],
                              "City 3": [1000.5, 1100.6]})

rainfall
Out[28]:
```

	City 1	City 2	City 3
0	300.1	400.3	1000.5
1	100.2	300.4	1100.6

```
In [29]: rainfall < 400
Out[29]:
```

	City 1	City 2	City 3
0	True	False	False
1	True	True	False

```
In [30]: rainfall[rainfall < 400]
Out[30]:
```

	City 1	City 2	City 3
0	300.1	NaN	NaN
1	100.2	300.4	NaN

Обратите внимание, в этом примере я использовал словарь для создания нового `DataFrame`, что оказывается гораздо удобнее, если данные уже существуют в такой форме. Таким образом, работа с логическими значениями чаще всего используется для фильтрации определенных значений, таких, которые выходят за рамки определенных значений.

В завершение этой части о выборе данных я введу специальный тип индекса, называемый `MultiIndex`.

Выбор с помощью `MultiIndex`

MultiIndex — это многоуровневая индексация. `MultiIndex` позволяет группировать данные по иерархии и обеспечивает легкий доступ к подмножествам. Например, если определить индекс нашего образца `DataFrame` `df` как комбинацию `continent` и `country`, вы можете легко выбрать все строки, в которых есть упоминание `continent`:

```
In [31]: # MultiIndex должен быть отсортирован
df_multi = df.reset_index().set_index(["continent", "country"])
df_multi = df_multi.sort_index()
df_multi
```

```
Out[31]: properties      user_id  name  age  score
continent country
America  USA           1000  John   33    6.7
         USA           1002   Tim   41    3.9
Europe   Germany       1003  Jenny   12    9.0
         Italy          1001   Mark   55    4.5
```

```
In [32]: df_multi.loc["Europe", :]
```

```
Out[32]: properties  user_id  name  age  score
country
Germany           1003  Jenny   12    9.0
Italy              1001   Mark   55    4.5
```

Обратите внимание, что pandas упрощает вывод MultiIndex, не повторяя самый левый уровень индекса (continents) для каждой строки. Вместо этого он выводит continent только при его изменении. Выбор из нескольких уровней индекса осуществляется с помощью кортежа:

```
In [33]: df_multi.loc[("Europe", "Italy"), :]
```

```
Out[33]: properties      user_id  name  age  score
continent country
Europe   Italy          1001  Mark   55    4.5
```

Если вы хотите выборочно обнулить часть MultiIndex, в качестве аргумента укажите уровень. Ноль — это первый столбец слева:

```
In [34]: df_multi.reset_index(level=0)
```

```
Out[34]: properties continent  user_id  name  age  score
country
USA      America          1000  John   33    6.7
USA      America          1002   Tim   41    3.9
Germany  Europe           1003  Jenny   12    9.0
Italy    Europe           1001   Mark   55    4.5
```

Хотя мы в этой книге не будем вручную создавать MultiIndex, есть определенные операции, такие как groupby, которые позволят pandas вернуть DataFrame с MultiIndex. Поэтому, что это такое, полезно знать. Мы с groupby познакомимся в этой главе, но позже.

Теперь, когда вы знаете различные способы *выбора* данных, пришло время узнать, как эти данные *изменять*.

Изменение данных

Самый простой способ изменения данных в DataFrame — присвоить значения определенным элементам с помощью атрибутов loc или iloc. Это, прежде чем мы перейдем к другим способам изменения существующих DataFrames, таких как замена значений и добавление новых столбцов, отправная точка данного раздела.

Изменение данных по метке или позиции

Как указывалось в этой главе ранее, когда вы вызываете методы `DataFrame`, такие как `df.reset_index()`, метод всегда будет применяться не к оригиналу `DataFrame`, а к его копии. Однако присвоение значений с помощью атрибутов `loc` и `iloc` изменяет исходный `DataFrame`. Поскольку мне хотелось оставить наш `DataFrame` `df` без изменений, я для экспериментов создал его копию, которой дал имя `df2`. Если вы хотите изменить только одно значение, сделайте следующее:

```
In [35]: # Сначала создайте копию DataFrame, чтобы
        # не вносить изменения в оригинал
        df2 = df.copy()
In [36]: df2.loc[1000, "name"] = "JOHN"
        df2
Out[36]: properties  name  age  country  score  continent
        user_id
        1001      Mark   55    Italy    4.5    Europe
        1000      JOHN   33     USA    6.7    America
        1002       Tim   41     USA    3.9    America
        1003     Jenny   12   Germany    9.0    Europe
```

Вы также можете изменять несколько значений одновременно. Один из способов изменить оценку пользователей с ID 1000 и 1001 — использовать список:

```
In [37]: df2.loc[[1000, 1001], "score"] = [3, 4]
        df2
Out[37]: properties  name  age  country  score  continent
        user_id
        1001      Mark   55    Italy    4.0    Europe
        1000      JOHN   33     USA    3.0    America
        1002       Tim   41     USA    3.9    America
        1003     Jenny   12   Germany    9.0    Europe
```

Изменение данных по позиции с помощью `iloc` работает подобным образом. Давайте теперь перейдем к рассмотрению, как изменять данные с помощью логического индексирования.

Изменение данных с помощью булевой индексации

Логическая (булева) индексация, которую мы использовали для фильтрации строк, также может быть использована для присвоения значений в `DataFrame`. Представьте, что вам нужно сделать анонимными все имена людей из США, которым меньше 20 лет:

```
In [38]: tf = (df2["age"] < 20) | (df2["country"] == "USA")
        df2.loc[tf, "name"] = "xxx"
        df2
Out[38]: properties  name  age  country  score  continent
        user_id
```

1001	Mark	55	Italy	4.0	Europe
1000	xxx	33	USA	3.0	America
1002	xxx	41	USA	3.9	America
1003	xxx	12	Germany	9.0	Europe

Иногда в наборе данных требуется заменить определенные значения по всем столбцам, т. е. не ограничиваясь отдельными столбцами. В этом случае снова воспользуйтесь специальным синтаксисом и снабдите весь DataFrame логическими значениями следующим образом (в примере снова используется DataFrame с данными об осадках):

```
In [39]: # Сначала создайте копию DataFrame, чтобы
        # не вносить изменения в оригинал
        rainfall2 = rainfall.copy()
        rainfall2

Out[39]:   City 1  City 2  City 3
0    300.1  400.3  1000.5
1    100.2  300.4  1100.6

In [40]: # Задайте значение 0 везде, где оно ниже 400
        rainfall2[rainfall2 < 400] = 0
        rainfall2

Out[40]:   City 1  City 2  City 3
0         0.0  400.3  1000.5
1         0.0    0.0  1100.6
```

Если вы просто хотите заменить одно значение на другое, для этого существует более простой способ. Как это сделать, я покажу дальше.

Изменение данных с помощью замены значений

Если вы хотите заменить определенное значение во всем DataFrame или в выбранных столбцах, используйте метод `replace`:

```
In [41]: df2.replace("USA", "U.S.")

Out[41]: properties  name  age  country  score  continent
        user_id
1001         Mark   55    Italy    4.0    Europe
1000         xxx   33     U.S.    3.0    America
1002         xxx   41     U.S.    3.9    America
1003         xxx   12  Germany    9.0    Europe
```

Чтобы обработать данные только в колонке `country`, используйте следующий синтаксис:

```
df2.replace({"country": {"USA": "U.S."}})
```

В данном случае, поскольку `USA` появляется только в столбце `country`, это дает тот же результат, как и в предыдущей выборке. Завершая этот раздел, давайте посмотрим, как в DataFrame можно добавить дополнительные столбцы.

Изменение данных с помощью добавления нового столбца

Чтобы добавить новый столбец в `DataFrame`, присвойте ему соответствующее имя. Например, вы можете добавить новый столбец в `DataFrame` в виде скаляра или списка:

```
In [42]: df2.loc[:, "discount"] = 0
         df2.loc[:, "price"] = [49.9, 49.9, 99.9, 99.9]
         df2
```

```
Out[42]: properties  name  age  country  score  continent  discount  price
user_id
1001          Mark   55    Italy    4.0    Europe         0    49.9
1000           xxx   33     USA    3.0    America         0    49.9
1002           xxx   41     USA    3.9    America         0    99.9
1003           xxx   12  Germany    9.0    Europe         0    99.9
```

При добавлении нового столбца часто используются векторные вычисления:

```
In [43]: df2 = df.copy() # Сначала создадим новую копию
         df2.loc[:, "birth year"] = 2021 - df2["age"]
         df2
```

```
Out[43]: properties  name  age  country  score  continent  birth year
user_id
1001          Mark   55    Italy    4.5    Europe       1966
1000          John   33     USA    6.7    America       1988
1002           Tim   41     USA    3.9    America       1980
1003          Jenny   12  Germany    9.0    Europe       2009
```

В ближайшее время я расскажу вам о вычислениях с помощью `DataFrames`, но прежде чем мы приступим к этому, помните ли вы, что я уже несколько раз использовал `NaN`? Следующий раздел, наконец, даст вам больше информации об отсутствующих данных.

Отсутствующие данные

Отсутствующие данные могут стать источником серьезных проблем, поскольку они способны исказить результаты анализа данных, тем самым делая ваши выводы менее достоверными. Тем не менее, очень часто в ваших наборах данных есть пропуски, с которыми вам придется разбираться. В Excel обычно приходится иметь дело с пустыми ячейками или ошибками `#N/A`, но pandas для недостающих данных использует NumPy's `np.nan`, отображаемых как `NaN`. `NaN` — это стандарт числа с плавающей запятой, обозначающий *не-число*. Для временных меток вместо этого используется `pd.NaT`, а для текста pandas использует `None`. Используя `None` или `np.nan`, вы можете ввести недостающие значения:

```
In [44]: df2 = df.copy() # В начале создадим новую копию
         df2.loc[1000, "score"] = None
         df2.loc[1003, :] = None
         df2
```

```
Out[44]: properties  name  age country  score continent
         user_id
1001      Mark  55.0   Italy    4.5    Europe
1000      John  33.0    USA     NaN    America
1002       Tim  41.0    USA     3.9    America
1003      None   NaN    None     NaN     None
```

Для очистки DataFrame часто требуется удалить строки с отсутствующими данными. Это делается просто:

```
In [45]: df2.dropna()
Out[45]: properties  name  age country  score continent
         user_id
1001      Mark  55.0   Italy    4.5    Europe
1002       Tim  41.0    USA     3.9    America
```

Если же вы хотите удалить только те строки, в которых отсутствуют *все* значения, используйте параметр `how`:

```
In [46]: df2.dropna(how="all")
Out[46]: properties  name  age country  score continent
         user_id
1001      Mark  55.0   Italy    4.5    Europe
1000      John  33.0    USA     NaN    America
1002       Tim  41.0    USA     3.9    America
```

Чтобы получить логический DataFrame или Series в зависимости от того, есть NaN или нет, используйте `isna`:

```
In [47]: df2.isna()
Out[47]: properties  name  age country  score  continent
         user_id
1001      False  False  False  False    False
1000      False  False  False   True    False
1002      False  False  False  False    False
1003       True   True   True   True     True
```

Для заполнения отсутствующих значений используйте `fillna`. Например, чтобы заменить NaN в столбце баллов на среднее значение (описательную статистику типа `mean` я представлю в ближайшее время):

```
In [48]: df2.fillna({"score": df2["score"].mean()})
Out[48]: properties  name  age country  score continent
         user_id
1001      Mark  55.0   Italy    4.5    Europe
1000      John  33.0    USA     4.2    America
1002       Tim  41.0    USA     3.9    America
1003      None   NaN    None     4.2     None
```

Отсутствующие данные не единственная причина, из-за которой требуется очистка набора данных. То же самое относится и к дубликатам данных, поэтому давайте посмотрим, какие у нас есть варианты!

Дубликаты данных

Как и отсутствующие данные, дубликаты негативно влияют на точность вашего анализа. Чтобы избавиться от дубликатов строк, используйте метод `drop_duplicates`. По желанию, в качестве аргумента вы можете ввести подмножество столбцов:

```
In [49]: df.drop_duplicates(["country", "continent"])
Out[49]: properties  name  age  country  score  continent
user_id
1001           Mark   55    Italy    4.5    Europe
1000           John   33     USA    6.7    America
1003           Jenny  12   Germany   9.0    Europe
```

По умолчанию будет оставлено первое вхождение. Чтобы выяснить, содержит ли определенный столбец дубликаты, или получить его уникальные значения, используйте две следующие команды (используйте `df.index` вместо `df["country"]`, если вы собираетесь выполнить индексацию):

```
In [50]: df["country"].is_unique
Out[50]: False
In [51]: df["country"].unique()
Out[51]: array(['Italy', 'USA', 'Germany'], dtype=object)
```

И, наконец, чтобы понять, какие строки являются дубликатами, используйте метод `df.index.duplicated`, который возвращает логическую серию: по умолчанию используется параметр `keep="first"`, который сохраняет первое вхождение и отмечает только дубликаты `True`. Если задать параметр `keep=False`, он будет возвращать `True` для всех строк, включая первое вхождение, что позволяет легко получить `DataFrame` со всеми дублирующимися строками. В следующем примере мы проверяем столбец `country` на наличие дубликатов, но в действительности часто просматривается или индекс, или все строки. В этом случае вам придется использовать `df.index.duplicated()` или `df.duplicated()`:

```
In [52]: # По умолчанию он отмечает только дубликаты
         # типа True, т. е. без первого вхождения
         df["country"].duplicated()
Out[52]: user_id
1001    False
1000    False
1002     True
1003    False
Name: country, dtype: bool

In [53]: # Чтобы получить все строки, в которых country
         # дублируется, используйте keep=False
         df.loc[df["country"].duplicated(keep=False), :]
Out[53]: properties  name  age  country  score  continent
user_id
1000           John   33     USA    6.7    America
1002           Tim   41     USA    3.9    America
```

После очистки DataFrames и удаления отсутствующих и повторяющихся данных у вас может возникнуть необходимость выполнить некоторые арифметические операции. В следующем разделе вы узнаете, как использовать арифметические операции.

Арифметические операции

Как и массивы NumPy, DataFrames и Series используют векторизацию. Например, чтобы добавить в DataFrame число к каждому значению rainfall, выполните следующие действия:

```
In [54]: rainfall
Out[54]:
```

	City 1	City 2	City 3
0	300.1	400.3	1000.5
1	100.2	300.4	1100.6

```
In [55]: rainfall + 100
Out[55]:
```

	City 1	City 2	City 3
0	400.1	500.3	1100.5
1	200.2	400.4	1200.6

Однако настоящая ценность pandas заключается в механизме автоматического совмещения данных: когда вы используете арифметические операторы с более чем одним DataFrame, pandas автоматически совмещает их по индексам столбцов и строк. Давайте создадим второй DataFrame с теми же метками строк и столбцов. Затем мы все просуммируем:

```
In [56]: more_rainfall = pd.DataFrame(data=[[100, 200], [300, 400]],
                                      index=[1, 2],
                                      columns=["City 1", "City 4"])

more_rainfall
Out[56]:
```

	City 1	City 4
1	100	200
2	300	400

```
In [57]: rainfall + more_rainfall
Out[57]:
```

	City 1	City 2	City 3	City 4
0	NaN	NaN	NaN	NaN
1	200.2	NaN	NaN	NaN
2	NaN	NaN	NaN	NaN

Индекс и столбцы итогового DataFrame являются объединением индексов и столбцов двух DataFrame: поля, имеющие значение в обоих DataFrames, показывают сумму, а остальная часть DataFrame показывает NaN. К этому, если вы работаете в Excel, где пустые ячейки, когда вы используете их в арифметических операциях, автоматически превращаются в нули, пожалуй, придется привыкнуть. Чтобы получить те же изменения, как в Excel, используйте для замены значений NaN нулями метод add с параметром fill_value:

```
In [58]: rainfall.add(more_rainfall, fill_value=0)
Out[58]:
```

	City 1	City 2	City 3	City 4
--	--------	--------	--------	--------

0	300.1	400.3	1000.5	NaN
1	200.2	300.4	1100.6	200.0
2	300.0	NaN	NaN	400.0

Как показано в табл. 5.4, этот метод работает и для других арифметических операторов.

Таблица 5.4. Арифметические операторы

Оператор	Метод
*	mul (умножение)
+	add (сложение)
-	sub (вычитание)
/	div (деление)
**	pow (возведение в степень)

Когда у вас в вычислении есть DataFrame и Series, по умолчанию Series транслируется вдоль индекса:

```
In [59]: # Series, взятый из ряда
rainfall.loc[1, :]
Out[59]: City 1    100.2
City 2    300.4
City 3   1100.6
Name: 1, dtype: float64
In [60]: rainfall + rainfall.loc[1, :]
Out[60]:   City 1  City 2  City 3
0    400.3    700.7  2101.1
1    200.4    600.8  2201.2
```

Следовательно, чтобы добавить Series по столбцам, необходимо использовать метод add с явным аргументом axis:

```
In [61]: # Series, взятый из столбца
rainfall.loc[:, "City 2"]
Out[61]: 0    400.3
1    300.4
Name: City 2, dtype: float64
In [62]: rainfall.add(rainfall.loc[:, "City 2"], axis=0)
Out[62]:   City 1  City 2  City 3
0    700.4    800.6  1400.8
1    400.6    600.8  1401.0
```

Этот раздел был посвящен DataFrames с числами и тому, как они ведут себя при выполнении арифметических операций. В следующем разделе мы рассмотрим способы обработки текста в DataFrames.

Работа с текстовой колонкой

В начале этой главы я уже показал, что столбцы с текстовыми или смешанными типами данных имеют тип данных `object`. Для выполнения операций над столбцами с текстовыми строками используйте атрибут `str`, который дает вам доступ к методам для обработки строк Python. Мы уже познакомились с несколькими методами для работы со строками в главе 3, но не помешает взглянуть на доступные методы, описанные в документации Python³. Например, чтобы удалить пробелы в начале и конце строки, используйте метод `strip`; чтобы сделать все первые буквы заглавными, существует метод `capitalize`. Соединив их вместе, вы очистите беспорядочные текстовые колонки, которые часто появляются при ручном вводе данных:

```
In [63]: # Сначала создадим новый фрейм DataFrame
         users = pd.DataFrame(data=[" mArk ", "JOHN ", "Tim", " jenny"],
                             columns=["name"])

         users
Out[63]:   name
0  mArk
1  JOHN
2   Tim
3  jenny

In [64]: users_cleaned = users.loc[:, "name"].str.strip().str.capitalize()
         users_cleaned
Out[64]: 0   Mark
         1   John
         2   Tim
         3  Jenny
         Name: name, dtype: object
```

Или найти все имена, которые начинаются на букву "J":

```
In [65]: users_cleaned.str.startswith("J")
Out[65]: 0   False
         1    True
         2   False
         3    True
         Name: name, dtype: bool
```

Строковые методы просты в использовании, но иногда вам может потребоваться выполнить операции с DataFrame способом, которого нет во встроенных методах. В этом случае, как показано в следующем разделе, создайте собственную функцию и примените ее к DataFrame.

³ <https://oreil.ly/-e7SC>. — Примеч. пер.

Использование функции

DataFrames использует метод `applymap`, который будет применять функцию к каждому отдельному элементу, что полезно, если нет доступных универсальных функций в NumPy. Например, для форматирования строк универсальных функций нет. И в этом случае мы можем отформатировать каждый элемент DataFrame следующим образом:

```
In [66]: rainfall
Out[66]:   City 1  City 2  City 3
          0   300.1   400.3 1000.5
          1   100.2   300.4 1100.6

In [67]: def format_string(x):
          return f"{x:,.2f}"

In [68]: # Обратите внимание, что мы передаем функцию без ее
          # вызова, т.е. format_string, а не format_string()!
rainfall.applymap(format_string)
Out[68]:   City 1  City 2  City 3
          0  300.10  400.30 1,000.50
          1  100.20  300.40 1,100.60
```

Разберем это подробнее: приведенная f-строка возвращает `x` как строку: `f"{x}"`. Чтобы включить форматирование, добавьте к переменной двоеточие, за которым следует строка форматирования `,.2f`. Запятая является разделителем тысяч, а `.2f` подразумевает обозначение с фиксированной запятой с двумя цифрами после десятичной точки. Для получения более подробной информации о том, как форматировать строки, обратитесь к Format Specification Mini-Language (Мини-язык форматирования в Python)⁴, который является частью документации Python.

Для таких случаев широко используются *лямбда-выражения* (см. врезку «Лямбда-выражения»), поскольку они позволяют написать то же самое в одной строке без необходимости определять отдельную функцию. С помощью лямбда-выражений мы можем переписать предыдущий пример следующим образом:

```
In [69]: rainfall.applymap(lambda x: f"{x:,.2f}")
Out[69]:   City 1  City 2  City 3
          0  300.10  400.30 1,000.50
          1  100.20  300.40 1,100.60
```

Лямбда-выражения

Python позволяет определить функцию в одной строке с помощью *лямбда-выражений*. Лямбда-выражения являются анонимными функциями, что означает, что это функция без имени. Рассмотрим данную функцию:

```
def function_name(arg1, arg2, ...):
    return return_value
```

⁴ <https://oreil.ly/NgsG8>.—Примеч. пер.

Эта функция может быть переписана в виде лямбда-выражения следующим образом:

```
lambda arg1, arg2, ...: return_value
```

По сути вы заменяете `def` на `lambda`, убираете ключевое слово `return` и имя функции и помещаете все в одну строку. Как мы видели на примере метода `applymap`, в данном случае это очень удобно, поскольку нам не нужно определять функцию, которая используется только один раз.

На данный момент я упомянул все важные методы работы с данными, но прежде чем двигаться дальше, важно понять, когда `pandas` использует представление `DataFrame`, а когда — копию.

Просмотр и копирование

Из предыдущей главы вы наверняка помните, что нарезка массивов `NumPy` возвращает представление. С `DataFrames`, к сожалению, не так все просто: не очень понятно, что возвращают `loc` и `iloc` — представления или копии. И поэтому здесь очень легко запутаться. Так как существует большая разница, в какой файл вы вносите изменения — в представление или в копию `DataFrame`, `pandas`, когда считает, что изменения вносятся недопустимым способом, выводит следующее предупреждение: `SettingWithCopyWarning` (Изменение конфигурации с предупреждением о копировании). Чтобы обойти это довольно загадочное предупреждение, даю несколько советов:

- ◆ устанавливайте значения в исходном `DataFrame`, а не в `DataFrame`, который был создан на основе другого `DataFrame`;
- ◆ если после нарезки вы хотите получить уникальный `DataFrame`, сделайте копию:

```
selection = df.loc[:, ["country", "continent"]].copy()
```

Хотя с `loc` и `iloc` все обстоит сложнее, следует помнить, что все методы `DataFrame`, такие как `df.dropna()` или `df.sort_values("column_name")`, *всегда* возвращают копию.

До сих пор мы работали в основном только с одним `DataFrame`. В следующем разделе показаны различные способы объединения нескольких `DataFrames` в один файл — очень распространенная задача, для решения которой `pandas` предлагает мощные инструменты.

Объединение DataFrames

Объединение различных наборов данных в Excel может быть трудоемкой задачей и, как правило, включает в себя множество формул `VLOOKUP`. К счастью, объединение `DataFrames` является одной из самых сильных функций `pandas`, где возможности совмещения данных действительно облегчат вам жизнь, значительно снизив

вероятность внесения ошибок. Объединение DataFrames может быть выполнено различными способами; в этом разделе рассматриваются только наиболее распространенные — с использованием `concat`, `join` и `merge`. Несмотря на то, что функции частично совпадают, каждая из них значительно упрощает выполнение определенной задачи. Я начну с функции `concat`, затем объясню различные возможности функции `join` и в заключение представлю функцию `merge`, наиболее универсальную из этих трех функций.

```
Out[72]:      quizzes  logins
        1000      3      4
        2000      5      6
```

```
In [73]: pd.concat([df, more_categories], axis=1)
```

```
Out[73]:      name  age  country  score  continent  quizzes  logins
        1000  John  33.0     USA    6.7   America    3.0    4.0
        1001  Mark  55.0    Italy    4.5   Europe     NaN     NaN
        1002  Tim   41.0     USA    3.9   America     NaN     NaN
        1003  Jenny 12.0  Germany  9.0   Europe     NaN     NaN
        2000   NaN   NaN     NaN    NaN     NaN     5.0    6.0
```

Особой и очень полезной особенностью `concat` является то, что он может работать более чем с двумя `DataFrames`. Мы будем использовать данную функцию в следующей главе для создания единого `DataFrame` из нескольких CSV-файлов:

```
pd.concat([df1, df2, df3, ...])
```

С другой стороны, `join` и `merge` (объединение и слияние), как мы увидим далее, работают только с двумя `DataFrames`.

Объединение и слияние

Когда вы объединяете два `DataFrame`, вы объединяете столбцы каждого `DataFrame` в новый `DataFrame` и, опираясь на теорию множеств, решаете, что будет со строками. Если вы уже работали с реляционными базами данных⁵, это та же концепция, что и `JOIN` в SQL-запросах. На двух примерах рис. 5.3, в которых использованы два `DataFrames`, `df1` и `df2`, показано, как работают четыре типа объединения (то есть внутреннее, левое, правое и внешнее).

При применении `join` в `pandas` для совмещения строк используются индексы обоих `DataFrames`. Воспользовавшись *inner join*, мы получим `DataFrame`, в котором содержатся только те строки, индексы которых совпадают. Применяя *left join*, мы действуем все строки из левого `DataFrame` `df1`, которые объединяются только со строками из правого `DataFrame` `df2` с общим для `df1` и `df2` индексом. Если в `df2` нет строки с таким же индексом, `pandas` в общей строке подставит значение `NaN`. *Left join* соответствует случаю `VLOOKUP` в Excel. При *right join* берутся все строки из правого `DataFrame` `df2` и объединяются со строками из `DataFrame` `df1` по совпадающему в обоих `DataFrame` индексу. И, наконец, *full outer join*, который сокращенно называется *outer join*, использует индексы из обоих `DataFrames` и объединяет значения там, где это возможно. Представим рис. 5.3 в текстовой форме в виде табл. 5.5.

⁵ Реляционные базы данных — это базы, которые используются для хранения и предоставления доступа к взаимосвязанным элементам информации. Такие базы данных основаны на наглядном, интуитивно понятном, в виде таблиц, способе представления информации. Каждая строка в таблице такой базы данных представляет собой запись с уникальным идентификатором, называемым ключом. — *Примеч. пер.*

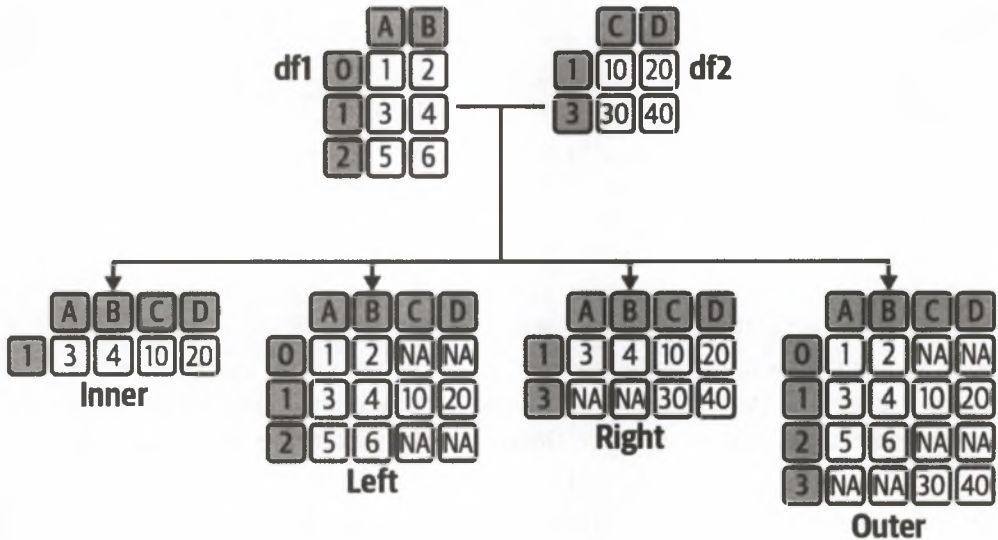


Рис. 5.3. Типы объединений

Таблица 5.5. Типы объединений

Тип	Значение
inner (внутренний)	Только строки, индекс которых присутствует в обоих DataFrames
left (левый)	Все строки из левого DataFrame, совпадающие со строками из правого DataFrame
right (правый)	Все строки из правого DataFrame, совпадающие со строками из левого DataFrame
outer (внешний)	Объединение индексов строк из обоих фреймов данных

Давайте посмотрим, как каждый тип объединения работает на практике, воспользовавшись примерами, показанными на рис. 5.3:

```
In [74]: df1 = pd.DataFrame(data=[[1, 2], [3, 4], [5, 6]],
                             columns=["A", "B"])

df1
Out[74]:   A  B
0  1  2
1  3  4
2  5  6

In [75]: df2 = pd.DataFrame(data=[[10, 20], [30, 40]],
                             columns=["C", "D"], index=[1, 3])

df2
Out[75]:   C  D
1  10 20
3  30 40
```

```
In [76]: df1.join(df2, how="inner")
```

```
Out[76]:   A  B  C  D
         1  3  4  10  20
```

```
In [77]: df1.join(df2, how="left")
```

```
Out[77]:   A  B    C    D
         0  1  2   NaN   NaN
         1  3  4  10.0  20.0
         2  5  6   NaN   NaN
```

```
In [78]: df1.join(df2, how="right")
```

```
Out[78]:   A    B  C  D
         1  3.0  4.0  10  20
         3  NaN  NaN  30  40
```

```
In [79]: df1.join(df2, how="outer")
```

```
Out[79]:   A    B    C    D
         0  1.0  2.0   NaN   NaN
         1  3.0  4.0  10.0  20.0
         2  5.0  6.0   NaN   NaN
         3  NaN  NaN  30.0  40.0
```

Если вы хотите объединить один или несколько столбцов DataFrame вместе того, чтобы использовать индекс, воспользуйтесь `merge` вместо `join`. `merge` для предоставления одного или нескольких столбцов в качестве условия объединения принимает аргумент `on`: эти общие для обоих DataFrames столбцы, используются для объединения строк:

```
In [80]: # Добавление столбца под названием "category"
```

```
        # в оба DataFrames
```

```
        df1["category"] = ["a", "b", "c"]
```

```
        df2["category"] = ["c", "b"]
```

```
In [81]: df1
```

```
Out[81]:   A  B category
         0  1  2         a
         1  3  4         b
         2  5  6         c
```

```
In [82]: df2
```

```
Out[82]:   C  D category
         1  10  20         c
         3  30  40         b
```

```
In [83]: df1.merge(df2, how="inner", on=["category"])
```

```
Out[83]:   A  B category  C  D
         0  3  4         b  30  40
         1  5  6         c  10  20
```

```
In [84]: df1.merge(df2, how="left", on=["category"])
```

```
Out[84]:   A  B category  C    D
         0  1  2         a  NaN  NaN
         1  3  4         b  30.0  40.0
         2  5  6         c  10.0  20.0
```

Поскольку `join` и `merge` для реализации более сложных сценариев принимают большое количество необязательных аргументов, чтобы узнать больше, я приглашаю вас заглянуть в официальную документацию⁶.

Теперь вы знаете, как оперировать одним или несколькими `DataFrames`, и это подводит нас к следующему шагу в нашем путешествии по анализу данных: осмыслению данных.

Описательная статистика и агрегация данных

Одним из способов анализа больших наборов данных является расчет описательной статистики, такой как сумма или среднее значение, либо по всему набору данных, либо по значимым подмножествам. Этот раздел начинается с рассмотрения того, как такой анализ работает в `pandas`, а затем представляет два способа объединить данные в подмножества: метод `groupby` и функция `pivot_table`.

Описательная статистика

Описательная статистика позволяет обобщать наборы данных с помощью количественных показателей. Например, количество точек данных — это простая описательная статистика. Другими популярными примерами являются средние значения, такие как среднее, медиана или мода⁷. `DataFrames` и `Series` позволяют получить удобный доступ к описательной статистике с помощью таких методов, как `sum`, `mean` и `count` и т. д. Со многими из этих методов вы познакомитесь в этой книге, а полный список можно найти в документации `pandas`⁸. По умолчанию эти методы возвращают `Series` вдоль `axis=0`, и это значит, что возвращается статистика столбцов:

```
In [85]: rainfall
Out[85]:
```

	City 1	City 2	City 3
0	300.1	400.3	1000.5
1	100.2	300.4	1100.6

```
In [86]: rainfall.mean()
Out[86]:
```

	City 1	City 2	City 3
	200.15	350.35	1050.55

```
dtype: float64
```

Если вам нужна статистика по строкам, введите аргумент `axis=1`:

```
In [87]: rainfall.mean(axis=1)
Out[87]:
```

	0	1
	566.966667	500.400000

```
dtype: float64
```

⁶ <https://oreil.ly/OZ4WV>. — Примеч. пер.

⁷ Характеристика дискретной случайной величины, равная наиболее часто принимаемому значению.

⁸ <https://oreil.ly/t2q9Q>. — Примеч. пер.

По умолчанию в описательные статистики, такие как `sum` или `mean`, отсутствующие значения не включаются. Это соответствует тому, как Excel трактует пустые ячейки, поэтому использование формулы `AVERAGE` в Excel в диапазоне с пустыми ячейками даст вам тот же результат, что и метод `mean`, примененный к серии с теми же числами и значениями `NaN` вместо пустых ячеек.

Полученная статистика по всем строкам `DataFrame` иногда недостаточно подробно, и вам нужна более детальная информация, например среднее значение по категориям. Давайте посмотрим, как это делается!

Группировка

Мы снова воспользуемся нашим `DataFrame` `df` и вычислим средний балл по континентам! Для этого сначала сгруппируем строки по `continent`, а затем применим метод `mean`, который рассчитает среднее значение для *каждой группы*. Все столбцы без чисел автоматически исключаются:

```
In [88]: df.groupby(["continent"]).mean()
Out[88]: properties  age  score
continent
America          37.0   5.30
Europe           33.5   6.75
```

Если вы в расчетах используете данные более чем одного столбца, результирующий `DataFrame` будет иметь многоуровневый индекс — `MultiIndex`, с которым мы познакомились ранее:

```
In [89]: df.groupby(["continent", "country"]).mean()
Out[89]: properties      age  score
continent country
America   USA          37    5.3
Europe   Germany      12    9.0
          Italy        55    4.5
```

Вместо метода `mean` можно использовать большинство описательных статистик, которые предлагает `pandas`, а если вы хотите использовать собственную функцию, воспользуйтесь методом `agg`. Например, вот как можно получить разницу между максимальным и минимальным значением для каждой группы:

```
In [90]: df.groupby(["continent"]).agg(lambda x: x.max() - x.min())
Out[90]: properties  age  score
continent
America           8    2.8
Europe           43    4.5
```

Одним из популярных способов получения статистики по группам в Excel является использование сводных таблиц. Они привносят второе измерение и позволяют взглянуть на данные с разных точек зрения. В `pandas`, как мы увидим далее, есть функция `pivot table` (сводная таблица).

Pivoting и Melting

Если вы используете сводные таблицы в Excel, вам не составит труда в pandas применить функцию `pivot_table`, потому что она работает практически так же, как и в Excel. Данные в следующем DataFrame упорядочены так же, как записи обычно хранятся в базе данных; каждая строка показывает сделку по продаже определенного фрукта в определенном регионе:

```
In [91]: data = [{"Fruit": "Oranges", "Region": "North", "Revenue": 12.30},
                {"Fruit": "Apples", "Region": "South", "Revenue": 10.55},
                {"Fruit": "Oranges", "Region": "South", "Revenue": 22.00},
                {"Fruit": "Bananas", "Region": "South", "Revenue": 5.90},
                {"Fruit": "Bananas", "Region": "North", "Revenue": 31.30},
                {"Fruit": "Oranges", "Region": "North", "Revenue": 13.10}]

sales = pd.DataFrame(data=data,
                     columns=["Fruit", "Region", "Revenue"])

sales
```

```
Out[91]:
```

	Fruit	Region	Revenue
0	Oranges	North	12.30
1	Apples	South	10.55
2	Oranges	South	22.00
3	Bananas	South	5.90
4	Bananas	North	31.30
5	Oranges	North	13.10

Для создания сводной таблицы следует в качестве первого аргумента функции `pivot_table` предоставить DataFrame. `index` и `columns` определяют, какой столбец DataFrame станет меткой строки и, соответственно, столбца сводной таблицы. `values` с помощью `aggfunc` — функции, которая может быть представлена в виде строки или NumPy `ufunc`, будет объединяться в часть данных результирующего DataFrame. И, наконец, `margins` соответствует Grand Total в Excel, т. е. если убрать `margins` и `margins_name`, то столбец и строка Total не будут отображаться:

```
In [92]: pivot = pd.pivot_table(sales,
                                index="Fruit", columns="Region",
                                values="Revenue", aggfunc="sum",
                                margins=True, margins_name="Total")

pivot
```

```
Out[92]:
```

Region	North	South	Total
Fruit			
Apples	NaN	10.55	10.55
Bananas	31.3	5.90	37.20
Oranges	25.4	22.00	47.40
Total	56.7	38.45	95.15

В целом преобразование данных в сводную таблицу означает, что вы берете уникальные значения столбца (в нашем случае Region) и преобразуете их в заголовки столбцов сводной таблицы, тем самым объединяя значения из другого столбца.

Это позволяет легко считывать сводную информацию по интересующим измерениям. В нашей сводной таблице вы сразу увидите, что в северном регионе не было продано ни одного яблока, а в южном регионе большая часть выручки приходится на апельсины. Если вы хотите поступить наоборот и превратить заголовки столбцов в значения одного столбца, используйте функцию `melt`. В этом случае функция `melt` является противоположностью функции `pivot_table`:

```
In [93]: pd.melt(pivot.iloc[:,-1].reset_index(),
                id_vars="Fruit",
                value_vars=["North", "South"], value_name="Revenue")

Out[93]:
```

	Fruit	Region	Revenue
0	Apples	North	NaN
1	Bananas	North	31.30
2	Oranges	North	25.40
3	Apples	South	10.55
4	Bananas	South	5.90
5	Oranges	South	22.00

Здесь я представляю нашу сводную таблицу в виде входных данных, но использую `iloc`, чтобы избавиться от итоговой строки и столбца. Я также сброшу индекс, чтобы вся информация была доступна в виде обычных столбцов. Затем я предоставляю `id_vars` для указания идентификаторов и `value_vars` для определения столбцов, которые я хочу «развернуть». Это может быть полезно, если вы хотите подготовить данные, чтобы их можно было сохранить обратно в базе данных, которая ожидает их в этом формате.

Работа с обобщенной статистикой поможет вам понять ваши данные, но никому не нравится читать страницу, полную цифр. Чтобы сделать информацию легко воспринимаемой, нет ничего лучше, чем использование наглядных представлений, о которых мы поговорим в следующей главе. Хотя в Excel используется термин *диаграммы*, в pandas их принято называть *графиками*. В этой книге я буду использовать эти термины как взаимозаменяемые.

Построение графиков

С помощью графиков мы наглядно представляем результаты анализа данных, что может оказаться самым важным шагом во всем процессе. Для построения графиков мы будем использовать две библиотеки: начнем с Matplotlib — библиотеки для построения графиков, используемой по умолчанию в pandas. А затем познакомимся с Plotly, современной библиотеки для построения графиков, которая при помощи блокнотов Jupyter позволяет нам работать в диалоговом режиме.

Matplotlib

Matplotlib — это пакет для создания чертежей, существующий уже долгое время и входящий в состав в дистрибутив Anaconda. С его помощью можно создавать гра-

фики в различных форматах, включая векторную графику для высококачественной печати. Когда вы вызываете метод `plot` для `DataFrame`, pandas по умолчанию создаст график Matplotlib.

Чтобы использовать Matplotlib в блокноте Jupyter, необходимо сначала выполнить одну из двух магических команд: `%matplotlib inline` или `%matplotlib notebook` (см. врезку «Магические команды»). Эти команды конфигурируют приложение так, чтобы графики можно было отображать в самом блокноте. Последняя команда добавляет больше возможностей, позволяя изменять размер или коэффициент масштабирования графика. Давайте начнем и создадим первый график с помощью pandas и Matplotlib (рис. 5.4):

```
In [94]: import numpy as np
          %matplotlib inline
          # Или %matplotlib notebook

In [95]: data = pd.DataFrame(data=np.random.rand(4, 4) * 100000,
                              index=["Q1", "Q2", "Q3", "Q4"],
                              columns=["East", "West", "North", "South"])

          data.index.name = "Quarters"
          data.columns.name = "Region"
          data
```

```
Out[95]: Region      East      West      North      South
Quarters
Q1      23254.220271  96398.309860  16845.951895  41671.684909
Q2      87316.022433  45183.397951  15460.819455  50951.465770
Q3      51458.760432  3821.139360   77793.393899  98915.952421
Q4      64933.848496  7600.277035   55001.831706  86248.512650
```

```
In [96]: data.plot() # Ярлык для data.plot.line().

Out[96]: <AxesSubplot:xlabel='Quarters'>
```

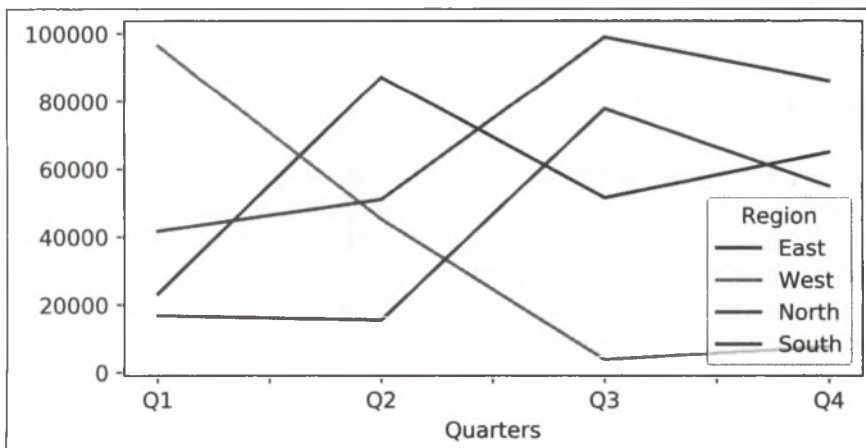


Рис. 5.4. График Matplotlib

Обратите внимание, что в этом примере для построения pandas DataFrame я использовал массив NumPy. Предоставление массивов NumPy позволяет вам воспользоваться конструкторами NumPy, с которыми мы познакомились в прошлой главе; здесь мы применяем NumPy для генерации pandas DataFrame на основе псевдослучайных чисел.

Магические команды

Команда `%matplotlib inline`, которую мы использовали, чтобы обеспечить корректную работу Matplotlib с блокнотами Jupyter, является магической командой. *Магические команды* — это набор простых команд, которые обеспечивают определенное поведение ячейки блокнота Jupyter и настолько упрощают выполнение громоздких задач, что это похоже на волшебство. Вы пишете магические команды в ячейках, как код Python, но они начинаются либо с `%`, либо с `.`. Команды, которые влияют на всю ячейку, начинаются с `%`, а команды, которые влияют только на одну строку в ячейке, начинаются с `.`.

Мы познакомимся с другими магическими командами в следующих главах, но если вы хотите получить список всех доступных на данный момент магических команд, выполните `%lsmagic`, а для получения подробного описания выполните `%magic`.

Даже если вы используете магическую команду `%matplotlib notebook`, вы, вероятно, заметите, что Matplotlib изначально был разработан для статических графиков, а не для диалогового взаимодействия на веб-странице. Поэтому далее мы будем использовать Plotly, библиотеку для построения чертежей и графиков, разработанную для работы в Интернете.

Plotly

Plotly — это библиотека на базе JavaScript, которая с версии 4.8.0 может использоваться в pandas как бэкэнд⁹ при построении графиков в диалоговом режиме: вы можете легко увеличить масштаб, выбрать легенду или отменить выбор категории, а также получить всплывающие подсказки с дополнительной информацией о конкретных данных, на которые вы навели курсор. Plotly не входит в состав дистрибутива Anaconda, поэтому, если вы еще не установили его, сделайте это сейчас, выполнив следующую команду:

```
(base)> conda install plotly
```

Как только вы перейдете к следующей ячейке, бэкэнд, отвечающая за построение графиков, и библиотека Plotly будут задействованы для всех ячеек блокнота, и если вы повторно выполните предыдущую ячейку, она отобразится в виде графика

⁹ Программно-аппаратная часть сервиса, которая работает на сервере, а не в браузере или на компьютере. — Примеч. ред.

Plotly. Для Plotly, прежде чем вы сможете отобразить рис. 5.5 и 5.6, установите магическую команду в бэкэнд, чтобы каждый раз ее не выполнять:

```
In [97]: # Установите сервер для создания чертежей Plotly
pd.options.plotting.backend = "plotly"
In [98]: data.plot()
```

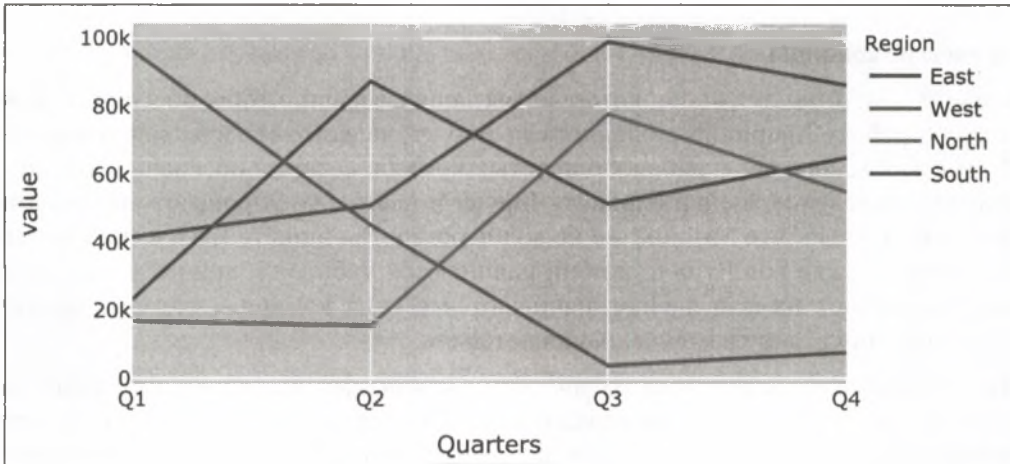


Рис. 5.5. Линейный график Plotly

```
In [99]: # Отображение тех же данных в виде гистограммы
data.plot.bar(barmode="group")
```

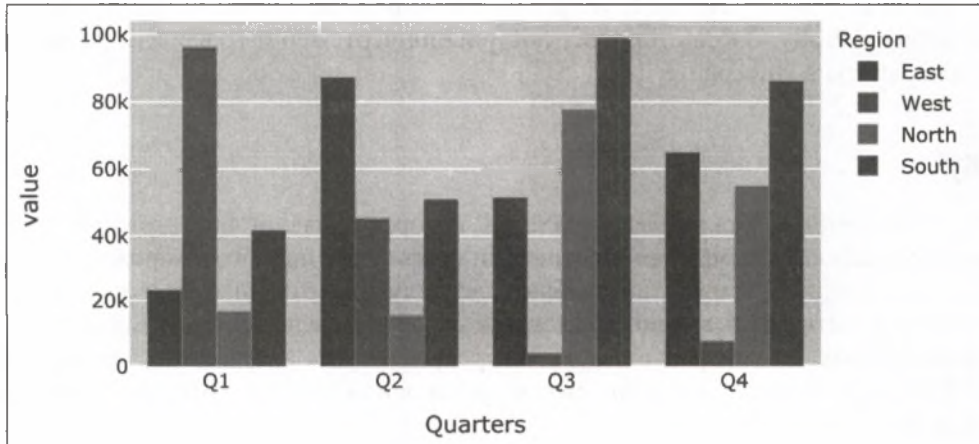


Рис. 5.6. Гистограмма Plotly



Различия в построении бэкэнда

Если вы при построении графика используете Plotly в качестве бэкэнда, вам нужно будет проверить принятые аргументы методов `plot` непосредственно в документах Plotly. Например, вы можете ознакомиться с аргументом `barmode=group` в документации по гистограммам Plotly.

pandas и базовые библиотеки черчения предлагают множество типов графиков и опций для их форматирования и позволяют отображать графики в желаемом формате. Также возможно несколько отдельных графиков преобразовать в набор вложенных графиков. В табл. 5.6 кратко описаны доступные типы графиков.

Таблица 5.6. Типы графиков pandas

Тип	Значение
line	Линейный график, отображаемый по умолчанию при запуске <code>df.plot()</code>
bar	Вертикальная гистограмма
barh	Горизонтальная гистограмма
hist	Гистограмма
box	Прямоугольная диаграмма
kde	График плотности, для его построения следует выполнить <code>density</code>
area	Диаграмма с областями
scatter	Точечный график
hexbin	Гексагональный биннинг
pie	Круговая диаграмма

Кроме того, pandas предлагает некоторые инструменты и методы построения графиков более высокого уровня, которые состоят из множества отдельных компонентов. Подробности см. в документации по визуализации¹⁰ pandas.

Другие библиотеки для построения графиков

Область научной визуализации в Python очень развита, и помимо Matplotlib и Plotly, существует множество других высококачественных приложений, которые могут в определенных случаях быть полезными.

Seaborn¹¹

Разработан на основе Matplotlib. Приложение по умолчанию улучшает стиль и создает дополнительные графики, такие как тепловые карты, которые часто упрощают вашу работу: вы можете создавать улучшенные статистические графики, написав всего несколько строк кода.

¹⁰ <https://oreil.ly/FxYg9>. — Примеч. пер.

¹¹ <https://oreil.ly/a3U1t>. — Примеч. пер.

*Bokeh*¹²

По технологии и функциональности похож на Plotly: он основан на JavaScript и поэтому хорошо подходит для создания интерактивных диаграмм в блокнотах Jupyter. Bokeh входит в состав Anaconda.

*Altair*¹³

Библиотека для статистической визуализации, основанная на проекте Vega¹⁴. Altair также основывается на JavaScript и предлагает некоторые возможности взаимодействия, например масштабирование.

*HoloViews*¹⁵

Пакет также на основе JavaScript, который фокусируется на упрощении анализа и визуализации данных. С помощью нескольких строк кода можно получить сложные статистические графики.

В следующей главе мы создадим больше графиков для анализа временных рядов, но перед этим давайте завершим эту главу, узнав, как можно импортировать и экспортировать данные с помощью pandas!

Импорт и экспорт DataFrames

До сих пор мы создавали DataFrames с нуля, используя вложенные списки, словари или массивы NumPy. Эти методы важно знать, но, как правило, эти данные доступны, и вам просто нужно преобразовать их в DataFrame. Для этого pandas предлагает различные функции чтения. Но даже если вам нужно получить доступ к проприетарной системе, для которой pandas не предлагает встроенный модуль чтения, у вас наверняка есть пакет Python для подключения к этой системе, и как только у вас появятся данные, их достаточно легко превратить в DataFrame. В Excel такой тип задач, как импорт данных, обычно выполняется с помощью Power Query.

После анализа и изменения набора данных вы, возможно, захотите вернуть результаты в базу данных, экспортировать их в CSV-файл или — учитывая название книги — представить их своему руководителю в виде рабочей книге Excel. Чтобы экспортировать pandas DataFrames, используйте один из методов экспорта, которые предлагают DataFrames. В табл. 5.7 приведен обзор наиболее распространенных методов импорта и экспорта.

¹² <https://docs.bokeh.org>. — Примеч. пер.

¹³ <https://oreil.ly/t06t7>. — Примеч. пер.

¹⁴ <https://oreil.ly/RN6A7>. — Примеч. пер.

¹⁵ <https://holoviews.org>. — Примеч. пер.

Таблица 5.7. Импорт и экспорт DataFrames

Формат данных/система	Импорт: функция pandas (pd)	Экспорт: метод DataFrame (df)
CSV files	pd.read_csv	df.to_csv
JSON	pd.read_json	df.to_json
HTML	pd.read_html	df.to_html
Clipboard	pd.read_clipboard	df.to_clipboard
Excel files	pd.read_excel	df.to_excel
SQL Databases	pd.read_sql	df.to_sql

Мы познакомимся с `pd.read_sql` и `pd.to_sql` в *главе 11*, где воспользуемся ими в рамках тематического исследования. И поскольку я собираюсь посвятить всю *главу 7* теме чтения и записи файлов Excel с помощью pandas, в этом разделе я сосредоточусь на импорте и экспорте файлов CSV. Давайте начнем с экспорта существующего DataFrame!

Экспорт CSV файлов

Если вам нужно передать DataFrame коллеге, который, возможно, не использует Python или pandas, правильно будет сохранить его как CSV-файл: практически каждая программа знает, как их импортировать. Чтобы экспортировать наш образец DataFrame `df` в CSV-файл, используйте метод `to_csv`:

```
In [100]: df.to_csv("course_participants.csv")
```

Если вы хотите сохранить файл в другом каталоге, укажите полный путь в виде необработанной строки, например, `r"C:\path\to\desired\location\msft.csv"`.



Использование необработанных строк как пути к файлам в Windows

В строках для экранирования определенных символов используется *обратная косая черта*. Поэтому для работы с путями к файлам в Windows нужно либо использовать двойные обратные косые черты (`C:\\path\\to\\file.csv`), либо префикс строки с буквой `r`, чтобы превратить ее в *необработанную строку*, которая интерпретирует символы буквально. В macOS или Linux такой проблемы не возникает, так как в этих операционных системах в путях используются просто косые черты.

Если указать только имя файла, как это делаю я, будет создан файл `course_participants.csv` в том же каталоге, где находится и блокнот, со следующим содержимым:

```
user_id,name,age,country,score,continent
1001,Mark,55,Italy,4.5,Europe
1000,John,33,USA,6.7,America
```

```
1002, Tim, 41, USA, 3.9, America
1003, Jenny, 12, Germany, 9.0, Europe
```

Теперь, когда вы знаете, как использовать метод `df.to_csv`, давайте посмотрим, как работает импорт файла CSV файла!

Импорт CSV-файлов

Импортировать локальный CSV-файл так же просто, как указать путь к нему в функции `read_csv`. `MSFT.csv` — это CSV-файл, который я загрузил из Yahoo! Finance содержит ежедневные исторические цены акций Microsoft — вы найдете его в сопутствующем репозитории, в папке `csv`:

```
In [101]: msft = pd.read_csv("csv/MSFT.csv")
```

Часто, кроме имени файла, вам для `read_csv` потребуется указать еще несколько параметров. Например, `sep` позволяет указать pandas, какой разделитель или ограничитель используется в CSV-файле, если ограничитель не является запятой по умолчанию. Мы в следующей главе воспользуемся еще несколькими параметрами, а полный обзор вы найдете в документации pandas¹⁶.

Теперь, когда мы занимаемся большими `DataFrame`, содержащими многие тысячи строк, обычно, чтобы получить сводную информацию о `DataFrame`, сначала запускается метод `info`. Далее, воспользовавшись методами `head` и `tail`, вы можете просмотреть первые и последние строки `DataFrame`. По умолчанию эти методы возвращают только пять строк, но количество возвращаемых строк можно изменить, указав в качестве аргумента желаемое значение. Чтобы получить некоторые основные статистические данные, вы также можете запустить метод `describe`:

```
In [102]: msft.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8622 entries, 0 to 8621
Data columns (total 7 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   Date        8622 non-null  object
1   Open        8622 non-null  float64
2   High        8622 non-null  float64
3   Low         8622 non-null  float64
4   Close       8622 non-null  float64
5   Adj Close   8622 non-null  float64
6   Volume      8622 non-null  int64
dtypes: float64(5), int64(1), object(1)
memory usage: 471.6+ KB

In [103]: # Я выбираю несколько столбцов из-за нехватки места.
          # Вы можете просто выполнить: msft.head()
          msft.loc[:, ["Date", "Adj Close", "Volume"]].head()
```

¹⁶ <https://oreil.ly/2GMhW>. — Примеч. пер.

```

Out[103]:      Date  Adj Close  Volume
0  1986-03-13  0.062205  1031788800
1  1986-03-14  0.064427  308160000
2  1986-03-17  0.065537  133171200
3  1986-03-18  0.063871  67766400
4  1986-03-19  0.062760  47894400

In [104]: msft.loc[:, ["Date", "Adj Close", "Volume"]].tail(2)
Out[104]:      Date  Adj Close  Volume
8620  2020-05-26  181.570007  36073600
8621  2020-05-27  181.809998  39492600

In [105]: msft.loc[:, ["Adj Close", "Volume"]].describe()
Out[105]:      Adj Close  Volume
count  8622.000000  8.622000e+03
mean    24.921952  6.030722e+07
std     31.838096  3.877805e+07
min      0.057762  2.304000e+06
25%     2.247503  3.651632e+07
50%    18.454313  5.350380e+07
75%    25.699224  7.397560e+07
max    187.663330  1.031789e+09

```

Adj Close означает скорректированную цену при закрытии и корректирует цену акций с учетом корпоративных действий, таких как дробление акций. Volume — это количество акций, которыми торговали. Я обобщил различные методы исследования DataFrame, которые мы рассматривали в этой главе, в табл. 5.8.

Таблица 5.8. Методы и атрибуты исследования DataFrame

DataFrame (df) метод/атрибут	Описание
df.info()	Указывает количество точек данных, тип индекса, тип и использование памяти
df.describe()	Предоставляет основные статистические данные, включая подсчет, среднее значение, среднее значение, мин, макс и процентиля
df.head(n=5)	Возвращает первые <i>n</i> строк фрейма DataFrame
df.tail(n=5)	Возвращает последние <i>n</i> строк фрейма DataFrame
df.dtypes	Возвращает dtype каждого столбца

Функция `read_csv` также принимает URL вместо локального CSV-файла. Так можно прочитать CSV-файл непосредственно из партнерского репозитория:

```

In [106]: # Разрыв строки в URL - чтобы он поместился на странице
url = ("https://raw.githubusercontent.com/fzumstein/"
      "python-for-excel/1st-edition/csv/MSFT.csv")
msft = pd.read_csv(url)

```

```
In [107]: msft.loc[:, ["Date", "Adj Close", "Volume"]].head(2)
Out[107]:
```

	Date	Adj Close	Volume
0	1986-03-13	0.062205	1031788800
1	1986-03-14	0.064427	308160000

Мы продолжим работу с этим набором данных и функцией `read_csv` в следующей главе о временных рядах, где мы превратим столбец `Date` в `DatetimeIndex`.

Заключение

Эта глава была наполнена новыми концепциями и инструментами для анализа наборов данных в pandas. Мы узнали, как загружать файлы CSV, как работать с отсутствующими или дублирующимися данными, и как использовать описательную статистику. Мы также увидели, как легко превратить ваши `DataFrames` в интерактивные графики. Хотя на то, чтобы все осознать, может потребоваться некоторое время, вы, вероятно, не сразу поймете, какую огромную силу вы получаете, добавляя pandas в свой инструментарий. Попутно мы сравнили pandas с следующими функциями Excel:

- ◆ Функциональность автофильтра (см. разд. «Выбор с помощью булевой индексации» на стр. 114).
- ◆ Формула `VLOOKUP` (см. разд. «Объединение и слияние» на стр. 129).
- ◆ Сводная таблица (см. разд. «Pivoting и Melting» на стр. 134).
- ◆ Power Query (см. разд. «Импорт и экспорт `DataFrames`» на стр. 140, «Манипулирование данными» на стр. 111 и «Объединение `DataFrames`» на стр. 127).

Следующая глава посвящена анализу временных рядов — функционалу, который привел к широкому внедрению pandas в финансовой индустрии. Давайте посмотрим, почему эта часть pandas имеет такое преимущество перед Excel!

Анализ временных рядов с помощью pandas

Временной ряд — это последовательность точек данных вдоль оси времени. Он выполняет центральную роль в различных сценариях: трейдеры для расчета меры риска используют исторические цены акций; прогноз погоды основан на временных рядах, созданных датчиками, измеряющими температуру, влажность и атмосферное давление. А отдел цифрового маркетинга полагается на временные ряды, создаваемые веб-страницами, например источник и количество просмотров страниц в час, и использует их, чтобы делать выводы о своих маркетинговых кампаниях.

Анализ временных рядов — один из главных мотивов, побудивших специалистов по анализу данных и аналитиков начать искать более подходящую замену Excel. В следующих пунктах кратко изложены несколько причин, из-за которых было принято это решение:

Большие массивы данных

Временные ряды могут быстро превысить ограничение Excel примерно в один миллион строк на лист. Например, если вы работаете с ценами на акции, которые формируются на протяжении этого дня, на уровне тиковых данных¹, вы часто имеете дело с сотнями тысяч записей на акцию в день!

Дата и время

Как мы видели в *главе 3*, в Excel имеются различные ограничения, когда приходится работать с датой и временем — основой временных рядов. Среди них — отсутствие поддержки часовых поясов и формат чисел, ограниченный миллисекундами. pandas поддерживает часовые пояса и использует тип данных NumPy `datetime64[ns]`, который обеспечивает точность до нескольких наносекунд.

Недостаточная функциональность

В Excel отсутствуют даже базовые инструменты для полноценной работы с данными временных рядов. Например, преобразование дневного временного ряда в месячный непростая задача, несмотря на то, что это очень распространенная операция.

¹ Тиковые данные — это данные, которые отображают все изменения цены. Например, изменение цены 48 раз в течение 1 минуты. — *Примеч. пер.*

DataFrames позволяет работать с различными индексами, основанными на времени: индекс `DatetimeIndex`, содержащий временные метки, является наиболее распространенным. Другие типы индексов, например `PeriodIndex`, основаны на временных интервалах, таких, например, как час или месяц. В этой главе мы рассмотрим только индекс `DatetimeIndex`, о котором я сейчас и расскажу более подробно.

DatetimeIndex

В этом разделе мы узнаем, как построить `DatetimeIndex`, как отфильтровать такой индекс по определенному временному диапазону и как работать с часовыми поясами.

Создание `DatetimeIndex`

Для создания `DatetimeIndex` pandas предлагает функцию `date_range`. Он принимает дату начала, частоту и, далее на выбор: или количество периодов, или дату окончания:

```
In [1]: # Начнем с импорта пакетов, которые мы используем в этой главе,
        # и установим серверную часть для черчения в Plotly
        import pandas as pd
        import numpy as np
        pd.options.plotting.backend = "plotly"

In [2]: # Так будет создан индекс DatetimeIndex, основанный на начальной
        # временной метке, количества периодов и частоту
        # ("D" = ежедневно).
        daily_index = pd.date_range("2020-02-28", periods=4, freq="D")
        daily_index

Out[2]: DatetimeIndex(['2020-02-28', '2020-02-29', '2020-03-01', '2020-03-02'],
                        dtype='datetime64[ns]', freq='D')

In [3]: # Создание индекса DatetimeIndex на основе временной метки
        # начала/окончания.
        # Частота установлена на "еженедельно по воскресеньям" ("W-SUN").
        weekly_index = pd.date_range("2020-01-01", "2020-01-31", freq="W-SUN")
        weekly_index

Out[3]: DatetimeIndex(['2020-01-05', '2020-01-12', '2020-01-19', '2020-01-26'],
                        dtype='datetime64[ns]', freq='W-SUN')

In [4]: # Постройте DataFrame, основываясь на weekly_index.
        # Это может быть количество посетителей музея,
        # который открывается только по воскресеньям.
        pd.DataFrame(data=[21, 15, 33, 34],
                      columns=["visitors"], index=weekly_index)

Out[4]:
```

	visitors
2020-01-05	21
2020-01-12	15
2020-01-19	33
2020-01-26	34

Теперь я предлагаю вернуться к временному ряду акций Microsoft из предыдущей главы. Если внимательно посмотреть на типы данных столбцов, то можно заметить, что столбец `Date` имеет тип `object`. Это означает, что `pandas` интерпретирует временные метки как строки:

```
In [5]: msft = pd.read_csv("csv/MSFT.csv")
In [6]: msft.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8622 entries, 0 to 8621
Data columns (total 7 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   Date        8622 non-null   object
1   Open        8622 non-null   float64
2   High        8622 non-null   float64
3   Low         8622 non-null   float64
4   Close       8622 non-null   float64
5   Adj Close   8622 non-null   float64
6   Volume      8622 non-null   int64
dtypes: float64(5), int64(1), object(1)
memory usage: 471.6+ KB
```

Есть два способа исправить неправильную интерпретацию данных и превратить их в тип данных `datetime`. Первый способ заключается в том, чтобы для этого столбца запустить функцию `to_datetime`. Если вы хотите внести изменения в столбец в источнике, не забудьте вернуть преобразованный столбец обратно в исходный `DataFrame`:

```
In [7]: msft.loc[:, "Date"] = pd.to_datetime(msft["Date"])
In [8]: msft.dtypes
Out[8]: Date          datetime64[ns]
       Open          float64
       High          float64
       Low           float64
       Close         float64
       Adj Close     float64
       Volume        int64
       dtype: object
```

Другая возможность — с помощью аргумента `parse_dates` указать `read_csv` столбцы, содержащие временные метки. `parse_dates` принимает список имен столбцов или индексов. Кроме того, почти всегда требуется преобразовать временные метки в индекс `DataFrame`, поскольку, как мы увидим через некоторое время, такое преобразование позволит легко фильтровать данные. Чтобы лишний раз не вызывать `set_index`, через аргумент `index_col` укажите столбец, который будет использован в качестве индекса, представив его в виде имени столбца или индекса:

```
In [9]: msft = pd.read_csv("csv/MSFT.csv",
                           index_col="Date", parse_dates=["Date"])
```

```
In [10]: msft.info()
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 8622 entries, 1986-03-13 to 2020-05-27
Data columns (total 6 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Open        8622 non-null   float64
1   High        8622 non-null   float64
2   Low         8622 non-null   float64
3   Close       8622 non-null   float64
4   Adj Close   8622 non-null   float64
5   Volume      8622 non-null   int64
dtypes: float64(5), int64(1)
memory usage: 471.5 KB
```

По сведениям `info` теперь вы имеете дело с `DataFrame`, содержащим `DatetimeIndex`. Если потребуется выбрать другой тип данных (допустим, вы хотите, чтобы `Volume` стал `float` вместо `int`), у вас снова имеется два варианта: или предоставить `dtype={"Volume": float}` в качестве аргумента функции `read_csv`, или применить метод `astype`:

```
In [11]: msft.loc[:, "Volume"] = msft["Volume"].astype("float")
         msft["Volume"].dtype
Out[11]: dtype('float64')
```

При работе с временными рядами перед началом анализа желательно проверить, что индекс отсортирован правильно:

```
In [12]: msft = msft.sort_index()
```

И наконец, если вам нужно получить доступ только к части `DatetimeIndex`, например к части даты без времени, обратитесь к атрибуту `date` следующим образом:

```
In [13]: msft.index.date
Out[13]: array([datetime.date(1986, 3, 13), datetime.date(1986, 3, 14),
               datetime.date(1986, 3, 17), ..., datetime.date(2020, 5, 22),
               datetime.date(2020, 5, 26), datetime.date(2020, 5, 27)],
              dtype=object)
```

Вместо `date` можно также использовать элементы даты, такие как `year`, `month`, `day` и т. д. Чтобы получить доступ к таким же функциям для обычного столбца с типом данных `datetime`, необходимо использовать атрибут `dt`, например `df["column_name"].dt.date`. Имея отсортированный индекс `DatetimeIndex`, давайте посмотрим, как можно отфильтровать `DataFrame` по определенным периодам времени!

Фильтрация `DatetimeIndex`

Если ваш `DataFrame` имеет индекс `DatetimeIndex`, есть простой способ выбрать строки из определенного периода времени, используя `loc` со строкой в формате

YYYY-MM-DD HH:MM:SS. pandas превратит эту строку в срез, чтобы она захватывала весь период. Например, чтобы выбрать все строки с 2019 года, укажите год в виде строки, а не числа:

```
In [14]: msft.loc["2019", "Adj Close"]
Out[14]: Date
2019-01-02    99.099190
2019-01-03    95.453529
2019-01-04    99.893005
2019-01-07   100.020401
2019-01-08   100.745613
...
2019-12-24   156.515396
2019-12-26   157.798309
2019-12-27   158.086731
2019-12-30   156.724243
2019-12-31   156.833633
Name: Adj Close, Length: 252, dtype: float64
```

Давайте сделаем еще один шаг вперед и построим график данных с июня 2019 года по май 2020 года (рис. 6.1):

```
In [15]: msft.loc["2019-06":"2020-05", "Adj Close"].plot()
```

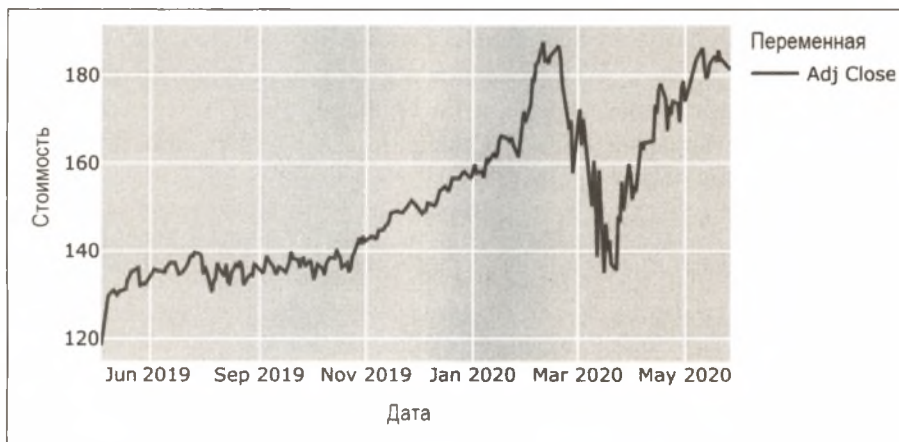


Рис. 6.1. Скорректированная цена закрытия для MSFT

Чтобы получить значение в виде подсказки, установите курсор на график Plotly и увеличьте масштаб, очертив с помощью мыши прямоугольник. Чтобы вернуться к прежнему виду, дважды щелкните мышью.

В следующем разделе мы, чтобы рассмотреть, как обращаться с часовыми поясами, используем скорректированную цену закрытия.

Работа с часовыми поясами

Компания Microsoft котируется на фондовой бирже Nasdaq. Nasdaq находится в Нью-Йорке, и рынки закрываются в 16:00. Чтобы добавить эту дополнительную информацию в индекс DataFrame, сначала добавьте час закрытия к дате через `DateOffset`, а затем прикрепите правильный часовой пояс к меткам времени через `tz_localize`. Поскольку час закрытия применим только к цене закрытия, давайте создадим новый DataFrame с данным параметром:

```
In [16]: # Добавьте информацию о времени к дате
         msft_close = msft.loc[:, ["Adj Close"]].copy()
         msft_close.index = msft_close.index + pd.DateOffset(hours=16)
         msft_close.head(2)
```

```
Out[16]:
```

Date	Adj Close
1986-03-13 16:00:00	0.062205
1986-03-14 16:00:00	0.064427

```
In [17]: # Сделать временные метки зависящими от временной зоны
         msft_close = msft_close.tz_localize("America/New_York")
         msft_close.head(2)
```

```
Out[17]:
```

Date	Adj Close
1986-03-13 16:00:00-05:00	0.062205
1986-03-14 16:00:00-05:00	0.064427

Если вы хотите преобразовать временные метки в часовой пояс UTC, используйте метод DataFrame `tz_convert`. UTC означает всемирное координированное время и является преемником среднего времени по Гринвичу (GMT). Обратите внимание, как меняются часы закрытия в UTC в зависимости от того, действует или нет в Нью-Йорке переход на летнее время (DST):

```
In [18]: msft_close = msft_close.tz_convert("UTC")
         msft_close.loc["2020-01-02", "Adj Close"] # 21:00 без учета DST
```

```
Out[18]: Date
         2020-01-02 21:00:00+00:00    159.737595
         Name: Adj Close, dtype: float64
```

```
In [19]: msft_close.loc["2020-05-01", "Adj Close"] # 20:00 по местному
         # времени
```

```
Out[19]: Date
         2020-05-01 20:00:00+00:00    174.085175
         Name: Adj Close, dtype: float64
```

Подготовка временных рядов, таким образом, позволит вам сравнить цены закрытия бирж в разных часовых поясах, даже если информация о времени отсутствует или указана в местном часовом поясе.

Теперь, когда вы знаете, что такое `DatetimeIndex`, давайте в следующем разделе опробуем несколько типичных манипуляций с временными рядами, рассчитав и сравнив показатели акций.

Общие манипуляции с временными рядами

В этом разделе я покажу вам, как выполнять общие операции анализа временных рядов, такие как вычисление доходности акций, построение графика доходности различных акций и визуализация корреляции их доходности в виде тепловой карты. Мы также рассмотрим, как изменять частоту временных рядов и как вычислять из статистики скользящую среднюю.

Смещение и процентные изменения

В финансовой сфере часто предполагается, что *логарифмическая доходность* акций имеет нормальное распределение. Под логарифмической доходностью я подразумеваю натуральный логарифм отношения текущей и предыдущей цены. Чтобы получить представление о распределении ежедневной доходности, построим гистограмму. Однако сначала нам нужно рассчитать логарифмическую доходность. В Excel это обычно делается с помощью формулы, которая содержит ячейки из двух строк, как показано на рис. 6.2.

	A	B	C
1	Date	Adj Close	
2	3/13/1986	0.062205	
3	3/14/1986	0.064427	=LN(B3/B2)
4	3/17/1986	0.065537	0.017082

Рис. 6.2. Расчет логарифмической доходности в Excel



Логарифмы в Excel и Python

Excel для обозначения натурального логарифма использует LN, а для логарифма с основанием 10 применяется обозначение LOG. Математический модуль Python и NumPy используют log для натурального логарифма и log10 для логарифма с основанием 10.

В pandas вместо того, чтобы формула обращалась к двум разным строкам, для сдвига значений вниз на одну строку используется метод `shift`. Это позволяет вам оперировать одной строкой, чтобы ваши вычисления могли использовать векторизацию. `shift` принимает целое положительное или отрицательное число, которое сдвигает временной ряд вниз или вверх на соответствующее количество рядов. Давайте сначала посмотрим, как работает `shift`:

```
In [20]: msft_close.head()
Out[20]:
```

Date	Adj Close
1986-03-13 21:00:00+00:00	0.062205


```

1986-03-14 21:00:00+00:00    0.064427
1986-03-17 21:00:00+00:00    0.065537
1986-03-18 21:00:00+00:00    0.063871
1986-03-19 21:00:00+00:00    0.062760

```

```
In [21]: msft_close.shift(1).head()
```

```

Out[21]:                               Adj Close
Date
1986-03-13 21:00:00+00:00                NaN
1986-03-14 21:00:00+00:00    0.062205
1986-03-17 21:00:00+00:00    0.064427
1986-03-18 21:00:00+00:00    0.065537
1986-03-19 21:00:00+00:00    0.063871

```

Теперь вы можете написать единую векторную формулу, которую легко прочесть и понять. Чтобы получить натуральный логарифм, используйте универсальную функцию NumPy `log`, которая применяется к каждому элементу. Затем мы сможем построить гистограмму (рис. 6.3):

```

In [22]: returns = np.log(msft_close / msft_close.shift(1))
         returns = returns.rename(columns={"Adj Close": "returns"})
         returns.head()

```

```

Out[22]:                               returns
Date
1986-03-13 21:00:00+00:00                NaN
1986-03-14 21:00:00+00:00    0.035097
1986-03-17 21:00:00+00:00    0.017082
1986-03-18 21:00:00+00:00   -0.025749
1986-03-19 21:00:00+00:00   -0.017547

```

```

In [23]: # Plot a histogram with the daily log returns
         returns.plot.hist()

```

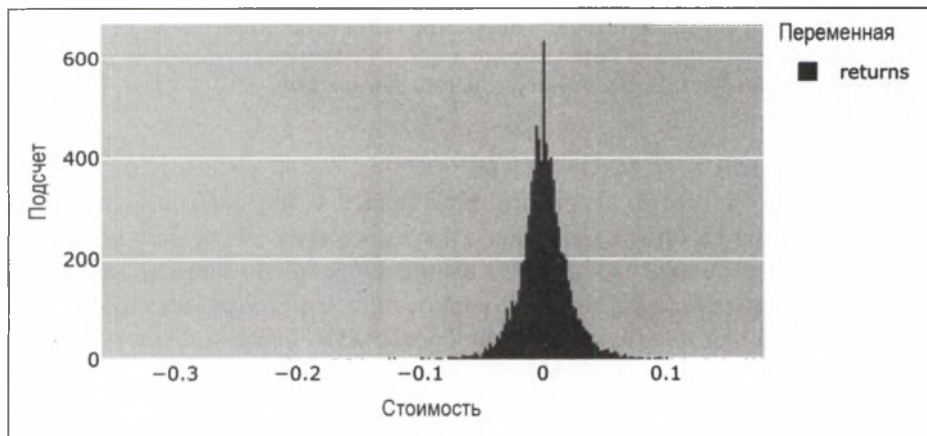


Рис. 6.3. Гистограмма

Чтобы получить *простую доходность*, используйте встроенный в pandas метод `pct_change`. По умолчанию он рассчитывает процентное изменение по сравнению с предыдущим рядом, что также является определением простой доходности:

```
In [24]: simple_rets = msft_close.pct_change()
         simple_rets = simple_rets.rename(columns={"Adj Close": "simple_rets"})
         simple_rets.head()
```

```
Out[24]:
```

Date	simple_rets
1986-03-13 21:00:00+00:00	NaN
1986-03-14 21:00:00+00:00	0.035721
1986-03-17 21:00:00+00:00	0.017229
1986-03-18 21:00:00+00:00	-0.025421
1986-03-19 21:00:00+00:00	-0.017394

До сих пор мы рассматривали только акции Microsoft. В следующем разделе будет загружено больше временных рядов, чтобы мы могли ознакомиться с другими методами `DataFrame`, для работы которых требуется несколько временных рядов.

Пересчет и корреляция

Ситуация становится интереснее, когда мы работаем более чем с одним временным рядом. Давайте загрузим на момент закрытия несколько дополнительных скорректированных цен для Amazon (AMZN), Google (GOOGL) и Apple (AAPL), также загруженных из Yahoo! Finance:

```
In [25]: parts = [] # Список для сбора отдельных DataFrames
         for ticker in ["AAPL", "AMZN", "GOOGL", "MSFT"]:
             # "usecols" позволяет нам считывать только дату и Adj Close
             adj_close = pd.read_csv(f"csv/{ticker}.csv",
                                     index_col="Date", parse_dates=["Date"],
                                     usecols=["Date", "Adj Close"])
             # Переименуйте столбец, присвоив имя биржевого символа
             adj_close = adj_close.rename(columns={"Adj Close": ticker})
             # Добавьте DataFrame акций к списку деталей
             parts.append(adj_close)
```

```
In [26]: # Объедините 4 DataFrame в один DataFrame
         adj_close = pd.concat(parts, axis=1)
         adj_close
```

```
Out[26]:
```

	AAPL	AMZN	GOOGL	MSFT
Date				
1980-12-12	0.405683	NaN	NaN	NaN
1980-12-15	0.384517	NaN	NaN	NaN
1980-12-16	0.356296	NaN	NaN	NaN
1980-12-17	0.365115	NaN	NaN	NaN
1980-12-18	0.375698	NaN	NaN	NaN
...	...	...	...	...

```

2020-05-22  318.890015  2436.879883  1413.239990  183.509995
2020-05-26  316.730011  2421.860107  1421.369995  181.570007
2020-05-27  318.109985  2410.389893  1420.280029  181.809998
2020-05-28  318.250000  2401.100098  1418.239990      NaN
2020-05-29  317.940002  2442.370117  1433.520020      NaN

```

```
[9950 rows x 4 columns]
```

Вы заметили возможности `concat`? `pandas` автоматически совмещает отдельные временные ряды по датам. Вот почему вы получаете значения `NaN` для тех акций, которые не уходят в прошлое так далеко, как `Apple`. И поскольку `MSFT` имеет в самых поздних датах значения `NaN`, вы могли догадаться, что я загрузил `MSFT.csv` на два дня раньше других. Совмещение временных рядов по дате — это обычная операция, которую очень обременительно выполнять в `Excel`, и при выполнении этой операции может возникнуть большая вероятность ошибок. Отбрасывая все строки, содержащие пропущенные значения, можно убедиться, что все акции имеют одинаковое количество точек данных:

```

In [27]: adj_close = adj_close.dropna()
         adj_close.info()
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 3970 entries, 2004-08-19 to 2020-05-27
Data columns (total 4 columns):
#   Column  Non-Null Count  Dtype
---  ---
0   AAPL    3970 non-null     float64
1   AMZN    3970 non-null     float64
2   GOOGL   3970 non-null     float64
3   MSFT    3970 non-null     float64
dtypes: float64(4)
memory usage: 155.1 KB

```

Давайте теперь пересчитаем цены так, чтобы все временные ряды начинались со 100. Это позволяет нам сопоставить на графике их относительную производительность (рис. 6.4). Чтобы пересчитать временной ряд, разделите каждое значение на его начальное значение и умножьте на 100 (новую базу). Если бы вы делали это в `Excel`, то, как правило, написали бы формулу с комбинацией абсолютных и относительных ссылок на ячейки, затем скопировали бы формулу для каждой строки и каждого временного ряда. В `pandas` благодаря векторизации и трансляции вы имеете дело с единственной формулой:

```

In [28]: # Используйте образец с июня 2019 года по май 2020 года
         adj_close_sample = adj_close.loc["2019-06":"2020-05", :]
         rebased_prices = adj_close_sample / adj_close_sample.iloc[0, :] * 100
         rebased_prices.head(2)

Out[28]:
           AAPL      AMZN      GOOGL      MSFT
Date
2019-06-03  100.000000  100.000000  100.000000  100.000000
2019-06-04  103.658406  102.178197  101.51626  102.770372

In [29]: rebased_prices.plot()

```

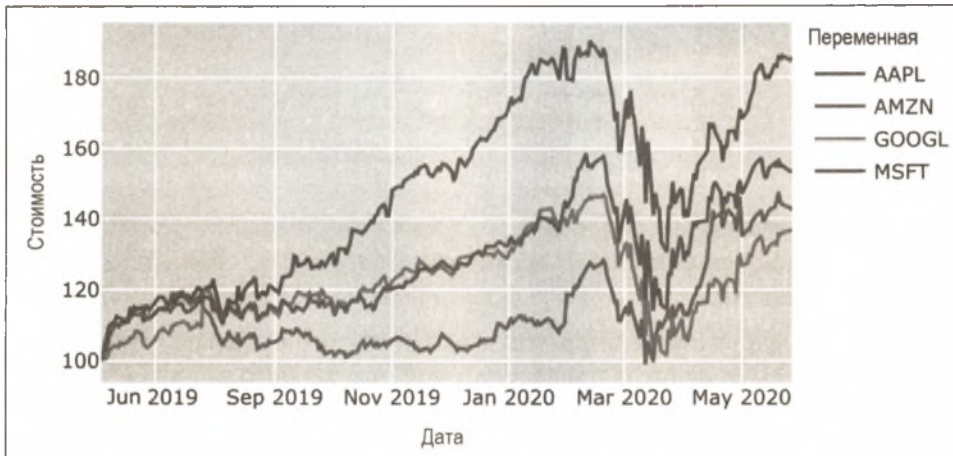


Рис. 6.4. Пересмотренный временной ряд

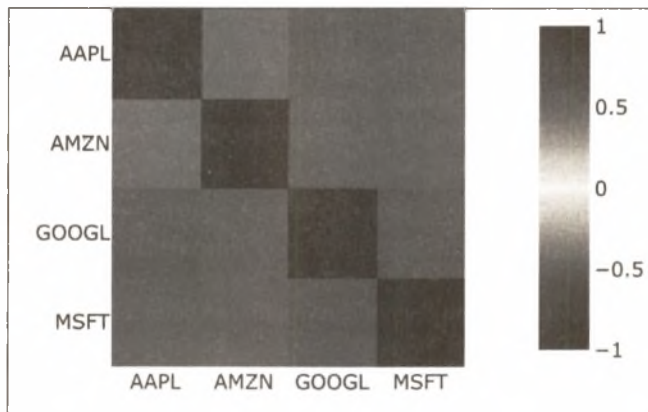


Рис. 6.5. Тепловая карта корреляции

Чтобы узнать, насколько независимы доходы различных акций, посмотрите на их корреляции с помощью метода `corr`. К сожалению, pandas не содержит встроенный тип графика для визуализации корреляционной матрицы в виде тепловой карты, поэтому нам необходимо использовать Plotly напрямую через интерфейс `plotly.express` (рис. 6.5):

```
In [30]: # Корреляция ежедневных логарифмических доходностей
         returns = np.log(adj_close / adj_close.shift(1))
         returns.corr()
```

```
Out[30]:
```

	AAPL	AMZN	GOOGL	MSFT
AAPL	1.000000	0.424910	0.503497	0.486065
AMZN	0.424910	1.000000	0.486690	0.485725
GOOGL	0.503497	0.486690	1.000000	0.525645
MSFT	0.486065	0.485725	0.525645	1.000000

```
In [31]: import plotly.express as px
In [32]: fig = px.imshow(returns.corr(),
                        x=adj_close.columns,
                        y=adj_close.columns,
                        color_continuous_scale=list(
                            reversed(px.colors.sequential.RdBu)),
                        zmin=-1, zmax=1)

fig.show()
```

Если вы хотите подробно разобраться, как работает `imshow`, обратитесь к документации Plotly Express API².

На данном этапе мы уже узнали довольно много нового о временных рядах, включая способы их объединения и очистки, а также расчет доходности и корреляции. Но что, если вы решите, что ежедневные доходы не являются подходящей базой для вашего анализа, и вам нужны ежемесячные доходы? Как изменить частоту данных временного ряда — тема следующего раздела.

Повторная выборка

Типичной задачей при работе с временными рядами является *повышающая и понижающая дискретизация*. Повышающая дискретизация означает, что временной ряд преобразуется в ряд с более высокой частотой, а понижающая дискретизация означает, что он преобразуется в ряд с более низкой частотой. В финансовых отчетах вы часто приводите, например, месячные или квартальные показатели. Чтобы превратить ежедневный временной ряд в месячный, используйте метод `resample`, который принимает строку частоты, например `M`, для *конца календарного месяца* или `BM` для *конца бизнес-месяца*. Вы можете найти список всех частотных строк в документации pandas³. Так же, как и в `groupby`, создается цепочка методов, определяющих способ выборки. Я использую `last`, чтобы всегда работать с последним обзором за этот месяц:

```
In [33]: end_of_month = adj_close.resample("M").last()
         end_of_month.head()

Out[33]:
```

	AAPL	AMZN	GOOGL	MSFT
Date				
2004-08-31	2.132708	38.139999	51.236237	17.673630
2004-09-30	2.396127	40.860001	64.864868	17.900215
2004-10-31	3.240182	34.130001	95.415413	18.107374
2004-11-30	4.146072	39.680000	91.081078	19.344421
2004-12-31	3.982207	44.290001	96.491493	19.279480

Вместо `last` вы можете выбрать любой другой метод, работающий с `groupby`, например `sum` или `mean`. Существует также функция `ohlc`, которая возвращает значе-

² <https://oreil.ly/O86Li>. — Примеч. пер.

³ <https://oreil.ly/zStpt>. — Примеч. пер.

ния открытия, максимума, минимума и значение на закрытие за этот период. Это может послужить источником для создания типовых интервальных графиков, которые часто используются совместно с ценами на акции. Если временной ряд на конец месяца — это все, что у вас есть, и вам нужно получить из него недельный временной ряд, вам придется увеличить выборку временного ряда. Используя `asfreq`, вы даете указание pandas не выполнять никаких преобразований, и, как следствие, увидите, что для большинства значений отображается `NaN`. Если вы хотите перенаправить *последнее известное значение*, используйте метод `ffill`:

```
In [34]: end_of_month.resample("D").asfreq().head() # Без изменений
Out[34]:
```

	AAPL	AMZN	GOOGL	MSFT
Date				
2004-08-31	2.132708	38.139999	51.236237	17.67363
2004-09-01	NaN	NaN	NaN	NaN
2004-09-02	NaN	NaN	NaN	NaN
2004-09-03	NaN	NaN	NaN	NaN
2004-09-04	NaN	NaN	NaN	NaN

```
In [35]: end_of_month.resample("W-FRI").ffill().head() # Увеличение
# выборки данных
Out[35]:
```

	AAPL	AMZN	GOOGL	MSFT
Date				
2004-09-03	2.132708	38.139999	51.236237	17.673630
2004-09-10	2.132708	38.139999	51.236237	17.673630
2004-09-17	2.132708	38.139999	51.236237	17.673630
2004-09-24	2.132708	38.139999	51.236237	17.673630
2004-10-01	2.396127	40.860001	64.864868	17.900215

Уменьшение выборки данных — один из способов сглаживания временного ряда. Другим способом, как мы увидим далее, является подсчет статистики в скользящем окне.

Скользящее окно

При расчете статистики временных рядов часто требуется скользящая статистика, например *скользящее среднее*. Скользящее среднее рассматривает подмножество временного ряда (допустим, 25 дней) и берет среднее значение из этого подмножества, а затем перемещает окно вперед на один день. В результате получится новый временной ряд, более гладкий и менее подверженный выбросам. Если вы занимаетесь алгоритмической торговлей, то, возможно, следите за пересечением скользящей средней с ценой акции и воспринимаете это (или какую-то вариацию этого) как торговый сигнал. DataFrames содержат метод `rolling`, который принимает в качестве аргумента количество наблюдений. Затем вы соединяете его со статистическим методом, который хотите использовать, — в случае со скользящей средней это среднее значение. Взглянув на рис. 6.6, вы с легкостью сможете сравнить исходный временной ряд со сглаженным скользящим средним:

```
In [36]: # Построение скользящего среднего для MSFT с данными за 2019 год
msft19 = msft.loc["2019", ["Adj Close"]].copy()
```



```
# Добавьте 25-дневное скользящее среднее в качестве нового
# столбца в DataFrame
msft19.loc[:, "25day average"] = msft19["Adj Close"].rolling(25).mean()
msft19.plot()
```

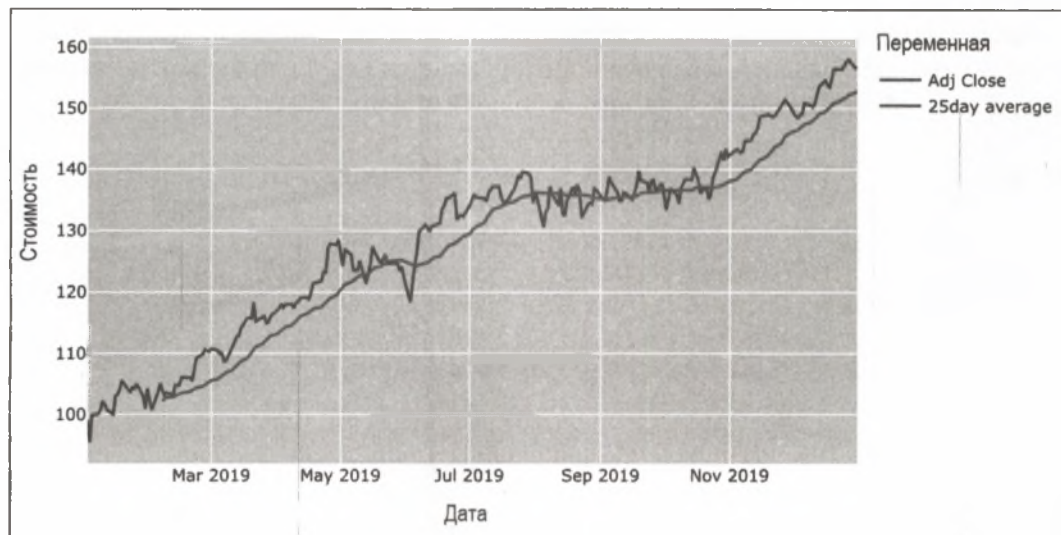


Рис. 6.6. График скользящего среднего

Вместо среднего значения можно использовать множество других статистических показателей, включая `count`, `sum`, `median`, `min`, `max`, `std` (стандартное отклонение) или `var` (дисперсия).

На данном этапе мы познакомились с наиболее важными функциональными возможностями `pandas`. Однако не менее важно понимать, какие у `pandas` есть ограничения, даже если до их достижения пока еще очень далеко.

Ограничения при работе с pandas

При увеличении ваших `DataFrames` полезно знать верхний предел ограничения `DataFrame`. В отличие от Excel, где у вас есть жесткое ограничение — примерно миллион строк и 12 000 столбцов на лист, у `pandas` есть только мягкое ограничение: все данные должны поместиться в доступную память вашей машины. Если это не так, имеется несколько простых решений: загружайте только те столбцы из набора данных, которые вам нужны, или удаляйте промежуточные результаты, чтобы освободить память. Если это не поможет, есть несколько проектов, с которыми пользователи `pandas` могут быть знакомы, — проекты, имеющими дело с большими объемами данных. Один из таких проектов, `Dask`, работает поверх `NumPy` и `pandas` и позволяет оперировать с большими объемами данных, разделяя их на несколько `pandas DataFrames` и распределяя нагрузку на несколько ядер процессора или не-

сколько компьютеров. Проекты для работы с большими объемами данных, которые работают с различными типами DataFrame, — это Modin⁴, Koalas⁵, Vaex⁶, PySpark⁷, cuDF⁸, Ibis⁹ и PyArrow¹⁰. В следующей главе мы вкратце рассмотрим Modin, но это не основная тема, которую мы будем изучать дальше в этой книге.

Заключение

Анализ временных рядов — это та область, где, по моему мнению, Excel отстает больше всего. Поэтому, после прочтения этой главы вы, вероятно, поймете, почему pandas имеет такой большой успех в финансовой сфере — отрасли, которая в значительной степени опирается на временные ряды. Мы видели, как легко работать с временными зонами, повторно дискретизировать временные ряды или создавать корреляционные матрицы — функции, которые либо не поддерживаются в Excel, либо требуют громоздких обходных путей.

Однако навыки работы с pandas не означают, что вам придется избавиться от Excel, поскольку эти два мира могут очень хорошо взаимодействовать друг с другом. pandas DataFrames — отличный способ передачи данных из одного мира в другой. Об этих возможностях я расскажу в следующей части, посвященной чтению и записи файлов Excel способами, позволяющими полностью обойти приложение Excel. Это очень полезные возможности, потому что вы с помощью Python можете работать с файлами Excel на любой операционной системе, поддерживающей Python, включая Linux. В начале этого пути, в следующей главе, вы узнаете, как можно использовать pandas для автоматизации утомительных ручных процессов, таких как объединение файлов Excel в сводные отчеты.

⁴ <https://oreil.ly/Wd8gi>. — Примеч. пер.

⁵ <https://oreil.ly/V13Be>. — Примеч. пер.

⁶ <https://vaex.io/>. — Примеч. пер.

⁷ <https://oreil.ly/E7kmX>. — Примеч. пер.

⁸ <https://oreil.ly/zaeWz>. — Примеч. пер.

⁹ <https://oreil.ly/Gw4wn>. — Примеч. пер.

¹⁰ <https://oreil.ly/DQQGD>. — Примеч. пер.

ЧТЕНИЕ И ЗАПИСЬ ФАЙЛОВ EXCEL БЕЗ EXCEL

Манипулирование файлами Excel с помощью pandas

После шести глав интенсивного знакомства с инструментами Python и pandas я предоставляю передышку и начну эту главу с практического примера, который позволит вам применить полученные навыки на практике: с помощью всего десяти строк кода pandas вы объедините десятки файлов Excel в отчет Excel, который можно будет отправлять вашим менеджерам. Далее я более подробно познакомлю вас с инструментами, которые pandas предлагает для работы с файлами Excel: функция `read_excel` и класс `ExcelFile` для чтения, метод `to_excel` и класс `ExcelWriter` для записи файлов Excel. pandas не использует приложение Excel для чтения и записи файлов Excel, и это означает, что все примеры кода в этой главе работают с любым компьютером, на котором работает Python, включая Linux.

Тематическое исследование: отчетность в Excel

Это тематическое исследование навеяно несколькими реальными проектами по отчетности, в которых я в течение последних нескольких лет принимал участие. Несмотря на то, что проекты осуществлялись в совершенно разных отраслях — телекоммуникационной, цифрового маркетинга и финансовой — они все равно удивительно похожи: отправной точкой был каталог с файлами Excel, которые необходимо обработать в отчет Excel — часто на ежемесячной, еженедельной или ежедневной основе. В сопутствующем репозитории, в каталоге `sales_data`, вы найдете файлы Excel с фиктивными транзакциями продаж для оператора связи, продающего различные тарифные планы (Bronze, Silver, Gold) в нескольких торговых центрах по всей территории США. Для каждого месяца имеется два файла, один во вложенной папке `new` для новых контрактов и один в предназначенной для действующих клиентов. Поскольку отчеты поступают из разных систем, они представлены в разных форматах: отчеты для новых клиентов поставляются в виде файлов `xlsx`, в то время как отчеты существующих клиентов приходят в старом формате `xls`. Каждый из файлов содержит до 10 000 транзакций, и наша цель — создать отчет Excel, который показывает общий объем продаж по магазинам и по месяцам. Для начала давайте проанализируем файл `January.xlsx` из новой подпапки, рис. 7.1.

	A	B	C	D	E	F	G
1	transaction_id	store	status	transaction_date	plan	contract_type	amount
2	abfbdd6d	Chicago	ACTIVE	1/1/2019	Silver	NEW	14.25
3	136a9997	San Francisco	ACTIVE	1/1/2019	Gold	NEW	19.35
4	c6688f32	San Francisco	ACTIVE	1/1/2019	Bronze	NEW	12.2
5	6ef349c1	Chicago	ACTIVE	1/1/2019	Gold	NEW	19.35
6	22066f29	San Francisco	ACTIVE	1/1/2019	Silver	NEW	14.25

Рис. 7.1. Первые несколько строк файла January.xlsx

Файлы Excel в существующей вложенной папке выглядят практически так же, за исключением того, что в них отсутствует колонка `status` и они хранятся в устаревшем формате `xls`.

```
In [1]: import pandas as pd
```

```
In [2]: df = pd.read_excel("sales_data/new/January.xlsx")
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9493 entries, 0 to 9492
Data columns (total 7 columns):
#   Column                Non-Null Count  Dtype
---  -
0   transaction_id         9493 non-null  object
1   store                  9493 non-null  object
2   status                 9493 non-null  object
3   transaction_date       9493 non-null  datetime64[ns]
4   plan                   9493 non-null  object
5   contract_type          9493 non-null  object
6   amount                 9493 non-null  float64
dtypes: datetime64[ns](1), float64(1), object(5)
memory usage: 519.3+ KB
```



Функция `read_excel` в Python 3.9

Это то же предупреждение, что и в *главе 5*: если вы запускаете `pd.read_excel` с Python 3.9 или выше, убедитесь, что используете как минимум `pandas 1.2`, иначе вы получите ошибку при чтении файлов `xlsx`.

Как видите, `pandas` правильно распознал типы данных всех столбцов, включая формат даты `transaction_date`. Это позволяет нам работать с данными без дополнительной их подготовки. Поскольку этот пример заведомо упрощен, мы можем перейти к созданию короткого сценария под названием `sales_report_pandas.py`, как показано в примере 7.1. Этот сценарий прочитает все файлы Excel из обоих каталогов, объединит данные и запишет в новый файл Excel сводную таблицу. Используйте VS Code, чтобы написать сценарий самостоятельно, или откройте его из партнерского репозитория. Для получения более подробной информации о том, как создавать или открывать файлы в VS Code, еще раз просмотрите *главу 2*. Если вы

создайте сценарий самостоятельно, убедитесь, что он находится в одном каталоге с папкой `sales_data` — это позволит вам запустить сценарий без необходимости корректировать пути к файлам.

Пример 7.1. `sales_report_pandas.py`

```
from pathlib import Path

import pandas as pd

# Каталог данного файла
this_dir = Path(__file__).resolve().parent 1

# Считывание всех файлов Excel из всех вложенных папок sales_data
parts = []

for path in (this_dir / "sales_data").rglob("*.xls*"): 2
    print(f'Reading {path.name}')
    part = pd.read_excel(path, index_col="transaction_id")
    parts.append(part)

# Объедините DataFrame из каждого файла в один DataFrame
# pandas позаботится о правильном совмещении столбцов
df = pd.concat(parts)

# Выделите каждый магазин в отдельный столбец и просуммируйте все
# транзакции по датам
pivot = pd.pivot_table(df,
                        index="transaction_date", columns="store",
                        values="amount", aggfunc="sum")

# Повторите выборку до конца месяца и назначьте имя индекса
summary = pivot.resample("M").sum()
summary.index.name = "Month"

# Запись сводного отчета в файл Excel
summary.to_excel(this_dir / "sales_report_pandas.xlsx")
```

- 1 До этой главы для указания путей к файлам я использовал строки. Применяя класс `Path` из модуля `pathlib` стандартной библиотеки, вы получаете доступ к мощному набору инструментов: объекты `path` позволяют легко конструировать пути, объединяя отдельные части через прямые слэши (вертикальная черточка), как это сделано четырьмя строками ниже с `this_dir / "sales_data"`. Эти пути работают на разных платформах и позволяют использовать фильтры типа `rglob`, о чем будет рассказано в следующем пункте. `__file__` преобразуется в путь к файлу исходного кода, когда вы запускаете его — использование параметра `parent` обеспечит имя каталога этого

файла. Метод `resolve`, который мы используем перед вызовом `parent`, превращает путь в полный путь. Если бы вы запускали это из Jupyter, вам пришлось бы заменить эту строку на `this_dir = Path(".").resolve()` с точкой, обозначающей текущий каталог. В большинстве случаев функции и классы, которые принимают путь в виде строки, также принимают объект пути.

- 2 Самый простой способ рекурсивного чтения всех файлов Excel из определенного направления — это использовать метод `rglob` объекта `path`. `Glob` — это сокращение от *globbing*, что означает расширение имени пути с помощью подстановочных знаков. Подстановочный знак `?` обозначает ровно один символ, а `*` обозначает любое количество символов (включая ноль). `r` в `rglob` означает, что расширение имени пути с помощью подстановочных знаков будет осуществляться рекурсивно, то есть поиск подходящих файлов будет происходить во всех вложенных каталогах, соответственно, `glob` будет игнорировать вложенные каталоги. Использование `*.xls*` в качестве глобального выражения гарантирует, что старые и новые файлы Excel будут найдены, поскольку оно соответствует как `.xls`, так и `.xlsx`. Обычно рекомендуется слегка улучшить выражение следующим образом: `[!~$]*.xls*`. При этом игнорируются временные файлы Excel (их имя файлов начинается с `~$`). Для получения дополнительной информации о том, как использовать *globbing* в Python, смотрите документацию по Python¹.

	A	B	C	D	E	F	G
1	Month	Boston	Chicago	Las Vegas	New York	San Francisco	Washington DC
2	#####	21784.1	51187.7	23012.75	49872.85	58629.85	14057.6
3	#####	21454.9	52330.85	25493.1	46669.85	55218.65	15235.4
4	#####	20043	48897.25	23451.1	41572.25	52712.95	14177.05
5	#####	18791.05	47396.35	22710.15	41714.3	49324.65	13339.15
6	#####	18036.75	45117.05	21526.55	40610.4	47759.6	13147.1
7	#####	21556.25	49460.45	21985.05	47265.65	53462.4	14284.3
8	#####	19853	47993.8	23444.3	40408.3	50181.6	14161.5
9	#####	22332.9	50838.9	24927.65	45396.85	55336.35	16127.05
10	#####	19924.5	49096.25	24410.7	42830.6	49931.45	14994.4
11	#####	16550.95	42543.8	22827.5	34090.05	44311.65	12846.7
12	#####	21312.9	52011.6	24860.25	46959.85	55056.45	14057.6
13	#####	19722.6	49355.1	24535.75	42364.35	50933.45	14702.15

Рис. 7.2. `sales_report_pandas.xlsx` (представлено без регулировки ширины столбцов)

Запустите сценарий, например нажав кнопку **Run File** в правом верхнем углу VS Code. Выполнение сценария займет некоторое время, после чего рабочая книга Excel `sales_report_pandas.xlsx` появится в том же каталоге, что и сценарий. Содержание Листа1 должно выглядеть так, как показано на рис. 7.2. Это впечатляющий

¹ <https://oreil.ly/fY0qG>. — Примеч. пер.

результат для всего десяти строк кода, даже если вам придется изменить ширину первого столбца, чтобы увидеть даты!

Для таких простых случаев, как этот, pandas предлагает действительно простое решение по работе с файлами Excel. Однако мы можем сделать отчет намного качественнее — в конце концов, заголовков, небольшое форматирование (включая ширину столбцов и постоянное количество десятичных цифр) и диаграмма не помешают. Именно об этом мы и поговорим в следующей главе, напрямую используя библиотеки сценариев, которые pandas реализует, скрывая под оболочкой. Однако прежде чем перейти к этой теме, давайте более подробно рассмотрим, как читать и записывать файлы Excel с помощью pandas.

Чтение и запись файлов Excel с помощью pandas

В примере использовались `read_excel` и `to_excel`, для упрощения работы с аргументами по умолчанию. В этом разделе я покажу вам наиболее часто используемые аргументы и опции для чтения и записи файлов Excel с помощью pandas. Мы начнем с функции `read_excel` и класса `ExcelFile`, а затем рассмотрим метод `to_excel` и класс `ExcelWriter`. Попутно я также познакомлю вас с оператором `with` в Python.

Функция `read_excel` и класс `ExcelFile`

В исследовании использовались рабочие книги Excel, в которых данные удобно располагались в ячейке A1 первого листа. В действительности ваши файлы Excel, вероятно, не так хорошо организованы. В этом случае pandas предлагает параметры для точной настройки процесса чтения. Для следующих нескольких примеров мы будем использовать файл `stores.xlsx`, который вы найдете в папке `x1` партнерского репозитория. Первый лист показан на рис. 7.3.

	A	B	C	D	E	F
1						
2		Store	Employees	Manager	Since	Flagship
3		New York	10	Sarah	7/20/2018	FALSE
4		San Francisco	12	Neriah	11/2/2019	MISSING
5		Chicago	4	Katelin	1/31/2020	
6		Boston	5	Georgiana	4/1/2017	TRUE
7		Washington DC	3	Evan		FALSE
8		Las Vegas	11	Paul	1/6/2020	FALSE
9						

Рис. 7.3. Первый лист файла `stores.xlsx`

Используя параметры `sheet_name`, `skiprows` и `usecols`, мы сообщим pandas о диапазоне ячеек, который мы хотим прочитать. Как обычно, неплохо взглянуть на типы данных возвращаемого `DataFrame`, выполнив метод `info`:

```
In [3]: df = pd.read_excel("xl/stores.xlsx",
                           sheet_name="2019", skiprows=1, usecols="B:F")
```

df

```
Out[3]:
```

	Store	Employees	Manager	Since	Flagship
0	New York	10	Sarah	2018-07-20	False
1	San Francisco	12	Neriah	2019-11-02	MISSING
2	Chicago	4	Katelin	2020-01-31	NaN
3	Boston	5	Georgiana	2017-04-01	True
4	Washington DC	3	Evan	NaT	False
5	Las Vegas	11	Paul	2020-01-06	False

```
In [4]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 6 entries, 0 to 5
```

```
Data columns (total 5 columns):
```

#	Column	Non-Null Count	Dtype
0	Store	6 non-null	object
1	Employees	6 non-null	int64
2	Manager	6 non-null	object
3	Since	5 non-null	datetime64[ns]
4	Flagship	5 non-null	object

```
dtypes: datetime64[ns](1), int64(1), object(3)
```

```
memory usage: 368.0+ bytes
```

Все выглядит хорошо, за исключением колонки `Flagship` — ее тип данных должен быть `bool`, а не `object`. Чтобы исправить это, мы можем предоставить функцию-конвертер, которая работает с выключенными ячейками в этом столбце (вместо написания функции `fix_missing` мы могли бы также использовать лямбда-выражение):

```
In [5]: def fix_missing(x):
```

```
    return False if x in ["", "MISSING"] else x
```

```
In [6]: df = pd.read_excel("xl/stores.xlsx",
                           sheet_name="2019", skiprows=1, usecols="B:F",
                           converters={"Flagship": fix_missing})
```

df

```
Out[6]:
```

	Store	Employees	Manager	Since	Flagship
0	New York	10	Sarah	2018-07-20	False
1	San Francisco	12	Neriah	2019-11-02	False
2	Chicago	4	Katelin	2020-01-31	False
3	Boston	5	Georgiana	2017-04-01	True
4	Washington DC	3	Evan	NaT	False
5	Las Vegas	11	Paul	2020-01-06	False

```
In [7]: # Колонка Flagship теперь имеет тип Dtype "bool".
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6 entries, 0 to 5
Data columns (total 5 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Store        6 non-null     object
1   Employees    6 non-null     int64
2   Manager      6 non-null     object
3   Since        5 non-null     datetime64[ns]
4   Flagship     6 non-null     bool
dtypes: bool(1), datetime64[ns](1), int64(1), object(2)
memory usage: 326.0+ bytes
```

Функция `read_excel` также принимает список имен листов. В этом случае в качестве ключа он возвращает словарь с `DataFrame` в виде значения и имени листа. Чтобы считать все листы, нужно указать `sheet_name=None`.

```
In [8]: sheets = pd.read_excel("xl/stores.xlsx", sheet_name=["2019", "2020"],
                               skiprows=1, usecols=["Store", "Employees"])

        sheets["2019"].head(2)
Out[8]:
```

	Store	Employees
0	New York	10
1	San Francisco	12

Если исходный файл не содержит заголовков столбцов, установите `header=None` и предоставьте их через `names`. Обратите внимание, что `sheet_name` также принимает индексы листов:

```
In [9]: df = pd.read_excel("xl/stores.xlsx", sheet_name=0,
                           skiprows=2, skipfooter=3,
                           usecols="B:C,F", header=None,
                           names=["Branch", "Employee_Count", "Is_Flagship"])

df
Out[9]:
```

	Branch	Employee_Count	Is_Flagship
0	New York	10	False
1	San Francisco	12	MISSING
2	Chicago	4	NaN

Для обработки значений `NaN` используйте комбинацию `na_values` и `keep_default_na`. Следующий пример предписывает pandas интерпретировать ячейки со словом `MISSING` только как `NaN` и никак иначе:

```
In [10]: df = pd.read_excel("xl/stores.xlsx", sheet_name="2019",
                             skiprows=1, usecols="B,C,F", skipfooter=2,
                             na_values="MISSING", keep_default_na=False)

df
Out[10]:
```

	Store	Employees	Flagship
0	New York	10	False
1	San Francisco	12	NaN
2	Chicago	4	
3	Boston	5	True

pandas предлагает альтернативный способ чтения файлов Excel с помощью класса `ExcelFile`. В основном это имеет значение, если вы хотите считать несколько листов из файла в устаревшем формате xls: в этом случае применение `ExcelFile` будет работать быстрее, так как не позволит pandas считывать весь файл несколько раз. `ExcelFile` можно использовать в качестве контекстного менеджера (см. врезку «Контекстные менеджеры и оператор `with`»), чтобы файл был корректно закрыт.

Контекстные менеджеры и оператор `with`

Во-первых, оператор `with` в Python не имеет ничего общего с оператором `with` в VBA: в VBA он используется для выполнения серии операторов над одним и тем же объектом, а в Python — для управления ресурсами, такими как файлы или подключения к базе данных. Если вы хотите загрузить последние данные о продажах, чтобы иметь возможность их проанализировать, вам, возможно, придется открыть файл или установить соединение с базой данных. После завершения чтения данных лучше всего как можно быстрее закрыть файл или соединение. В противном случае не исключена ситуация, когда вы не сможете открыть другой файл или установить другое соединение с базой данных, так как обработчики файлов и соединения с базой данных это ограниченные ресурсы. Открытие и закрытие текстового файла вручную работает следующим образом (`w` означает открытие файла в режиме `write`, который заменяет существующий файл, если он был создан):

```
In [11]: f = open("output.txt", "w")
         f.write("Some text")
         f.close()
```

Запуск этого кода создаст файл `output.txt` в том же каталоге, что и блокнот, из которого вы его запускаете, и запишет в него «некоторый текст». Чтобы *прочитать* файл, вы используете `r` вместо `w`, а чтобы что-то дописать в конец файла, используйте `a`. Поскольку файлами можно манипулировать и извне вашей программы, такая операция может завершиться неудачно. Вы можете справиться с этим, используя механизм `try/except`, который я представлю в *главе 11*. Однако, поскольку это распространенная операция, Python предоставляет оператор `with`, чтобы упростить задачу:

```
In [12]: with open("output.txt", "w") as f:
         f.write("Some text")
```

Когда выполнение кода в теле оператора `with` завершается, файл автоматически закрывается, независимо от того, происходит исключение или нет. Это гарантирует, что ресурсы очищаются должным образом. Объекты, поддерживающие оператор `with`, называются менеджерами контекста; к ним относятся объекты `ExcelFile` и `ExcelWriter`, о которых пойдет речь в этой главе, а также объекты подключения к базе данных, которые мы рассмотрим в *главе 11*.

Давайте посмотрим класс `ExcelFile` в действии:

```
In [13]: with pd.ExcelFile("xl/stores.xls") as f:
          df1 = pd.read_excel(f, "2019", skiprows=1, usecols="B:F", nrows=2)
          df2 = pd.read_excel(f, "2020", skiprows=1, usecols="B:F", nrows=2)
```

df1

```
Out[13]:      Store  Employees Manager      Since Flagship
0      New York      10 Sarah 2018-07-20      False
1 San Francisco      12  Neriah 2019-11-02     MISSING
```

`ExcelFile` также дает вам доступ к именам всех листов:

```
In [14]: stores = pd.ExcelFile("xl/stores.xlsx")
          stores.sheet_names
```

```
Out[14]: ['2019', '2020', '2019-2020']
```

Наконец, `pandas` позволяет читать файлы Excel из URL-адреса, подобно тому, как мы делали это с файлами CSV в главе 5. Давайте прочитаем его непосредственно из партнерского репозитория:

```
In [15]: url = ("https://raw.githubusercontent.com/fzumstein/"
                "python-for-excel/1st-edition/xl/stores.xlsx")
          pd.read_excel(url, skiprows=1, usecols="B:E", nrows=2)
```

```
Out[15]:      Store  Employees Manager      Since
0      New York      10 Sarah 2018-07-20
1 San Francisco      12  Neriah 2019-11-02
```



Чтение файлов `xlsb` с помощью `pandas`

Если вы используете `pandas` версии ниже 1.3, чтение файлов `xlsb` требует явного указания механизма в функции `read_excel` или в классе `ExcelFile`:

```
pd.read_excel("xl/stores.xlsb", engine="pyxlsb")
```

Для этого необходимо установить пакет `pyxlsb`, который не входит в состав Anaconda, — его, а также другие механизмы, мы рассмотрим в следующей главе.

Подводя итоги, в табл. 7.1 приведем наиболее часто используемые параметры `read_excel`. Полный список вы найдете в официальной документации².

Таблица 7.1. Выбранные параметры для `read_excel`

Параметр	Описание
<code>sheet_name</code>	Вместо имени листа можно также указать индекс листа (на основе нуля), например <code>sheet_name=0</code> . Если задать <code>sheet_name=None</code> , <code>pandas</code> прочитает всю рабочую книгу и вернет словарь в виде <code>{"sheetname": df}</code> . Чтобы прочитать несколько листов, предоставьте список с именами листов или индексами

² <https://oreil.ly/v8Yes>. — Примеч. пер.

Таблица 7.1 (окончание)

Параметр	Описание
<code>skiprows</code>	Позволяет пропустить указанное количество строк
<code>usecols</code>	Если файл Excel содержит имена заголовков столбцов, укажите их в виде списка для выбора столбцов, например <code>["Store", "Employees"]</code> . Также это может быть список индексов столбцов, например <code>[1, 2]</code> , или строка (не список!) имен столбцов Excel, включая диапазоны, например <code>"B:D,G"</code> . Вы также можете указать функцию: например, чтобы включить только столбцы, начинающиеся с <code>Manager</code> , используйте: <code>usecols=lambda x: x.startswith("Manager")</code>
<code>nrows</code>	Количество строк, которые вы хотите прочитать
<code>index_col</code>	Указывает, какой столбец должен быть индексом, принимает имя столбца или индекс, например <code>index_col=0</code> . Если вы предоставите список с несколькими столбцами, будет создан иерархический индекс
<code>header</code>	Если вы зададите <code>header=None</code> , по умолчанию назначаются однотипные заголовки, за исключением случаев, когда вы указываете нужные имена с помощью параметра <code>names</code> . Если вы предоставите список индексов, будут созданы иерархические заголовки столбцов
<code>names</code>	Укажите желаемые названия колонок в виде списка
<code>na_values</code>	Pandas по умолчанию интерпретирует следующие значения ячеек как NaN (я рассказал о NaN в <i>главе 5</i>): пустые ячейки, <code>#NA</code> , <code>NA</code> , <code>null</code> , <code>#N/A</code> , <code>N/A</code> , <code>NaN</code> , <code>n/a</code> , <code>-NaN</code> , <code>1.#IND</code> , <code>nan</code> , <code>#N/A N/A</code> , <code>-1.#QNAN</code> , <code>- nan</code> , <code>NULL</code> , <code>-1.#IND</code> , <code><NA></code> , <code>1.#QNAN</code> . Если вы хотите добавить одно или несколько значений в этот список, укажите их через <code>na_values</code>
<code>keep_default_na</code>	Если вы хотите игнорировать значения по умолчанию, которые pandas интерпретирует как NaN, установите <code>keep_default_na=False</code>
<code>convert_float</code>	Excel хранит все числа в формате <code>float</code> , и по умолчанию pandas преобразует числа без значимых десятичных знаков в целые числа. Если вы хотите изменить это действие, установите <code>convert_float=False</code> (это может работать немного быстрее)
<code>converters</code>	Позволяет вам предоставить функцию для преобразования его значений каждого столбца. Например, чтобы сделать текст в определенной колонке прописным, используйте следующее: <code>converters={"column_name": lambda x: x.upper() }</code>

Вот и все, что касается чтения файлов Excel с помощью `pandas`, давайте теперь изменим направление и в следующем разделе узнаем о записи файлов Excel!

Метод `to_excel` и класс `ExcelWriter`

Самый простой способ записать файл Excel с помощью pandas — использовать метод `to_excel` из `DataFrame`. Этот метод позволяет указать, в какую ячейку какого листа вы хотите записать `DataFrame`. Вы также можете решить, включать или нет заголовки столбцов и индекс `DataFrame` и как обращаться с такими типами данных, как `np.nan` и `np.inf`, которые не имеют эквивалентного представления в Excel. Начнем с создания `DataFrame` с различными типами данных и использования метода `to_excel`:

```
In [16]: import numpy as np
import datetime as dt

In [17]: data=[(dt.datetime(2020,1,1, 10, 13), 2.222, 1, True),
               (dt.datetime(2020,1,2), np.nan, 2, False),
               (dt.datetime(2020,1,2), np.inf, 3, True)]
df = pd.DataFrame(data=data,
                  columns=["Dates", "Floats", "Integers", "Booleans"])
df.index.name="index"
df

Out[17]:
```

	Dates	Floats	Integers	Booleans
index				
0	2020-01-01 10:13:00	2.222	1	True
1	2020-01-02 00:00:00	NaN	2	False
2	2020-01-02 00:00:00	inf	3	True

```
In [18]: df.to_excel("written_with_pandas.xlsx", sheet_name="Output",
                    startrow=1, startcol=1, index=True, header=True,
                    na_rep="<NA>", inf_rep="<INF>")
```

Выполнение команды `to_excel` создаст файл Excel, как показано на рис. 7.4 (чтобы правильно видеть даты, вам нужно будет сделать столбец с шире):

	A	B	C	D	E	F
1						
2		index	Dates	Floats	Integers	Booleans
3		0	2020-01-01 10:13:00	2.222	1	TRUE
4		1	2020-01-02 00:00:00	<NA>	2	FALSE
5		2	2020-01-02 00:00:00	<INF>	3	TRUE

Рис. 7.4. `written_with_pandas.xlsx`

Если вы хотите записать несколько `DataFrames` в один и тот же лист или на разные листы, вам необходимо использовать класс `ExcelWriter`. В следующем примере один и тот же `DataFrame` записывается в два разных места на `Sheet1` (Лист1) и повторно на `Sheet2` (Лист2):

```
In [19]: with pd.ExcelWriter("written_with_pandas2.xlsx") as writer:
df.to_excel(writer, sheet_name="Sheet1", startrow=1, startcol=1)
df.to_excel(writer, sheet_name="Sheet1", startrow=10, startcol=1)
df.to_excel(writer, sheet_name="Sheet2")
```

Поскольку мы используем класс `ExcelWriter` в качестве контекстного менеджера, файл автоматически записывается на диск при выходе из контекстного менеджера, то есть, когда отступы заканчиваются. В противном случае вам придется явно вызывать `writer.save()`. Краткое описание наиболее часто используемых параметров, которые принимает `to_excel`, приведено в табл. 7.2. Полный список параметров вы найдете в официальных документах³.

Таблица 7.2. Избранные параметры для `to_excel`

Параметр	Описание
<code>sheet_name</code>	Название листа, на который производится запись
<code>startrow</code> and <code>startcol</code>	<code>Startrow</code> — первая строка, в которую будет записан <code>DataFrame</code> , а <code>startcol</code> — первый столбец
<code>index</code> and <code>header</code>	Если вы хотите скрыть индекс и/или заголовок, установите для них значения <code>index=False</code> и <code>header=False</code> , соответственно
<code>na_rep</code> and <code>inf_rep</code>	По умолчанию <code>np.nan</code> будет преобразован в пустую ячейку, а <code>np.inf</code> , отображение бесконечности в NumPy, будет преобразовано в строку <code>inf</code> . Предоставление значений позволяет вам изменить это поведение
<code>freeze_panes</code>	Заморозка первой пары строк и столбцов. Для этого нужно указать кортеж: например, <code>(2, 1)</code> заморозит первые две строки и первый столбец

Как вы можете заметить, чтение и запись простых файлов Excel с помощью `pandas` работает хорошо. Однако есть и ограничения — давайте посмотрим, какие!

Ограничения при работе `pandas` с файлами Excel

Использование интерфейса `pandas` для чтения и записи файлов Excel отлично подходит для простых случаев, но есть и ограничения:

- ◆ при записи `DataFrame` в файл вы не можете включить в него заголовок или график;
- ◆ в Excel нет возможности по умолчанию изменить формат заголовка и индекса;
- ◆ при чтении файлов `pandas` автоматически преобразует ячейки с ошибками типа `#REF!` или `#NUM!` в `NaN`, что не позволяет искать в ваших электронных таблицах конкретные ошибки;
- ◆ работа с большими файлами Excel, как мы увидим в следующей главе, может потребовать дополнительных настроек, которые легче контролировать, используя непосредственно пакеты чтения и записи.

³ <https://oreil.ly/ESKAG>. — Примеч. пер.

Заключение

Преимущество pandas состоит в том, что он предлагает согласованный интерфейс для работы со всеми поддерживаемыми форматами файлов Excel, будь то xls,xlsx, xlsm или xlsb. Это позволило нам легко прочитать каталог файлов Excel, сгруппировать данные и вывести сводку в отчет Excel, используя всего десять строк кода.

Однако pandas выполняет тяжелую работу не самостоятельно: скрытно под оболочкой он выбирает пакет `reader` или `writer` для выполнения этой работы. В следующей главе я продемонстрирую вам, какие пакеты для чтения и записи использует pandas и как их можно использовать непосредственно или в сочетании с pandas. Это позволит нам обойти ограничения, которые были описаны в предыдущем разделе.

Манипулирование файлами Excel с помощью пакетов reader и writer

В этой главе вы познакомитесь с OpenPyXL, XlsxWriter, pyxlsb, xlrd и xlwt: это пакеты, которые могут читать и записывать файлы Excel и используются pandas скрытно, под оболочкой, когда вы вызываете функции `read_excel` или `to_excel`. Использование напрямую пакетов чтения и записи позволяет создавать комплексные отчеты Excel и точнее настроить процесс чтения. Кроме того, если вы когда-нибудь будете работать над проектом, в котором нужно только читать и записывать файлы Excel без необходимости использования остальной функциональности pandas, установка полного набора NumPy/pandas будет излишней.

Мы начнем эту главу с изучения того, когда следует использовать тот или иной пакет и как работает их синтаксис, а затем рассмотрим несколько более сложных тем, включая работу с большими файлами Excel и то, как объединить pandas с пакетами reader и writer для улучшения отображения стиля DataFrames. В заключение мы вновь воспользуемся примером из начала предыдущей главы и улучшим отчет Excel, отформатировав таблицу и добавив диаграмму. Как и в предыдущей главе, нам не потребуется устанавливать Excel. А это означает, что все примеры кода работают под Windows, macOS и Linux.

Пакеты reader и writer

Набор программ для чтения и записи может быть весьма обширным: в этом разделе мы рассмотрим не меньше шести таких пакетов, так как почти для каждого типа файлов Excel требуется свой пакет чтения/записи. Тот факт, что каждый пакет использует свой синтаксис, который в большинстве случаев значительно отличается от оригинальной объектной модели Excel, не делает использование этих пакетов проще — подробнее я расскажу об объектной модели Excel в следующей главе. Это означает, что вам, даже если вы опытный разработчик VBA, скорее всего, придется изучить множество команд. Этот раздел начинается с обзора того, в каких случаях вам нужен тот или иной пакет, а затем будет представлен вспомогательный модуль, который немного упрощает работу с этими пакетами. После этого я представлю каждый из пакетов в стиле «кулинарной книги», где вы сможете посмотреть, как работают наиболее часто используемые команды.

В каких случаях какой пакет используется

В этом разделе представлены следующие шесть пакетов для чтения, записи и редактирования файлов Excel:

- ◆ OpenPyXL¹
- ◆ XlsxWriter²
- ◆ pyxlsb³
- ◆ xlrd⁴
- ◆ xlwt⁵
- ◆ xlutils⁶

Чтобы понять возможности каждого пакета, ознакомьтесь с табл. 8.1. Например, чтобы прочитать формат файла `xlsx`, вам придется использовать пакет OpenPyXL:

Таблица 8.1. Когда какой пакет использовать

Формат файла Excel	Чтение	Запись	Редактирование
xlsx	OpenPyXL	OpenPyXL, XlsxWriter	OpenPyXL
xlsm	OpenPyXL	OpenPyXL, XlsxWriter	OpenPyXL
xltx, xltm	OpenPyXL	OpenPyXL	OpenPyXL
xlsb	pyxlsb	—	—
xls, xlt	xlrd	xlwt	xlutils

Если вы хотите записывать файлы в формате `xlsx` или `xlsm`, вам нужно выбрать один из двух пакетов: OpenPyXL или XlsxWriter. Оба пакета имеют сходную функциональность, но каждый из них может содержать несколько уникальных функций, которых нет у другого пакета. Поскольку обе библиотеки активно совершенствуются, со временем ситуация может измениться. Рассмотрим их различия:

- ◆ OpenPyXL может читать, записывать и редактировать, а XlsxWriter умеет только записывать;
- ◆ OpenPyXL упрощает создание файлов Excel с макросами VBA;
- ◆ XlsxWriter лучше документирован;

¹ <http://oreil.ly/3jHQM>. — Примеч. пер.

² <http://oreil.ly/7jI3T>. — Примеч. пер.

³ <http://oreil.ly/sEHXS>. — Примеч. пер.

⁴ <http://oreil.ly/tSam7>. — Примеч. пер.

⁵ <http://oreil.ly/wPSLe>. — Примеч. пер.

⁶ <http://oreil.ly/MTFOL>. — Примеч. пер.

- ◆ XlsxWriter, как правило, более быстрый, чем OpenPyXL, но в зависимости от размера рабочей книги, которую вы пишете, разница может быть незначительной.



Что такое xlwings?

Если вам интересно, где в табл. 8.1 находится xlwings, то ответ — либо нигде, либо везде. Это зависит от конкретного случая: в отличие от других пакетов, рассмотренных в этой главе, использование xlwings зависит от приложения Excel, которое зачастую недоступно. Например, в случае, если вам нужно запустить свои сценарии на Linux или если вы запускаете свои сценарии на Windows или macOS, где у вас есть доступ к установленному Excel, xlwings действительно, поэтому его можно использовать в качестве альтернативы всем пакетам. Поскольку зависимость от Excel является таким фундаментальным отличием xlwings от всех других пакетов Excel, я представляю xlwings в следующей главе, с которой начинается *часть IV* этой книги.

pandas использует найденный им пакет writer, но, если у вас установлены и OpenPyXL, и lxxWriter, то по умолчанию будет использоваться XlsxWriter. Если вы хотите определить, какой пакет должен использовать pandas, укажите параметр `engine` в функциях `read_excel` или `to_excel` или классах `ExcelFile` и `ExcelWriter` соответственно. Engine — это имя пакета, которого пишется в нижнем регистре, поэтому чтобы записать файл с помощью OpenPyXL, а не XlsxWriter, выполните следующее:

```
df.to_excel("filename.xlsx", engine="openpyxl")
```

Как только вы поймете, какой пакет вам нужен, вас ожидает следующая задача: большинство из этих пакетов требуют написания довольно большого количества строк кода для чтения или записи нужного диапазона ячеек, и каждый пакет использует свой синтаксис. Чтобы облегчить вам жизнь, я создал вспомогательный модуль, который представлю далее.

Модуль excel.py

Я создал модуль `excel.py`, чтобы облегчить вам работу при использовании пакетов `reader` и `writer`, поскольку он решает следующие задачи:

Переключение между пакетами

Необходимость переключения между пакетами для чтения или записи является наиболее распространенной задачей. Например, файлы Excel имеют тенденцию со временем увеличиваться в размерах, с чем многие пользователи борются, меняя формат файла с `xlsx` на `xlsb`, так как таким образом можно существенно уменьшить размер файла.

Преобразование типов данных

Эта операция имеет отношение к предыдущему пункту: при переходе от одного пакета к другому необходимо не только корректировать синтаксис кода, но и следить за разными типами данных, которые эти пакеты возвращают для одного и того же содержимого ячеек. Например, OpenPyXL для пустых ячеек возвращает `None`, а `xlrd` возвращает пустую строку.

За цикливание ячеек

Пакеты reader и writer относятся к *низкоуровневым* пакетам: это означает, что в них отсутствуют удобные функции, которые позволили бы вам легко справляться с обычными задачами. Например, большинство пакетов требуют от вас циклического просмотра каждой отдельной ячейки, которую вы собираетесь прочитать или записать.

Вы найдете модуль excel.py в партнерском репозитории, и мы будем использовать его в последующих разделах, но в качестве предварительного обзора представлю синтаксис для чтения и записи значений:

```
import excel
values = excel.read(sheet_object, first_cell="A1", last_cell=None)
excel.write(sheet_object, values, first_cell="A1")
```

Функция read принимает объект sheet из одного из следующих пакетов: xlrd, OpenPyXL или pyxlsb. Функция также принимает необязательные аргументы first_cell и last_cell. Они могут быть представлены либо в нотации A1, либо в виде строки-столбца-кортежа с индексами Excel, основанными на единице: (1, 1). Значением по умолчанию для first_cell является A1, а значением по умолчанию для last_cell — правый нижний угол используемого диапазона. Следовательно, если вы предоставите только объект sheet, он прочитает весь лист. Функция write работает аналогично: она ожидает объект sheet от xlwt, OpenPyXL или XlsxWriter вместе со значениями в виде вложенного списка и необязательным first_cell, который маркирует левый верхний угол, где будет записан вложенный список. Модуль excel.py, как показано в табл. 8.2, дополнительно согласовывает преобразование типов данных.

Таблица 8.2. Преобразование типов данных

Представление в Excel	Тип данных Python
Пустая клетка	None
Ячейка с форматом даты	datetime.datetime (за исключением pyxlsb)
Ячейка с логическим значением	bool
Ячейка с ошибкой	str (сообщение об ошибке)
Строка	str
С плавающей запятой	float или int

Имея модуль excel.py, мы готовы к изучению пакетов OpenPyXL, XlsxWriter, pyxlsb и xlrd/xlwt/xlutils, которые рассмотрены в следующих разделах. Они составлены в стиле «кулинарных книг», что позволяет быстро приступить к работе с каждым пакетом. Вместо того чтобы читать все разделы друг за другом, я бы рекомендовал вам выбрать необходимый пакет на основе табл. 8.1, а затем перейти непосредственно к соответствующему разделу.

Оператор with



В этой главе мы в различных ситуациях будем использовать оператор `with`. Если вам нужно освежить информацию, посмотрите врезку «Контекстные менеджеры и оператор `with`» на стр. 170 в *главе 7*.

OpenPyXL

OpenPyXL — единственный пакет в этом разделе, который может как читать, так и записывать файлы Excel. Вы даже можете использовать его для редактирования простых файлов Excel. Для начала рассмотрим, как работает функция чтения.

Чтение с помощью OpenPyXL

Следующий пример кода показывает, как выполнять общие задачи, воспользовавшись OpenPyXL для чтения файлов Excel. Чтобы получить значения ячеек, необходимо открыть рабочую книгу с параметром `data_only=True`. По умолчанию используется значение `False`, которое возвращает формулы ячеек:

```
In [1]: import pandas as pd
        import openpyxl
        import excel
        import datetime as dt

In [2]: # Открыть рабочую книгу для чтения значений ячеек.
        # После загрузки данных файл снова автоматически закроется.
        book = openpyxl.load_workbook("xl/stores.xlsx", data_only=True)

In [3]: # Получить объект рабочего листа по имени или индексу
        # (на основе 0)
        sheet = book["2019"]
        sheet = book.worksheets[0]

In [4]: # Получить список с именами всех листов
        book.sheetnames

Out[4]: ['2019', '2020', '2019-2020']

In [5]: # Просмотреть все объекты листа.
        # Вместо "name" openpyxl использует "title".
        for i in book.worksheets:
            print(i.title)

2019
2020
2019-2020

In [6]: # Получение размеров, т.е. используемый диапазон листа
        sheet.max_row, sheet.max_column

Out[6]: (8, 6)

In [7]: # Чтение значения одной ячейки с использованием нотации
        # "A1" и индексов ячеек (на основе 1)
```

```

sheet["B6"].value
sheet.cell(row=6, column=2).value
Out[7]: 'Boston'
In [8]: # Считывание диапазона значений ячеек с помощью модуля excel
data = excel.read(book["2019"], (2, 2), (8, 6))
data[:e] # Вывести первые две строки
Out[8]: [['Store', 'Employees', 'Manager', 'Since', 'Flagship'],
         ['New York', 10, 'Sarah', datetime.datetime(2018, 7, 20, 0, 0), False]]

```

Запись с помощью OpenPyXL

OpenPyXL создает в памяти файл Excel и записывает его после вызова метода `save`. Следующий код, как показано на рис. 8.1, создает файл:

```

In [9]: import openpyxl
        from openpyxl.drawing.image import Image
        from openpyxl.chart import BarChart, Reference
        from openpyxl.styles import Font, colors
        from openpyxl.styles.borders import Border, Side
        from openpyxl.styles.alignment import Alignment
        from openpyxl.styles.fills import PatternFill
        import excel

In [10]: # Создать рабочую книгу
book = openpyxl.Workbook()

        # Получить первый лист и дать ему имя
sheet = book.active
sheet.title = "Sheet1"

        # Запись отдельных ячеек с использованием нотации A1 и индексов
#ячеек (на основе 1)
sheet["A1"].value = "Hello 1"
sheet.cell(row=2, column=1, value="Hello 2")

        # Форматирование: цвет заливки, выравнивание, границы и шрифт
font_format = Font(color="FF0000", bold=True)
thin = Side(border_style="thin", color="FF0000")
sheet["A3"].value = "Hello 3"
sheet["A3"].font = font_format
sheet["A3"].border = Border(top=thin, left=thin,
                             right=thin, bottom=thin)
sheet["A3"].alignment = Alignment(horizontal="center")
sheet["A3"].fill = PatternFill(fgColor="FFFF00", fill_type="solid")

        # Форматирование чисел (с использованием строк форматирования
# Excel)
sheet["A4"].value = 3.3333
sheet["A4"].number_format = "0.00"

```

```

# Форматирование даты (с использованием строк
# форматирования Excel)
sheet["A5"].value = dt.date(2016, 10, 13)
sheet["A5"].number_format = "mm/dd/yy"

# Формула: вы должны использовать английское название формулы
# с запятыми в качестве разделителей

sheet["A6"].value = "=SUM(A4, 2)"

# Изображение
sheet.add_image(Image("images/python.png"), "C1")

# Двумерный список (мы используем наш модуль excel)
data = [[None, "North", "South"],
        ["Last Year", 2, 5],
        ["This Year", 3, 6]]
excel.write(sheet, data, "A10")

# График
chart = BarChart()
chart.type = "col"
chart.title = "Sales Per Region"
chart.x_axis.title = "Regions"
chart.y_axis.title = "Sales"
chart_data = Reference(sheet, min_row=11, min_col=1,
                        max_row=12, max_col=3)
chart_categories = Reference(sheet, min_row=10, min_col=2,
                             max_row=10, max_col=3)
# from_rows интерпретирует данные так же, как если бы вы
# добавили диаграмму вручную в Excel
chart.add_data(chart_data, titles_from_data=True,
               from_rows=True)
chart.set_categories(chart_categories)
sheet.add_chart(chart, "A15")

# Сохранить рабочую книгу - создать файл на диске
book.save("openpyxl.xlsx")

```

Если вы хотите записать файл шаблона Excel, то перед сохранением нужно установить атрибут шаблона `True`:

```

In [11]: book = openpyxl.Workbook()
        sheet = book.active
        sheet["A1"].value = "This is a template"
        book.template = True
        book.save("template.xlsx")

```

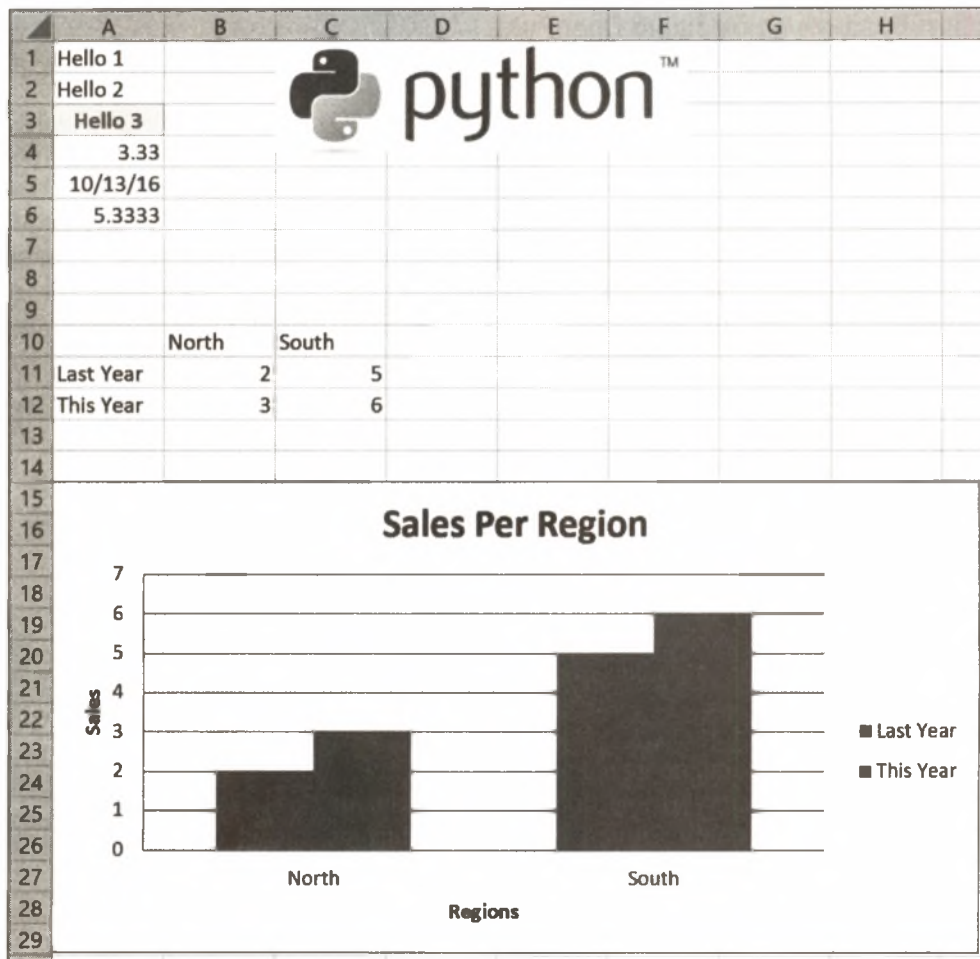


Рис. 8.1. Файл, записанный OpenPyXL (openpyxl.xlsx)

Как видно из кода, OpenPyXL устанавливает цвета, предоставляя строку вида FF0000. Это значение состоит из трех шестнадцатеричных значений (FF, 00 и 00), которые соответствуют значениям красного/зеленого/синего цветов для выбранного цвета. Нех означает *шестнадцатеричную* систему и представляет числа, в которых вместо основания десять, которое используется в нашей стандартной десятичной системе, используется основание шестнадцать.



Определение цвета в шестнадцатеричной системе

Чтобы найти нужное шестнадцатеричное значение цвета в Excel, нажмите на открывающийся список красок, используемых для изменения цвета заливки ячейки, затем выберите пункт **Другие цвета**. Теперь выберите необходимый цвет и прочитайте его шестнадцатеричное значение в меню.

Редактирование с помощью OpenPyXL

Не существует пакета чтения/записи, который действительно мог бы редактировать файлы Excel: в действительности OpenPyXL читает файл со всем содержимым, которое он понимает, а затем снова записывает файл с нуля — включая все изменения, которые были внесены. Такой метод может быть очень эффективен, когда он применяется в отношении для простых файлов Excel, содержащих в основном отформатированные ячейки с данными и формулами. Но метод имеет свои ограничения: если в вашей таблице есть диаграммы и другое более сложное содержимое, OpenPyXL либо изменит их, либо вообще удалит. Например, начиная с версии 3.0.5, OpenPyXL будет переименовывать графики и удалять их названия. Вот простой пример редактирования:

```
In [12]: # Прочитайте файл stores.xlsx, измените ячейку и сохраните ее
        # под новым расположением/именем.
        book = openpyxl.load_workbook("xl/stores.xlsx")
        book["2019"]["A1"].value = "modified"
        book.save("stores_edited.xlsx")
```

Если вы хотите написать файл xlsm, OpenPyXL должен работать с существующим файлом, который нужно загрузить с параметром `keep_vba`, установленным как `True`:

```
In [13]: book = openpyxl.load_workbook("xl/macro.xlsm", keep_vba=True)
        book["Sheet1"]["A1"].value = "Click the button!"
        book.save("macro_openpyxl.xlsm")
```

Кнопка в файле примера вызывает макрос, который отображает окно сообщения. OpenPyXL обладает гораздо большей функциональностью, чем я могу описать в этом разделе; поэтому для получения дополнительных сведений вам стоит заглянуть в официальную документацию⁷. Мы рассмотрим дополнительные функциональные возможности в конце этой главы, когда снова обратимся к примеру из предыдущей главы.

XlsxWriter

Как следует из названия, XlsxWriter только записывает файлы Excel. С помощью следующего кода мы создадим рабочую книгу, показанную на рис. 8.1. Она была создана ранее с помощью OpenPyXL. Обратите внимание, что XlsxWriter использует нулевые индексы ячеек, а OpenPyXL — единичные индексы ячеек. Обязательно учитывайте это при переключении между пакетами:

```
In [14]: import datetime as dt
        import xlsxwriter
        import excel

In [15]: # Создание рабочей книги
        book = xlsxwriter.Workbook("xlsxwriter.xlsx")
```

⁷ <http://oreil.ly/7qfYL>. — Примеч. пер.

```

# Добавление листа и присваивание ему имени
sheet = book.add_worksheet("Sheet1")

# Запись отдельных ячеек с использованием нотации A1
# и индексов ячеек (на основе 0)
sheet.write("A1", "Hello 1")
sheet.write(1, 0, "Hello 2")

# Форматирование: цвет заливки, выравнивание, границы и шрифт
formatting = book.add_format({"font_color": "#FF0000",
                              "bg_color": "#FFFF00",
                              "bold": True, "align": "center",
                              "border": 1, "border_color": "#FF0000"})
sheet.write("A3", "Hello 3", formatting)

# Форматирование чисел (с использованием строк форматирования
# Excel)
number_format = book.add_format({"num_format": "0.00"})
sheet.write("A4", 3.3333, number_format)

# Форматирование даты (с использованием строк форматирования
# Excel)
date_format = book.add_format({"num_format": "mm/dd/yy"})
sheet.write("A5", dt.date(2016, 10, 13), date_format)

# Формула: вы должны использовать английское название формулы
# с запятыми в качестве разделителей
sheet.write("A6", "=SUM(A4, 2)")

# Изображение
sheet.insert_image(0, 2, "images/python.png")

data = [[None, "North", "South"],
        ["Last Year", 2, 5],
        ["This Year", 3, 6]]
excel.write(sheet, data, "A10")

# Диаграмма: смотрите файл "sales_report_xlsxwriter.py" в
# сопутствующем репозитории, чтобы увидеть, как можно работать
# с индексами вместо адресов ячеек
chart = book.add_chart({"type": "column"})
chart.set_title({"name": "Sales per Region"})
chart.add_series({"name": "=Sheet1!A11",
                  "categories": "=Sheet1!B10:C10",
                  "values": "=Sheet1!B11:C11"})
chart.add_series({"name": "=Sheet1!A12",
                  "categories": "=Sheet1!B10:C10",
                  "values": "=Sheet1!B12:C12"})

```

```

chart.set_x_axis({"name": "Regions"})
chart.set_y_axis({"name": "Sales"})
sheet.insert_chart("A15", chart)

# При закрытии рабочей книги на диске создается файл
book.close()

```

По сравнению с OpenPyXL, XlsxWriter должен использовать более сложный подход для записи файлов в формате xlsx, так как XlsxWriter является пакетом только с функцией записи. Сначала необходимо извлечь код макроса из существующего в Anaconda Prompt файла Excel (в примере используется файл macro.xlsx, который вы найдете в папке xl сопутствующего репозитория):

Windows

Вначале перейдите в каталог xl, затем найдите путь к vba_extract.py, сценарию, который поставляется вместе с XlsxWriter:

```

(base)> cd C:\Users\username\python-for-excel\xl
(base)> where vba_extract.py
C:\Users\username\Anaconda3\Scripts\vba_extract.py

```

Далее используйте этот путь в следующей команде:

```

(base)> python C:\...\Anaconda3\Scripts\vba_extract.py macro.xlsx

```

macOS

В macOS команда доступна в виде исполняемого сценария и может быть запущена следующим образом:

```

(base)> cd /Users/username/python-for-excel/xl
(base)> vba_extract.py macro.xlsx

```

Это сохранит файл vbaProject.bin в каталоге, в котором вы выполняете команду. Я также включил извлеченный файл в папку xl в партнерском репозитории. Мы будем использовать его в следующем примере, чтобы написать рабочую книгу с кнопкой макроса:

```

In [16]: book = xlsxwriter.Workbook("macro_xlsxwriter.xlsx")
        sheet = book.add_worksheet("Sheet1")
        sheet.write("A1", "Click the button!")
        book.add_vba_project("xl/vbaProject.bin")
        sheet.insert_button("A3", {"macro": "Hello", "caption": "Button
                                1", "width": 130, "height": 35})
        book.close()

```

pyxlsb

По сравнению с другими библиотеками чтения pyxlsb предлагает меньшую функциональность, но это ваш единственный вариант, когда речь идет о чтении файлов Excel в двоичном формате xlsb. pyxlsb не входит в состав Anaconda, поэтому вам

придется установить его, если вы еще не сделали этого. В настоящее время через Conda его установить не получится, поэтому для установки используйте pip:

```
(base)> pip install pyxlsb
```

Листы и значения ячеек читаются следующим образом:

```
In [17]: import pyxlsb
```

```
import excel
```

```
In [18]: # Перебор листов. В pyxlsb объекты workbook и sheet можно
```

```
# использовать в качестве контекстного менеджера.
```

```
# book.sheets возвращает список имен листов, а не объектов!
```

```
# Чтобы получить объект листа, используйте get_sheet().
```

```
with pyxlsb.open_workbook("xl/stores.xlsb") as book:
```

```
    for sheet_name in book.sheets:
```

```
        with book.get_sheet(sheet_name) as sheet:
```

```
            dim = sheet.dimension
```

```
            print(f"Sheet '{sheet_name}' has "
```

```
                  f"{dim.h} rows and {dim.w} cols")
```

```
Sheet '2019' has 7 rows and 5 cols
```

```
Sheet '2020' has 7 rows and 5 cols
```

```
Sheet '2019-2020' has 20 rows and 5 cols
```

```
In [19]: # Считывание значений из диапазона ячеек с помощью нашего модуля
```

```
# excel.
```

```
# Вместо "2019" можно также использовать его индекс
```

```
# (на основе 1).
```

```
with pyxlsb.open_workbook("xl/stores.xlsb") as book:
```

```
    with book.get_sheet("2019") as sheet:
```

```
        data = excel.read(sheet, "B2")
```

```
data[:2] # Печать первых двух строк
```

```
Out[19]: [['Store', 'Employees', 'Manager', 'Since', 'Flagship'],
```

```
          ['New York', 10.0, 'Sarah', 43301.0, False]]
```

В настоящее время в pyxlsb отсутствует способ распознавания ячеек с датами, поэтому вам придется вручную преобразовывать значения из ячеек, отформатированных под дату, в объекты datetime следующим образом:

```
In [20]: from pyxlsb import convert_date
```

```
convert_date(data[1][3])
```

```
Out[20]: datetime.datetime(2018, 7, 20, 0, 0)
```

Помните, что при чтении файлов формата xlsb с помощью pandas версии ниже 1.3 вам необходимо явно указать используемый модуль:

```
In [21]: df = pd.read_excel("xl/stores.xlsb", engine="pyxlsb")
```

xlrd, xlwt, and xlutils

Комбинация xlrd, xlwt и xlutils предлагает примерно ту же функциональность для устаревшего формата xls, что и OpenPyXL для формата xlsx: xlrd считывает, xlwt

записывает, а xlutils редактирует файлы xls. Эти пакеты больше активно не разрабатываются, но они, вероятно, будут актуальны до тех пор, пока существуют файлы xls. xlutils не является частью Anaconda, поэтому, если вы еще этого не сделали, установите его:

```
(base)> conda install xlutils
```

Приступим к чтению!

Чтение с помощью xlrd

Следующий пример кода показывает, как читать значения из рабочей книги Excel с помощью xlrd:

```
In [22]: import xlrd
         import xlwt
         from xlwt.Utils import cell_to_rowcol2
         import xlutils
         import excel

In [23]: # Откройте рабочую книгу для чтения значений ячеек. После
         # загрузки данных файл автоматически закрывается.
         book = xlrd.open_workbook("xl/stores.xls")

In [24]: # Получить список с именами всех листов
         book.sheet_names()

Out[24]: ['2019', '2020', '2019-2020']

In [25]: # Перебор объектов листа
         for sheet in book.sheets():
             print(sheet.name)

2019
2020
2019-2020

In [26]: # Получение объекта листа по имени или индексу (на основе 0)
         sheet = book.sheet_by_index(0)
         sheet = book.sheet_by_name("2019")

In [27]: # Размеры
         sheet.nrows, sheet.ncols

Out[27]: (8, 6)

In [28]: # Чтение значения одной ячейки с
         # использованием нотации "A1" и индексов ячеек (на основе 0).
         # Символ "*" распаковывает кортеж, возвращаемый cell_to_rowcol2,
         # в отдельные аргументы
         sheet.cell(*cell_to_rowcol2("B3")).value
         sheet.cell(2, 1).value

Out[28]: 'New York'

In [29]: # Считывание диапазона значений ячеек с помощью нашего модуля
         # excel
         data = excel.read(sheet, "B2")
         data[:2] # Вывести первые две строки
```

```
Out[29]: [['Store', 'Employees', 'Manager', 'Since', 'Flagship'],
          ['New York', 10.0, 'Sarah', datetime.datetime(2018, 7, 20, 0, 0),
           False]]
```



Используемый диапазон

В отличие от OpenPyXL и pyxlsb, xlrd при использовании `sheet.nrows` и `sheet.ncols` возвращает размеры ячеек со значением, а не *используемый диапазон* листа. То, что Excel возвращает в качестве используемого диапазона, часто в нижней части и на правой границе диапазона содержит пустые строки и столбцы. Это может произойти, например, когда вы удаляете содержимое строк (нажав клавишу <Delete>), а не сами строки (щелкнув правой кнопкой мыши и выбрав из появившегося контекстного меню команду **Delete**).

Работа с xlwt

Следующий код воспроизводит то, что мы сделали ранее с помощью OpenPyXL и XlsxWriter, а результат был показан на рис. 8.1.

```
In [30]: import xlwt
         from xlwt.Utills import cell_to_rowcol2
         import datetime as dt
         import excel

In [31]: # Создание рабочей книги
         book = xlwt.Workbook()

         # Добавление листа и присвоение ему имени
         sheet = book.add_sheet("Sheet1")

         # Запись отдельных ячеек с использованием нотации A1
         # и индексов ячеек (на основе 0)
         sheet.write(*cell_to_rowcol2("A1"), "Hello 1")
         sheet.write(r=1, c=0, label="Hello 2")

         # Форматирование: цвет заливки, выравнивание, границы и шрифт
         formatting = xlwt.easyxf("font: bold on, color red;"
                                   "align: horiz center;"
                                   "borders: top_color red, bottom_color red,"
                                   "right_color red, left_color red,"
                                   "left thin, right thin,"
                                   "top thin, bottom thin;"
                                   "pattern: pattern solid, fore_color yellow;")
         sheet.write(r=2, c=0, label="Hello 3", style=formatting)

         # Форматирование чисел (с использованием строк форматирования
         # Excel)
         number_format = xlwt.easyxf(num_format_str="0.00")
         sheet.write(3, 0, 3.3333, number_format)
```



```
# Форматирование даты (с использованием строк форматирования
# Excel)
date_format = xlwt.easyxf(num_format_str="mm/dd/yyyy")
sheet.write(4, 0, dt.datetime(2012, 2, 3), date_format)

# Формула: вы должны использовать английское название формулы
# с запятыми в качестве разделителей
sheet.write(5, 0, xlwt.Formula("SUM(A4, 2)"))

# Двумерный список (мы используем наш модуль excel)
data = [[None, "North", "South"],
        ["Last Year", 2, 5],
        ["This Year", 3, 6]]
excel.write(sheet, data, "A10")

# Изображение (позволяет добавить только формат bmp)
sheet.insert_bitmap("images/python.bmp", 0, 2)
# Запись файла на диск
book.save("xlwt.xls")
```

Редактирование с помощью xlutils

xlutils действует как связующий элемент между xlrd и xlwt. Таким образом, становится очевидным, что это не настоящая операция редактирования: электронная таблица считывается с учетом форматирования через xlrd (используя `formatting_info=True`), а затем снова записывается xlwt, включая изменения, которые были сделаны в заданных пределах:

```
In [32]: import xlutils.copy
In [33]: book = xlrd.open_workbook("xl/stores.xls", formatting_info=True)
        book = xlutils.copy.copy(book)
        book.get_sheet(0).write(0, 0, "changed!")
        book.save("stores_edited.xls")
```

На данном этапе вы узнали, как читать и записывать рабочую книгу Excel в определенном формате. В следующем разделе мы рассмотрим расширенный круг задач, включая работу с большими файлами Excel и совместное использование `pandas` и пакетов `reader` и `writer`.

Расширенный круг задач для reader и writer

Если ваши файлы больше и сложнее, чем простые файлы Excel, которые мы до сих пор использовали в примерах, полагаться на параметры, задаваемые по умолчанию, скорее всего и не совсем правильно. Поэтому мы начнем этот раздел с рассмотрения способов работы с большими файлами. Затем мы узнаем, как использовать `pandas` вместе с пакетами `reader` и `writer`: это позволит оформлять ваши `pandas DataFrames` так, как вы пожелаете. В завершение этого раздела мы используем все

полученные в этой главе знания, чтобы отчет Excel из примера прошлой главы выглядел более профессионально.

Работа с большими файлами Excel

Работа с большими файлами сопряжена с двумя возможными неприятностями: процесс чтения и записи может быть медленным или на вашем компьютере может закончиться свободный объем памяти. Обычно наибольшую обеспокоенность вызывает проблема с памятью, поскольку она может привести к аварийному завершению работы вашей программы. В какой именно момент файл будет считаться *большим*, всегда зависит от доступных ресурсов в вашей системе и вашего определения *медлительности*. В этом разделе показаны методы оптимизации, предлагаемыми отдельными пакетами, которые позволяют работать с файлами Excel, достигая предельных параметров. Я начну с рассмотрения опций для библиотек, обеспечивающих запись файлов, а затем рассмотрю опции для библиотек, позволяющих эти файлы считывать. В конце этого раздела я покажу вам, как читать листы рабочей книги параллельно, что сократит время их обработки.

Запись с помощью OpenPyXL

При записи больших файлов с помощью OpenPyXL убедитесь, что пакет lxml, ускоряющий процесс записи, установлен. Он входит в состав Anaconda, поэтому вам ничего не нужно дополнительно устанавливать. Но наиболее важным параметром является флаг `write_only=True`, с помощью которого обеспечивается низкое потребление памяти. Однако вы будете вынуждены записывать строку за строкой с помощью метода `append` и не сможете создавать отдельные ячейки:

```
In [34]: book = openpyxl.Workbook(write_only=True)
        # При write_only=True, book.active не работает
        sheet = book.create_sheet()
        # В результате получится лист с ячейками 1000 x 200
        for row in range(1000):
            sheet.append(list(range(200)))
        book.save("openpyxl_optimized.xlsx")
```

Запись с помощью XlsxWriter

XlsxWriter, как и OpenPyXL, содержит подобную опцию, которая называется `constant_memory`. Она вынуждает вас записывать строки последовательно. Вы включаете опцию, предоставляя словарь опций следующим образом:

```
In [35]: book = xlsxwriter.Workbook("xlsxwriter_optimized.xlsx",
        options={"constant_memory": True})
        sheet = book.add_worksheet()
        # В результате получится лист с ячейками 1000 x 200
        for row in range(1000):
            sheet.write_row(row, 0, list(range(200)))
        book.close()
```

Чтение с помощью xlrd

При чтении больших файлов в устаревшем формате xls xlrd позволяет загружать листы по требованию. Например, так:

```
In [36]: with xlrd.open_workbook("xl/stores.xls", on_demand=True) as book:
          sheet = book.sheet_by_index(0) # Загружает только первый
          # лист
```

Если вы, как мы это делаем здесь, не будете использовать рабочую книгу в качестве контекстного менеджера, вам, чтобы правильно закрыть рабочую книгу, придется вызвать `book.release_resources()` вручную. Чтобы использовать xlrd в данном режиме с pandas, действуйте следующим образом:

```
In [37]: with xlrd.open_workbook("xl/stores.xls", on_demand=True) as book:
          with pd.ExcelFile(book, engine="xlrd") as f:
              df = pd.read_excel(f, sheet_name=0)
```

Чтение с помощью OpenPyXL

Чтобы при чтении больших файлов Excel с помощью OpenPyXL держать память под контролем, вам необходимо загружать рабочую книгу с `read_only=True`. Так как в OpenPyXL не поддерживается оператор `with`, вам следует убедиться, что по окончании вы снова закрыли файл. Если ваш файл содержит ссылки на внешние рабочие книги, вы, возможно, для ускорения работы захотите дополнительно использовать `keep_links=False`. `keep_links` обеспечивает сохранение ссылок на внешние рабочие книги, что, если вас интересует только чтение значений рабочей книги, может неоправданно замедлить процесс:

```
In [38]: book = openpyxl.load_workbook("xl/big.xlsx",
                                         data_only=True, read_only=True,
                                         keep_links=False)
          # Выполните необходимые операции чтения
          book.close() # Требуется при read_only=True
```

Параллельное чтение листов

Когда вы начнете использовать функцию pandas `read_excel` для чтения нескольких листов большой рабочей книги, вы увидите, что это занимает много времени (мы через некоторое время перейдем к конкретному примеру). Дело в том, что pandas читает листы последовательно, т. е. один за другим. Чтобы ускорить работу, можно читать листы параллельно. Хотя не существует простого способа распараллелить запись рабочих книг из-за внутренней структуры файлов, параллельное чтение нескольких листов достаточно просто организовать. Однако, поскольку распараллеливание — это довольно сложная тема и я оставил ее за рамками введения в Python, здесь я не буду вдаваться в подробности.

В Python, если вы хотите воспользоваться преимуществами нескольких ядер процессора, которые есть в каждом современном компьютере, вы используете пакет `multiprocessing`, который является частью стандартной библиотеки. Это приведет к

вызову нескольких интерпретаторов Python (обычно по одному на ядро процессора), которые будут работать над задачей параллельно. Вместо того чтобы обрабатывать один лист за другим, один интерпретатор Python обрабатывает первый лист, в то время как второй интерпретатор Python обрабатывает второй лист и т. д. Однако каждый дополнительный интерпретатор Python требует определенного времени на запуск и использует дополнительную память, поэтому если у вас небольшие файлы, то при распараллеливании процесса чтения они, скорее всего, будут работать медленнее, а не быстрее. В случае обработки большого файла с несколькими большими листами многопроцессорная обработка может значительно ускорить процесс, хотя только при условии, что в вашей системе имеется необходимый объем памяти для выполнения такой работы. Если вы запустите Jupyter notebook на Binder, как показано в *главе 2*, вам не хватит памяти, и, следовательно, распараллеленная версия будет работать медленнее. В сопутствующем репозитории вы найдете файл `parallel_pandas.py`, который является простой реализацией для параллельного чтения листов и использует OpenPyXL в качестве движка. Файл прост в использовании, поэтому вам не потребуется ничего знать о многопроцессорной обработке:

```
import parallel_pandas
parallel_pandas.read_excel(filename, sheet_name=None)
```

По умолчанию `parallel_pandas.py` будет считывать все листы, но вы можете указать список имен листов, которые вы хотите обработать. Как и `pandas`, он возвращает словарь в следующем виде: `{"имя листа": df}`, т. е. ключами являются имена листов, а значениями — `DataFrames`.

Магическая команда `%%time`

В следующих примерах я собираюсь использовать магическую ячейку `%%time`. Я познакомил вас с магическими командами в *главе 5*, рассказывая о Matplotlib. `%%time` — это магическая ячейка, которая может быть полезна для простой настройки производительности, поскольку она позволяет легко сравнить время выполнения двух ячеек с разными фрагментами кода. *Wall time* — это время, прошедшее от начала до конца исполнения программы, т. е. ячейки. Если вы работаете под управлением macOS или Linux, вы получите не только «Wall time», но и дополнительную строку для процессорного времени:

```
CPU times: user 49.4 s, sys: 108 ms, total: 49.5 s
```

`CPU times` измеряет время, затраченное на CPU, которое может быть меньше, чем «Wall time» (если программе приходится ждать, пока CPU освободится), или больше (если программа работает на нескольких ядрах CPU параллельно). Для более точного измерения времени используйте `%%timeit` вместо `%%time`, который запускает ячейку несколько раз и высчитывает среднее значение времени, затраченное на эти запуски. `%%time` и `%%timeit`, которые должны находиться в первой строке ячейки, будут измерять время работы всей ячейки. Если, вместо этого вы хотите измерить время работы только одной строки, начните ее с `%time` или `%timeit`.

Давайте посмотрим, насколько быстрее распараллеленная версия читает файл `big.xlsx`, который вы найдете в папке `x1` партнерского репозитория:

```
In [39]: %%time
          data = pd.read_excel("x1/big.xlsx",
                               sheet_name=None, engine="openpyxl")

Wall time: 49.5 s

In [40]: %%time
          import parallel_pandas
          data = parallel_pandas.read_excel("x1/big.xlsx", sheet_name=None)

Wall time: 12.1 s
```

Чтобы получить `DataFrame`, представляющий Лист1, в обоих случаях нужно написать `data["Sheet1"]`. Если посмотреть на время выполнения обоих примеров, то получится, что распараллеленная версия работает в несколько раз быстрее, чем `pd.read_excel`, с этой конкретной рабочей книгой и на моем ноутбуке с 6 ядрами CPU. Если вы хотите ускориться, распараллельте `OpenPyXL` напрямую: вы найдете эту реализацию в сопутствующем репозитории (`parallel_openpyxl.py`), а также реализацию для `xlrd` для параллельного чтения унаследованного формата `xls` (`parallel_xlrd.py`). Обращение к базовым пакетам вместо `pandas` позволит вам не выполнять преобразование в `DataFrame` или применять только те шаги очистки, которые вам необходимы, что, если это ваша главная задача, скорее всего, ускорит все процессы.

Параллельное чтение листа с помощью Modin

Если вы читаете только с одного огромного листа, стоит обратить внимание на `Modin`, проект, который выступает в качестве замены `pandas`. Он распараллеливает процесс чтения одного листа и обеспечивает значительное повышение скорости. Поскольку `Modin` требует определенной версии `pandas`, он при установке может обновить версию, установленную с `Anaconda`. Если вы хотите протестировать `Modin`, я бы рекомендовал вам создать для этого отдельную среду `Conda`, чтобы не испортить свою базовую среду. Более подробные инструкции по созданию среды `Conda` см. в *приложении А*:

```
(base)> conda create --name modin python=3.8 -y
(base)> conda activate modin
(modin)> conda install -c conda-forge modin -y
```

На моей машине при использовании файла `big.xlsx` выполнение следующего кода заняло около пяти секунд, а у `pandas` — около двенадцати секунд:

```
import modin.pandas
data = modin.pandas.read_excel("x1/big.xlsx",
                               sheet_name=0, engine="openpyxl")
```

Теперь, когда вы знаете, как работать с большими файлами, давайте продолжим и рассмотрим, как можно использовать совместно `pandas` и низкоуровневые пакеты, чтобы улучшить форматирование по умолчанию при записи `DataFrames` в файлы Excel!

Форматирование данных в Excel

Чтобы отформатировать DataFrames в Excel так, как нам нужно, мы можем написать код, который использует pandas вместе с OpenPyXL или XlsxWriter. Сначала мы применим эту комбинацию для добавления заголовка к экспортированному DataFrame, затем мы отформатируем заголовок и индекс DataFrame, после чего завершим этот раздел форматированием части данных DataFrame. Для чтения комбинация pandas с OpenPyXL может быть иногда полезной, поэтому давайте начнем с этого:

```
In [41]: with pd.ExcelFile("xl/stores.xlsx", engine="openpyxl") as xfile:
        # Чтение DataFrame
        df = pd.read_excel(xfile, sheet_name="2020")

        # Получить объект рабочей книги OpenPyXL
        book = xfile.book

        # С этого момента это код OpenPyXL
        sheet = book["2019"]
        value = sheet["B3"].value # Считывание одного значения
```

При написании рабочих книг это работает аналогично, позволяя нам легко добавить заголовок к нашему отчету DataFrame:

```
In [42]: with pd.ExcelWriter("pandas_and_openpyxl.xlsx",
        engine="openpyxl") as writer:
        df = pd.DataFrame({"col1": [1, 2, 3, 4], "col2": [5, 6, 7, 8]})
        # Запись DataFrame
        df.to_excel(writer, "Sheet1", startrow=4, startcol=2)

        # Получение объектов рабочей книги и листа OpenPyXL
        book = writer.book
        sheet = writer.sheets["Sheet1"]

        # С этого момента это код OpenPyXL
        sheet["A1"].value = "This is a Title" # Запись значения
        # одной ячейки
```

В этих примерах используется OpenPyXL, но концептуально он работает одинаково и с другими пакетами. Теперь продолжим выяснять, как можно отформатировать индекс и заголовок DataFrame.

Форматирование индексов и заголовков DataFrame

Самый простой способ получить полный контроль над форматированием индекса и заголовков колонок — это просто написать их самостоятельно. В приведенном ниже примере показано, как это сделать с помощью OpenPyXL и XlsxWriter соответственно (результат показан на рис. 8.2).

Давайте начнем с создания DataFrame:

```
In [43]: df = pd.DataFrame({"col1": [1, -2], "col2": [-3, 4]},
                             index=["row1", "row2"])
df.index.name = "ix"
df
```

```
Out[43]:
```

	col1	col2
ix		
row1	1	-3
row2	-2	4

Чтобы отформатировать индекс и заголовки с помощью OpenPyXL, сделайте следующее:

```
In [44]: from openpyxl.styles import PatternFill
In [45]: with pd.ExcelWriter("formatting_openpyxl.xlsx",
                             engine="openpyxl") as writer:
    # Запишите df с форматированием по умолчанию в A1
    df.to_excel(writer, startrow=0, startcol=0)

    # Запишите df с пользовательским форматированием
    # индексов/заголовков в A6
    startrow, startcol = 0, 5
    # 1. Запишите часть данных из DataFrame
    df.to_excel(writer, header=False, index=False,
                startrow=startrow + 1, startcol=startcol + 1)
    # Выберите объект листа и создайте объект стиля
    sheet = writer.sheets["Sheet1"]
    style = PatternFill(fgColor="D9D9D9", fill_type="solid")

    # 2. Запишите стилизованные заголовки столбцов
    for i, col in enumerate(df.columns):
        sheet.cell(row=startrow + 1, column=i + startcol + 2,
                  value=col).fill = style

    # 3. Запишите индекс стиля
    index = [df.index.name if df.index.name else None] + list(df.index)
    for i, row in enumerate(index):
        sheet.cell(row=i + startrow + 1, column=startcol + 1,
                  value=row).fill = style
```

Чтобы отформатировать индекс и заголовки с помощью XlsxWriter, необходимо немного изменить код:

```
In [46]: # Форматирование индекса/заголовков с помощью XlsxWriter
with pd.ExcelWriter("formatting_xlsxwriter.xlsx",
                    engine="xlsxwriter") as writer:
    # Запишите df с форматированием по умолчанию в A1
    df.to_excel(writer, startrow=0, startcol=0)
```

```

# Запишите df с пользовательским форматированием
# индексов/заголовков в A6
startrow, startcol = 0, 5
# 1. Запишите часть данных из DataFrame
df.to_excel(writer, header=False, index=False,
            startrow=startrow + 1, startcol=startcol + 1)
# Получите объект книги и листа и создайте объект стиля
book = writer.book
sheet = writer.sheets["Sheet1"]
style = book.add_format({"bg_color": "#D9D9D9"})

# 2. Запишите стилизованные заголовки столбцов
for i, col in enumerate(df.columns):
    sheet.write(startrow, startcol + i + 1, col, style)

# 3. Запишите индекс стиля
index = [df.index.name if df.index.name else None] + list(df.index)
for i, row in enumerate(index):
    sheet.write(startrow + i, startcol, row, style)

```

Когда индекс и заголовок отформатированы, давайте посмотрим, как мы можем отформатировать часть данных.

	A	B	C	D	E	F	G	H
1	ix	col1	col2			ix	col1	col2
2	row1	1	3			row1	1	3
3	row2	2	4			row2	2	4

Рис. 8.2. DataFrame с форматированием по умолчанию (слева) и с пользовательским форматированием (справа)

Форматирование части данных DataFrame

Возможности форматирования части данных в DataFrame зависят от используемого пакета: если вы используете метод pandas `to_excel`, OpenPyXL может применять формат к каждой ячейке, в то время как пакет XlsxWriter только на основании строки или столбца. Например, чтобы установить формат чисел в ячейках на три десятичных знака и выровнять содержимое по центру, как показано на рис. 8.3, сделайте в OpenPyXL следующее:

```

In [47]: from openpyxl.styles import Alignment
In [48]: with pd.ExcelWriter("data_format_openpyxl.xlsx",
                        engine="openpyxl") as writer:

    # Запишите DataFrame
    df.to_excel(writer)

    # Получите объекты книги и листа
    book = writer.book
    sheet = writer.sheets["Sheet1"]

```

```
# Форматирование отдельных ячеек
nrows, ncols = df.shape
for row in range(nrows):
    for col in range(ncols):
        # +1 для учета заголовка/индекса
        # +1, поскольку OpenPyXL основан на 1
        cell = sheet.cell(row=row + 2,
                           column=col + 2)
        cell.number_format = "0.000"
        cell.alignment = Alignment(horizontal="center")
```

Для XlsxWriter настройте код следующим образом:

```
In [49]: with pd.ExcelWriter("data_format_xlsxwriter.xlsx",
                             engine="xlsxwriter") as writer:

    # Запишите DataFrame
    df.to_excel(writer)

    # Получите объекты книги и листа
    book = writer.book
    sheet = writer.sheets["Sheet1"]

    # Форматирование столбцов (отдельные ячейки не могут быть
    # отформатированы)
    number_format = book.add_format({"num_format": "0.000",
                                      "align": "center"})
    sheet.set_column(first_col=1, last_col=2,
                     cell_format=number_format)
```

	A	B	C
1		col1	col2
2	row1	1.000	-3.000
3	row2	-2.000	4.000

Рис. 8.3. DataFrame с форматированной частью данных

В качестве альтернативы pandas предлагает экспериментальную поддержку свойства `style` для DataFrames. Экспериментальная, потому что синтаксис может измениться в любой момент времени. Поскольку стили были введены для форматирования DataFrames в формате HTML, они используют синтаксис CSS. CSS расшифровывается как Cascading Style Sheets (каскадные таблицы стилей) и используется для определения стилей элементов HTML. Чтобы применить тот же формат, что и в предыдущем примере (три десятичных знака и выравнивание по центру), нужно применить функцию к каждому элементу объекта `Styler` через `applymap`. Вы получаете объект `Styler` через атрибут `df.style`:

```
In [50]: df.style.applymap(lambda x: "number-format: 0.000;"
                             "text-align: center")\
        .to_excel("styled.xlsx")
```

Результат работы этого кода такой же, как показано на рис. 8.3. Для получения более подробной информации о методе стилизации DataFrame, пожалуйста, обрати-

тесь непосредственно к документации по стилизации⁸. Не прибегая к атрибуту `style`, `pandas` предлагает поддержку форматирования объектов `date` и `datetime`, как показано на рис. 8.4:

```
In [51]: df = pd.DataFrame({"Date": [dt.date(2020, 1, 1)],
                             "Datetime": [dt.datetime(2020, 1, 1, 10)]})
        with pd.ExcelWriter("date.xlsx",
                             date_format="yyyy-mm-dd",
                             datetime_format="yyyy-mm-dd hh:mm:ss") as writer:
            df.to_excel(writer)
```

	A	B	C
1		Date	Datetime
2	0	2020-01-01	2020-01-01 10:00:00

Рис. 8.4. DataFrame с отформатированными датами

Другие пакеты для reader и writer

Помимо пакетов, рассмотренных в этой главе, есть еще несколько, которые могут быть интересны для специфических случаев применения:

*pyexcel*⁹

Предлагает согласованный синтаксис для различных пакетов Excel и других форматов файлов, включая файлы CSV и файлы OpenOffice.

*PyExcelerate*¹⁰

Записывает файлы Excel самым быстрым из возможных способов.

*pylightxl*¹¹

Может читать файлы `xlsx` и `xlsm` и записывать файлы `xlsx`.

*styleframe*¹²

Использует `pandas` и `OpenPyXL` для создания файлов Excel с красиво отформатированным DataFrames.

*oletools*¹³

Не является классическим пакетом для чтения или записи, но может использоваться для анализа документов Microsoft Office, например для анализа вредоносных программ. Он предлагает удобный способ извлечения кода VBA из книг Excel.

⁸ http://oreil.ly/_JzFP. — Примеч. пер.

⁹ <http://pyexcel.org/>. — Примеч. пер.

¹⁰ <http://oreil.ly/yJax7>. — Примеч. пер.

¹¹ <http://oreil.ly/efjt4>. — Примеч. пер.

¹² <http://oreil.ly/nQUg9>. — Примеч. пер.

¹³ <http://oreil.ly/SG-Jy>. — Примеч. пер.

Теперь, когда вы знаете, как форматировать DataFrames в Excel, пришло время еще раз попробовать рассмотреть пример из предыдущей главы и посмотреть, сможем ли мы улучшить отчет Excel с помощью знаний, полученных в этой главе!

Тематическое исследование (повторное): отчетность в Excel

Дойдя до конца этой главы, вы знаете достаточно, чтобы вернуться к отчету Excel из примера, рассмотренного нами в прошлой главе, и сделать его визуально более привлекательным. Если хотите, вернитесь к файлу `sales_report_pandas.py` в партнерском репозитории и попытайтесь преобразовать его в отчет, как показано на рис. 8.5.

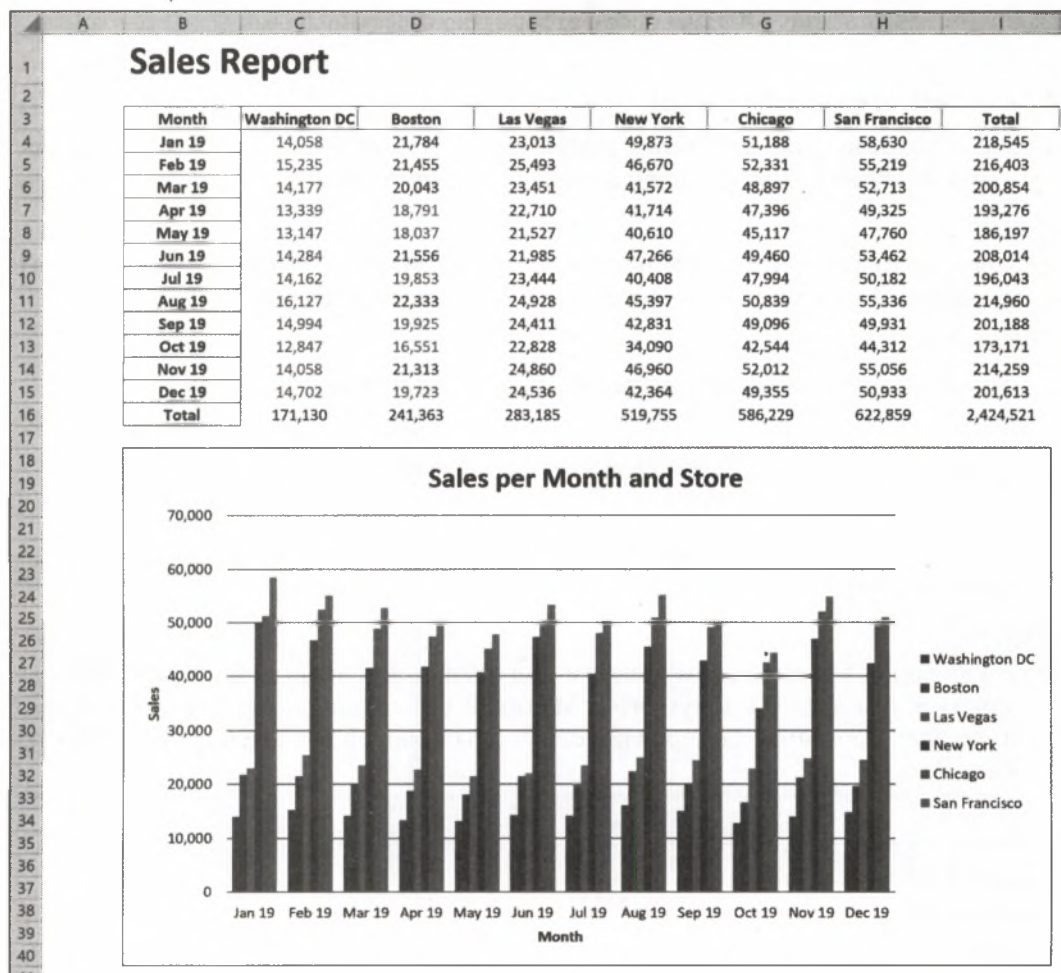


Рис. 8.5. Обновленный отчет о продажах, созданный `sales_report_openpyxl.py`

Красные цифры — это показатели продаж, которые ниже 20 000. В этой главе я затронул не все аспекты форматирования (например, как применять условное форматирование), поэтому вам придется воспользоваться документацией пакета, с которым вы решили работать. Чтобы сопоставить с вашим вариантом, я в сопутствующий репозиторий включил две версии сценария, которые создают этот отчет. Первая версия основана на OpenPyXL (`sales_report_openpyxl.py`), а вторая — на XlsxWriter (`sales_report_xlsxwriter.py`). Просмотрев два сценария, находящиеся рядом друг с другом, вы сможете принять обоснованное решение, какой пакет вы хотите выбрать для своей следующей задачи. Мы еще раз вернемся к этому примеру в следующей главе, где для работы с шаблонами отчетов воспользуемся установленным Microsoft Excel.

Заключение

В этой главе я познакомил вас с пакетами reader и writer, которые используются в pandas скрытно, под оболочкой. Их прямое использование позволяет нам читать и записывать рабочие книги Excel без необходимости установки pandas. В то же время их использование в сочетании с pandas позволяет нам улучшать отчеты Excel DataFrame, добавляя заголовки, диаграммы и форматирование. Хотя текущие пакеты reader и writer невероятно мощные, я все еще надеюсь, что однажды мы увидим NumPy, который объединит усилия всех разработчиков в единый проект. Было бы хорошо знать, какой пакет использовать, не заглядывая сначала в таблицу и не используя разный синтаксис для каждого типа файла Excel. В этом смысле имеет смысл начать с pandas и возвращаться к пакетам reader и writer только тогда, когда вам нужна дополнительная функциональность, которую pandas не обеспечивает.

Однако Excel — это гораздо больше, чем просто файл данных или отчет: приложение Excel — это один из самых интуитивно понятных пользовательских интерфейсов, в котором пользователь может ввести несколько цифр и получить на экране нужную информацию. Автоматизация приложения Excel вместо чтения и записи файлов Excel открывает целый ряд новых функциональных возможностей, которые мы рассмотрим в *части IV*. Следующая глава начинается этот путь, показывая, как управлять Excel из Python удаленно.

ПРОГРАММИРОВАНИЕ ПРИЛОЖЕНИЯ EXCEL С ПОМОЩЬЮ XLWINGS

Автоматизация Excel

На данный момент мы узнали, как выполнять типичные задачи Excel с помощью *pandas* (*часть II*) и как использовать файлы Excel в качестве источника данных и в качестве файлового формата для ваших отчетов (*часть III*). Эта глава — начало *части IV*, в которой мы для работы с *файлами* Excel не будем использовать пакеты чтения и записи (*reader* и *writer*) а начнем автоматизацию *приложения Excel* с помощью *xlwings*. Основное назначение *xlwings* — создание интерактивных приложений, в которых электронные таблицы Excel используются в качестве пользовательского интерфейса, позволяя вам вызывать Python нажатием кнопки или вызовом пользовательской функции — это тот тип функциональности, который отсутствует в пакетах *reader* и *writer*. Но это не значит, что *xlwings* не может использоваться для чтения и записи файлов, если вы работаете под macOS или Windows и у вас установлен Excel. Одним из преимуществ *xlwings* в этой области является возможность редактировать файлы Excel всех форматов без изменения или потери существующего содержимого или оформления. Другим преимуществом является то, что вы можете читать значения ячеек из рабочей книги Excel без необходимости ее предварительного сохранения. Тем не менее, как будет показано далее, совместное использование пакетов для чтения/записи Excel и *xlwings* также имеет смысл, в чем мы убедимся при повторном рассмотрении примера с отчетностью из *главы 7*.

Я начну эту главу со знакомства с объектной моделью Excel, а также с *xlwings*: сначала мы рассмотрим базовые операции, такие как подключение к рабочей книге или чтение и запись значений ячеек, а затем копнем немного глубже, чтобы понять, как конвертеры и опции позволяют нам работать с *pandas DataFrames* и массивами *NumPy*. И прежде чем перейти к последнему разделу, в котором объясняется, как *xlwings* работает «под оболочкой», мы рассмотрим, как взаимодействовать с графиками, изображениями и определенными именами: это даст вам необходимые знания, чтобы сделать ваши скрипты работоспособными, а также работать с отсутствующими функциями.

Начиная с этой главы, вам нужно будет запускать примеры кода под управлением операционной системой Windows или macOS, поскольку они зависят от локальной установки Microsoft Excel¹.

¹ В Windows вам потребуется как минимум Excel 2007, а в macOS — как минимум Excel 2016. В качестве альтернативы можно установить десктопную версию Excel, которая является частью подписки Microsoft 365. Подробную информацию о том, как это сделать, можно найти в подписке.

Начало работы с xlwings

Одна из целей xlwings — стать полноценной заменой VBA, позволяющей под управлением операционных систем Windows и macOS взаимодействовать с Excel из Python. Поскольку таблица Excel идеально подходит для отображения структур данных Python, таких как вложенные списки, массивы NumPy и pandas DataFrames, одна из основных функций xlwings — максимально упростить чтение/запись данных в Excel. Я начну этот раздел со знакомства с Excel как со средством просмотра данных — это полезно, когда вы взаимодействуете с DataFrames в блокноте Jupyter. Затем я объясню объектную модель Excel, после чего с помощью xlwings рассмотрю ее в интерактивном режиме. В завершение данного раздела я покажу вам, как вызывать код VBA, который, возможно, все еще хранится в старых рабочих книгах. Поскольку xlwings является частью Anaconda, нам не нужно устанавливать его вручную.

Использование Excel в качестве средства просмотра данных

Вероятно, в предыдущих главах вы заметили, что по умолчанию блокноты Jupyter скрывают большую часть данных крупных DataFrames и показывают только верхнюю и нижнюю строки, а также несколько первых и последних столбцов. Один из способов получить более полное представление о данных — построить, базируясь на этих данных, график, что позволяет выявить ошибки или другие отклонения от нормы. Иногда бывает очень полезно иметь возможность пролистать таблицу данных. После прочтения главы 7 вы знаете, как применять метод `to_excel` к вашему DataFrame. Хотя данный метод работает, его применение может быть довольно хлопотным: вам нужно дать файлу Excel имя, найти его в файловой системе, открыть его, а после внесения изменений в DataFrame требуется закрыть файл Excel и запустить весь процесс заново. Более правильным решением может быть выполнение `df.to_clipboard()`, который копирует DataFrame `df` в буфер обмена, что позволяет вставить его в Excel. Но есть еще более простой способ — использовать функцию `view`, которая поставляется с xlwings:

```
In [1]: # Сначала импортируем пакеты, которые будем использовать в этой
# главе
import datetime as dt
import xlwings as xw
import pandas as pd
import numpy as np

In [2]: # Давайте создадим DataFrame, основанный на псевдослучайных
# числах, с достаточным количеством строк, чтобы отображались
# только заголовок и конец таблицы
df = pd.DataFrame(data=np.random.randn(100, 5),
                  columns=[f"Trial {i}" for i in range(1, 6)])

df
```

```

Out[2]:      Trial 1   Trial 2   Trial 3   Trial 4   Trial 5
0  -1.313877  1.164258 -1.306419 -0.529533 -0.524978
1  -0.854415  0.022859 -0.246443 -0.229146 -0.005493
2  -0.327510 -0.492201 -1.353566 -1.229236  0.024385
3  -0.728083 -0.080525  0.628288 -0.382586 -0.590157
4  -1.227684  0.498541 -0.266466  0.297261 -1.297985
..      ***      ***      ***      ***      ***
95 -0.903446  1.103650  0.033915  0.336871  0.345999
96 -1.354898 -1.290954 -0.738396 -1.102659  0.115076
97 -0.070092 -0.416991 -0.203445 -0.686915 -1.163205
98 -1.201963  0.471854 -0.458501 -0.357171  1.954585
99  1.863610  0.214047 -1.426806  0.751906 -2.338352

[100 rows x 5 columns]
In [3]: # Просмотр DataFrame в Excel
        xw.view(df)

```

Функция `view` принимает все распространенные объекты Python, включая числа, строки, списки, словари, кортежи, массивы NumPy и pandas DataFrames. По умолчанию данная функция открывает новую рабочую книгу и вставляет объект в ячейку A1 первого листа. Она же с помощью функции Excel AutoFit регулирует ширину столбцов. И, чтобы каждый раз не открывать новую рабочую книгу, можно использовать повторно рабочую книгу, с которой работали ранее, предоставив функции просмотра объект листа `xlwings` в качестве второго аргумента: `xw.view(df, mysheet)`. О том, как получить доступ к такому объекту `sheet` и как он вписывается в объектную модель Excel, я расскажу далее².



macOS: Права доступа и параметры

В операционной системе macOS убедитесь, что блокноты Jupyter и VS Code запускаются из Anaconda Prompt (т. е. через терминал), как было показано в *главе 2*. Благодаря этому при первом использовании `xlwings` на экране появятся два всплывающих окна: первое — «Терминал получает доступ к управлению Событиями системы», а второе — «Терминалу нужен доступ к управлению Microsoft Excel». Вам нужно будет дать свое согласие на выполнение предлагаемых действий в обоих всплывающих окнах, чтобы разрешить Python автоматизировать Excel. Теоретически эти всплывающие окна должны вызываться в любом приложении, из которого вы запускаете код `xlwings`, но на практике это часто не так, поэтому запуск их через терминал уберет вас от неприятностей. Кроме того, необходимо зайти в меню **Параметры Excel** и в категории **Общие** снять флажок **Показывать галерею рабочей книги при открытии Excel**. В результате вместо того, чтобы сначала открывать галерею, которая будет мешать вам при открытии нового экземпляра Excel через `xlwings`, Excel откроет непосредственно пустую рабочую книгу.

² Обратите внимание, что в `xlwings` 0.22.0 появилась функция `xw.load`, которая похожа на `xw.view`, но работает в обратном направлении: она позволяет легко загрузить диапазон Excel в блокнот Jupyter в виде pandas DataFrame, см. документацию.

Объектная модель Excel

Когда вы работаете с Excel в режиме программирования, вы взаимодействуете с его компонентами, такими как рабочая книга или лист. Эти компоненты организованы в объектной модели Excel — иерархической структуре, которая представляет собой графический интерфейс пользователя Excel (рис. 9.1). Компания Microsoft в основном использует одну и ту же объектную модель во всех языках программирования, которые она официально поддерживает, будь то VBA, Office Scripts (интерфейс JavaScript для Excel в Интернете) или C#. В отличие от пакетов reader и writer, рассмотренных в главе 8, xlwings полностью повторяет объектную модель Excel, только с «глотком свежего воздуха»: xlwings, например, использует имена `app` вместо `application` и `book` вместо `workbook`:

- ◆ `app` содержит коллекцию `books`;
- ◆ `book` содержит коллекцию `sheets`;
- ◆ `sheet` предоставляет доступ к объектам `range` и коллекциям, таким как `charts`;
- ◆ `range` в качестве своих элементов содержит одну или несколько смежных ячеек.

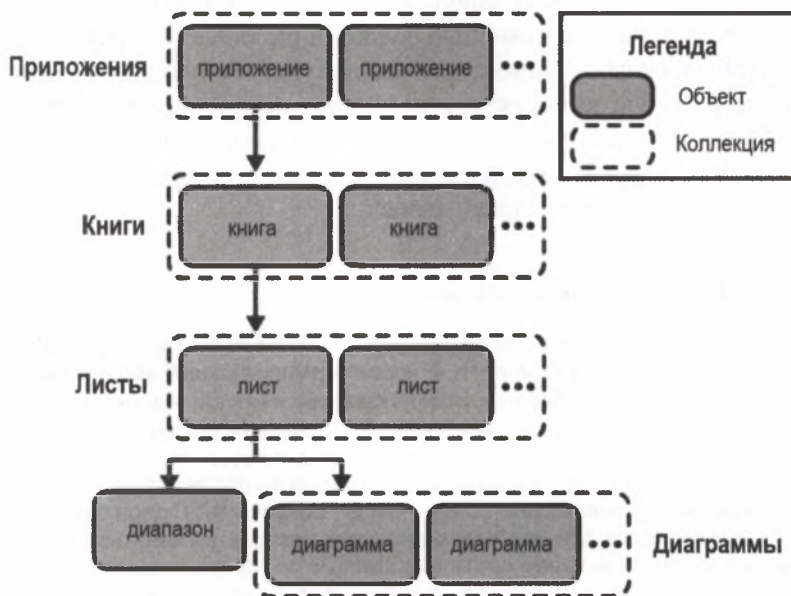


Рис. 9.1. Объектная модель Excel в реализации xlwings (фрагмент)

Пунктирные рамки являются *коллекциями* и содержат один или несколько объектов одного типа. Опытные пользователи иногда используют несколько экземпляров Excel параллельно, чтобы открыть одну и ту же рабочую книгу во втором экземпляре, например для параллельного расчета рабочей книги с различными исходными данными. В последних версиях Excel компания Microsoft немного усложнила процедуру открытия нескольких экземпляров Excel вручную: требуется запустить Excel, затем щелкнуть правой кнопкой мыши на значке этой программы на панели

задач Windows. Далее в появившемся меню, удерживая нажатой клавишу <Alt> (обязательно удерживайте клавишу <Alt> нажатой до тех пор, пока не отпустите кнопку мыши), щелкните левой кнопкой мыши на пункте Excel — появится всплывающее окно с вопросом, хотите ли вы запустить новый экземпляр Excel. В macOS не существует способа запустить вручную несколько экземпляров одной программы, но вы, как будет показано ниже, можете запустить несколько экземпляров Excel программно, с помощью xlwings. Подводя итог, можно сказать, что один экземпляр Excel представляет собой изолированную среду, то есть один экземпляр не может взаимодействовать с другим³. Объект sheet предоставляет доступ к таким коллекциям, как диаграммы, рисунки и определенные имена, — темы, которые мы рассмотрим во втором разделе этой главы.

Язык и региональные настройки

Эта книга основана на американско-английской версии Excel. Иногда я буду ссылаться на имена, предлагаемые по умолчанию, такие как «Book1» («Книга1» на рус.) или «Sheet1» («Лист1» на рус.), которые, если вы используете Excel на другом языке, будут отличаться от предложенных мною. Например, «Sheet1» на французском языке называется «Feuille1», а на испанском — «Hoja1». Кроме того, разделитель списка, который Excel использует в формулах ячеек, зависит от ваших настроек: я буду использовать запятую, но ваша версия может потребовать точку с запятой или другой символ. Например, вместо того, чтобы написать =SUM(A1, A2), на компьютере с немецкими региональными настройками, нужно написать =SUMME(A1; A2). В Windows, если вы хотите изменить разделитель списка с точки с запятой на запятую, вам нужно изменить его вне Excel через настройки Windows: нажмите на кнопку **Пуск**, найдите **Настройки** (или нажмите на значок шестеренки), затем выберите пункт **Время и язык > Регион и язык > Дополнительные параметры даты, времени и региона**, там выберите пункт **«Регион» > Изменить местоположение**. В разделе **Разделитель списка** вы сможете изменить разделитель с точки с запятой на запятую. Имейте в виду, что это работает только в том случае, если ваш «Десятичный символ» (в том же меню) не является запятой. Чтобы переопределить общесистемные десятичный и тысячный разделители (но не разделитель списка), в Excel перейдите в меню **Параметры > Дополнительно**, там в разделе **Параметры редактирования** вы найдете необходимые настройки.

В macOS это происходит примерно так же, за исключением того, что вы не можете напрямую изменить разделитель списка: в разделе **Системные настройки** вашей macOS (не Excel) выберите **Язык и регион**. И в этом разделе выберите конкретный регион либо глобально (на вкладке **Общие**), либо специально для Excel (на вкладке **Приложения**).

³ Дополнительные сведения об отдельных экземплярах Excel см. в статье «What are Excel instances, and why is this important?» на <https://oreil.ly/L2FDT>.

Чтобы получить представление об объектной модели Excel, как обычно, лучше всего поэкспериментировать с ней в интерактивном режиме. Начнем с класса `Book`: он позволяет создавать новые рабочие книги и подключаться к существующим. В табл. 9.1 дан общий обзор класса `Book`.

Таблица 9.1. Работа с рабочими книгами Excel

Команда	Описание
<code>xw.Book()</code>	Возвращает объект <code>book</code> в виде новой рабочей книги Excel и открывает ее в активном экземпляре Excel. Если активного экземпляра нет, будет запущен Excel
<code>xw.Book("Book1")</code>	Возвращает объект <code>book</code> , представляющий несохраненную рабочую книгу с именем <code>Book1</code> (имя отображается без расширения файла)
<code>xw.Book("Book1.xlsx")</code>	Возвращает объект <code>book</code> , представляющий ранее сохраненную рабочую книгу с именем <code>Book1.xlsx</code> (имя с расширением файла). Файл должен быть либо открыт, либо находиться в текущем рабочем каталоге
<code>xw.Book(r"C:\path\Book1.xlsx")</code>	Возвращает объект <code>book</code> ранее сохраненной рабочей книги (полный путь к файлу). Файл может быть открытым или закрытым. Символ <code>r</code> превращает строку в необработанную строку, поэтому обратные слэши (<code>\</code>) в пути интерпретируются буквально в Windows (я представил необработанные строки в главе 5). В macOS <code>r</code> не требуется, так как в путях к файлам используются прямые косые черты вместо обратных косых черт
<code>xw.books.active</code>	Возвращает объект <code>book</code> , представляющий активную рабочую книгу в активном экземпляре Excel

Давайте посмотрим, как мы можем пройти по иерархии объектной модели от объекта `book` до объекта `range`:

```
In [4]: # Создайте новую пустую рабочую книгу и введите ее имя.
        # Именно эту книгу мы будем использовать для выполнения
        # большинства примеров кода в этой главе.
        book = xw.Book()
        book.name

Out[4]: 'Book2'

In [5]: # Доступ к коллекции листов
        book.sheets

Out[5]: Sheets([<Sheet {Book2}Sheet1>])

In [6]: # Получение объекта Sheet по индексу или имени. Вам нужно будет
        # настроить "Sheet1", если ваш лист называется по-другому.
```

```

sheet1 = book.sheets[0]
sheet1 = book.sheets["Sheet1"]
In [7]: sheet1.range("A1")
Out[7]: <Range [Book2]Sheet1!$A$1>

```

С объектом `range` мы добрались до самого низа иерархии. Строка, которая выводится между угловыми скобками, дает вам полезную информацию об этом объекте, но чтобы что-то сделать, как показано в следующем примере, вы обычно используете объект с атрибутом:

```

In [8]: # Наиболее распространенные задачи: написать значения...
        sheet1.range("A1").value = [[1, 2],
                                     [3, 4]]
        sheet1.range("A4").value = "Hello!"
In [9]: # ...и прочитать значения
        sheet1.range("A1:B2").value
Out[9]: [[1.0, 2.0], [3.0, 4.0]]
In [10]: sheet1.range("A4").value
Out[10]: 'Hello!'

```

Как видите, по умолчанию атрибут `value` объекта `range xlwings` принимает и возвращает вложенный список для двумерных диапазонов и скаляр для одной ячейки. Все, что мы использовали до сих пор, практически идентично VBA: если предположить, что `book` — это объект рабочей книги VBA или `xlwings` соответственно, то таким образом можно получить доступ к атрибуту `value` из ячеек A1–B2 в VBA и в `xlwings`:

```

book.Sheets(1).Range("A1:B2").Value # VBA
book.sheets[0].range("A1:B2").value # xlwings

```

Различия заключаются в следующем:

Атрибуты

Python использует строчные буквы, возможно с подчеркиванием, как предложено в PEP 8, руководстве по стилю Python. Данное руководство было представлено в главе 3.

Индексация

Python для доступа к элементу в коллекции `sheets` использует квадратные скобки, и индексы, основанные на нулях.

В табл. 9.2 приведен обзор строк, которые принимает `range` (диапазон) `xlwings`.

Таблица 9.2. Строки для определения диапазона в нотации A1

Адрес	Описание	Адрес	Описание
"A1"	Одна ячейка	"A:B"	Столбцы A–B
"A1:B2"	Ячейки от A1 до B2	"1:1"	Строка 1
"A:A"	Столбец A	"1:2"	Строки 1 - 2

Индексирование и нарезка работают с объектами `range` `xlwings` — посмотрите на адрес между угловыми скобками (печатное представление объекта), чтобы увидеть, какой диапазон ячеек вы получите в итоге:

```
In [11]: # Индексирование
        sheet1.range("A1:B2")[0, 0]
Out[11]: <Range [Book2]Sheet1!$A$1>
In [12]: # Нарезка
        sheet1.range("A1:B2")[:, 1]
Out[12]: <Range [Book2]Sheet1!$B$1:$B$2>
```

Индексирование соответствует использованию свойства `Cells` в VBA:

```
book.Sheets(1).Range("A1:B2").Cells(1, 1) # VBA
book.sheets[0].range("A1:B2")[0, 0] # xlwings
```

Вместо того чтобы использовать диапазон явно в качестве атрибута объекта листа, вы также можете получить объект диапазона путем индексирования и нарезки объекта листа. Использование нотации `A1` позволит вам набирать меньше текста, а использование целочисленных индексов делает лист Excel похожим на массив NumPy:

```
In [13]: # Одна ячейка: Условное обозначение A1
        sheet1["A1"]
Out[13]: <Range [Book2]Sheet1!$A$1>
In [14]: # Несколько ячеек: Условное обозначение A1
        sheet1["A1:B2"]
Out[14]: <Range [Book2]Sheet1!$A$1:$B$2>
In [15]: # Отдельная ячейка: индексация
        sheet1[0, 0]
Out[15]: <Range [Book2]Sheet1!$A$1>
In [16]: # Несколько ячеек: нарезка
        sheet1[:, 2]
Out[16]: <Range [Book2]Sheet1!$A$1:$B$2>
```

Тем не менее, иногда может оказаться более понятным определить диапазон, указав левую верхнюю и правую нижнюю ячейки диапазона. Представленные ниже примеры относятся к диапазонам ячеек `D10` и `D10:F11` соответственно, что позволяет понять разницу между индексированием/нарезкой объекта листа и работой с объектом диапазона:

```
In [17]: # D10 через индексацию листов
        sheet1[9, 3]
Out[17]: <Range [Book2]Sheet1!$D$10>
In [18]: # D10 через объект диапазона
        sheet1.range((10, 4))
Out[18]: <Range [Book2]Sheet1!$D$10>
In [19]: # D10:F11 через нарезку листов
        sheet1[9:11, 3:6]
Out[19]: <Range [Book2]Sheet1!$D$10:$F$11>
In [20]: # D10:F11 через объект диапазона
        sheet1.range((10, 4), (11, 6))
Out[20]: <Range [Book2]Sheet1!$D$10:$F$11>
```

Определение объектов диапазона с помощью кортежей очень напоминает работу свойства `Cells` в VBA, как показано в следующем сравнении — здесь снова предполагается, что `book` — это либо объект рабочей книги VBA, либо объект `book xlwings`. Для начала рассмотрим версию VBA:

```
With book.Sheets(1)
    myrange = .Range(.Cells(10, 4), .Cells(11, 6))
End With
```

Это эквивалентно следующему выражению `xlwings`:

```
myrange = book.sheets[0].range((10, 4), (11, 6))
```



Нулевые индексы против индексов, основанных на единице

Как и пакет Python, когда вы обращаетесь к элементам с помощью синтаксиса Python `index` или `slice`, то есть через квадратные скобки, так и `xlwings` всякий раз последовательно использует индексацию на основе нуля. Однако объекты `xlwings` `range` используют индексы строк и столбцов Excel, основанные на единице. Иногда полезно иметь те же индексы строк/столбцов, что и в пользовательском интерфейсе Excel. Если вы предпочитаете только нулевую индексацию Python, просто используйте синтаксис `sheet[row_selection, column_selection]`.

В следующем примере показано, как перейти от объекта `range(sheet1["A1"])` к объекту `app`. Помните, что объект `app` представляет экземпляр Excel (вывод между угловыми скобками означает идентификатор процесса Excel и поэтому будет отличаться на вашей машине):

```
In [21]: sheet1["A1"].sheet.book.app
Out[21]: <Excel App 9092>
```

После того как мы добрались до самой вершины объектной модели Excel, настал подходящий момент посмотреть, как можно работать с несколькими экземплярами Excel. Вам нужно будет явно использовать объект `app`, если вы хотите открыть одну и ту же рабочую книгу в нескольких экземплярах Excel или если вы хотите распределить рабочие книги по разным экземплярам в целях повышения производительности. Еще один распространенный случай использования объекта `app` — открытие рабочей книги в скрытом экземпляре Excel: это позволяет запустить сценарий `xlwings` в фоновом режиме, не мешая одновременно выполнять другую работу в Excel:

```
In [22]: # Получить один объект приложения из открытой рабочей книги и
        # создать дополнительный фоновый экземпляр приложения
        visible_app = sheet1.book.app
        invisible_app = xw.App(visible=False)

In [23]: # Перечислить имена книг, которые открыты в каждом экземпляре,
        # просмотрев весь список
        [book.name for book in visible_app.books]

Out[23]: ['Book1', 'Book2']

In [24]: [book.name for book in invisible_app.books]

Out[24]: ['Book3']
```

```

In [25]: # Ключ приложения обозначает идентификатор процесса (PID).
         xw.apps.keys()
Out[25]: [5996, 9092]
In [26]: # Доступ к нему также можно получить через атрибут pid
         xw.apps.active.pid
Out[26]: 5996
In [27]: # Работа с книгой в скрытом экземпляре Excel
         invisible_book = invisible_app.books[0]
         invisible_book.sheets[0]["A1"].value = "Created by an invisible app."
In [28]: # Сохранить рабочую книгу Excel в каталоге xl
         invisible_book.save("xl/invisible.xlsx")
In [29]: # Выход из скрытого экземпляра Excel
         invisible_app.quit()

```



macOS: программный доступ к файловой системе

Если вы выполните в операционной системе macOS команду `save`, в Excel появится всплывающее окно **Grant File Access**, в котором, прежде чем выбрать **Grant Access**, вам нужно будет подтвердить свои действия, нажав кнопку **Select**. В macOS Excel находится в «песочнице», и это означает, что ваша программа может получить доступ к файлам и папкам только за пределами приложения Excel. А для этого следует подтвердить свои действия в окне **Grant File Access**. После подтверждения Excel запомнит указанные местоположения и при следующем запуске сценария больше не будет беспокоить вас.

Если одна и та же рабочая книга открыта в двух экземплярах Excel, или если вы хотите указать, в каком экземпляре Excel открывать рабочую книгу, вам больше не следует использовать `xw.Book`. Вместо этого нужно использовать коллекцию `books`, как показано в табл. 9.3. Обратите внимание, что `myapp` означает объект `app` в `xlwings`. Если вы замените `myapp.books` на `xw.books`, `xlwings` будет использовать активный `app`.

Таблица 9.3. Работа с коллекцией `books`

Команда	Описание
<code>myapp.books.add()</code>	Создает новую рабочую книгу Excel в экземпляре Excel, на который ссылается <code>myapp</code> , и возвращает соответствующий объект <code>book</code>
<code>myapp.books.open(r"C:\path\Book.xlsx")</code>	Возвращает <code>book</code> , если он уже открыт, в ином случае открывает его первым в экземпляре Excel, на который ссылается <code>myapp</code> . Не забывайте, что символ <code>r</code> превращает путь к файлу в необработанную строку, что позволяет интерпретировать обратные слэши буквально
<code>myapp.books["Book1.xlsx"]</code>	Возвращает объект <code>book</code> , если он открыт. Если объект <code>book</code> еще не открыт, будет выдана ошибка <code>KeyError</code> . Убедитесь, что при открытии файла используется имя, а не полный путь. Используйте эту функцию, если вам нужно узнать, открыта ли рабочая книга в Excel

Прежде чем углубиться в изучение, как xlwings может заменить ваши макросы VBA, давайте посмотрим, как xlwings может взаимодействовать с существующим кодом VBA: это может быть полезно, если у вас много устаревшего кода и нет времени на перенос всего на Python.

Запуск кода VBA

Если у вас сохранились старые проекты Excel с большим количеством кода VBA, перенос всех этих проектов на Python может потребовать больших трудозатрат. В этом случае для запуска макросов VBA вы можете задействовать Python. В следующем примере используется файл vba.xlsm, который вы найдете в папке xl сопутствующего репозитория, где в модуле Module1 содержится следующий код:

```
Function MySum(x As Double, y As Double) As Double
    MySum = x + y
End Function
Sub ShowMsgBox(msg As String)
    MsgBox msg
End Sub
```

Чтобы вызвать эти функции через Python, сначала нужно создать макрообъект xlwings, который при последующих вызовах создает впечатление, что это собственная функция Python:

```
In [30]: vba_book = xw.Book("xl/vba.xlsm")
In [31]: # Создайте объект макроса с помощью функции VBA
        mysum = vba_book.macro("Module1.MySum")
        # Вызов функции VBA
        mysum(5, 4)
Out[31]: 9.0
In [32]: # То же самое можно сделать с помощью процедуры VBA Sub
        show_msgbox = vba_book.macro("Module1.ShowMsgBox")
        show_msgbox("Hello xlwings!")
In [33]: # Снова закройте книгу (не забудьте сначала закрыть MessageBox)
        vba_book.close()
```



Не храните функции VBA в модулях Sheet и ThisWorkbook

Если вы храните функцию VBA MySum в модуле рабочей книги This Workbook или в модуле листа (например, Sheet1), вы должны обращаться к ней как ThisWorkbook.MySum или Sheet1.MySum. Однако поскольку вы не сможете получить доступ к возвращаемому значению функции из Python, обязательно храните функции VBA в стандартном модуле кода VBA, который вы вставляете, щелкнув правой кнопкой мыши на папке Modules в редакторе VBA.

Теперь, когда вы знаете, как взаимодействовать с существующим кодом VBA, мы можем продолжить изучение xlwings, рассмотрев, как использовать его с DataFrames, массивами NumPy и коллекциями, такими как графики, изображения и определенные имена.

Конвертеры, опции и коллекции

Во вступительных примерах кода этой главы мы уже как считывали, так и записывали строку и вложенный список как из Excel, так и в Excel с помощью атрибута `value` объекта `range` `xlwings`. Я начну этот раздел с демонстрации того, как эти операции выполняются с `pandas DataFrames`, а затем более подробно рассмотрю метод `options`, который позволяет нам влиять на способ чтения и записи значений `xlwings`. Мы переходим к диаграммам, картинкам и определенным именам — коллекциям, к которым вы обычно обращаетесь из объекта `sheet`. Получив базовые знания по `xlwings`, мы еще раз рассмотрим пример с отчетностью из главы 7.

Работа с DataFrames

Запись `DataFrame` в Excel ничем не отличается от записи скаляра или вложенного списка в Excel: просто поместите `DataFrame` в левую верхнюю ячейку диапазона Excel:

```
In [34]: data=[["Mark", 55, "Italy", 4.5, "Europe"],
               ["John", 33, "USA", 6.7, "America"]]
df = pd.DataFrame(data=data,
                  columns=["name", "age", "country",
                          "score", "continent"],
                  index=[1001, 1000])
df.index.name = "user_id"
df
```

```
Out[34]:
```

	name	age	country	score	continent
user_id					
1001	Mark	55	Italy	4.5	Europe
1000	John	33	USA	6.7	America

```
In [35]: sheet1["A6"].value = df
```

Если, однако, вы хотите скрыть заголовки столбцов и/или индекс, используйте метод `options` следующим образом:

```
In [36]: sheet1["B10"].options(header=False, index=False).value = df
```

Чтение диапазонов Excel в виде `DataFrame` требует предоставления класса `DataFrame` в качестве параметра `convert` в методе `options`. По умолчанию ожидается, что ваши данные имеют заголовок и индекс, но вы можете использовать параметры `index` и `header`, чтобы изменить это. Вместо использования конвертера вы также можете сначала считать значения в виде вложенного списка, а затем вручную построить свой `DataFrame`, но использование конвертера значительно упрощает работу с индексом и заголовком.



Метод расширения

В следующем примере кода я представляю метод `expand`, который позволяет легко читать непрерывный блок ячеек, обеспечивая тот же диапазон, как если бы вы

использовали комбинацию клавиш <Shift>+<Ctrl>+<Down-Arrow>+<Right-Arrow> в Excel, за исключением того, что expand перемещается через пустую ячейку в левом верхнем углу.

```
In [37]: df2 = sheet1["A6"].expand().options(pd.DataFrame).value
df2
Out[37]:
```

	name	age	country	score	continent
user_id					
1001.0	Mark	55.0	Italy	4.5	Europe
1000.0	John	33.0	USA	6.7	America

```
In [38]: # Если вы хотите, чтобы индекс был целочисленным, можете
# изменить тип его данных
df2.index = df2.index.astype(int)
df2
Out[38]:
```

	name	age	country	score	continent
1001	Mark	55.0	Italy	4.5	Europe
1000	John	33.0	USA	6.7	America

```
In [39]: # Если установить index=False, то все значения
# попадут в часть DataFrame и будет использована функция
sheet1["A6"].expand().options(pd.DataFrame, index=False).value
Out[39]:
```

	user_id	name	age	country	score	continent
0	1001.0	Mark	55.0	Italy	4.5	Europe
1	1000.0	John	33.0	USA	6.7	America

Чтение и запись DataFrames стали первым примером того, как работают конвертеры и опции. Как они формально определяются и как использовать их с другими структурами данных, мы рассмотрим далее.

Конвертеры и опции

Как мы только что видели, метод options объекта range xlwings позволяет вам влиять на способ считывания и записи значений как из Excel, так и в Excel. То есть, options анализируются только при вызове атрибута value объекта range. Синтаксис следующий (myrange — объект range xlwings):

```
myrange.options(convert=None, option1=value1, option2=value2, ...).value
```

В табл. 9.4 показаны встроенные конвертеры, т. е. значения, которые принимает аргумент convert. Они называются *встроенными*, поскольку xlwings предлагает способ написания собственных конвертеров. Это может быть полезно, если вам приходится многократно применять дополнительные преобразования перед записью или после чтения значений — чтобы узнать, как это работает, загляните в xlwings-документацию.

Таблица 9.4. Встроенные конвертеры

Конвертер	Описание
dict	Простые словари без вложений, т. е. в форме {key1: value1, key2: value2, ...}

Таблица 9.4 (окончание)

Конвертер	Описание
np.array	Массивы NumPy, необходим <code>import numpy as np</code>
pd.Series	pandas Series, необходим <code>import pandas as pd</code>
pd.DataFrame	DataFrame pandas, необходим <code>import pandas as pd</code>

Мы уже использовали опции `index` и `header` в примере DataFrame, но есть, как показано в табл. 9.5, и другие опции.

Таблица 9.5. Встроенные опции

Опция	Описание
empty	По умолчанию пустые ячейки считываются как None. Чтобы изменить это условие, укажите значение для <code>empty</code>
date	Принимает функцию, которая применяется к значениям из ячеек, отформатированных по дате
number	Принимает функцию, которая применяется к числам
ndim	<i>Количество размерностей</i> : при чтении используйте <code>ndim</code> , чтобы значения диапазона поступали в определенной размерности. Должно быть либо None, либо 1, либо 2. Может использоваться при чтении значений в виде списков или массивов NumPy
transpose	Транспонирует значения, т. е. превращает столбцы в строки или наоборот
index	Используется с pandas DataFrames и Series: при чтении используйте этот параметр, чтобы определить, содержит ли диапазон Excel данный индекс. Может принимать значения True/False или целое число. Целое число определяет, сколько столбцов должно быть превращено в мультииндекс. Например, 2 будет использовать два крайних левых столбца в качестве индекса. При записи можно решить, нужно ли записывать индекс, задав для параметра <code>index</code> значение True или False
header	Работает так же, как и индекс, но применяется к заголовкам столбцов

Познакомимся с `ndim` поближе: по умолчанию, когда вы читаете из Excel одну ячейку, вы получаете скаляр (например, float или строку); когда вы читаете столбец или строку, вы получаете простой список; и, наконец, когда вы читаете двумерный диапазон, вы получаете вложенный (т. е. двумерный) список. Такой подход не только последователен сам по себе, но и эквивалентен тому, как нарезка работает с массивами NumPy, что было показано в главе 4. Одномерный вариант — это особый случай: иногда столбец может быть просто крайним в двумерном диапазоне.

В этом случае имеет смысл принудительно указать диапазон в виде двухмерного списка, используя `ndim=2`:

```
In [40]: # Горизонтальный диапазон (одномерный)
         sheet1["A1:B1"].value
Out[40]: [1.0, 2.0]
In [41]: # Вертикальный диапазон (одномерный)
         sheet1["A1:A2"].value
Out[41]: [1.0, 3.0]
In [42]: # Горизонтальный диапазон (двухмерный)
         sheet1["A1:B1"].options(ndim=2).value
Out[42]: [[1.0, 2.0]]
In [43]: # Вертикальный диапазон (двухмерный)
         sheet1["A1:A2"].options(ndim=2).value
Out[43]: [[1.0], [3.0]]
In [44]: # Использование конвертера массивов NumPy ведет себя так же:
         # вертикальный диапазон обеспечивает одномерный массив
         sheet1["A1:A2"].options(np.array).value
Out[44]: array([1., 3.])
In [45]: # Сохранение ориентации столбцов
         sheet1["A1:A2"].options(np.array, ndim=2).value
Out[45]: array([[1.],
                [3.]])
In [46]: # Если вам нужно выписать список по вертикали,
         # пригодится опция "transpose".
         sheet1["D1"].options(transpose=True).value = [100, 200]
```

Используйте `ndim=1`, чтобы заставить значение одной ячейки считываться как список, а не как скаляр. В pandas вам не понадобится `ndim`, поскольку `DataFrame` всегда двумерный, а `Series` всегда одномерный. Вот еще один пример, показывающий, как работают опции `empty`, `date`, и `number`:

```
In [47]: # Записать некоторые выборочные данные
         sheet1["A13"].value = [dt.datetime(2020, 1, 1), None, 1.0]
In [48]: # Прочитать данные, используя параметры по умолчанию
         sheet1["A13:C13"].value
Out[48]: [datetime.datetime(2020, 1, 1, 0, 0), None, 1.0]
In [49]: # Прочитать данные повторно, используя опции,
         # не предусмотренные по умолчанию
         sheet1["A13:C13"].options(empty="NA",
                                   dates=dt.date,
                                   numbers=int).value
Out[49]: [datetime.date(2020, 1, 1), 'NA', 1]
```

До сих пор мы работали с объектами `book`, `sheet` и `range`. Теперь давайте перейдем к изучению того, как работать с коллекциями, такими как диаграммы, к которым вы получаете доступ из объекта `sheet`!

Диаграммы, рисунки и определенные имена

В этом разделе я покажу вам, как работать с тремя коллекциями, доступ к которым осуществляется через объект `sheet` или `book`: диаграммы, рисунки и определенные имена⁴. `xlwings` поддерживает только основные функции диаграмм, но поскольку вы можете работать с шаблонами, вы потеряли не много. А чтобы компенсировать этот недостаток, `xlwings` дает возможность встраивать графики `Matplotlib` в виде картинок. Вы наверняка помните из *главы 5*, что `Matplotlib` по умолчанию является сервером `pandas` для построения графиков. Давайте начнем с создания первой диаграммы Excel!

Диаграммы Excel

Чтобы добавить новый график, используйте метод `add` коллекции `charts`, а затем установите тип графика и исходные данные:

```
In [50]: sheet1["A15"].value = [[None, "North", "South"],
                                ["Last Year", 2, 5],
                                ["This Year", 3, 6]]

In [51]: chart = sheet1.charts.add(top=sheet1["A19"].top,
                                   left=sheet1["A19"].left)
        chart.chart_type = "column_clustered"
        chart.set_source_data(sheet1["A15"].expand())
```

В результате получится график, показанный в левой части рис. 9.2. Чтобы узнать о доступных типах графиков, загляните в документацию `xlwings`. Если вам нравится работать с графиками `pandas` больше, чем с диаграммами Excel, или если вы хотите использовать тип диаграммы, недоступный в Excel, `xlwings` поможет вам — давайте посмотрим, как это сделать!

Изображения: Графики Matplotlib

Когда вы используете стандартный сервер `pandas` для построения графиков, вы создаете график `Matplotlib`. Чтобы перенести такой график в Excel, вам сначала нужно получить объект `figure`, который вы предоставляете в качестве аргумента для `pictures.add`, — это позволит преобразовать график в рисунок и отправить его в Excel:

```
In [52]: # Считывание данных графика как DataFrame
        df = sheet1["A15"].expand().options(pd.DataFrame).value
        df

Out[52]:      North  South
Last Year    2.0    5.0
This Year    3.0    6.0

In [53]: # Включите Matplotlib с помощью команды notebook magic
        # и переключитесь на стиль "seaborn"
```

⁴ Еще одна популярная коллекция — таблицы. Для их использования вам потребуется версия `xlwings` 0.21.0; см. документацию.

```
%matplotlib inline
import matplotlib.pyplot as plt
plt.style.use("seaborn")
```

```
In [54]: # Метод pandas plot возвращает объект "axis",
# из которого можно получить рисунок. "T" трансформирует
# DataFrame, чтобы придать диаграмме нужную ориентацию
ax = df.T.plot.bar()
fig = ax.get_figure()
```

```
In [55]: # Отправить диаграмму в Excel
plot = sheet1.pictures.add(fig, name="SalesPlot",
                           top=sheet1["H19"].top,
                           left=sheet1["H19"].left)

# Масштабирование диаграммы до 70%
plot.width, plot.height = plot.width * 0.7, plot.height * 0.7
```

Чтобы обновить рисунок новым сюжетом, просто используйте метод `update` с другим объектом `figure` — технически это заменит рисунок в Excel, но сохранит все свойства, такие как расположение, размер и имя:

```
In [56]: ax = (df + 1).T.plot.bar()
plot = plot.update(ax.get_figure())
```

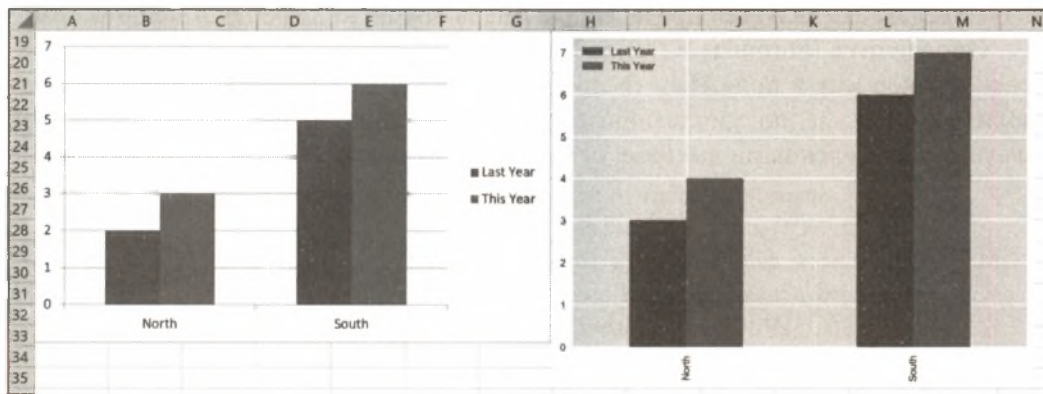


Рис. 9.2. Диаграмма Excel (слева) и график Matplotlib (справа)

На рис. 9.2 показано сравнение диаграммы Excel и графика Matplotlib после выполнения обновления.



Убедитесь, что Pillow установлена

При работе с изображениями убедитесь, что библиотека Python для работы с изображениями Pillow установлена: данная библиотека обеспечит правильный размер и пропорции картинок в Excel. Pillow является частью Anaconda, поэтому если вы используете другой дистрибутив, вам нужно будет установить его, выполнив `conda install pillow` или `pip install pillow`. Обратите внимание, что `pictures.add` также получает путь к изображению на диске вместо рисунка Matplotlib.

Диаграммы и рисунки — это коллекции, доступ к которым осуществляется через объект `sheet`. Определенные имена и коллекция, которую мы рассмотрим далее, могут быть доступны как из объекта `sheet`, так и из объекта `book`. Давайте посмотрим, в чем отличие!

Определенные имена

В Excel вы создаете *определенное имя*, присваивая его диапазону, формуле или константе⁵. Присвоение имени диапазону, вероятно, является наиболее распространенным случаем и называется *именованным диапазоном*. При использовании именованного диапазона вы можете ссылаться на диапазон Excel в формулах и коде, используя описательное имя, а не абстрактный адрес в виде `A1:B2`. Использование таких имен с `xlwings` делает ваш код более гибким и надежным: чтение и запись значений из именованных диапазонов и в именованные диапазоны дает вам гибкость при реструктуризации рабочей книги без необходимости корректировать код Python: имя сохраняется за ячейкой, даже если вы перемещаете ее, например вставляя новый ряд. Определенные имена могут быть установлены как в глобальной области значений книги, так и в локальной области значений листа. Преимущество имен с областью действия листа заключается в том, что вы можете копировать лист, не сталкиваясь с конфликтами с дублирующимися именованными диапазонами. В Excel вы добавляете определенные имена вручную, перейдя в меню **Формулы > Определить** (`Formulas > Define Name`) имя или выбрать диапазон, а затем написать нужное имя в поле **Имя** (`Name`) — это текстовое поле находится слева от строки формул, где по умолчанию отображается адрес ячейки. Вот как можно управлять определенными именами с помощью `xlwings`:

```
In [57]: # Область применения книги — это область применения по умолчанию
        sheet1["A1:B2"].name = "matrix1"

In [58]: # Для области действия листа перед именем листа
        # поставьте восклицательный знак
        sheet1["B10:E11"].name = "Sheet1!matrix2"

In [59]: # Теперь вы можете получить доступ к диапазону по имени
        sheet1["matrix1"]

Out[59]: <Range [Book2]Sheet1!$A$1:$B$2>

In [60]: # Если вы обращаетесь к коллекции имен через объект "sheet1",
        # в ней содержатся только имена с областью обзора этого листа
        sheet1.names

Out[60]: [<Name 'Sheet1!matrix2': =Sheet1!$B$10:$E$11>]

In [61]: # Если вы обращаетесь к коллекции имен через объект "book",
        # в ней содержатся все имена, включая область действия book
        # и sheet
        book.names
```

⁵ Определенные имена с формулами также используются для лямбда-функций — нового способа определения пользовательских функций без VBA или JavaScript, о котором в декабре 2020 года Microsoft объявила как о новой функции для подписчиков Microsoft 365.

```
Out[61]: [<Name 'matrix1': =Sheet1!$A$1:$B$2>, <Name 'Sheet1!matrix2':  
        =Sheet1!$B$10:$E$11>]
```

```
In [62]: # Имена включают в себя различные методы и атрибуты.  
        # Вы можете, например, получить соответствующий объект  
        # диапазона.  
        book.names["matrix1"].refers_to_range
```

```
Out[62]: <Range [Book2]Sheet1!$A$1:$B$2>
```

```
In [63]: # Если вы хотите присвоить имя константе  
        # или формуле, используйте метод "add".  
        book.names.add("EURUSD", "=1.1151")
```

```
Out[63]: <Name 'EURUSD': =1.1151>
```

Взгляните на созданные определенные имена в Excel, открыв менеджер имен через **Формулы > Менеджер имен** (Formulas > Name Manager) (рис. 9.3). Обратите внимание, что в macOS Excel не имеет менеджера имен, поэтому перейдите в меню **Формулы > Определить имя** (Formulas > Define Name), где отобразятся имеющиеся имена.

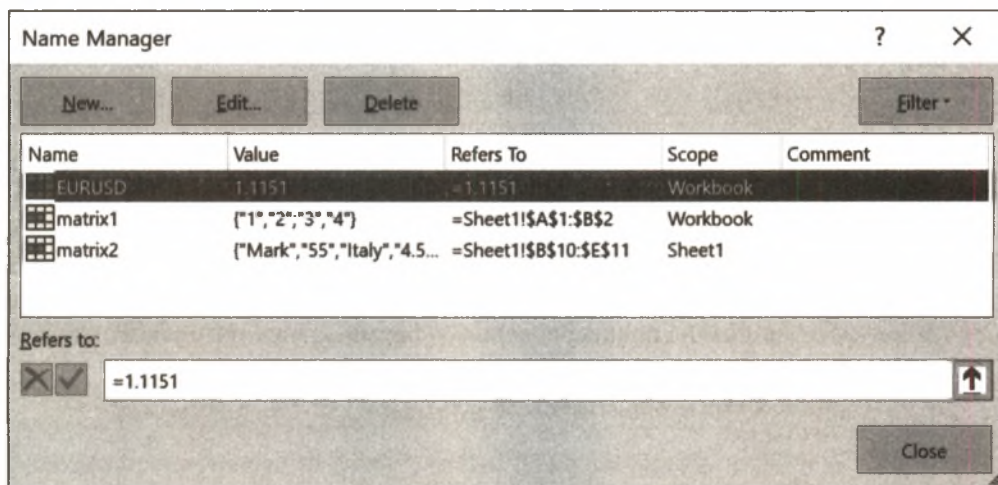


Рис. 9.3. Менеджер имен Excel
после добавления нескольких определенных имен с помощью xlwings

Теперь вы знаете, как работать с наиболее часто используемыми компонентами рабочей книги Excel. Это означает, что мы можем еще раз рассмотреть пример с отчетами из главы 7: давайте посмотрим, что изменится, когда мы добавим в картину xlwings!

Случай из практики (повторный анализ): отчетность в Excel

Возможность реального редактирования файлов Excel с помощью xlwings позволяет нам работать с файлами шаблонов, которые будут сохранены на 100% независимо от того, насколько они сложны или в каком формате хранятся. Например, вы

можете легко редактировать файл `xlsb`, случай, который в настоящее время не поддерживается ни одним из пакетов `writer` и с которыми мы познакомились в предыдущей главе. Если посмотреть на `sales_report_openpyxl.py` в сопутствующем репозитории, можно увидеть, что после подготовки сводного `DataFrame` нам пришлось написать почти сорок строк кода для создания с помощью `OpenPyXL` одного графика и стилизации только одного `DataFrame`. С помощью `xlwings` вы достигнете того же результата всего за шесть строк кода, что и показано в примере 9.1. Возможность управлять форматированием в шаблоне Excel сэкономит вам много времени. Однако за это приходится платить: `xlwings` для запуска требует установки Excel — это вполне приемлемо, если вам на своей машине приходится создавать такие отчеты нечасто, но может оказаться не совсем удачным решением, если вы пытаетесь создавать отчеты на сервере как составной части веб-приложения.

Во-первых, вы должны убедиться, что ваша лицензия Microsoft Office распространяется на установку на сервере, а во-вторых, Excel не был создан для автоматизации без посторонней помощи, что означает, что вы можете столкнуться с проблемами стабильности, особенно если вам нужно создавать много отчетов за короткий промежуток времени. Тем не менее, я видел многих клиентов, которые успешно справились с этой задачей, поэтому если вы по каким-то причинам не можете использовать пакет `writer`, запуск `xlwings` на сервере может оказаться вполне подходящим вариантом. Только не забудьте каждый сценарий запустить в новом экземпляре Excel с помощью `app = xw.App()`, чтобы обойти типичные проблемы со стабильностью. Вы найдете полный сценарий `xlwings` в файле `sales_report_xlwings.py` в партнерском репозитории (первая часть — та же, что мы использовали с `OpenPyXL` и `XlsxWriter`). Это отличный пример сочетания пакета `reader` с `xlwings`: в то время как `pandas` (через `OpenPyXL` и `xlrd`) быстрее считывает множество файлов с диска, `xlwings` облегчает заполнение предварительно отформатированного шаблона.

Пример 9.1. `sales_report_xlwings.py` (только вторая часть)

```
# Откройте шаблон, вставьте данные, выполните автоподгонку столбцов
# и настройте источник диаграммы. Затем сохраните шаблон под другим
# именем.
template = xw.Book(this_dir / "xl" / "sales_report_template.xlsx")
sheet = template.sheets["Sheet1"]
sheet["B3"].value = summary
sheet["B3"].expand().columns.autofit()
sheet.charts["Chart 1"].set_source_data(sheet["B3"].expand()[:-1, :-1])
template.save(this_dir / "sales_report_xlwings.xlsx")
```

Когда вы впервые запустите этот сценарий на macOS (например, открыв его в VS Code и нажав кнопку **Run File**), вам снова придется подтвердить свои намерения о предоставлении доступа к файловой системе во всплывающем окне, с чем мы уже сталкивались ранее в этой главе.

С помощью шаблонов форматирования Excel вы можете очень быстро создавать красивые отчеты Excel. Вы также получаете доступ к таким методам, как `autofit`,

что недоступно в пакетах `writer`, поскольку они основаны на расчетах, выполняемых приложением `Excel`: это позволяет вам правильно установить ширину и высоту ячеек в соответствии с их содержимым.

На рис. 9.4 показана верхняя часть отчета о продажах, созданная `xlwings`, с настроенным заголовком таблицы, а также столбцами, к которым был применен метод автоподбора.

Когда вы начнете использовать `xlwings` не только для заполнения пары ячеек в шаблоне, полезно будет узнать немного о его внутреннем устройстве: в следующем разделе мы рассмотрим, как `xlwings` работает «под оболочкой».

	A	B	C	D	E	F	G	H	I
1	Sales Report								
2									
3		Month	Washington DC	Boston	Las Vegas	New York	Chicago	San Francisco	Total
4		Jan 19	14,058	21,784	23,013	49,873	51,188	58,630	218,545
5		Feb 19	15,235	21,455	25,493	46,670	52,331	55,219	216,403
6		Mar 19	14,177	20,043	23,451	41,572	48,897	52,713	200,854
7		Apr 19	13,339	18,791	22,710	41,714	47,396	49,325	193,276
8		May 19	13,147	18,037	21,527	40,610	45,117	47,760	186,197
9		Jun 19	14,284	21,556	21,985	47,266	49,460	53,462	208,014
10		Jul 19	14,162	19,853	23,444	40,408	47,994	50,182	196,043
11		Aug 19	16,127	22,333	24,928	45,397	50,839	55,336	214,960
12		Sep 19	14,994	19,925	24,411	42,831	49,096	49,931	201,188
13		Oct 19	12,847	16,551	22,828	34,090	42,544	44,312	173,171
14		Nov 19	14,058	21,313	24,860	46,960	52,012	55,056	214,259
15		Dec 19	14,702	19,723	24,536	42,364	49,355	50,933	201,613
16		Total	171,130	241,363	283,185	519,755	586,229	622,859	2,424,521

Рис. 9.4. Таблица отчета о продажах на основе предварительно отформатированного шаблона

Расширенные темы `xlwings`

В этом разделе рассказывается о том, как сделать код `xlwings` более производительным и как восполнить недостающие функции. Однако для понимания этих тем сначала нужно сказать несколько слов о том, как `xlwings` взаимодействует с `Excel`.

Основы `xlwings`

`xlwings` зависима от других пакетов `Python`, с помощью которых осуществляется связь с механизмом автоматизации соответствующей операционной системы:

Windows

В `Windows` `xlwings` полагается на технологию `COM`, сокращенно от *Component Object Model*. `COM` — это стандарт, который позволяет двум процессам взаимо-

действовать друг с другом — в нашем случае Excel и Python. xlwings использует пакет Python `pywin32`⁶ для обработки вызовов COM.

macOS

В macOS xlwings использует AppleScript. AppleScript — это язык сценариев Apple для автоматизации приложений с поддержкой сценариев, и, к сожалению, Excel является таким приложением. Для выполнения команд AppleScript xlwings использует пакет Python `appscript`⁷.

Windows: как предотвратить появление зомби-процессов

Когда вы работаете с xlwings в Windows, вы можете заметить, что работа Excel вроде бы полностью завершена, но когда вы откроете диспетчер задач (щелкните правой кнопкой мыши на панели задач Windows, затем выберите **Диспетчер задач**), то на вкладке **Процессы**, в разделе **Фоновые процессы** вы увидите строки Microsoft Excel. Если не отображается ни одна вкладка, нажмите кнопку **Подробнее**. Или перейдите на вкладку **Подробности**, где Excel в списке будет отображен как «EXCEL.EXE». Для завершения данного *зомби-процесса* щелкните правой кнопкой мыши по соответствующей строке и выберите команду **Завершить задачу**, чтобы принудительно закрыть Excel.

Поскольку эти процессы *не завершены*, а лишь *остановлены*, их часто называют зомби-процессами. Оставляя их незавершенными, вы расходуете ресурсы и рискуете получить нежелательные последствия: например, при открытии нового экземпляра Excel файлы могут быть заблокированы или надстройки не будут загружены должным образом. Причина, по которой Excel порой не удается завершить работу правильно, заключается в том, что процессы завершаются только после того, как больше нет ссылок на COM, например в виде объекта `app` xlwings. Чаще всего после завершения работы интерпретатора Python вы получаете зомби-процесс Excel, поскольку ему не удастся правильно очистить ссылки COM. Рассмотрим этот пример в Anaconda Prompt:

```
(base)> python
>>> import xlwings as xw
>>> app = xw.App()
```

После запуска нового экземпляра Excel снова выйдите из него через пользовательский интерфейс Excel: пока Excel закрывается, процесс Excel в диспетчере задач будет продолжать выполняться. Если вы правильно завершили сеанс Python, выполнив `quit()` или воспользовавшись комбинацией клавиш `<Ctrl>+<Z>`, Excel в конечном итоге будет закрыт. Однако если вы завершите Anaconda Prompt, нажав на «х» (крестик) в правом верхнем углу окна, вы заметите, что процесс остается в виде зомби-процесса.

⁶ <https://oriel.ly/tm7sK>. — Примеч. пер.

⁷ <https://oriel.ly/tlsDd>. — Примеч. пер.

То же самое произойдет, если вы завершите Anaconda Prompt перед закрытием Excel или во время работы сервера Jupyter и будете держать объект приложения `xlwings` в одной из ячеек блокнота Jupyter. Чтобы свести к минимуму шансы получить в итоге зомби-процессы Excel, ниже приведено несколько рекомендаций:

- ◆ запустите `app.quit()` из Python вместо того, чтобы закрывать Excel вручную. Это гарантирует, что ссылки будут очищены должным образом;
- ◆ не закрывайте интерактивные сессии Python при работе с `xlwings`, например если вы запускаете Python REPL в Anaconda Prompt, завершите интерпретатор Python надлежащим образом, выполнив `quit()` или используя комбинацию клавиш `<Ctrl>+<Z>`. Когда вы работаете с блокнотами Jupyter, выключите сервер, нажав Quit в веб-интерфейсе;
- ◆ в интерактивных сессиях Python полезно избегать прямого использования объекта `app`, например используя `xw.Book()` вместо `myapp.books.add()`. Это позволит правильно завершить Excel, даже если процесс Python будет завершен.

Теперь, когда вы имеете представление о технологии, лежащей в основе `xlwings`, попробуем разобраться, как мы можем ускорить медленные скрипты!

Улучшение производительности

Чтобы обеспечить производительность ваших скриптов `xlwings`, есть несколько стратегий: самая важная из них — свести к абсолютному минимуму межприкладные вызовы. Использование необработанных значений может стать еще одним способом, и, кроме того, поможет сделать правильный выбор соответствующих параметров приложения. Рассмотрим эти варианты последовательно!

Минимизация вызовов между приложениями

Очень важно знать, что каждый межприкладной вызов между приложениями из Python в Excel является «ресурсоемким», вследствие чего — медленным. Поэтому количество таких вызовов должно быть максимально сокращено. Самый простой способ сделать это — читать и записывать целые диапазоны Excel вместо того, чтобы выполнять циклический просмотр через отдельные ячейки. В следующем примере мы считываем и записываем 150 ячеек, сначала перебирая каждую ячейку, а затем обрабатывая весь диапазон за один вызов:

```
In [64]: # Добавьте новый лист и запишите 150 значений
         # чтобы было с чем работать
         sheet2 = book.sheets.add()
         sheet2["A1"].value = np.arange(150).reshape(30, 5)

In [65]: %time
         # В результате получается 150 кросс-прикладных вызовов
         for cell in sheet2["A1:E30"]:
             cell.value += 1
```



```
Wall time: 909 ms
```

```
In [66]: %%time
```

```
# В результате происходит всего два межприкладных вызова
values = sheet2["A1:E30"].options(np.array).value
sheet2["A1"].value = values + 1
```

```
Wall time: 97.2 ms
```

Эти цифры еще более впечатляют на macOS, где на моей машине второй вариант выполняется примерно в 50 раз быстрее первого.

Необработанные значения

xlwings был разработан в первую очередь с акцентом на практичность, а не на скорость. Однако, если вы работаете с огромными диапазонами ячеек, то можете столкнуться с ситуациями, когда можно сэкономить время, пропустив шаг очистки данных в xlwings: xlwings перебирает каждое значение при чтении и записи данных, например для унификации типов данных между Windows и macOS. Используя строку `raw` в качестве конвертера в методе `options`, вы пропускаете этот шаг. Хотя в результате все операции должны выполняться быстрее, разница может оказаться несущественной, если только вы не создаете большие массивы в операционной системе Windows. Использование необработанных значений, однако, означает, что вы больше не можете напрямую работать с DataFrames. Вместо этого вам необходимо предоставлять значения в виде вложенных списков или кортежей. Кроме того, вам придется указать полный адрес диапазона, в который вы записываете, — предоставления левой верхней ячейки уже недостаточно:

```
In [67]: # При использовании необработанных значений вы должны
# предоставить полный диапазон объектов, sheet["A35"] больше не
# работает
sheet1["A35:B36"].options("raw").value = [[1, 2], [3, 4]]
```

Свойства приложений

В зависимости от содержания рабочей книги изменение свойств объектов приложений также может помочь ускорить выполнение кода. Обычно вы обращаете внимание на следующие свойства (`myapp` — это объект приложения xlwings):

- `myapp.screen_updating = False`
- `myapp.calculation = "manual"`
- `myapp.display_alerts = False`

В конце сценария обязательно верните атрибуты в исходное состояние. Если вы работаете под Windows, вы также можете увидеть небольшое улучшение производительности, запустив сценарий в скрытом экземпляре Excel с помощью `xw.App(visible=False)`.

Теперь, когда вы знаете, как контролировать производительность, давайте рассмотрим, как расширить функциональность xlwings.

Как действовать при отсутствии недостающих функций

xlwings предоставляет Pythonic интерфейс для наиболее часто используемых команд Excel и обеспечивает их работу в операционных системах Windows и macOS. Однако существует множество методов и атрибутов объектной модели Excel, которые еще не поддерживаются xlwings. Но не все потеряно! xlwings с помощью атрибута `api` предоставляет вам доступ к базовому объекту `pywin32` в Windows и объекту `appscript` в macOS для любого объекта xlwings. Таким образом, вы получаете доступ ко всей объектной модели Excel, но, в свою очередь, теряете кроссплатформенную совместимость. Например, допустим, вы хотите очистить форматирование ячейки. Вот как это можно сделать:

- ◆ проверьте, доступен ли метод для объекта диапазона xlwings. Для этого, например, используя клавишу `<Tab>`, когда поставите точку в конце объекта диапазона в блокноте Jupyter, выполните `dir(sheet["A1"])` или поиск по ссылке API xlwings⁸. В VS Code доступные методы должны автоматически отображаться во всплывающей подсказке;
- ◆ если нужные функции отсутствуют, используйте атрибут `api` для получения базового объекта: в Windows `sheet["A1"].api` выдаст вам объект `pywin32`, а в macOS вы получите объект `appscript`;
- ◆ сверьтесь с объектной моделью Excel в справочнике Excel VBA⁹. Чтобы очистить формат диапазона, нужно выполнить `Range.ClearFormats`¹⁰;
- ◆ в Windows в большинстве случаев можно использовать метод или свойство VBA непосредственно с объектом `api`. Если это метод, не забудьте добавить круглые скобки в Python: `sheet["A1"].api.ClearFormats()`. Если вы работаете с операционной системой macOS, ситуация усложняется, поскольку в `appscript` используется синтаксис, который трудно определить. Лучше всего обратиться к руководству для разработчиков¹¹, являющемуся частью исходного кода xlwings. Однако очистить форматирование ячеек довольно просто: достаточно применить правила синтаксиса Python к имени метода, используя символы нижнего регистра с подчеркиванием: `sheet["A1"].api.clear_formats()`.

Если вам нужно убедиться, что `ClearFormats` работает на обеих платформах, вы можете сделать это следующим образом (Darwin — ядро macOS и используется в качестве имени в `sys.platform`):

```
import sys
if sys.platform.startswith("darwin"):
    sheet["A10"].api.clear_formats()
elif sys.platform.startswith("win"):
    sheet["A10"].api.ClearFormats()
```

⁸ <https://oriel.ly/EiXVc>. — Примеч. пер.

⁹ <https://oriel.ly/UlLPo>. — Примеч. пер.

¹⁰ <https://oriel.ly/kcEsw>. — Примеч. пер.

¹¹ <https://oriel.ly/VSS0Y>. — Примеч. пер.

В любом случае, чтобы включить эту функциональность в будущую версию, стоит задать вопрос в репозитории xlwings¹² на GitHub.

Заключение

В этой главе вы познакомились с концепцией автоматизации Excel: с помощью xlwings вы можете использовать Python для задач, которые традиционно выполняются на VBA. Мы узнали об объектной модели Excel и о том, как xlwings позволяет взаимодействовать с ее компонентами, такими как объекты `sheet` и `range`. Вооружившись этими знаниями, мы снова обратились к примеру с отчетами из главы 7 и использовали xlwings для заполнения предварительно отформатированного шаблона отчета; это показало вам, что существует возможность использовать пакеты для чтения и xlwings параллельно. Мы также узнали о библиотеках, которые xlwings использует «под оболочкой», чтобы понять, как мы можем улучшить производительность и обойти недостающие функции. Моя излюбленная фишка xlwings в том, что он одинаково хорошо работает как на macOS, так и на Windows. Это становится еще более интересным, поскольку Power Query на macOS пока не обладает всеми возможностями версии для Windows: то, чего не хватает, вы сможете легко заметить комбинацией pandas и xlwings.

Теперь, когда вы познакомились с основами xlwings, можно переходить к следующей главе: в ней мы сделаем очередной шаг и будем вызывать скрипты xlwings из самого Excel, что позволит вам создавать инструменты Excel, работающие на Python.

¹² <https://oriel.ly/kFkD0>. — Примеч. пер.

Инструменты Excel на основе технологии Python

В прошлой главе мы рассмотрели, как писать сценарии Python для автоматизации работы Microsoft Excel. Это очень мощный инструмент, используя для запуска сценариев либо командную строку Anaconda, либо редактор, подобный VS Code, пользователь должен чувствовать себя комфортно. Однако, если ваши инструменты используются бизнес-пользователями, скорее всего, так не получится. Для этих пользователей необходимо скрыть часть Python, и сделать так, чтобы инструмент Excel стал похож на обычную рабочую книгу с поддержкой макросов. В этой главе как раз и пойдет речь о том, как достичь этот результат с помощью xlwings. Я начну с демонстрации кратчайшего пути к запуску Python-кода из Excel, а затем рассмотрю проблемы развертывания инструментов xlwings — это также позволит нам более подробно рассмотреть доступные настройки, которые предлагает xlwings. Как и в предыдущей главе, здесь потребуется, чтобы на вашем компьютере, работающем под управлением операционной системы Windows или macOS, была установлена Microsoft Excel.

Использование Excel в качестве интерфейса xlwings

Интерфейс — это часть приложения, которую пользователь видит и с которой взаимодействует. Другие распространенные названия интерфейса — *графический пользовательский интерфейс* (GUI), или просто *пользовательский интерфейс* (UI). Когда я спрашиваю пользователей xlwings, почему они разрабатывают свой инструмент с помощью Excel, а не создают современное веб-приложение, обычно я слышу следующее: «Excel — это интерфейс, с которым знакомы наши пользователи». Использование ячеек электронных таблиц позволяет пользователям быстро и интуитивно понятно вводить данные, что делает их работу более продуктивной, чем, если бы они использовали разработанный на скорую руку веб-интерфейс. В начале этого раздела я познакомлю вас с настройкой xlwings Excel и xlwings CLI (интерфейс командной строки), а затем с помощью команды `quickstart` мы создадим наш первый проект. В завершение этого раздела я представлю два способа, с помощью которых можно вызвать код Python из Excel: нажатием в надстройке

кнопки Run main или используя функцию RunPython в VBA. Начнем с установки надстройки xlwings Excel!

Надстройка Excel

Поскольку xlwings включен в дистрибутив Anaconda, в предыдущей главе мы могли запускать команды xlwings в Python прямо из приложения. Однако, если вам требуется вызывать сценарии Python из Excel, потребуется либо установить надстройку Excel, либо настроить рабочую книгу для работы в автономном режиме. Хотя я представляю автономный режим в разд. «Развертывание» на стр. 240, в этом разделе показано, как работать с надстройкой. Чтобы установить надстройку, выполните в командной строке Anaconda следующие действия:

```
(base)> xlwings addin install
```

Вам при каждом обновлении xlwings нужно будет синхронизировать версию пакета Python и версию надстройки. Поэтому при обновлении xlwings всегда следует выполнять две команды — одну для пакета Python и одну для надстройки Excel. В зависимости от того, какой у вас установлен менеджер пакетов: Conda или pip, обновлять вашу установку xlwings следует так:

Conda (используйте эти команды с дистрибутивом Anaconda Python)

```
(base)> conda update xlwings
```

```
(base)> xlwings addin install
```

pip (используйте эти команды с остальными дистрибутивами Python)

```
(base)> pip install --upgrade xlwings
```

```
(base)> xlwings addin install
```



Антивирусное программное обеспечение

К сожалению, дополнение xlwings иногда распознается антивирусными программами как вредоносное дополнение, особенно если вы используете новый релиз. Если это произошло на вашей машине, зайдите в настройки вашего антивирусного программного обеспечения и отметьте xlwings как безопасный для запуска. Обычно о таких ложных срабатываниях можно сообщать через домашнюю страницу программы.

Когда вы в приглашении Anaconda вводите xlwings, вы используете интерфейс командной строки xlwings. Помимо упрощения установки надстройки xlwings, она предлагает еще несколько команд: я буду вводить их всякий раз, когда они нам понадобятся, но вы, чтобы вывести на экран все доступные параметры, всегда можете ввести xlwings в командной строке Anaconda и нажать <Enter>. Давайте теперь подробнее рассмотрим, что делает xlwings addin install:

Установка

Установка надстройки выполняется с помощью копирования xlwings.xlam из каталога пакета Python в специальную папку XLSTART Excel. Excel при каждом своем запуске открывает все файлы, находящиеся в этой папке. Когда вы запус-

тите `xlwings addin status` в **Anaconda Prompt**, приложение отобразит, где находится каталог **XLSTART** в вашей системе и установлено ли дополнение.

Конфигурация

Когда вы устанавливаете надстройку в первый раз, система также настроит ее на использование интерпретатора Python или среды Conda, из которой выполняется команда `install`: как показано на рис. 10.1. Значения `Conda Path` и `Conda Env` `xlwings CLI`¹ вводит автоматически и хранятся данные значения в вашем домашнем каталоге, в файле `xlwings.conf` папки `.xlwings`. В Windows это обычно `C:\Users<username>\.xlwings\xlwings.conf`, а в macOS `/Users/<username>/.xlwings/xlwings.conf`. В macOS папки и файлы с лидирующей точкой по умолчанию скрыты. В Finder, чтобы включить их отображение, нажмите комбинацию клавиш `<Command>->Shift-<.>`.

После выполнения команды установки вам придется перезапустить Excel, чтобы на ленте появилась вкладка **xlwings**, которая показана на рис. 10.1.

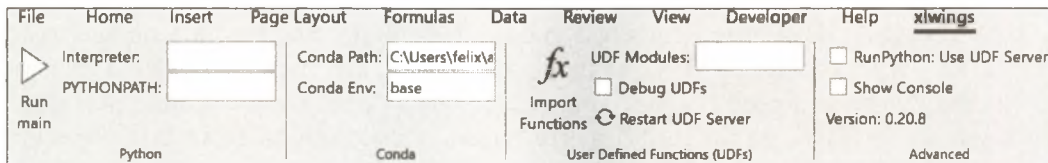


Рис. 10.1. Надстройка `xlwings` ribbon после выполнения команды установки



Надстройка для ленты в macOS

На macOS лента выглядит немного иначе, поскольку в ней отсутствуют разделы с пользовательскими функциями и Conda: в то время как пользовательские функции в macOS не поддерживаются, среды Conda не требуют особого внимания, потому что они настроены как интерпретатор в группе Python.

Теперь, когда у вас установлена надстройка `xlwings`, нам понадобится рабочая книга и небольшой Python-код, чтобы протестировать его. Самый быстрый способ добиться этого, как я покажу дальше — использовать команду `quickstart`.

Команда Quickstart

Чтобы максимально упростить процесс создания вашего первого инструмента `xlwings`, CLI `xlwings CLI` предлагает команду `quickstart`. В **Anaconda Prompt** для перехода в каталог, в котором вы хотите создать свой первый проект, используйте команду `cd` (например, `cd Desktop`), затем, чтобы создать проект с именем `first_project`, выполните следующие действия:

```
(base)> xlwings quickstart first_project
```

¹ Если вы работаете под macOS или используете дистрибутив Python, отличный от **Anaconda**, будет настроен интерпретатор, а не параметры Conda.

Имя проекта должно быть действительным именем модуля Python: оно может содержать символы, цифры и знаки подчеркивания, но не пробелы или тире, и не должно начинаться с цифры. В разд. «Функция RunPython» на стр. 235 я покажу вам, как можно изменить имя файла Excel на то, которое не обязательно следует этим правилам. Выполнение команды `quickstart` создаст папку с названием `first_project` в вашем текущем каталоге. Открыв эту папку в File Explorer (в Windows) или Finder (в macOS), вы увидите два файла: `first_project.xlsm` и `first_project.py`. Откройте оба файла — файл Excel в Excel или файл Python в VS Code. Самый простой способ запустить код Python из Excel — это использовать основную кнопку **Run main** в надстройке — давайте посмотрим, как это работает!

Run Main

Прежде чем рассмотреть файл `first_project.py` более подробно, если файл `first_project.xlsm` является активным, нажмите основную кнопку **Run main** в левой части надстройки xlwings; в результате в ячейку A1 первого листа будет вписано «Hello xlwings!». Нажмите основную кнопку **Run main** еще раз, и надпись изменится на «Bye xlwings!». Поздравляем, вы только что запустили свою первую функцию Python из Excel! В конце концов, это было не намного сложнее, чем написать макрос на VBA, не так ли? Давайте теперь посмотрим на файл `first_project.py` из примера 10.1.

Пример 10.1. `first_project.py`

```
import xlwings as xw

def main():
    wb = xw.Book.caller()
    sheet = wb.sheets[0]
    if sheet["A1"].value == "Hello xlwings!":
        sheet["A1"].value = "Bye xlwings!"
    else:
        sheet["A1"].value = "Hello xlwings!"

@xw.func
def hello(name):
    return f"Hello {name}!"

if __name__ == "__main__":
    xw.Book("first_project.xlsm").set_mock_caller()
    main()
```

- 1 `xw.Book.caller()` — это объект xlwings `book`, ссылающийся на рабочую книгу Excel, которая становится активной, когда вы нажимаете основную кнопку **Run main**. В нашем случае это соответствует `xw.Book("first_project.xlsm").`

Использование `xw.Book.caller()` дает возможность переименовывать и перемещать файл Excel по файловой системе без нарушения ссылки. `xw.Book.caller()` также гарантирует, что вы работаете с нужной рабочей книгой, если она открыта в нескольких экземплярах Excel.

- 2 В этой главе мы проигнорируем функцию `hello`, поскольку она станет темой главы 12. Если вы запустите команду `quickstart` на macOS, вы все равно не увидите функцию `hello`, поскольку определяемые пользователем функции поддерживаются только в Windows.
- 3 Я объясню последние три строки кода в следующей главе, когда мы будем рассматривать отладку. В этой главе не обращайтесь или даже удалите все, что находится ниже первой функции.

Кнопка **Run main** в надстройке Excel — это удобная функция: она позволяет вызывать функцию с именем `main` в модуле Python, который имеет то же имя, что и файл Excel, без необходимости каждый раз сначала добавлять кнопку в рабочую книгу. Она будет работать, даже если вы сохраните рабочую книгу в формате `xlsx` без макросов. Если же вы хотите вызвать одну или несколько функций Python, которые не называются `main` и не являются частью модуля с тем же именем, что и рабочая книга, тогда используйте функцию `RunPython` из VBA. Подробности в следующем разделе!

Функция `RunPython`

Если требуется дополнительный контроль над тем, как вызывается код Python, используйте функцию VBA `RunPython`. Соответственно, `RunPython` требует, чтобы ваша рабочая книга была сохранена как рабочая книга с поддержкой макросов.



Включение макросов

Необходимо нажать кнопку **Включить содержимое** (в Windows) или **Включить макросы** (в macOS) при открытии рабочей книги с поддержкой макросов (расширение `xlsm`), например, той, которая создается командой `quickstart`. В Windows при работе с файлами `xlsm` из партнерского репозитория необходимо дополнительно нажать кнопку **Включить редактирование**, иначе Excel не будет корректно открывать файлы, загруженные из интернета.

`RunPython` принимает строку с кодом Python: чаще всего вы импортируете модуль Python и запускаете одну из его функций. Открыв редактор VBA с помощью комбинации клавиш `<Alt>+<F11>` (в Windows) или `<Option>+<F11>` (в macOS), вы увидите, что команда `quickstart` добавляет макрос под названием `SampleCall` в модуль VBA под именем «Module1» (рис. 10.2). Если вы не видите макрос `SampleCall`, дважды щелкните на строке `Module1` в дереве проектов VBA с левой стороны окна.

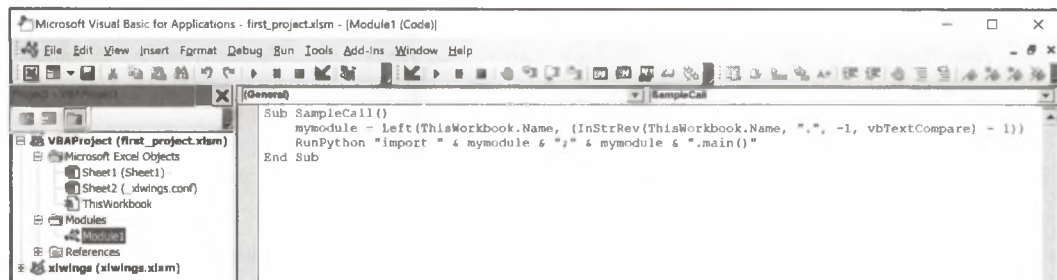


Рис. 10.2. Редактор VBA, отображающий Module1

Код выглядит довольно замысловатым, но это сделано, чтобы он действовал автоматически, независимо от того, какое имя проекта вы выберете при выполнении команды `quickstart`. Поскольку наш модуль Python носит название `first_project`, вы можете заменить этот код следующим простым для понимания эквивалентом:

```
Sub SampleCall()
    RunPython "import first_project; first_project.main()"
End Sub
```

Поскольку не очень хотелось бы писать многострочные строки в VBA, мы используем точку с запятой, которую Python принимает как перенос строки. Есть несколько способов запустить этот код: например, находясь в редакторе VBA, установите курсор на любую строку макроса `SampleCall` и нажмите клавишу `<F5>`. Однако обычно код выполняется из листа Excel, а не из редактора VBA. Поэтому закройте редактор VBA и снова переключитесь на рабочую книгу. Нажав комбинацию клавиш `<Alt>+<F8>` (в Windows) или `<Option>-<F8>` (в macOS), вы вызовете меню макроса: выберите `SampleCall` и нажмите кнопку **Run main**. Чтобы сделать макрос более удобным для использования, добавьте кнопку в рабочую книгу Excel и свяжите ее с `SampleCall`: но сначала убедитесь, что на ленте отображается вкладка **Developer** (Разработчик). Если этого не произошло, перейдите в меню **Файл > Параметры > Настроить ленту** и установите флажок рядом с пунктом **Разработчик** (в macOS этот пункт находится в меню **Excel > Параметры > Лента и панель инструментов**). Чтобы вставить кнопку, перейдите на вкладку **Developer** и в группе **Controls** нажмите **Insert > Button** (под **Form Controls**). На macOS кнопка будет отображаться сразу, без необходимости сначала переходить к вставке. Когда вы нажимаете на значок кнопки, ваш курсор превращается в маленький крестик: используйте его, чтобы нарисовать кнопку на листе. Рисуя прямоугольную форму, удерживайте левую кнопку мыши нажатой. Когда вы отпустите кнопку мыши, на экране появится меню **Assign Macro** (Назначить макрос). Выберите пункт `SampleCall` и нажмите **OK**. Нажмите на кнопку, которую вы только что создали (в моем случае она называется «Button 1»), и она снова запустит нашу главную функцию, как показано на рис. 10.3.

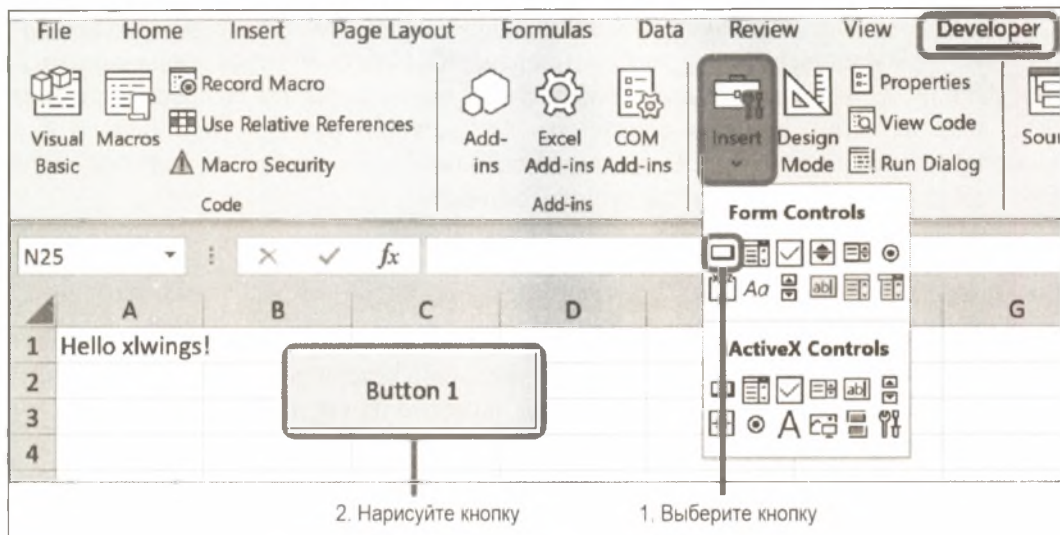


Рис. 10.3. Рисование кнопки на листе



Элементы управления формами в сравнении с элементами управления ActiveX

В Windows имеется два типа элементов управления: Элементы управления формами (Form Controls) и элементы управления ActiveX. Хотя для подключения к макросу `SampleCall` можно использовать кнопку из любой группы, на macOS будет работать только кнопка из элементов управления формами. В следующей главе мы будем использовать прямоугольники в качестве кнопок, чтобы придать им более современный вид.

Теперь давайте посмотрим, как можно изменить имена, присвоенные командой `quickstart` по умолчанию: вернемся к вашему файлу Python и переименуем его из `first_project.py` в `hello.py`. Также переименуем вашу функцию `main` в `hello_world`. Обязательно сохраните файл, затем снова откройте редактор VBA с помощью комбинации клавиш `<Alt>+<F11>` (в Windows) или `<Option>-<F11>` (в macOS) и отредактируйте `SampleCall` следующим образом, чтобы отразить изменения:

```
Sub SampleCall()
    RunPython "import hello; hello.hello_world()"
End Sub
```

Вернитесь на лист, нажмите на кнопку «Button 1», чтобы убедиться, что все по-прежнему работает. Наконец, у вас может возникнуть желание сохранить сценарий Python и файл Excel в двух разных каталогах. Чтобы понять смысл этих действий, сначала нужно сказать пару слов о пути поиска модулей в Python: если вы импортируете модуль в свой код, Python ищет его в различных каталогах. Сначала Python проверяет, существует ли встроенный модуль с таким именем, и если он не найден, переходит к поиску в текущем рабочем каталоге и в каталогах, указанных в так называемом `PYTHONPATH`. `xlwings` автоматически добавляет каталог рабочей книги

в PYTHONPATH и позволяет добавлять дополнительные пути с помощью надстройки. Чтобы проверить это, выберите сценарий Python, который теперь называется `hello.py`, и переместите его в папку `pyscripts`, которую вы создадите в своем домашнем каталоге: в моем случае это `C:\Users\felix\pyscripts` в Windows или `/Users/felix/pyscripts` в macOS. Когда вы снова нажмете на кнопку, во всплывающем окне отобразится сообщение о следующей ошибке:

```
Traceback (most recent call last):
```

```
File "<string>", line 1, in <module>
```

```
ModuleNotFoundError: No module named 'first_project'
```

Чтобы устранить ошибку, в ленте `xlwings` добавьте в настройку `PYTHONPATH` путь к каталогу `pyscripts`, как показано на рис. 10.4. Теперь после нажатия на кнопку еще раз она снова сработает. Повторное нажатие на кнопку приведет к новому срабатыванию.

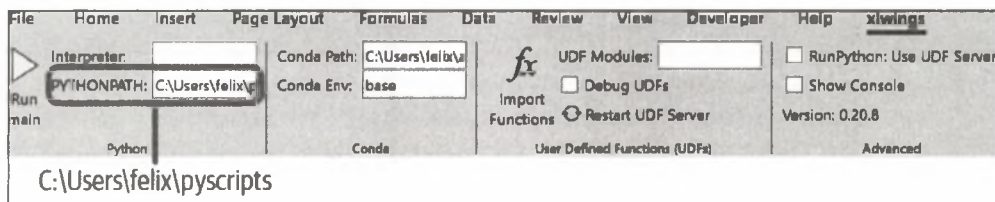


Рис. 10.4. Настройка PYTHONPATH

Чего я еще не коснулся, так это имени рабочей книги Excel: если ваш вызов функции `RunPython` использует явное имя модуля, например `first_project`, а не код, добавленный `quickstart`, вы можете переименовать свою рабочую книгу Excel в любое имя и как пожелаете.

Использование команды `quickstart` — самый простой вариант, если вы начинаете новый проект `xlwings`. Однако если у вас есть уже готовая рабочая книга, вы можете воспользоваться ручной настройкой. Давайте посмотрим, как это делается!

Запуск Python без команды `quickstart`

Если вы хотите использовать функцию `RunPython` с существующей рабочей книгой, которая не создавалась командой `quickstart`, вам нужно вручную выполнить те действия, которые команда `quickstart` выполняет за вас. Обратите внимание, что следующие шаги необходимы только для вызова `RunPython`, но не в случае, когда вы желаете использовать кнопку **Run main**:

1. Прежде всего убедитесь, что ваша рабочая книга сохранена как рабочая книга с поддержкой макросов с расширением `xlsm` или `xlsb`.
2. Добавьте модуль VBA. Для этого с помощью комбинации клавиш `<Alt>+<F11>` (в Windows) или `<Option>+<F11>` (в macOS) откройте редактор VBA и убедитесь, что в древовидном отображении с левой стороны рабочего окна выбран `VBAProject` вашей рабочей книги, затем щелкните на нем правой кнопкой мыши

и выберите команду **Insert > Module**, как показано на рис. 10.5. Это позволит вставить пустой модуль VBA, в котором вы сможете написать свой макрос VBA с вызовом RunPython.

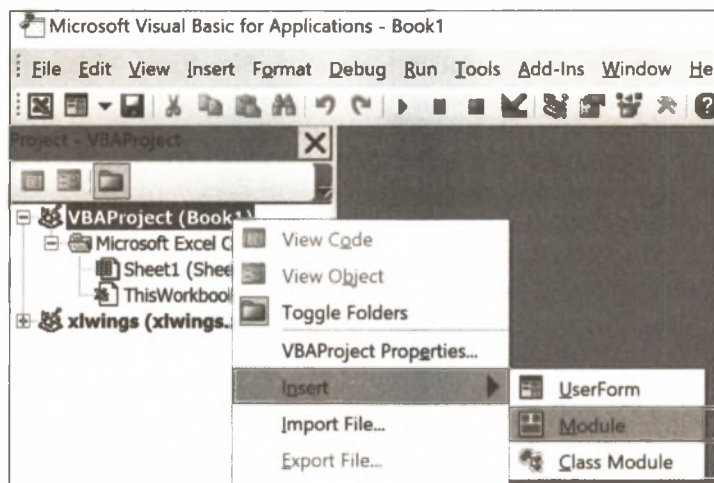


Рис. 10.5. Добавление модуля VBA

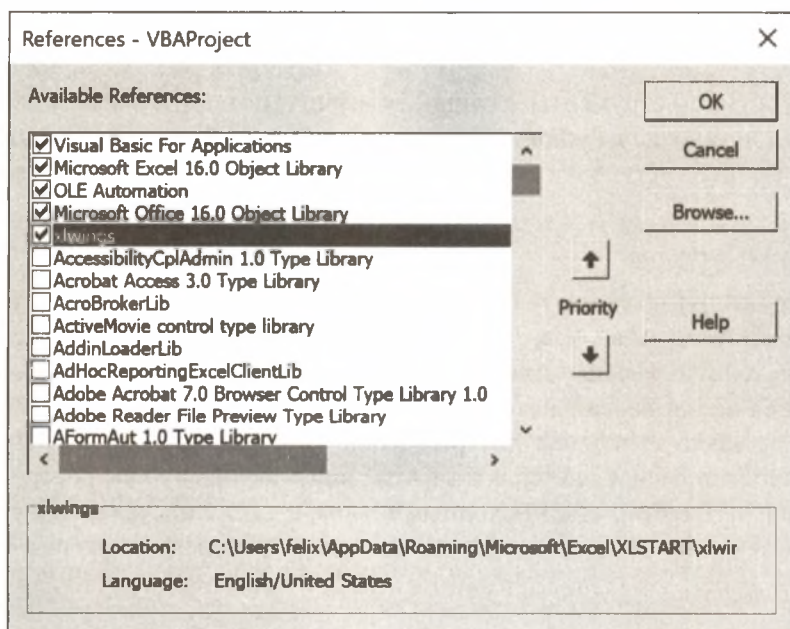


Рис. 10.6. RunPython нуждается в ссылке на xlwings

3. Добавьте ссылку на xlwings: RunPython — это функция, которая является частью надстройки xlwings. Чтобы ее использовать, следует убедиться, что в вашем проекте VBA установлена ссылка на xlwings. Опять же начните с выбора соот-

ветствующей рабочей книги в древовидном представлении с левой стороны редактора VBA, затем перейдите в меню **Инструменты > Ссылка** и установите флажок **xlwings**, как показано на рис. 10.6.

Теперь ваша рабочая книга готова к повторному использованию вызова `RunPython`. Когда все на вашей машине заработает, следующий шаг обычно заключается в том, чтобы заставить это работать на машине вашего коллеги — давайте рассмотрим несколько вариантов, которые облегчат эту часть работы!

Развертывание

В разработке программного обеспечения термин *развертывание* относится к распространению и установке программного обеспечения, чтобы пользователи могли его использовать. В случае с инструментами `xlwings` полезно знать, какие зависимости необходимы и какие настройки могут облегчить развертывание. Я начну с самой важной зависимости, а именно с Python, а затем рассмотрю рабочие книги, которые были настроены в автономном режиме, чтобы избавиться от надстройки `xlwings` Excel. И в завершение этого раздела я подробнее рассмотрю, как настроенная конфигурация используется в `xlwings`.

Зависимости Python

Чтобы запустить инструменты `xlwings`, у ваших потенциальных пользователей должен быть установлен Python. Но если у них еще Python не установлен, это не значит, что процесс установки нельзя упростить. Ниже приведены несколько вариантов:

Anaconda или WinPython

Проинструктируйте своих пользователей, как они должны загрузить и установить дистрибутив `Anaconda`. Чтобы подстраховаться, вам придется согласиться на определенную версию `Anaconda`, дабы убедиться, что у пользователей используются те же версии пакетов, что и у вас. Хороший вариант, если вы используете только те пакеты, которые входят в состав `Anaconda`. `WinPython`² — достаточно интересная альтернатива `Anaconda`, поскольку распространяется под лицензией MIT с открытым исходным кодом, а также поставляется с предустановленным `xlwings`. Как следует из названия, он доступен только под Windows.

Диск общего доступа

Если у вас есть доступ к достаточно быстрому диску общего доступа, вы можете установить Python прямо на него, что позволит всем использовать инструменты без локальной установки Python.

² <https://oriel.ly/A66KN>. — Примеч. пер.

Заморозка исполняемого файла

В Windows `xlwings` позволяет работать с замороженными исполняемыми файлами, которые представляют собой файлы с расширением `.exe`, содержащие Python и все зависимости. Популярным пакетом для создания замороженных исполняемых файлов является `PyInstaller`³. Преимущество замороженных исполняемых файлов в том, что они упаковывают только то, что нужно вашей программе, и способны создавать один файл, что может облегчить распространение. Для получения более подробной информации о том, как работать с замороженными исполняемыми файлами, ознакомьтесь с документацией `xlwings`⁴. Обратите внимание, что замороженные исполняемые файлы не будут работать, если вы используете `xlwings` для пользовательских функций — функциональности, которую я представляю в *главе 12*.

Если установка Python является жестким требованием, то установка дополнения `xlwings` не является обязательной, о чем я расскажу далее.

Автономные рабочие книги: избавление от надстройки `xlwings`

В этой главе мы все время использовали надстройку `xlwings` и вызывали код Python либо нажатием на кнопку **Run main**, либо с помощью функции `RunPython`. Даже если `xlwings CLI` упрощает установку дополнения, это все равно может быть хлопотно для менее технически подкованных пользователей, которые не чувствуют себя комфортно при работе с `Anaconda Prompt`. Кроме того, поскольку дополнение `xlwings` и пакет `xlwings Python` должны иметь одну и ту же версию, вы можете столкнуться с конфликтом, когда у ваших получателей уже установлено дополнение `xlwings`, но его версия отличается от той, которая требуется вашему программному обеспечению. Однако существует более простое решение: `xlwings` не требует установки надстройки Excel и может быть настроена как *самостоятельная рабочая книга*. В этом случае VBA-код надстройки хранится непосредственно в рабочей книге. Как обычно, самый простой способ все настроить — это использовать команду `quickstart`, на этот раз с флагом `--standalone`:

```
(base)> xlwings quickstart second_project --standalone
```

Когда вы откроете созданную рабочую книгу `second_project.xlsm` в Excel и нажмете комбинацию клавиш `<Alt>+<F11>` (в Windows) или `<Option>-<F11>` (в macOS), вы увидите модуль `xlwings` и модуль класса `Dictionary`, которые требуются взамен надстройки. Очень важно, чтобы в автономном проекте больше не было ссылок на `xlwings`. Хотя это настраивается автоматически при использовании флага `--standalone`, важно удалить ссылку в случае, если вы хотите преобразовать существующую рабочую книгу: перейдите в **Tools > References** в вашем редакторе VBA и снимите флажок для `xlwings`.

³ <https://oriel.ly/A6nYIV>. — Примеч. пер.

⁴ <https://oriel.ly/QWz7i>. — Примеч. пер.



Создание пользовательской надстройки

Хотя в этом разделе показано, как избавиться от зависимости надстройки xlwings, иногда вам может понадобиться создать собственную надстройку для развертывания. Это имеет смысл, если вы хотите использовать одни и те же макросы в разных рабочих книгах. Инструкции по созданию собственной надстройки вы найдете в документации xlwings⁵.

Затронув тему Python и надстройки, давайте теперь более подробно рассмотрим, как работает конфигурация xlwings.

Иерархия конфигурации

Как упоминалось в начале этой главы, лента хранит свою конфигурацию в домашнем каталоге пользователя в файле `.xlwings\xlwings.conf`. *Конфигурация* состоит из отдельных *параметров*, таких как `PYTHONPATH`, которые мы уже рассматривали в начале этой главы. Параметры, установленные в вашей надстройке, могут быть отменены на уровне каталогов и рабочих книг — xlwings ищет параметры в следующих местах и в следующем порядке:

Конфигурация рабочей книги

Сначала xlwings ищет лист под названием `xlwings.conf`. Это рекомендуемый способ настройки рабочей книги для развертывания, поскольку вам не придется работать с дополнительным файлом конфигурации. Когда вы запустите команду `quickstart`, она создаст образец конфигурации на листе под названием `"_xlwings.conf"`: удалите переднее подчеркивание в имени, чтобы активировать его. Если вы не хотите его использовать, не стесняйтесь удалить этот лист.

Конфигурация каталогов

Далее xlwings ищет файл `xlwings.conf` в том же каталоге, что и рабочая книга Excel.

Пользовательская конфигурация

Наконец, xlwings ищет файл `xlwings.conf` в папке `.xlwings` в домашнем каталоге пользователя. Обычно вы не редактируете этот файл напрямую — вместо этого он создается и редактируется надстройкой всякий раз, когда вы изменяете настройки.

Если xlwings не находит никаких настроек в этих трех местах, он возвращается к значениям по умолчанию.

Когда вы редактируете настройки с помощью надстройки Excel, автоматически редактируется файл `xlwings.conf`. Если вы хотите отредактировать файл напрямую, посмотрите его формат и доступные настройки в документации xlwings⁶, а я укажу на наиболее полезные настройки в процессе развертывания.

⁵ <https://oriel.ly/hFvIj>. — Примеч. пер.

⁶ <https://oriel.ly/U9JTY>. — Примеч. пер.

Настройки

Самой важной настройкой, безусловно, является интерпретатор Python — если ваш инструмент Excel не может найти правильный интерпретатор Python, ничего не будет работать. Параметр `PYTHONPATH` позволяет вам контролировать местоположение исходных файлов Python, а параметр `Use UDF Server` поддерживает работу интерпретатора Python между вызовами в операционной системе Windows, что может значительно повысить производительность.

Интерпретатор Python

`xlwings` основывается на локально установленном Python. Однако это не означает, что пользователю, получившему ваш инструмент `xlwings`, придется возиться с конфигурацией, прежде чем он сможет использовать инструмент. Как упоминалось ранее, вы можете предложить им произвести установку дистрибутива Anaconda с настройками по умолчанию, что позволит установить `xlwings` в домашний каталог пользователя. Если вы используете *переменные окружения* в конфигурации, `xlwings` найдет правильный путь к интерпретатору Python. Переменная среды — это переменная, установленная на компьютере пользователя, которая позволяет программам запрашивать информацию, специфичную для этой среды, например имя домашней папки текущего пользователя. Так, в Windows следует установить `Conda Path` в `%USERPROFILE%\anaconda3`, а в macOS — `Interpreter_Mac` в `$HOME/opt/anaconda3/bin/python`. Эти пути затем динамически преобразуются в путь установки Anaconda по умолчанию.

PYTHONPATH

По умолчанию `xlwings` ищет исходный файл Python в том же каталоге, что и файл Excel. Это может оказаться не очень удачным решением, если вы предоставите свой инструмент пользователям, не знакомым с Python, потому что они при перемещении файла Excel могут забыть сохранить эти два файла вместе. Вместо этого вы можете поместить исходные файлы Python в специальную папку (она может находиться на общем диске) и добавить путь к этой папке в настройку `PYTHONPATH`. В качестве альтернативы можно поместить исходные файлы по пути, который уже является частью пути поиска модулей Python. Например, можно распространять исходный код в виде пакета Python — при установке пакета код будет помещен в каталог `site-packages` Python, где Python и найдет ваш код. Для получения дополнительной информации о том, как собрать пакет Python, см. *Руководство пользователя*⁷.

RunPython: Использование UDF-сервера (только для Windows)

Возможно, вы заметили, что запуск `RunPython` может быть довольно медленным. Это происходит потому, что `xlwings` запускает интерпретатор Python, выполняет код Python и в заключение выключает интерпретатор. Во время разработки это может оказаться не так уж и плохо, поскольку гарантирует, что все модули загружаются с нуля каждый раз, когда вы вызываете команду `RunPython`. Однако когда ваш код стабилизируется, вы, возможно, захотите установить **флажок**

⁷ https://oriel.ly/_kJoJ. — Примеч. пер.

RunPython: Use UDF Server, который доступен только в Windows. Это позволит задействовать тот же сервер Python, который используют определяемые пользователем функции (см. главу 12), и поддерживать сеанс Python между вызовами, что будет намного быстрее. Обратите внимание, что после внесения изменений в код необходимо на ленте нажать кнопку Restart UDF Server.

xlwings PRO

Хотя в этой книге используется только бесплатная версия xlwings с открытым исходным кодом, существует также коммерческий пакет PRO, предназначенный для финансирования дальнейшего поддержания и развития пакета с открытым исходным кодом. Ниже показаны дополнительные функции, предлагаемые xlwings PRO:

- ◆ код Python можно внедрить в Excel, избавившись тем самым от внешних исходных файлов;
- ◆ пакет отчетов позволяет превратить рабочие книги в шаблоны с использованием заполнителей. Это дает пользователям, не обладающим техническими знаниями, возможность редактировать шаблон без необходимости изменять код Python;
- ◆ чтобы избавиться от головной боли при развертывании, программа позволяет без особых усилий создать установщик: с его помощью обычные пользователи могут установить Python, включая все зависимости, одним щелчком мыши, что дает им ощущение работы с обычными рабочими книгами Excel без необходимости настраивать что-либо вручную.

Для получения более подробной информации о xlwings PRO и запроса пробной лицензии см. *домашнюю страницу*⁸.

Заключение

В начале этой главы мы показали, как Python-код легко запускается из Excel: если установлена Anaconda, достаточно запустить `xlwings addin install`, затем `xlwings quickstart myproject`, и все готово — можно использовать кнопку **Run main** в надстройке xlwings или функцию `RunPython VBA`. Во второй части этой главы было рассмотрено несколько настроек, облегчающих развертывание инструмента xlwings для конечных пользователей. Тот факт, что xlwings выпускается с предустановленной Anaconda, в значительной степени способствует снижению начальных требований для новых пользователей.

В этой главе, чтобы показать, как все работает, мы использовали пример Hello World. В следующей главе на основе этих знаний будет создан Python Package Tracker (трекер пакетов Python) — полноценное бизнес-приложение.

⁸ <https://oriel.ly/QEuoo>. — Примеч. пер.

Трекер пакетов Python

В этой главе мы создадим типичное бизнес-приложение, которое загружает данные из Интернета и хранит их в базе данных, а затем визуализирует в Excel. Данное приложение поможет вам понять, какую роль играет `xlwings` в подобном проекте, и позволит увидеть, насколько легко с помощью Python можно подключаться к внешним системам. В попытке создать проект, близкий к реальному приложению и при этом относительно простой в исполнении, я придумал *Python Package Tracker*, инструмент Excel, который показывает количество релизов в год для данного пакета Python. Несмотря на то, что это только пример, вы можете, если хотите понять, насколько активно разрабатывается определенный пакет Python, посчитать этот инструмент полезным.

После знакомства с приложением мы рассмотрим несколько тем, которые нам необходимо изучить, чтобы иметь возможность следить за кодом приложения. Мы, прежде чем мы узнаем об обработке исключений в Python — важной концепции, когда речь идет о разработке приложений, увидим, как можно загружать данные из Интернета и как взаимодействовать с базами данных. После того как мы закончим с этими предварительными действиями, рассмотрим компоненты *Python Package Tracker*, чтобы понять, как все сочетается друг с другом. В заключение этой главы будет показано, как происходит отладка кода `xlwings`. Как и в двух предыдущих главах, в этой главе вам потребуется Microsoft Excel, установленный на компьютер под управлением операционных систем Windows или macOS. Давайте начнем с пробного запуска *Python Package Tracker*.

Что мы будем создавать

Перейдите в партнерский репозиторий, где находится папка `packagetracker`. В этой папке хранится несколько файлов, но пока просто откройте файл Excel `package tracker.xlsm` и перейдите к листу `Database`: сначала нам нужно внести некоторые данные в базу данных, чтобы было с чем работать. Введите, как показано на рис. 11.1, имя пакета, например «`xlwings`», затем нажмите кнопку **Add Package**. Вы можете выбрать любое имя пакета, существующее в *Python Package Index (PyPI)*.



macOS: подтверждение доступа к папке

При добавлении самого первого пакета в macOS необходимо во всплывающем окне подтвердить свои действия, чтобы приложение могло получить доступ к папке package tracker. С этим всплывающим окном мы сталкивались уже в *главе 9*.

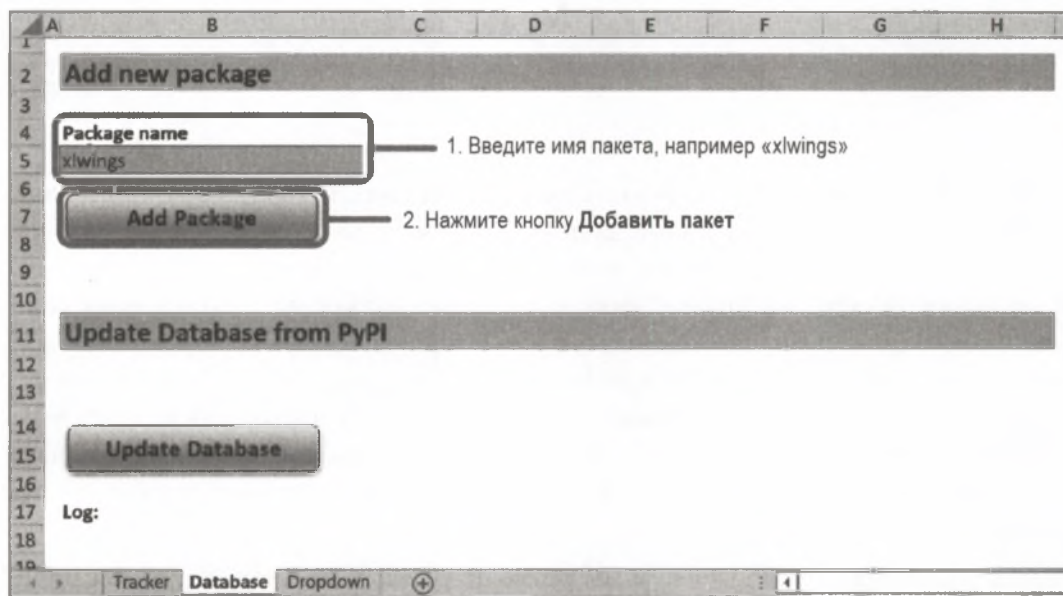


Рис. 11.1. Программа отслеживания пакетов Python
(лист базы данных)

Если все идет по плану, справа от того места, где вы вводили имя пакета, появится сообщение «Added xlwings successfully». Кроме того, в разделе **Update Database** вы увидите временную метку **Last updated**, а также обновленный раздел **Log**, в котором говорится об успешной загрузке xlwings и сохранении его в базе данных. Давайте еще добавим пакет pandas, чтобы у нас было больше данных для экспериментов. Далее перейдите на лист Tracker и в ячейке B5 выберите из выпадающего списка xlwings, а затем нажмите кнопку **Show History**. Теперь ваш экран должен выглядеть как на рис. 11.2, где показан последний выпуск пакета, а также график с количеством выпусков за несколько лет.

После этого вы можете вернуться к листу Database и добавить дополнительные пакеты. Всякий раз, когда вы пожелаете обновить базу данных, загрузив актуальную информацию из PyPI, нажмите на кнопку **Update Database**: это синхронизирует вашу базу данных с последними данными из PyPI.

Рассмотрев, как работает Python Package Tracker с точки зрения пользователя, давайте познакомимся с его основной функциональностью.

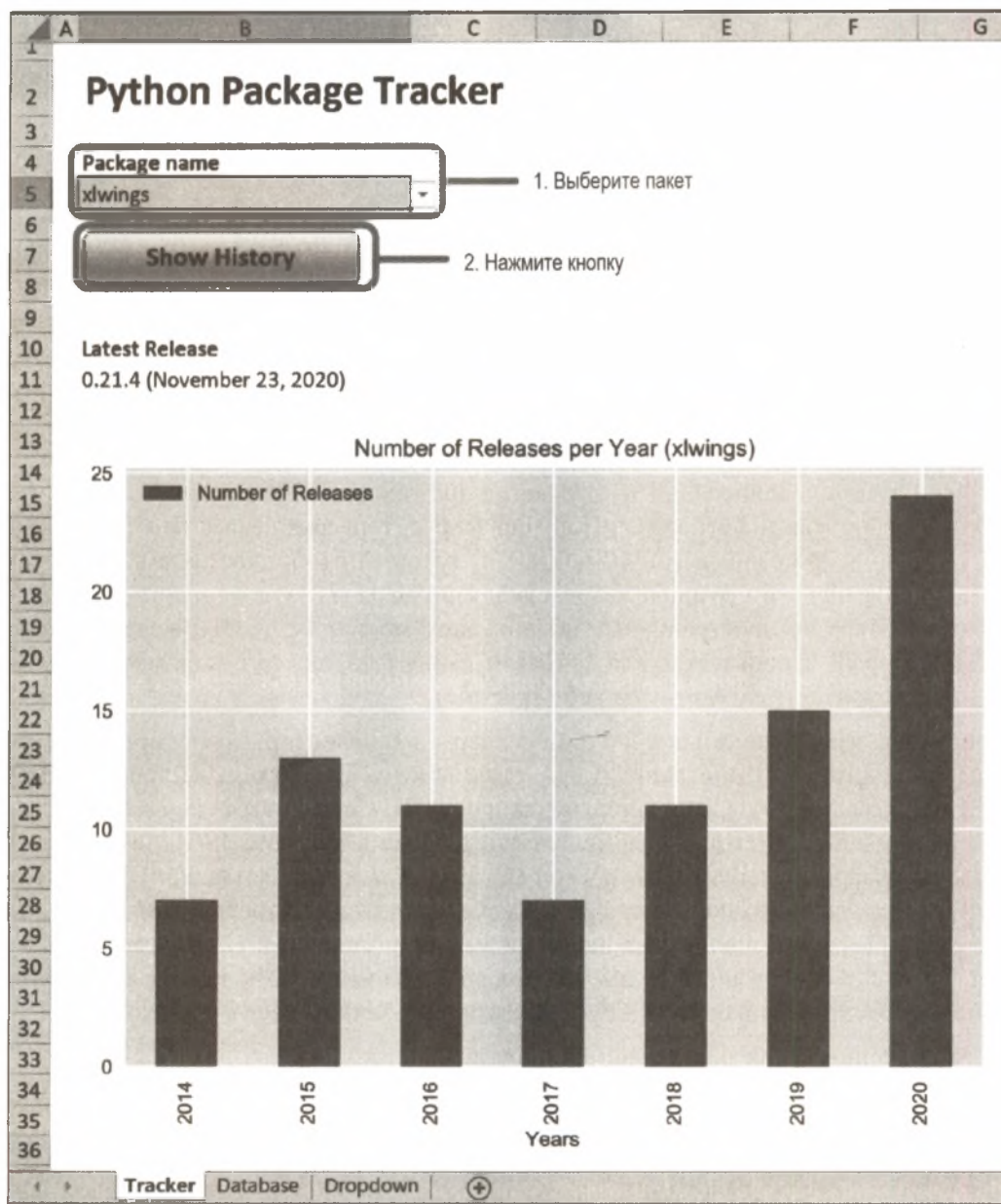


Рис. 11.2. Трекер пакетов Python (лист трекера)

Основной функционал

В этом разделе мы познакомимся с основными функциями Python Package Tracker: получение данных через веб-интерфейсы API и формирование запросов в базы данных. Мы также рассмотрим, как обрабатывать исключения — тему, которая не-

избежно возникнет при написании кода приложения. Давайте приступим к работе с веб-интерфейсами!

Web APIs

Веб-интерфейсы API являются одним из самых популярных способов получения данных из интернета: API означает *программный интерфейс приложения* (application programming interface) и определяет, как вы программно взаимодействуете с приложением. Таким образом, веб-API — это API, доступ к которому осуществляется через сеть, как правило, интернет. Чтобы понять, как работают веб-интерфейсы API, давайте сделаем шаг назад и посмотрим, что происходит (в упрощенном виде), когда вы обращаетесь к веб-странице в браузере: после ввода в адресную строку URL браузер отправляет GET-запрос на сервер, запрашивая нужную вам веб-страницу. Запрос GET — это метод протокола HTTP, который ваш браузер использует для связи с сервером. Получив запрос, сервер отвечает на него, отправляя обратно запрошенный HTML-документ, который отображается в вашем браузере: вуаля, ваша веб-страница загрузилась. Протокол HTTP содержит и другие методы; наиболее распространенным из них, помимо запроса GET, является запрос POST, который используется для отправки данных на сервер (например, когда вы заполняете контактную форму на веб-странице).

Но если для взаимодействия с людьми серверы должны отправлять обратно красиво оформленную HTML-страницу, приложения не заботятся о дизайне и интересуются только данными. Следовательно, запрос GET к веб-интерфейсу API работает как запрос веб-страницы, но вы получаете данные в формате JSON, а не HTML. JSON расшифровывается как *JavaScript Object Notation* и представляет собой структуру данных, которую понимает практически каждый язык программирования, что делает ее идеальной для обмена данными между различными системами. Хотя нотация использует синтаксис JavaScript, она очень близка к тому, как вы используете (вложенные) словари и списки в Python. Различия заключаются в следующем:

- ◆ JSON принимает только двойные кавычки для строк;
- ◆ В JSON используется `null`, а в Python — `None`;
- ◆ В JSON используются строчные `true` и `false`, в то время как в Python они пишутся с заглавной буквы;
- ◆ JSON принимает в качестве ключей только строки, в то время как словари Python принимают в качестве ключей широкий спектр объектов.

Модуль `json` из стандартной библиотеки позволяет преобразовать словарь Python в строку JSON и наоборот:

```
In [1]: import json
In [2]: # Словарь Python...
        user_dict = {"name": "Jane Doe",
                     "age": 23,
                     "married": False,
```

```

        "children": None,
        "hobbies": ["hiking", "reading"]}
In [3]: # ...конвертация в строку JSON.
        # json.dumps ("строка дампа"). Параметр "отступ" является
        # необязательным и просто улучшает печать.
        user_json = json.dumps(user_dict, indent=4)
        print(user_json)

{
    "name": "Jane Doe",
    "age": 23,
    "married": false,
    "children": null,
    "hobbies": [
        "hiking",
        "reading"
    ]
}
In [4]: # Преобразование строки JSON обратно в собственную структуру
        # данных Python
        json.loads(user_json)
Out[4]: {'name': 'Jane Doe',
        'age': 23,
        'married': False,
        'children': None,
        'hobbies': ['hiking', 'reading']}
```

REST API

Вместо web API часто можно встретить термин REST или RESTful API. REST расшифровывается как *representational state transfer* (передача состояния представления) и определяет веб-интерфейс API, который придерживается определенных ограничений. В своей основе идея REST заключается в том, что вы получаете доступ к информации в виде *ресурсов без статических данных*. Stateless (нестационарный) означает, что каждый запрос к REST API полностью независим от любого другого запроса и должен всегда предоставлять полный набор информации, которая запрашивается. Обратите внимание, что термин REST API часто неправильно используется для обозначения любого вида веб-интерфейса, даже если он не придерживается ограничений REST.

Использовать веб-интерфейсы API обычно очень просто (мы скоро увидим, как это работает с Python), и почти каждый сервис предлагает такие API. Если вы хотите загрузить свой любимый плейлист Spotify, вы должны отправить следующий запрос GET (см. *Spotify Web API Reference*¹):

```
GET https://api.spotify.com/v1/playlists/playlist_id
```

¹ <https://oriel.ly/zcyUh>. — Примеч. пер.

Или, если вы хотите получить информацию о своих последних поездках в Uber, выполните следующий запрос GET (см. *Uber REST API*²):

```
GET https://api.uber.com/v1.2/history
```

Однако для использования этих API необходимо пройти аутентификацию, а это означает наличие учетной записи и маркера, который можно отправить вместе с запросом. Для Python Package Tracker нам понадобится получить данные из PyPI, чтобы собрать информацию о релизах определенного пакета. К счастью, веб-интерфейс PyPI не требует аутентификации, поэтому нам не о чем беспокоиться. Если вы посмотрите на документацию PyPI JSON API, вы увидите, что есть только две конечные точки, то есть фрагменты URL, которые добавляются к общему базовому URL, **https://pypi.org/pypi**:

```
GET /project_name/json
```

```
GET /project_name/version/json
```

Вторая конечная точка дает вам ту же информацию, что и первая, но только для конкретной версии. Для Python Package Tracker первая конечная точка — это все, что нам нужно для получения подробной информации о прошлых выпусках пакета, поэтому давайте посмотрим, как это работает. В Python для взаимодействия с веб-API достаточно использовать пакет Requests, который предустановлен в Anaconda. Выполните следующие команды, чтобы получить данные PyPI о pandas:

```
In [5]: import requests
```

```
In [6]: response = requests.get("https://pypi.org/pypi/pandas/json")
        response.status_code
```

```
Out[6]: 200
```

Каждый ответ сопровождается кодом состояния HTTP: например, 200 означает «ОК», а 404 означает «Не найдено». Полный список кодов состояния ответа HTTP можно найти в веб-документации Mozilla³. Вам может быть знаком код состояния 404, поскольку ваш браузер иногда выводит его, когда вы пытаетесь перейти по недоступной ссылке или вводите несуществующий адрес. Точно так же вы получите код состояния 404, если выполните GET-запрос с именем пакета, которого не существует на PyPI. Чтобы просмотреть содержимое полученного ответа, удобнее всего вызвать метод `json` объекта ответа, который преобразует JSON-строку ответа в словарь Python:

```
In [7]: response.json()
```

Ответ очень длинный, поэтому я печатаю здесь короткую часть, чтобы вы могли разобраться в его структуре:

```
Out[7]: {
    'info': {
        'bugtrack_url': None,
        'license': 'BSD',
        'maintainer': 'The PyData Development Team',
```

² <https://oriel.ly/FTp-Y>. — Примеч. пер.

³ <https://oriel.ly/HySVq>. — Примеч. пер.


```

        'maintainer_email': 'pydata@googlegroups.com',
        'name': 'pandas'
    },
    'releases': {
        '0.1': [
            {
                'filename': 'pandas-0.1.tar.gz',
                'size': 238458,
                'upload_time': '2009-12-25T23:58:31'
            },
            {
                'filename': 'pandas-0.1.win32-py2.5.exe',
                'size': 313639,
                'upload_time': '2009-12-26T17:14:35'
            }
        ]
    }
}

```

Чтобы получить список со всеми релизами и их датами, необходимыми для Python Package Tracker, мы можем выполнить следующий код, чтобы просмотреть словарь релизов:

```

In [8]: releases = []
        for version, files in response.json()['releases'].items():
            releases.append(f"{version}: {files[0]['upload_time']}")
        releases[:3] # показать первые 3 элемента списка
Out[8]: ['0.1: 2009-12-25T23:58:31',
        '0.10.0: 2012-12-17T16:52:06',
        '0.10.1: 2013-01-22T05:22:09']

```

Обратите внимание, что мы выбираем первый появившийся в списке пакет с произвольной временной меткой. Конкретный выпуск часто содержит несколько пакетов для различных версий Python и операционных систем. Завершая эту тему, напомним главу 5: в pandas есть метод `read_json` для возврата DataFrame непосредственно из строки JSON. Однако в данном случае это нам не поможет, поскольку ответ от PyPI не имеет структуры, которая может быть непосредственно преобразована в DataFrame.

Это было краткое введение в веб-интерфейсы API, чтобы понять, как их использовать в кодовой базе Python Package Tracker. Давайте теперь посмотрим, как мы можем взаимодействовать с базами данных, другой внешней системой, которую мы используем в нашем приложении.

Базы данных

Чтобы иметь возможность использовать данные из PyPI, даже когда вы не подключены к интернету, вам необходимо сохранить их после загрузки. Хотя вы можете хранить ваши JSON ответы в виде текстовых файлов на диске, намного удобнее

использовать базу данных: это позволит вам легко запрашивать ваши данные. Трекер пакетов Python использует SQLite, *реляционную базу данных*. Реляционные системы баз данных получили свое название от слова *relation* (взаимосвязь), которое относится к самой таблице базы данных (а не к связи между таблицами, что является распространенным заблуждением). Их главная задача — целостность данных, которая достигается путем разбиения данных на различные таблицы (этот процесс называется нормализацией) и применением ограничений, чтобы избежать противоречивых и избыточных данных. Реляционные базы данных используют язык SQL (Structured Query Language, или Язык структурированных запросов) для выполнения запросов к базе данных и являются одними из самых популярных серверных систем реляционных баз данных — SQL Server⁴, Oracle⁵, PostgreSQL⁶ и MySQL⁷. Как пользователь Excel, вы также можете быть знакомы с файловой базой данных Microsoft Access⁸.

Базы данных NoSQL

В наши дни реляционные базы данных сталкиваются с сильной конкуренцией со стороны баз данных NoSQL, которые хранят избыточные данные, стремясь достичь следующих преимуществ:

Отсутствие объединений таблиц

Поскольку в реляционных базах данных данные распределяются по нескольким таблицам, часто требуется *объединить* информацию из двух или более таблиц методом соединения, что может оказаться медленным. В базах данных NoSQL этого не требуется, что может привести к повышению производительности для определенных типов запросов.

Отсутствие миграций баз данных

В реляционных системах баз данных каждый раз, когда вы вносите изменения в структуру таблицы, например добавляете новый столбец в таблицу, вы должны выполнить *миграцию* базы данных. Миграция — это сценарий, который приводит базу данных к желаемой новой структуре. Это усложняет развертывание новых версий приложения и может привести к простоям, которого легче избежать, используя базы данных NoSQL.

Более легкое масштабирование

Базы данных NoSQL легче распределять по нескольким серверам, поскольку в них нет таблиц, которые зависят друг от друга. Это означает, что приложение, использующее базу данных NoSQL, может легче расширяться при резком увеличении пользовательской базы.

⁴ <https://oriel.ly/XZOI9>. — Примеч. пер.

⁵ <https://oriel.ly/VKWE0>. — Примеч. пер.

⁶ <https://oriel.ly/VKEqY>. — Примеч. пер.

⁷ <https://www.mysql.com/>. — Примеч. пер.

⁸ <https://oriel.ly/bRh6Q>. — Примеч. пер.

Базы данных NoSQL бывают разных видов: некоторые представляют собой простые хранилища ключевых значений, то есть работают аналогично словарю в Python (например, Redis⁹); другие позволяют хранить документы, часто в формате JSON (например, MongoDB¹⁰). Некоторые базы данных даже объединяют реляционный и NoSQL миры: PostgreSQL, база данных, одна из самых популярных в сообществе Python, традиционно является реляционной базой данных, но при этом позволяет хранить данные в формате JSON — без потери возможности запрашивать их через SQL.

SQLite, база данных, которую мы будем использовать, — это база данных на основе файлов наподобие Microsoft Access. Однако, в отличие от Microsoft Access, работающего только под управлением операционной системы Windows, SQLite применим ко всем платформам, которые поддерживает Python. С другой стороны, SQLite не позволяет создавать, как Microsoft Access, пользовательский интерфейс, поэтому здесь пригодится Excel. Давайте теперь посмотрим на структуру базы данных Package Tracker, а затем узнаем, как мы можем использовать Python для подключения к базам данных и выполнения SQL-запросов. И в конце этого введения о базах данных мы рассмотрим SQL-инъекции, распространенную уязвимость приложений, управляемых базами данных.

База данных трекера пакетов

База данных трекера пакетов Python проще быть и не может, потому что состоит всего лишь из двух таблиц: в таблице `packages` хранится имя пакета, а в таблице `package_versions` — информация о версиях и датах загрузки. Эти две таблицы могут быть объединены по `package_id`: вместо того, чтобы хранить `package_name` с каждой строкой в таблице `package_versions`, это имя было нормализовано в таблице `packages`. Такой подход позволяет избавиться от лишних данных — например, при изменении имен нужно заполнять только одно поле во всей базе данных. Чтобы получить представление о том, как выглядит база данных с загруженными пакетами `xlwings` и `pandas`, посмотрите табл. 11.1 и 11.2.

Таблица 11.1. Таблица пакетов

<code>package_id</code>	<code>package_name</code>
1	xlwings
2	pandas

⁹ <https://redis.io/>. — Примеч. пер.

¹⁰ <https://mongodb.com/>. — Примеч. пер.

Таблица 11.2. Таблица package_versions (первые три строки каждого package_id)

package_id	version_string	uploaded_at
1	0.1.0	2014-03-19 18:18:49.000000
1	0.1.1	2014-06-27 16:26:36.000000
1	0.2.0	2014-07-29 17:14:22.000000
...	...	...
2	0.1	2009-12-25 23:58:31.000000
2	0.2beta	2010-05-18 15:05:11.000000
2	0.2b1	2010-05-18 15:09:05.000000
...	...	...

На рис. 11.3 показана диаграмма базы данных, на которой схематично изображены две таблицы, приведенные выше. Вы можете прочитать имена таблиц и столбцов и получить информацию о первичных и внешних ключах:

Первичный ключ

Реляционные базы данных требуют, чтобы каждая таблица имела *первичный ключ* (*primary key*). Первичный ключ — это один или несколько столбцов, которые однозначно идентифицируют строку (строка также называется записью). В случае таблицы packages первичным ключом является package_id, а в случае таблицы package_versions — так называемый составной ключ, т. е. комбинация package_id и version_string.

Внешний ключ

Колонка package_id в таблице package_versions является *внешним ключом* (*foreign key*) к той же колонке в таблице packages, символизируемой линией, соединяющей таблицы: внешний ключ — это ограничение, которое в нашем случае гарантирует, что каждый package_id в таблице package_versions существует в таблице packages — что гарантирует целостность данных. Ветви на правом конце соединительной линии символизируют природу отношений: один пакет может иметь много версий package_versions, что называется отношением *один ко многим*.

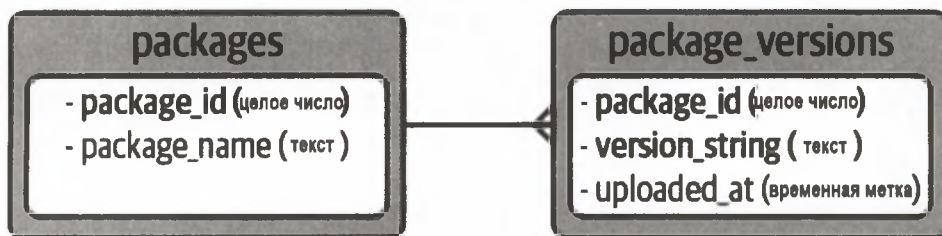


Рис. 11.3. Диаграмма базы данных (первичные ключи выделены жирным шрифтом)

Чтобы просмотреть содержимое таблиц базы данных и выполнить SQL-запросы, следует установить расширение в VS Code под названием SQLite (*подробнее см. документацию по расширению SQLite¹¹⁾*) или использовать специальное программное обеспечение для управления SQLite, которых существует множество. Однако мы для выполнения SQL-запросов будем использовать Python. Прежде всего, давайте посмотрим, как мы можем подключиться к базе данных!

Подключение к базе данных

Чтобы подключиться к базе данных из Python, вам нужен драйвер, т.е. пакет Python, который знает, как взаимодействовать с используемой базой данных. Для каждой базы данных требуется свой драйвер, и каждый драйвер использует свой синтаксис. Но, к счастью, существует мощный пакет под названием SQLAlchemy¹²⁾, который устраняет большинство различий между различными базами данных и драйверами. SQLAlchemy в основном используется как *объектно-реляционный картограф* (object relational Mapper или ORM), который переводит записи вашей базы данных в объекты Python, концепция, которую многие разработчики, хотя и не все, считают более естественной для работы. Для простоты мы игнорируем функциональность ORM и используем SQLAlchemy только для облегчения выполнения необработанных SQL-запросов. SQLAlchemy также работает скрытно, когда вы используете pandas для чтения и записи таблиц базы данных в виде DataFrames. Выполнение запроса к базе данных из pandas, как показано на рис. 11.4, включает три уровня пакетов pandas, SQLAlchemy и драйвера базы данных. Вы можете выполнять запросы к базе данных с каждого из этих трех уровней.

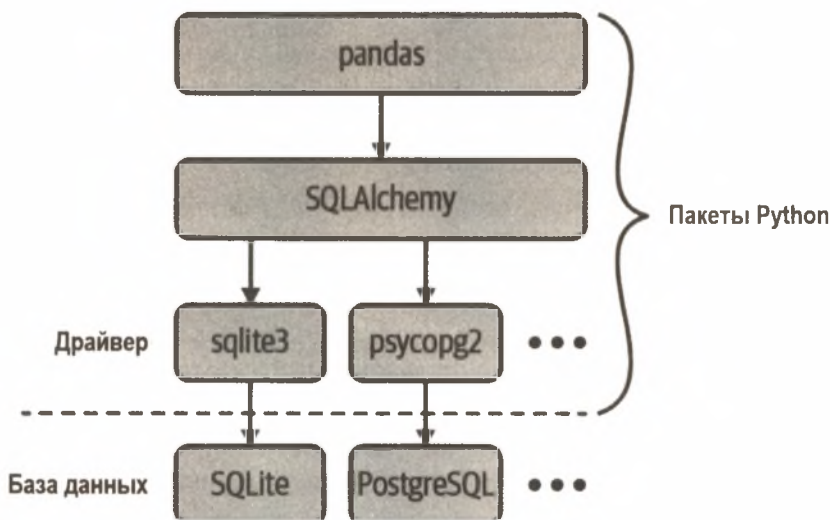


Рис. 11.4. Доступ к базам данных из Python

¹¹ <https://oriel.ly/nP4mC>. — Примеч. пер.

¹² <https://sqlalchemy.org/>. — Примеч. пер.

В табл. 11.3 показано, какой драйвер SQLAlchemy используется по умолчанию (некоторые базы данных могут использоваться с более чем одним драйвером). Здесь также указан формат строки подключения к базе данных — строку подключения мы будем использовать в ближайшее время, когда нам придется выполнять реальные SQL-запросы.

Таблица 11.3. Драйверы и строки подключения SQLAlchemy по умолчанию

База данных	Драйвер по умолчанию	Строка подключения
SQLite	sqlite3	sqlite:///filepath
PostgreSQL	psycopg2	postgresql://username:password@host:port/database
MySQL	mysql-python	mysql://username:password@host:port/database
Oracle	cx_oracle	oracle://username:password@host:port/database
SQL Server	pyodbc	mssql+pyodbc://username:password@host:port/database

За исключением SQLite, для подключения к базе данных обычно требуется пароль. А поскольку строки подключения — это URL-адреса, вам придется использовать кодированную версию ваших паролей, если в них есть специальные символы. Вот как можно вывести URL-кодированную версию пароля:

```
In [9]: import urllib.parse
In [10]: urllib.parse.quote_plus("pa$$word")
Out[10]: 'pa%24%24word'
```

Представив pandas, SQLAlchemy и драйвер базы данных как три уровня, с которых мы можем подключаться к базам данных, давайте посмотрим, как они взаимодействуют на практике, сделав несколько SQL-запросов.

SQL-запросы

Даже если вы новичок в SQL, у вас не должно возникнуть проблем с пониманием нескольких SQL-запросов, которые я буду использовать в следующих примерах и в трекере пакетов Python. SQL — это *декларативный язык*, а это значит, что вы говорите базе данных, *что вы хотите*, а не указываете, *что базе данных делать*. Некоторые запросы читаются почти как обычный английский язык:

```
SELECT * FROM packages
```

Таким образом, вы сообщаете базе данных, что хотите выбрать все столбцы из таблицы пакетов. В производственном коде не стоит использовать подстановочный знак *, который означает все столбцы. Гораздо лучше указать каждый столбец в явном виде, так как в этом случае запрос будет менее подвержен ошибкам:

```
SELECT package_id, package_name FROM packages
```




Запросы к базе данных и pandas DataFrames

SQL — язык, основанный на множествах, и это означает, что вы работаете с набором строк, а не перебираете отдельные строки. Это очень похоже на то, как вы работаете с pandas DataFrames. SQL-запрос:

```
SELECT package_id, package_name FROM packages
```

соответствует следующему выражению pandas (предполагается, что packages — это DataFrame):

```
packages.loc[:, ["package_id", "package_name"]]
```

В приведенных ниже примерах кода используется файл packagetracker.db, который вы найдете в папке packagetracker сопутствующего репозитория. В примерах предполагается, что вы уже добавили xlwings и pandas в базу данных через внешний интерфейс Excel в Python Package Tracker, как мы делали в начале этой главы — в противном случае вы получите только пустые значения. Следуя схеме на рис. 11.4 снизу вверх, мы сначала сделаем наш SQL-запрос непосредственно из драйвера, затем воспользуемся SQLAlchemy и, наконец, pandas:

```
In [11]: # Начнем с импорта
import sqlite3
from sqlalchemy import create_engine
import pandas as pd

In [12]: # Наш SQL-запрос: "выбрать все столбцы из таблицы пакетов"
sql = "SELECT * FROM packages"

In [13]: # Вариант 1: Драйвер базы данных (sqlite3 является частью
# стандартной библиотеки)
# Использование соединения в качестве менеджера контекста
# автоматически фиксирует транзакцию или откатывает ее в случае
# ошибки.
with sqlite3.connect("packagetracker/packagetracker.db") as con:
    cursor = con.cursor() # Нам нужен курсор для выполнения
    # SQL-запросов
    result = cursor.execute(sql).fetchall() # Возврат всех
    # записей
result
Out[13]: [(1, 'xlwings'), (2, 'pandas')]

In [14]: # Вариант 2: SQLAlchemy
# "create_engine" ожидает строку подключения вашей базы данных.
# Здесь мы можем выполнить запрос как метод объекта connection.
engine = create_engine("sqlite:///packagetracker/packagetracker.db")
with engine.connect() as con:
    result = con.execute(sql).fetchall()
result
Out[14]: [(1, 'xlwings'), (2, 'pandas')]

In [15]: # Вариант 3: pandas
# При предоставлении имени таблицы команде "read_sql"
# считывается полная таблица.
```



```

# Для Pandas требуется инструмент SQLAlchemy, который мы
# повторно используем из предыдущего примера.
df = pd.read_sql("packages", engine, index_col="package_id")
df
Out[15]:      package_name
package_id
1          xlwings
2          pandas

In [16]: # "read_sql" также принимает SQL-запрос
pd.read_sql(sql, engine, index_col="package_id")
Out[16]:      package_name
package_id
1          xlwings
2          pandas

In [17]: # Метод DataFrame "to_sql" записывает DataFrame в
# таблицы "if_exists" должен быть либо "fail", либо "append",
# либо "replace" и определяет, что произойдет, если таблица
# уже существует
df.to_sql("packages2", con=engine, if_exists="append")

In [18]: # Предыдущая команда создала новую таблицу "packages2" и
# вставила записи из DataFrame df, в чем мы можем убедиться,
# прочитав его в обратном порядке
pd.read_sql("packages2", engine, index_col="package_id")
Out[18]:      package_name
package_id
1          xlwings
2          pandas

In [19]: # Теперь давайте повторно избавимся от таблицы, выполнив команду
# "drop table" через SQLAlchemy
with engine.connect() as con:
    con.execute("DROP TABLE packages2")

```

Что использовать для выполнения запросов — драйвер базы данных, SQLAlchemy или pandas — во многом зависит от ваших предпочтений: я предпочитаю более точный контроль, который можно получить, воспользовавшись SQLAlchemy, и мне нравится, что я могу использовать один и тот же синтаксис с разными базами данных. С другой стороны, `read_sql` из pandas удобен для получения результата запроса в виде DataFrame.



Внешние ключи в SQLite

Как ни странно, SQLite по умолчанию не использует внешние ключи при выполнении запросов. Однако, если вы используете SQLAlchemy, вы можете легко обеспечить принудительное использование внешних ключей (см. документацию по SQLAlchemy¹³). Это также будет работать, если вы запустите запросы из pandas.

¹³ <https://oriel.ly/6YPvC>. — Примеч. пер.

Соответствующий код вы найдете в верхней части модуля `database.py` в папке `packagetracker` партнерского репозитория.

Теперь, когда вы знаете, как выполнять простые SQL-запросы, давайте завершим этот раздел рассмотрением SQL-инъекций, которые могут создавать угрозу безопасности вашего приложения.

SQL-инъекция

Если вы не обеспечите надлежащую защиту SQL-запросов, злоумышленник может запустить произвольный SQL-код, внедрив SQL-запросы в поля ввода данных: например, вместо выбора имени пакета, например `xlwings`, в открывающемся списке Python Package Tracker он может отправить SQL-запрос, который изменит ваш предполагаемый запрос. В результате может быть раскрыта конфиденциальная информация или выполнены деструктивные действия, например удаление таблицы. Как это можно предотвратить? Для начала давайте рассмотрим следующий запрос к базе данных, который Package Tracker выполняет, когда вы выбираете `xlwings` и нажимаете кнопку **Show History**¹⁴:

```
SELECT v.uploaded_at, v.version_string
FROM packages p
INNER JOIN package_versions v ON p.package_id = v.package_id
WHERE p.package_id = 1
```

Этот запрос объединяет две таблицы вместе и возвращает только те строки, в которых `package_id` равен 1. Чтобы облегчить понимание этого запроса на основе изученного в *главе 5*, если `package` и `package_versions` были бы `pandas DataFrames`, вы могли бы написать:

```
df = packages.merge(package_versions, how="inner", on="package_id")
df.loc[df["package_id"] == 1, ["uploaded_at", "version_string"]]
```

Очевидно, что `package_id`, чтобы возвращать правильные строки в зависимости от выбранного пакета, должен быть переменной, в которой сейчас хранится записанная 1. Зная из *главы 3* о форматировании строк, вы можете поддаться искушению и изменить последнюю строку SQL-запроса следующим образом:

```
f"WHERE p.package_id = {package_id}"
```

Несмотря на то, что с технической точки зрения такой вариант работает, вы никогда не должны так поступать, поскольку это открывает возможности для SQL-инъекций: например, кто-то может отправить `'1 OR TRUE'` вместо целого числа, представляющего `package_id`. Результирующий запрос вернет строки всей таблицы, а не только те, где `package_id` равен 1. Поэтому всегда используйте синтаксис, который SQLAlchemy предлагает вам для заполнителей (они начинаются с двоеточия):

```
In [20]: # Для начала импортируем текстовую функцию SQLAlchemy
         from sqlalchemy.sql import text
```

¹⁴ В действительности инструмент для упрощения кода вместо `package_id` использует `package_name`.

```
In [21]: # ":package_id" является заполнителем
        sql = """
        SELECT v.uploaded_at, v.version_string
        FROM packages p
        INNER JOIN package_versions v ON p.package_id = v.package_id
        WHERE p.package_id = :package_id
        ORDER BY v.uploaded_at
        """
```

```
In [22]: # С помощью SQLAlchemy
        with engine.connect() as con:
            result = con.execute(text(sql), package_id=1).fetchall()
            result[:3] # Вывести первые 3 записи
```

```
Out[22]: [('2014-03-19 18:18:49.000000', '0.1.0'),
          ('2014-06-27 16:26:36.000000', '0.1.1'),
          ('2014-07-29 17:14:22.000000', '0.2.0')]
```

```
In [23]: # С помощью pandas
        pd.read_sql(text(sql), engine, parse_dates=["uploaded_at"],
                    params={"package_id": 1},
                    index_col=["uploaded_at"]).head(3)
```

```
Out[23]:
            version_string
uploaded_at
2014-03-19 18:18:49      0.1.0
2014-06-27 16:26:36      0.1.1
2014-07-29 17:14:22      0.2.0
```

Использование текстовой функции SQLAlchemy для упаковки SQL-запроса имеет то преимущество, что вы можете использовать один и тот же синтаксис для заполнителей в разных базах данных. В обратном случае вам придется применить заполнитель, который использует драйвер базы данных: например, `sqlite3` для заполнения использует `?`, а `psycopg2` — `%s`.

Вы можете сказать, что SQL-инъекции не представляют особой проблемы, когда ваши пользователи имеют прямой доступ к Python и в любом случае могут выполнить произвольный код в базе данных. Но если вы возьмете свой прототип `xlwings` и однажды превратите его в веб-приложение, это станет огромной проблемой, поэтому лучше сделать все правильно с самого начала.

Помимо веб-интерфейсов API и баз данных есть еще одна тема, которую мы до сих пор обходили стороной и без которой невозможно обойтись при разработке надежных приложений: это обработка исключений. Давайте посмотрим, как это работает!

Исключения

В главе 1 я упомянул обработку исключений, как пример того, в чем VBA с его механизмом `GoTo` отстает от других. В этом разделе я покажу вам, как Python использует механизм `try/except` для обработки ошибок в ваших программах. Всякий раз,

когда что-то находится вне вашего контроля, могут и будут происходить ошибки. Например, почтовый сервер откажется работать, когда вы пытаетесь отправить письмо, или отсутствует файл, который ожидает ваша программа, — в случае с Python Package Tracker это может быть файл базы данных. Работа с пользовательским вводом — это еще одна область, где вы должны быть готовы к вводам, не имеющим никакого смысла. Давайте немного попрактикуемся — если следующая вызванная функция будет иметь значение ноль, то будет выдана ошибка `ZeroDivisionError`:

```
In [24]: def print_reciprocal(number):
```

```
    result = 1 / number
```

```
    print(f"The reciprocal is: {result}")
```

```
In [25]: print_reciprocal(0) # Это приведет к возникновению ошибки
```

```
-----
ZeroDivisionError                                Traceback (most recent call last)
```

```
<ipython-input-25-095f19ebb9e9> in <module>
```

```
----> 1 print_reciprocal(0) # This will raise an error
```

```
<ipython-input-24-88fd8a4711> in print_reciprocal(number)
```

```
    1 def print_reciprocal(number):
```

```
----> 2     result = 1 / number
```

```
    3     print(f"The reciprocal is: {result}")
```

```
ZeroDivisionError: division by zero
```

Чтобы ваша программа гибко реагировала на такие ошибки, используйте операторы `try/except` (это равносильно примеру из VBA в главе 1):

```
In [26]: def print_reciprocal(number):
```

```
    try:
```

```
        result = 1 / number
```

```
    except Exception as e:
```

```
        # "as e" делает объект Exception доступным в качестве
```

```
        # переменной "e"
```

```
        # "repr" означает "печатное представление" объекта
```

```
        # и возвращает вам строку с сообщением об ошибке
```

```
        print(f"There was an error: {repr(e)}")
```

```
        result = "N/A"
```

```
    else:
```

```
        print("There was no error!")
```

```
    finally:
```

```
        print(f"The reciprocal is: {result}")
```

При возникновении ошибки в блоке `try` выполнение кода переходит к блоку `except`, где можно обработать ошибку: это позволяет дать пользователю полезную информацию или записать ошибку в файл журнала. Условие `else` выполняется только в том случае, если в блоке `try` не возникла ошибка, а блок `finally` выполняется всегда, независимо от того, возникла ошибка или нет. Часто можно обойтись только

попыткой и исключением блоков. Давайте посмотрим на выход функции при различных входных данных:

```
In [27]: print_reciprocal(10)
There was no error!
The reciprocal is: 0.1
In [28]: print_reciprocal("a")
There was an error: TypeError("unsupported operand type(s) for /: 'int'
and 'str'")
The reciprocal is: N/A
In [29]: print_reciprocal(0)
There was an error: ZeroDivisionError('division by zero')
The reciprocal is: N/A
```

То, как я использовал оператор `except`, означает, что любое исключение, которое произойдет в блоке `try`, приведет к продолжению выполнения кода в блоке `except`. Обычно это не то, что вам нужно. Вам нужно проверять наличие ошибок настолько конкретно, насколько это возможно и обрабатывать только те, которые вы ожидаете. В противном случае ваша программа может не сработать по какой-то совершенно неожиданной причине, что затруднит ее отладку. Чтобы исправить это, перепишите функцию следующим образом, явно проверяя две ошибки, которые мы ожидаем (я убираю операторы `else` и `finally`):

```
In [30]: def print_reciprocal(number):
    try:
        result = 1 / number
        print(f"The reciprocal is: {result}")
    except (TypeError, ZeroDivisionError):
        print("Please type in any number except 0.")
```

Let's run the code again:

```
In [31]: print_reciprocal("a")
Please type in any number except 0.
```

Если вы хотите обрабатывать ошибки в зависимости от исключения по-разному, обрабатывайте их по отдельности:

```
In [32]: def print_reciprocal(number):
    try:
        result = 1 / number
        print(f"The reciprocal is: {result}")
    except TypeError:
        print("Please type in a number.")
    except ZeroDivisionError:
        print("The reciprocal of 0 is not defined.")
```

```
In [33]: print_reciprocal("a")
Please type in a number.
In [34]: print_reciprocal(0)
The reciprocal of 0 is not defined.
```

Теперь, когда вы узнали об обработке ошибок, веб-интерфейсах API и базах данных, вы готовы перейти к следующему разделу, где мы рассмотрим каждый компонент Python Package Tracker.

Структура приложения

В этом разделе мы заглянем за кулисы Python Package Tracker, чтобы понять, как все работает. Сначала мы рассмотрим интерфейс приложения, то есть файл Excel, а затем его внутреннюю часть — код на языке Python. В заключение этого раздела мы познакомимся с тем, как происходит отладка проекта xlwings, что весьма полезно при работе с проектами такого размера и сложности, как Package Tracker.

В каталоге packagetracker, в сопутствующем репозитории вы найдете четыре файла. Помните, в *главе 1* я говорил о разделении задач? Теперь мы можем сопоставить эти файлы с различными слоями, как показано в табл. 11.4:

Таблица 11.4. Разделение задач

Слой	Файл	Описание
Слой представления	packagetracker.xlsm	Это внешний интерфейс и соответственно единственный файл, с которым взаимодействует конечный пользователь
Слой бизнес-логики	packagetracker.py	Этот модуль обрабатывает загрузку данных через веб-интерфейс API и выполняет подсчет с помощью pandas
Слой данных	database.py	Этот модуль обрабатывает все запросы к базе данных
База данных	packagetracker.db	Это файл базы данных SQLite

В этом контексте стоит отметить, что слой представления, то есть файл Excel, не содержит ни одной ячейки с формулами, что значительно облегчает аудит и контроль инструмента.

Model-View-Controller (MVC)

У разделения задач много граней, и разбивка, показанная в табл. 11.4, является лишь одним из вариантов. Другой популярный шаблон проектирования, с которым вы можете столкнуться в ближайшее время, называется MVC. В среде MVC ядром приложения является модель, в которой обрабатываются все данные и, как правило, большая часть бизнес-логики. В то время как представление (View) соответствует слою представления, контроллер (Controller) — это лишь тонкий слой, который находится между моделью и представлением, чтобы обеспечить их постоянную синхронизацию. Чтобы упростить ситуацию, в этой книге я не использую модель MVC.

Теперь, когда вы знаете, за что отвечает каждый файл, давайте перейдем к более подробному рассмотрению настройки внешнего интерфейса Excel!

Внешний интерфейс

Когда вы создаете веб-приложение, необходимо разграничить *внешний интерфейс*, который представляет собой часть приложения, запускаемую в браузере, и *внутренний интерфейс*, представляющий собой код, который выполняется на сервере. Мы можем применить ту же терминологию к инструментам xlwings: внешний интерфейс — это файл Excel, а внутренний интерфейс — это код Python, который вы вызываете с помощью функции RunPython. Если вы хотите создать внешний интерфейс с нуля, начните с выполнения следующей команды в Anaconda Prompt (не забудьте сначала с помощью команды `cd` зайти в каталог по вашему выбору):

```
(base)> xlwings quickstart packagetracker
```

Перейдите в каталог `packagetracker` и откройте в Excel файл `packagetracker.xlsm`. Начните с добавления трех вкладок: `Tracker`, `Database` и `Dropdown`, как показано на рис. 11.5.

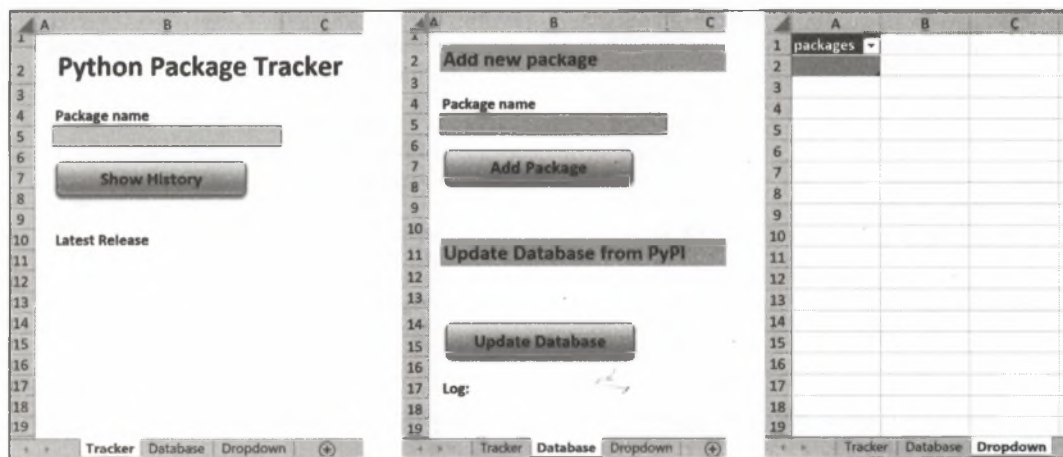


Рис. 11.5. Построение пользовательского интерфейса

Несмотря на то, что вы можете самостоятельно разобраться с текстом и его форматированием, чтобы у вас получилось, как показано на рис. 11.5, мне нужно дать вам несколько дополнительных сведений о том, что здесь не показано:

Кнопки

Чтобы инструмент меньше напоминал Windows 3.1, я не стал задействовать стандартные кнопки макросов, которые мы использовали в предыдущей главе. Вместо этого я перешел в меню **Вставка > Фигуры** и вставил **Скругленный прямоугольник**. Если вы хотите использовать стандартную кнопку, просто не назначайте макрос, создающий закругленные углы на кнопке.

Именованные диапазоны

Чтобы немного упростить обслуживание инструмента, мы будем использовать именованные диапазоны, а не адреса ячеек в коде Python. Поэтому добавьте именованные диапазоны согласно тому, как показано в табл. 11.5.

Таблица 11.5. Именованные диапазоны

Лист	Ячейка	Имя
Tracker	B5	package_selection
Tracker	B11	latest_release
Database	B5	new_package
Database	B13	updated_at
Database	B18	log

Один из способов добавления именованных диапазонов — выделить ячейку, затем ввести в поле **Name** (Имя) название диапазона и нажать клавишу <Enter>, это показано на рис. 11.6.

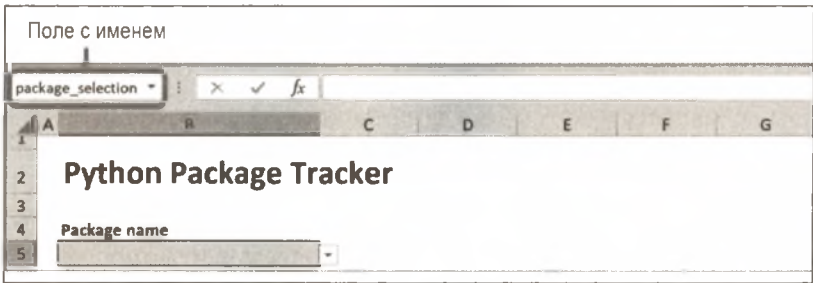


Рис. 11.6. Поле с Именем

Таблицы

На листе Dropdown после ввода слова «packages» в ячейку A1 выделите ячейку A1, затем перейдите к **Insert > Table** (Вставка > Таблица) и убедитесь, что установлен флажок **My table has headers** (В моей таблице есть заголовки). Для завершения работы, когда таблица выбрана, перейдите на вкладку **Table Design** (Дизайн таблицы) (в Windows) или **Table** (Таблица) (в macOS) и переименуйте таблицу из Table1 в dropdown_content, как показано на рис. 11.7.

Валидация данных

Для обеспечения раскрывающегося списка в ячейке B5 на листе Tracker используем проверку данных. Чтобы добавить этот элемент, выберите ячейку B5, затем перейдите в меню **Data > Data Validation** (Данные > Валидация данных) и в разделе **Allow** (Разрешить) выберите **List** (Список). В разделе **Source** (Источник) установите следующую формулу:

```
=INDIRECT("dropdown_content[packages]")
```

Затем нажмите кнопку **ОК**. Это просто ссылка на тело таблицы, но поскольку Excel не принимает прямую ссылку на таблицу, мы должны обернуть ее в формулу **INDIRECT**, которая преобразует таблицу в ее адрес. Тем не менее, использование таблицы позволяет правильно изменить размер диапазона, отображаемого в выпадающем списке, когда мы добавляем дополнительные пакеты.

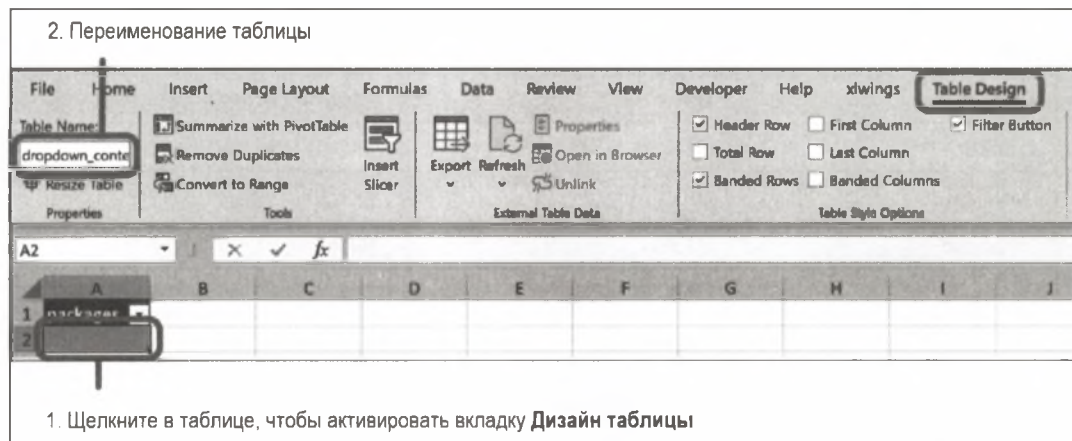


Рис. 11.7. Переименование таблицы Excel

Условное форматирование

Когда вы добавляете пакет, может возникнуть несколько ошибок, на которые мы хотели обратить внимание пользователя: поле может быть пустым, пакет может уже существовать в базе данных или он может отсутствовать на PyPI. Чтобы показать ошибку красным цветом, а другие сообщения — черным, мы воспользуемся простым приемом, основанным на условном форматировании: нам нужен красный шрифт всякий раз, когда сообщение содержит слово «ошибка». На листе **Database** выделите ячейку **C5**, в которой будет написано сообщение. Затем перейдите в меню **Home > Conditional Formatting > Highlight Cells Rules > Text That Contains** (Главная > Условное форматирование > Правила выделения ячеек > Текст, в котором содержится). Выберите значение **error** (ошибка) и в открывающемся списке выберите **Red Text** (Красный текст), как показано на рис. 11.8, затем нажмите **ОК**. Примените то же условное форматирование к ячейке **C5** на листе **Tracker**.

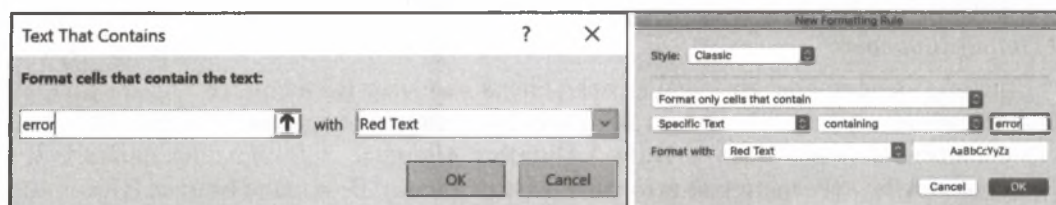


Рис. 11.8. Условное форматирование в Windows (слева) и macOS (справа)

Сетка

На листах Tracker и Database линии сетки не отображались, потому что флажок **View** (Вид) в разделе **Page Layout > Gridlines** (Разметка страницы > Сетка) был снят.

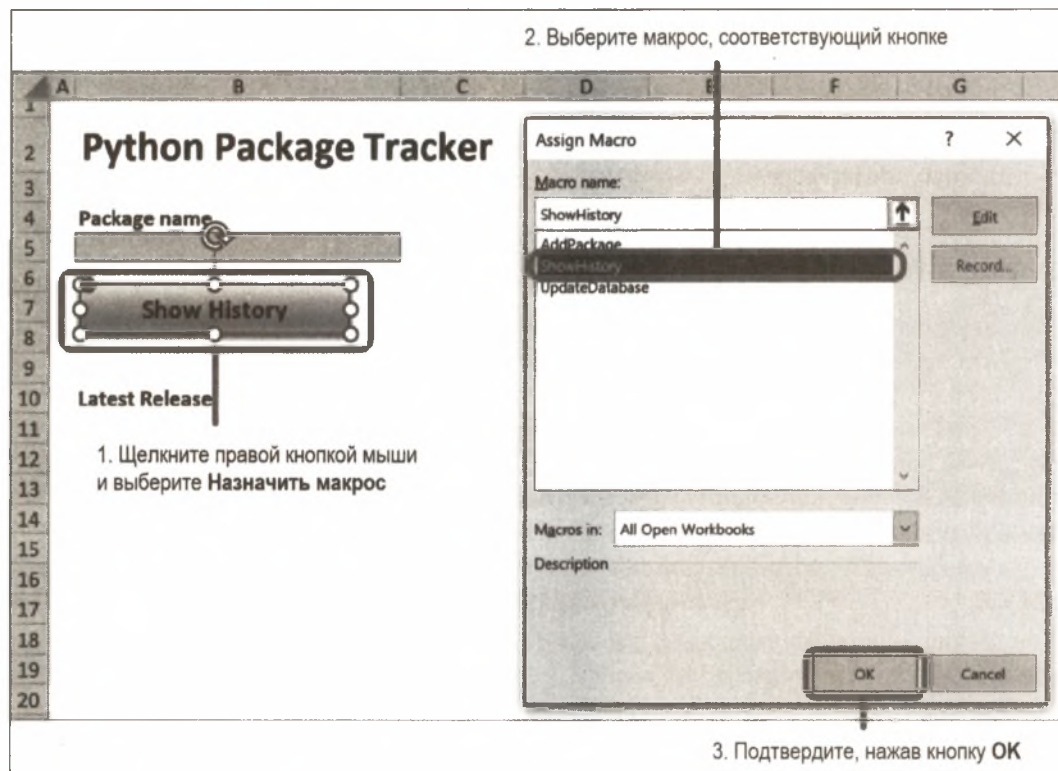


Рис. 11.9. Назначение макроса ShowHistory для кнопки Show History

Теперь пользовательский интерфейс полностью готов и должен выглядеть, как показано на рис. 11.5. Далее нам нужно добавить вызовы RunPython в редактор VBA и связать их с кнопками. Чтобы открыть редактор VBA, нажмите комбинацию клавиш <Alt>+<F11> (в Windows) или <Option>-<F11> (в macOS). Затем в VBAProject файла packagetracker.xlsm, в разделе Modules с левой стороны дважды щелкните мышью на Module1, чтобы открыть его. Удалите существующий код SampleCall и замените его следующими макросами:

```
Sub AddPackage()  
    RunPython "import packagetracker; packagetracker.add_package()"  
End Sub  
  
Sub ShowHistory()  
    RunPython "import packagetracker; packagetracker.show_history()"  
End Sub
```



```
Sub UpdateDatabase()
    RunPython "import packagetracker; packagetracker.update_database()"
End Sub
```

Затем щелкните правой кнопкой мыши на каждой кнопке, найдите в появившемся контекстном меню строку **Assign Macro** (Назначить макрос) и выберите макрос, который соответствует данной кнопке. На рис. 11.9 показана кнопка **Show History** (Показать историю), и по такому же принципу добавляются макросы для кнопок **Add Package** (Добавить пакет) и **Update Database** (Обновить базу данных).

Итак, внешний интерфейс готов, и мы можем перейти к созданию на языке Python внутреннего интерфейса.

Внутренний интерфейс

Код двух файлов Python `packagetracker.py` и `database.py` слишком длинный, чтобы показать его здесь, поэтому вам придется открыть их из сопутствующего репозитория в VS Code. Однако в этом разделе я приведу несколько фрагментов кода, чтобы объяснить несколько ключевых моментов. Давайте посмотрим, что произойдет, когда вы нажмете кнопку **Add Package** (Добавить пакет) на листе `Database`. Этой кнопке назначен следующий макрос VBA:

```
Sub AddPackage()
    RunPython "import packagetracker; packagetracker.add_package()"
End Sub
```

Как видно, функция `RunPython` вызывает Python-функцию `add_package` в модуле `packagetracker`, см. пример 11.1.



Отсутствие производственного кода

Для простоты использования приложение было максимально упрощено, и поэтому оно не проверяет все возможные ошибки. В производственной среде это приложение следует сделать более надежным: например, если приложение не может найти файл базы данных, то на экран выводится ошибка в удобном для пользователя виде.

Пример 11.1. Функция `add_package` в файле `packagetracker.py` (без комментариев)

```
def add_package():
    db_sheet = xw.Book.caller().sheets["Database"]
    package_name = db_sheet["new_package"].value
    feedback_cell = db_sheet["new_package"].offset(column_offset=1)

    feedback_cell.clear_contents()

    if not package_name:
        feedback_cell.value = "Error: Please provide a name!"
    return
```

```

if requests.get(f"{BASE_URL}/{package_name}/json",
                timeout=6).status_code != 200:
    feedback_cell.value = "Error: Package not found!"
    return

error = database.store_package(package_name)
db_sheet["new_package"].clear_contents()

if error:
    feedback_cell.value = f"Error: {error}"
else:
    feedback_cell.value = f"Added {package_name} successfully."
    update_database()
    refresh_dropdown()

```

- 1 Ошибка в сообщении обратной связи инициирует выделение красным шрифтом в Excel с помощью условного форматирования.
- 2 По умолчанию Requests ожидает ответа постоянно, что может привести к «зависанию» приложения в случаях, когда в PyPI возникают проблемы и отвечает медленно. Поэтому в производственном коде всегда следует включать явный параметр таймута.
- 3 Функция `store_package`, если операция прошла успешно, возвращает `None`. В ином случае вернется строка с сообщением об ошибке.
- 4 Чтобы упростить задачу, обновляется вся база данных. В производственной среде вы бы добавили только данные о новом пакете.
- 5 Таким образом, таблица на листе Dropdown, в которой содержится таблица пакетов, будет обновлена. Вместе с проверкой данных, которую мы настроили в Excel, это гарантирует, что все пакеты появятся в раскрывающемся списке на листе Tracker. Вам необходимо предоставить пользователям способ прямого вызова этой функции, если вы разрешаете заполнять базу данных не из вашего файла Excel. Такая ситуация возникает, когда несколько пользователей используют одну и ту же базу данных из разных файлов Excel.

Вам необходимо следить за другими функциями в файле `packagetracker.py` и использовать комментарии в коде. Теперь давайте обратимся к файлу `database.py`. Первые несколько строк показаны в примере 11.2.

Пример 11.2. `database.py` (фрагмент с необходимыми импортными данными)

```

from pathlib import Path

import sqlalchemy
import pandas as pd

```



```
# Пусть файл базы данных находится рядом с этим файлом.
# Здесь мы превращаем путь в абсолютный путь.
this_dir = Path(__file__).resolve().parent
db_path = this_dir / "packagetracker.db"

# Механизм базы данных
engine = sqlalchemy.create_engine(f"sqlite:///({db_path})")
```

1

1 Если вам нужно освежить в памяти, что делает эта строка, загляните в начало главы 7, где я объясняю ее в коде отчета о продажах.

Хотя этот фрагмент касается создания пути к файлу базы данных, он также показывает, как обойти распространенную ошибку при работе с любым файлом, будь то картинка, CSV-файл или, как в данном случае, файл базы данных. Когда вы составляете быстрый сценарий Python, вы можете просто использовать относительный путь, как я делал в большинстве примеров для блокнота Jupyter:

```
engine = sqlalchemy.create_engine("sqlite:///packagetracker.db")
```

Этот путь работает до тех пор, пока ваш файл находится в рабочей директории. Однако, когда вы запустите этот код из Excel через RunPython, рабочий каталог может быть другим, что заставит Python искать файл в не в том каталоге, и вы получите ошибку File not found (Файл не найден). Вы можете решить эту проблему, указав абсолютный путь или создав путь так, как мы это сделали в примере 11.2. Это гарантирует, что Python будет искать файл в том же каталоге, что и исходный файл, даже если вы выполняете код из Excel с помощью команды RunPython.

Если вы хотите создать Python Package Tracker с нуля, вам нужно будет создать базу данных вручную: запустите файл *database.py* как сценарий, например нажав кнопку Run File в VS Code. Это создаст файл базы данных *packagetracker.db* с двумя таблицами. Код, создающий базу данных, находится в конце файла *database.py*:

```
if __name__ == "__main__":
    create_db()
```

Хотя последняя строка вызывает функцию `create_db`, смысл предшествующего оператора `if` объясняется в следующем совете.



if __name__ == "__main__"

Это выражение `if` вы можете увидеть в нижней части многих файлов Python. Данное выражение гарантирует, что этот код будет выполняться только тогда, когда вы запустите файл как сценарий, например из Anaconda Prompt, выполнив `python database.py` или нажав кнопку Run File в VS Code. Однако код не работает, если вы запустите файл, импортировав его как модуль, то есть, введя в свой код `import database`. Причина этого заключается в том, что если вы запускаете его непосредственно как сценарий, то Python присваивает файлу имя `__main__`. Если же вы запускаете его через оператор `import`, он будет вызван по имени модуля (`database`). Поскольку Python отслеживает имя файла в переменной `__name__`, оператор `if` будет иметь значение `True` только тогда, когда вы запустите его как сценарий; и он не сработает, если вы импортируете его из файла *packagetracker.py*.

Остальная часть модуля `database` выполняет SQL-запросы как с помощью SQLAlchemy, так и с помощью методов `pandas to_sql` и `read_sql`. Это сделано, чтобы вы познакомились с двумя этими подходами.

Переход к PostgreSQL

Если вы хотите заменить SQLite на PostgreSQL (серверную базу данных), вам нужно изменить всего несколько элементов. Прежде всего, следует запустить `conda install psycopg2` (или `pip install psycopg2-binary`, если вы не используете дистрибутив Anaconda), чтобы установить драйвер PostgreSQL. Затем в файле `database.py` измените строку подключения в функции `create_engine` на версию PostgreSQL, как показано в табл. 11.3. Наконец, для создания таблиц необходимо изменить тип данных `INTEGER` в `packages.package_id` на специфическую для PostgreSQL нотацию `SERIAL`. Создание первичного ключа с автоинкрементацией — это пример того, как различаются диалекты SQL.

Когда вы создаете инструменты такой сложности, как Python Package Tracker, у вас, вероятно, могут возникать некоторые проблемы на этом пути: например, вы могли переименовать именованный диапазон в Excel и забыть соответствующим образом скорректировать код Python. Это хороший повод, чтобы узнать, как работает отладка!

Отладка

Чтобы легко отлаживать сценарии `xlwings`, запускайте функции непосредственно из VS Code, вместо того чтобы запускать их, нажимая кнопку в Excel. Следующие строки в самом низу файла `packagetracker.py` помогут вам в отладке функции `add_package` (этот же код вы найдете в нижней части проекта `quickstart`):

```
if __name__ == "__main__":
    xw.Book("packagetracker.xlsm").set_mock_caller()
    add_package()
```

- 1 Мы только что видели, как работает данный оператор `if`, когда рассматривали код `database.py` (см. предыдущий совет).
- 2 Поскольку этот код выполняется только при запуске файла непосредственно из Python как сценарий, команда `set_mock_caller()` предназначена только для отладки: при запуске файла в VS Code или из Anaconda Prompt она устанавливает `xw.Book.caller()` в `xw.Book("packagetracker.xlsm")`. Единственная цель этого — иметь возможность запускать ваш сценарий с двух приложений, Python и Excel, без необходимости переключать объект `book` в функции `add_package` между `xw.Book("packagetracker.xlsm")` (когда вы вызываете его из VS Code) и `xw.Book.caller()` (когда вы вызываете его из Excel).

Откройте файл *packagetracker.py* в VS Code и установите точку останова на любой строке в функции `add_package`, щелкнув слева от номера строки. Затем, чтобы запустить отладчик и заставить ваш код остановиться в точке останова, нажмите клавишу F5 и выберите в появившемся диалоговом окне **Python File**. Убедитесь, что вы нажимаете клавишу <F5>, а не используете кнопку **Run File** (Запустить файл), так как эта кнопка игнорирует точку останова.



Отладка в VS Code и Anaconda

В Windows при первом запуске отладчика VS Code с кодом, в котором используется `pandas`, вас может поджидать ошибка: «Exception has occurred: ImportError, Unable to import required dependencies: numpy» (Произошло исключение: Ошибка импорта, невозможно импортировать необходимые зависимости: numpy). Это происходит потому, что отладчик был запущен до того, как среда Conda активировалась должным образом. В качестве обходного пути остановите отладчик, нажав на значок «стоп», и снова нажмите клавишу <F5> — во второй раз это сработает.

Если вы не знаете, как работает отладчик в VS Code, загляните в *приложение В*, где я объясняю все функциональные возможности и кнопки. Мы также вернемся к этой теме в соответствующем разделе следующей главы. Если вы хотите отладить другую функцию, остановите текущий сеанс отладки, а затем откорректируйте имя функции в нижней части вашего файла. Например, чтобы отладить функцию `show_history`, прежде чем снова нажать клавишу <F5>, измените последнюю строку в файле *packagetracker.py* следующим образом:

```
if __name__ == "__main__":
    xw.Book("packagetracker.xlsm").set_mock_caller()
    show_history()
```

В Windows в дополнении `xlwings` вы также можете установить флажок **Show Console**, который отобразит командную строку во время выполнения вызова `RunPython`¹⁵. С помощью командной строки у вас появится возможность отобразить дополнительную информацию, которая поможет вам отладить возникшую неполадку. Например, вы сможете ввести значение переменной, чтобы проверить реакцию приложения в командной строке. Однако после выполнения кода командная строка будет закрыта. Если вам нужно, чтобы она оставалась открытой еще некоторое время, примените маленькую хитрость: добавьте в качестве последней строки в вашей функции `input()`. В результате Python, вместо того, чтобы сразу закрыть командную строку, будет ждать ввода данных пользователем. Когда вы закончите с проверкой вывода, нажмите клавишу <Enter> в командной строке, чтобы закрыть ее — только прежде чем снять флажок с опции **Show Console**, не забудьте снова удалить строку `input()`!

¹⁵ На момент написания статьи в macOS эта опция была еще недоступна.

Заключение

В этой главе вы узнали, что можно создавать достаточно сложные приложения с минимальными усилиями. Если сравнивать с VBA, где взаимодействовать с внешними системами гораздо сложнее, то возможность использовать мощные пакеты Python, такие как Requests или SQLAlchemy, имеет для меня огромное значение. Если у вас есть похожие сценарии для работы с данными, я бы настоятельно рекомендовал вам внимательнее изучить как Requests, так и SQL–Alchemy — возможность эффективно работать с внешними источниками данных. Это позволит вам сделать копирование/вставку делом прошлого.

Вместо того чтобы нажимать кнопки, некоторые пользователи предпочитают создавать свои инструменты Excel, используя формулы в ячейках. В следующей главе вы узнаете, как xlwings может помочь вам написать определяемые пользователем функции в Python, что позволит повторно использовать большинство концепций xlwings, которые мы изучили ранее.

Функции, определяемые пользователем (UDFs)

В предыдущих трех главах вы узнали, как с помощью сценария Python автоматизировать Excel и как запускать такой сценарий из Excel одним нажатием кнопки. В этой главе в качестве еще одного варианта вызова кода Python из Excel с помощью `xlwings` представлены функции, определяемые пользователем (user-defined functions или UDF). UDFs — это функции Python, которые работают в ячейках Excel так же, как и встроенные функции, такие как `SUM` или `AVERAGE`. Как и предыдущую главу, эту начнем с изучения команды `quickstart`, которая позволит нам быстро протестировать первую UDF. Затем мы перейдем к рассмотрению конкретного примера, который и используем как предлог для работы с более сложными UDF. Это будет пример, в котором мы получим и обработаем данные из Google Trends: научимся работать с `pandas DataFrames` и графиками, а также отлаживать UDF. В завершение этой главы мы рассмотрим несколько более сложных тем, в которых обратим особое внимание на производительность. К сожалению, `xlwings` в операционной системе macOS UDFs не поддерживает, и поэтому эта глава является единственной главой, требующей запуска примеров на компьютере под управлением операционной системы Windows. В Windows используется COM-сервер (я кратко познакомил вас с технологией COM в *главе 9*). Поскольку в операционной системе macOS COM-сервер отсутствует, UDF придется создавать заново, что требует много времени и больших трудозатрат, и, поэтому такие UDF еще не разработаны.



Памятка для пользователей macOS и Linux

Даже если ваш компьютер не работает под управлением операционной системы Windows, вы все равно можете ознакомиться с примером Google Trends, поскольку в операционной системе macOS вы с легкостью сможете адаптировать этот пример для работы с использованием вызова `RunPython`. Вы также можете создать отчет, используя одну из библиотек `writer` из *главы 8*, которая работает и под операционной системой Linux.

Начало работы с UDF

В этом разделе, прежде чем мы сможем использовать команду `quickstart` для запуска нашей первой UDF, рассмотрим условия, необходимые для написания UDF.

Чтобы следовать примерам, приведенным в этой главе, вам понадобятся установленная надстройка xlwings и активированный в программе Excel параметр **Режим доверительного доступа к объекту проекта VBA**:

Надстройка

Я предполагаю, что надстройка xlwings у вас уже установлена. Как установить надстройку, было описано в *главе 10*. Однако это не жесткое требование, и хотя эта надстройка облегчает разработку, особенно работу кнопки **Import Functions**, она не является обязательной для развертывания и может быть заменена надстройкой рабочей книги в автономном режиме (*подробнее см. в главе 10*).

Доверительный доступ к объектной модели проекта VBA

Чтобы написать свои первые UDF, вам нужно изменить настройки в Excel: перейдите в меню **File > Options > Trust Center > Trust Center Settings > Macro Settings** (Файл > Параметры > Центр доверия > Параметры центра доверия > Параметры макросов) и установите флажок **Trust access to the VBA project object model** (Доверять доступ к объектной модели проекта VBA), как показано на рис. 12.1. Как мы увидим далее, после нажатия кнопки **Import Functions** (Импортировать функции) настройка позволит xlwings автоматически вставлять модуль VBA в рабочую книгу. Поскольку данный параметр вы используете только в процессе импорта, вам следует рассматривать его как параметр разработчика, и конечным пользователям об этом не стоит знать.

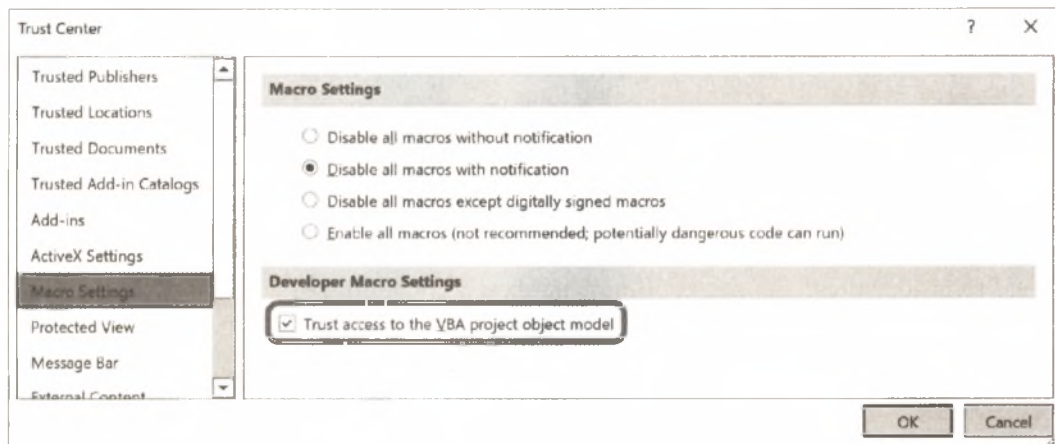


Рис. 12.1. Доверительный доступ к объектной модели проекта VBA

После выполнения этих двух условий вы можете запустить свой первый UDF!

UDF Quickstart

Как обычно, самый простой способ приступить к работе — использовать команду `quickstart`. Перед тем как выполнить следующие действия в Anaconda Prompt, убедитесь, что с помощью команды `cd` вы перешли в выбранный вами каталог. Напри-

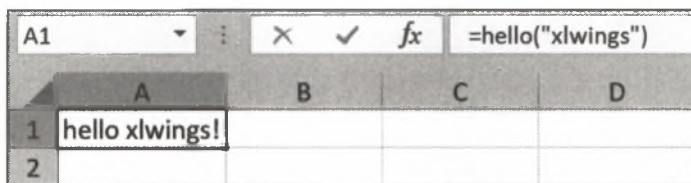
мер, если вы находитесь в своем домашнем каталоге и хотите перейти на Рабочий стол (Desktop), сначала выполните команду `cd Desktop`:

```
(base)> xlwings quickstart first_udf
```

Перейдите в проводнике в папку `first_udf` и откройте в Excel файл `first_udf.xlsm` и файл `first_udf.py` в VS Code. Затем в надстройке ленты `xlwings` нажмите кнопку **Import Functions** (Импорт функции). По умолчанию это действие никак не проявится. То есть вы что-то увидите только в случае возникновения ошибки. Однако, если вы установите флажок **Show Console in the Excel** (Показывать консоль в надстройке Excel) и снова нажмете кнопку **Import Functions**, откроется командная строка, в которой отобразится следующее:

```
xlwings server running [...]
Imported functions from the following modules: first_udf
(сервер xlwings работает [...])
(Импортированные функции из следующих модулей: first_udf)
```

В первой строке будет показано еще несколько деталей, на которые мы можем не обращать внимания. Главное, что после вывода этой строки Python будет запущен. Вторая строка подтверждает, что функции из модуля `first_udf` были правильно импортированы. Теперь в ячейку A1 активного листа `first_udf.xlsm` введите `=hello("xlwings")`, и после нажатия клавиши <Enter> вы увидите результат, как показано на рис. 12.2.



	A	B	C	D
1	hello xlwings!			
2				

Рис. 12.2. `first_udf.xlsm`

Давайте разберемся, как все работает: начнем с функции `hello` в файле `first_udf.py` (пример 12.1), это та часть кода `quickstart`, которую мы до сих пор игнорировали.

Пример 12.1. `first_udf.py` (выдержка)

```
import xlwings as xw

@xw.func
def hello(name):
    return f"Hello {name}!"
```

Каждая функция, которую вы поместили символом декоратора `@xw.func`, будет импортирована в Excel, когда в надстройке `xlwings` будет нажата кнопка **Import Functions**. Импорт функции делает добавляемую функцию доступной в Excel, чтобы вы ее могли использовать в формулах ячеек — к техническим деталям мы пе-

рейдем в ближайшее время. Если вы хотите узнать немного больше о том, как работают декораторы, смотрите врезку.

Декораторы функций

Декоратор — это название функции, размещаемой поверх определения функции, и начинается имя функции-декоратора со знака @. Это простой способ изменить поведение функции. Декоратор используется xlwings для распознавания функций, которые вы хотите сделать доступными в Excel. Чтобы вам было понятнее, как работает декоратор, в следующем примере мы приводим определение декоратора `verbose`, который будет печатать текст до и после выполнения функции `print_hello`. Технически декоратор принимает функцию (`print_hello`) и предоставляет ее в качестве аргумента `func` функции `verbose`. Внутренняя функция, называемая `wrapper`, может делать все, что нужно; в данном случае она печатает значение до и после вызова функции `print_hello`. Имя внутренней функции не имеет значения:

```
In [1]: # Это определение декоратора функции
def verbose(func):
    def wrapper():
        print("Before calling the function.")
        func()
        print("After calling the function.")
    return wrapper
```

```
In [2]: # Использование декоратора функций
@verbose
def print_hello():
    print("hello!")
```

```
In [3]: # Эффект от вызова декоратора функции
print_hello()
```

Before calling the function.

hello!

After calling the function.

(Перед вызовом функции

hello!

После вызова функции).

В конце этой главы вы найдете табл. 12.1 с кратким описанием всех декораторов, которые предлагает xlwings.

По умолчанию, если аргументами функции являются диапазоны ячеек, xlwings вместо объекта `xlwings.range` передает вам значения этих диапазонов ячеек. В подавляющем большинстве случаев это очень удобно и позволяет вызвать функцию

hello с ячейкой в качестве аргумента. Например, вы можете написать "xlwings" в ячейке A2, а затем изменить формулу в A1 на следующую:

```
=hello(A2)
```

Результат будет таким же, как на рис. 12.2. В конце этой главы я покажу вам, как изменить данное поведение и сделать так, чтобы аргументы поступали как объекты диапазона xlwings — как мы увидим далее, в некоторых случаях вам это понадобится. В VBA эквивалентная функция hello выглядела бы следующим образом:

```
Function hello(name As String) As String
```

```
    hello = "Hello " & name & "!"
```

```
End Function
```

Когда вы в надстройке нажимаете кнопку **Import Functions**, xlwings вставляет в вашу рабочую книгу Excel модуль VBA с названием xlwings_udfs. Этот модуль содержит функцию VBA для каждой импортируемой функции Python: такие функции-обертки VBA заботятся о запуске соответствующей функции в Python. Хотя никто не мешает вам посмотреть VBA-модуль xlwings_udfs, открыв с помощью комбинации клавиш <Alt>+<F11> редактор VBA. Вы можете игнорировать его, поскольку код автогенерируется и при повторном нажатии кнопки **Import Functions** любые изменения будут потеряны. Давайте теперь поэкспериментируем с нашей функцией hello в файле first_udf.py и в возвращаемом значении заменим Hello на Bye:

```
@xw.func
```

```
def hello(name):
```

```
    return f"Bye {name}!"
```

Чтобы пересчитать функцию в Excel, для редактирования формулы или дважды щелкните на ячейке A1 или выделите требуемую ячейку и нажмите клавишу <F2>, чтобы активировать режим редактирования, а затем нажмите клавишу <Enter>. В качестве альтернативы можно использовать комбинацию клавиш <Ctrl>+<Alt>+<F9>: таким образом, будет произведен *принудительный* пересчет всех рабочих листов во всех открытых рабочих книгах, включая формулу hello. Обратите внимание, что нажатие клавиши <F9> (пересчитать все рабочие листы во всех открытых рабочих книгах) или комбинации клавиш <Shift>+<F9> (пересчитать активный рабочий лист) не пересчитает UDF, поскольку Excel запускает пересчет UDF только в случае изменения зависимой ячейки. Чтобы изменить это поведение, вы можете сделать функцию *зависимой*, добавив соответствующий аргумент в декоратор func:

```
@xw.func(volatile=True)
```

```
def hello(name):
```

```
    return f"Bye {name}!"
```

Зависимые функции обрабатываются каждый раз, когда Excel выполняет перерасчет, независимо от того, изменились ли зависимости функции или нет. Некоторые из встроенных функций Excel являются зависимыми, например =RAND() или =NOW(), которые, при их использовании вашу рабочую книгу замедлят, поэтому не переусердствуйте. Если вы измените имя или аргументы функции, или декоратор func,

как мы только что сделали, вам нужно будет повторно импортировать функцию, еще раз нажав кнопку **Import Functions**: это действие перед импортом обновленной функции перезапустит интерпретатор Python. Теперь для обратного изменения функции с Bye на Hello достаточно использовать комбинации клавиш <Shift>+<F9> или <F9>, чтобы произошел перерасчет формулы, поскольку функция теперь зависимая.



Сохранение файла Python после его изменения

Часто встречающаяся ошибка заключается в том, что после внесения изменений пользователь забывает сохранить исходный файл Python. Поэтому всегда дважды проверяйте, сохранен ли файл Python, прежде чем нажимать кнопку **Import Functions** или выполнять пересчет UDF в Excel.

По умолчанию xlwings импортирует функции из файла Python в тот же каталог и с тем же именем, что и файл Excel. Переименование и перемещение исходного файла Python нуждается в таких же изменениях, как и в *главе 10*, когда мы делали то же самое с вызовами RunPython. Поэтому переименуйте файл first_udf.py, изменив его имя на hello.py. Чтобы сообщить xlwings об этом изменении, добавьте в надстройке xlwings в UDF Modules имя модуля, то есть hello (без расширения .py!), как показано на рис. 12.3.

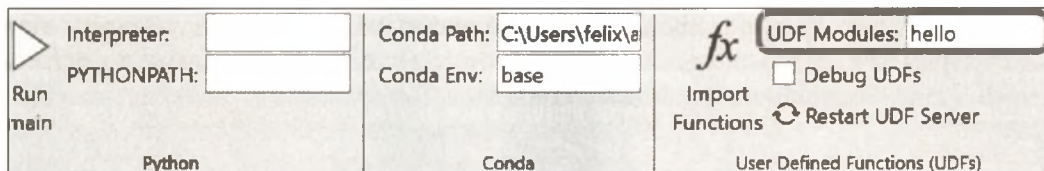


Рис. 12.3. Настройка модулей UDF

Чтобы повторно импортировать функцию, нажмите кнопку **Import Functions**. Затем, чтобы убедиться, что все по-прежнему работает, пересчитайте формулу в Excel.



Импорт функций из различных модулей Python

Если вы хотите импортировать функции из разных модулей, используйте в настройке UDF Modules в качестве разделителя точку с запятой между их именами. Например, hello;another_module.

Теперь переместите hello.py на рабочий стол: для этого необходимо добавить путь к вашему рабочему столу в PYTHONPATH в надстройке xlwings. Как было сказано в *главе 10*, для выполнения этой задачи можно использовать переменные среды, то есть, установить параметр PYTHONPATH в дополнении на %USERPROFILE%\Desktop. Если у вас остался путь к папке rscripts из *главы 10*, либо перепишите его, либо оставьте, разделив пути точкой с запятой. После этих изменений еще раз нажмите

кнопку Импорт функций, а затем пересчитайте функцию в Excel, чтобы убедиться, что все по-прежнему работает.



Конфигурация и развертывание

В этой главе я всегда подразумеваю изменение параметров в надстройке; однако все, что написано в *главе 10* о конфигурации и развертывании, можно применить и к этой главе. Это означает, что настройку можно также изменить в листе `xlwings.conf` или в файле конфигурации, находящемся в том же каталоге, что и файл Excel. А вместо использования надстройки `xlwings` можно использовать рабочую книгу, настроенную в автономном режиме. В случае с UDFs также имеет смысл создать собственную надстройку — это позволит вам совместно использовать UDFs во всех рабочих книгах без необходимости импортировать их в каждую отдельно. Для получения дополнительной информации о создании собственного пользовательского дополнения см. *документацию xlwings*¹.

Если вы изменяете Python-код вашего UDF, `xlwings` при каждом сохранении Python-файла автоматически подхватывает изменения. Как уже говорилось, импортировать повторно UDF нужно только в том случае, если вы изменили что-то в имени функции, аргументах или декораторах. Однако если ваш исходный файл импортирует код из других модулей, и вы что-то измените в этих модулях, самый простой способ позволить Excel получить все изменения — это нажать кнопку **Restart UDF Server** (Перезапустить UDF-сервер).

Теперь вы знаете, как в Python написать простую UDF и как использовать ее в Excel. Пример в следующем разделе познакомит вас с приближенными к реальности UDF, использующими `pandas DataFrames`.

Тематическое исследование: Google Trends

В этом примере мы будем использовать данные из Google Trends, чтобы научиться работать с `pandas DataFrames` и динамическими массивами — одной из самых интересных новых функций Excel, которую Microsoft официально запустила в 2020 году. Затем мы создадим UDF, которая подключается непосредственно к Google Trends, а также UDF, использующий метод `plot` для `DataFrame`. В заключение этого раздела мы рассмотрим, как работает отладка с UDF. Давайте начнем с краткого введения в Google Trends!

Введение в Google Trends

Google Trends² — это сервис Google, который позволяет анализировать популярность поисковых запросов Google с разбивкой по регионам и по временным интер-

¹ <https://oriel.ly/uNo0g>. — Примеч. пер.

² <https://oriel.ly/G6TpC>. — Примеч. пер.

валам. На рис. 12.4 показаны данные Google Trends после добавления нескольких популярных языков программирования, выбора региона «Весь мир» и временного диапазона 1/1/16–12/26/20. Для каждого поискового запроса был выбран контекст *языка программирования*, который появляется в открывающемся списке после ввода поискового запроса. Это позволяет нам не обращать внимания на такие названия, как «змея» вместо Python и «остров» вместо Java. Google индексирует данные в выбранные временные интервалы и по определенному местоположению, причем значение 100 представляет собой максимальный поисковый интерес. В нашей выборке это означает, что в данном временном интервале и местоположении наибольший поисковый интерес был проявлен к Java в феврале 2016 года. Более подробную информацию о Google Trends можно найти в официальном блоге компании³.

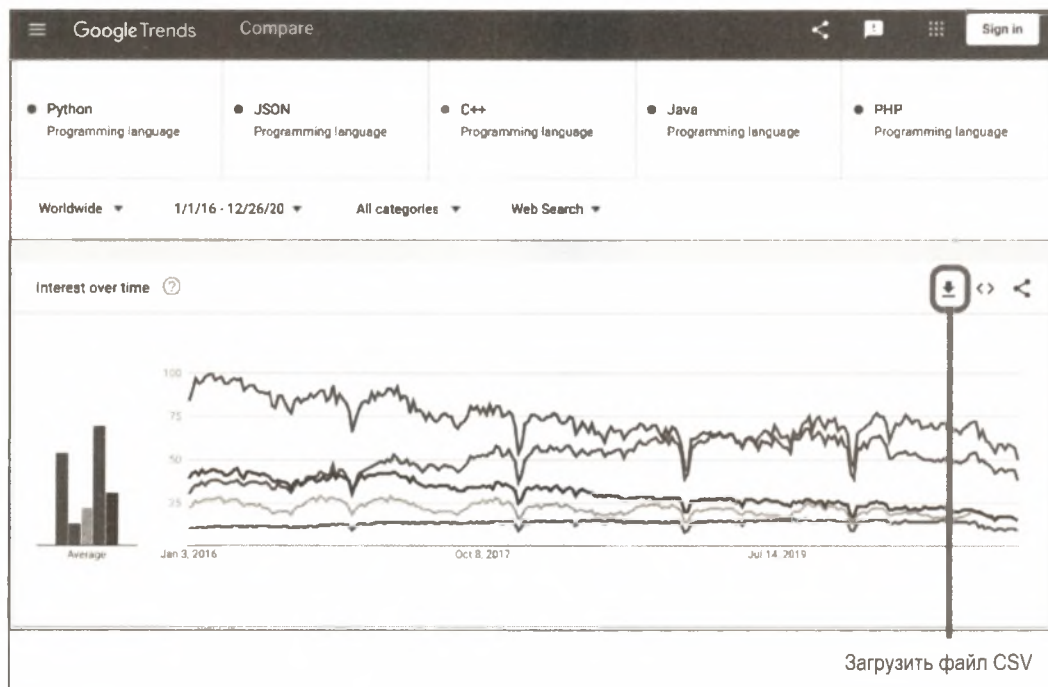


Рис. 12.4. Проценты с течением времени
(источник данных: Google Trends⁴)



Случайные выборки

Показатели Google Trends основаны на случайных выборках, что означает, что вы можете увидеть картину, немного отличающуюся от показанной на рис. 12.4, даже если используете то же расположение, временные интервалы и поисковые запросы, что и на снимке экрана.

³ https://oriel.ly/_aw8f. — Примеч. пер.

⁴ <https://oriel.ly/SR8zD>. — Примеч. пер.

Я нажал кнопку **download**, показанную на рис. 12.4, чтобы получить CSV-файл, из которого я далее скопировал данные в рабочую книгу Excel проекта `quickstart`. В следующем разделе я покажу вам, где найти эту рабочую книгу — мы будем использовать ее для анализа данных с помощью UDF прямо из Excel!

Работа с DataFrames и динамическими массивами

Прочитав эту книгу, вы не должны удивляться тому, что DataFrames в pandas также являются лучшим другом UDF. Чтобы увидеть, как DataFrames и UDF работают вместе, и узнать о динамических массивах, перейдите в папку `describe` в каталоге `udfs` репозитория-компаньона и откройте в Excel `describe.xlsm` и в VS Code — `describe.py`. Файл Excel содержит данные Google Trends, а в файле Python вы для начала найдете простую функцию, как показано в примере 12.2.

Пример 12.2. `describe.py`

```
import xlwings as xw
import pandas as pd

@xw.func
@xw.arg("df", pd.DataFrame, index=True, header=True)
def describe(df):
    return df.describe()
```

По сравнению с функцией `hello` из проекта `quickstart` здесь вы заметите второй декоратор:

```
@xw.arg("df", pd.DataFrame, index=True, header=True)
```

`arg` — это сокращение от *argument* и позволяет применять те же конвертеры и опции, которые я использовал в главе 9 при знакомстве с синтаксисом `xlwings`. Другими словами, декоратор предлагает ту же функциональность для UDF, что и метод `options` для объектов `range` `xlwings`. Формально синтаксис декоратора `arg` выглядит следующим образом:

```
@xw.arg("argument_name", convert=None, option1=value1, option2=value2, ...)
```

Чтобы помочь вам установить связь с главой 9, эквивалент функции `describe` в виде сценария выглядит следующим образом (здесь предполагается, что файл `describe.xlsm` открыт в Excel и что функция применяется к диапазону A3:F263):

```
import xlwings as xw
import pandas as pd

data_range = xw.Book("describe.xlsm").sheets[0]["A3:F263"]
df = data_range.options(pd.DataFrame, index=True, header=True).value
df.describe()
```

Опции `index` и `header` не нужны, поскольку аргументы используются по умолчанию, но я включил их, чтобы показать, как они применяются в UDF. В качестве активной рабочей книги выберите `describe.xlsm`, нажмите кнопку **Import Functions**,

затем в свободную ячейку, например в H3, введите формулу `=describe(A3:F263)`. Дальнейшие действия при нажатии клавиши <Enter> зависят от версии Excel — точнее, от того, является ли она достаточно свежей и поддерживает ли *динамические массивы*. Если динамические массивы поддерживаются, вы увидите картину, как на рис. 12.5, то есть вывод функции `describe` в ячейках H3:M11 будет обведен тонкой синей рамкой. Вы сможете увидеть синюю рамку, только если курсор находится внутри массива, да и рамка настолько тонкая, что у вас могут возникнуть проблемы с ее четким отображением, если вы посмотрите на рисунок в печатной версии книги. В ближайшее время мы увидим, как ведут себя динамические массивы, а больше узнать о них вы сможете, прочитав врезку «Динамические массивы» на стр. 287.

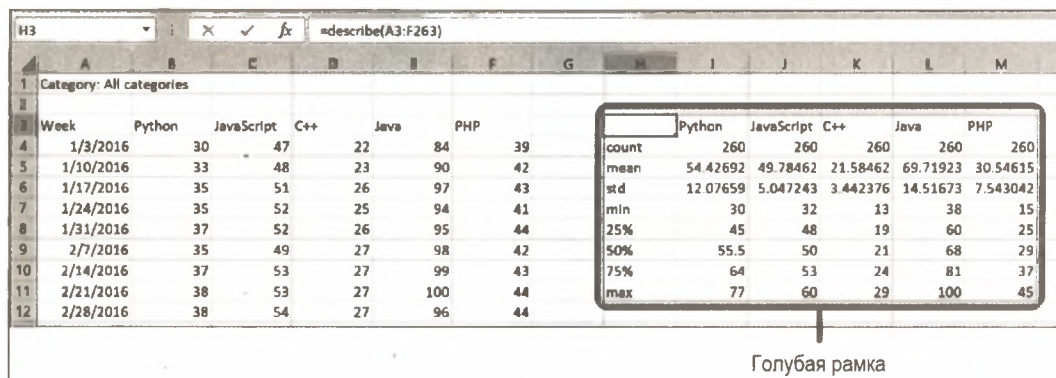


Рис. 12.5. Функция `describe` с динамическими массивами

Однако если вы используете версию Excel, которая не поддерживает динамические массивы, то все будет выглядеть так, как будто ничего не происходит: по умолчанию формула возвращает в H3 только пустую верхнюю ячейку. Чтобы исправить эту проблему, используйте то, что Microsoft сегодня называет устаревшими *массивами CSE*. Массивы CSE необходимо подтверждать, набирая комбинацию клавиш <Ctrl>+<Shift>+<Enter>, а не просто <Enter> — отсюда и их название. Давайте в деталях рассмотрим, как они работают:

- ♦ убедитесь, что ячейка H3 пуста, для чего выделите ее и нажмите клавишу <Delete>;
- ♦ выберите выходной диапазон, начав с ячейки H3, а затем — все ячейки так, чтобы в правом нижнем углу рамки выделения оказалась ячейка M11;
- ♦ выбрав диапазон H3:M11, введите формулу `=describe(A3:F263)`, а затем подтвердите ее, нажав комбинацию клавиш <Ctrl>+<Shift>+<Enter>.

Теперь вы должны увидеть почти такую же картину, как на рис. 12.5, но с некоторыми отличиями:

- ♦ вокруг диапазона H3:M11 отсутствует синяя рамка;
- ♦ в формуле массив заключен в фигурные скобки, чтобы обозначить его как массив CSE: `{=describe(A3:F263)}`;

- ◆ если динамические массивы удаляются просто переводом курсора в левую верхнюю ячейку и нажатием клавиши <Delete>, то в случае с массивами CSE весь массив сначала следует выделить, и это обязательно, иначе он не удалится.

Давайте теперь сделаем нашу функцию более практичной, введя дополнительный параметр `selection`, который позволит нам указать, какие столбцы мы хотим включить в результат в нашем выводе. Если у вас много столбцов, и вы хотите включить в функцию `describe` только их часть, такая функция может оказаться полезной. Измените функцию следующим образом:

```
@xw.func
@xw.arg("df", pd.DataFrame)                                     1
def describe(df, selection=None):                               2
    if selection is not None:
        return df.loc[:, selection].describe()                 3
    else:
        return df.describe()
```

- 1 Я опустил аргументы `index` и `header`, так как они используют значения по умолчанию, но вы можете не стесняться и их оставить.
- 2 Добавьте параметр `selection` и сделайте его необязательным, присвоив ему по умолчанию значение `None`.
- 3 Если предоставлен выбор, отфильтруйте столбцы `DataFrame` на основании `DataFrame`.

После изменения функции обязательно сохраните ее, а затем в надстройке `xlwings` нажмите кнопку **Import Functions** — это необходимо, поскольку мы добавили новый параметр. Запишите `Selection` в ячейку A2 и `TRUE` в ячейки B2:F2. В заключение настройте формулу в ячейке H3 в зависимости от того, имеются ли у вас динамические массивы или нет:

С динамическими массивами

Выберите ячейку H3, затем измените формулу на `=describe(A3:F263, B2:F2)` и нажмите клавишу <Enter>.

Без динамических массивов

Начиная с ячейки H3, выделите H3:M11, а затем нажмите клавишу <F2>, чтобы активировать режим редактирования ячейки H3, и измените формулу на `=describe(A3:F263, B2:F2)`. Чтобы завершить работу, нажмите комбинацию клавиш <Ctrl>+<Shift>+<Enter>.

Чтобы опробовать улучшенную функцию, давайте изменим в ячейке E2 значение `Java` с `TRUE` на `FALSE` и посмотрим, что произойдет: с помощью динамических массивов вы увидите, как таблица волшебным образом уменьшится на один столбец. Однако при использовании устаревших массивов CSE вы получите безобразный столбец, заполненный значениями `#N/A`, как показано на рис. 12.6.

Чтобы решить эту проблему, xlwings может изменить размер унаследованных массивов CSE, используя декоратор `return`. Добавьте его, изменив свою функцию следующим образом:

```
@xw.func
@xw.arg("df", pd.DataFrame)
@xw.ret(expand="table")
def describe(df, selection=None):
    if selection is not None:
        return df.loc[:, selection].describe()
    else:
        return df.describe()
```

1

1 Если добавить декоратор `return` с опцией `expand="table"`, xlwings изменит размер массива CSE в соответствии с размерами возвращаемого DataFrame.

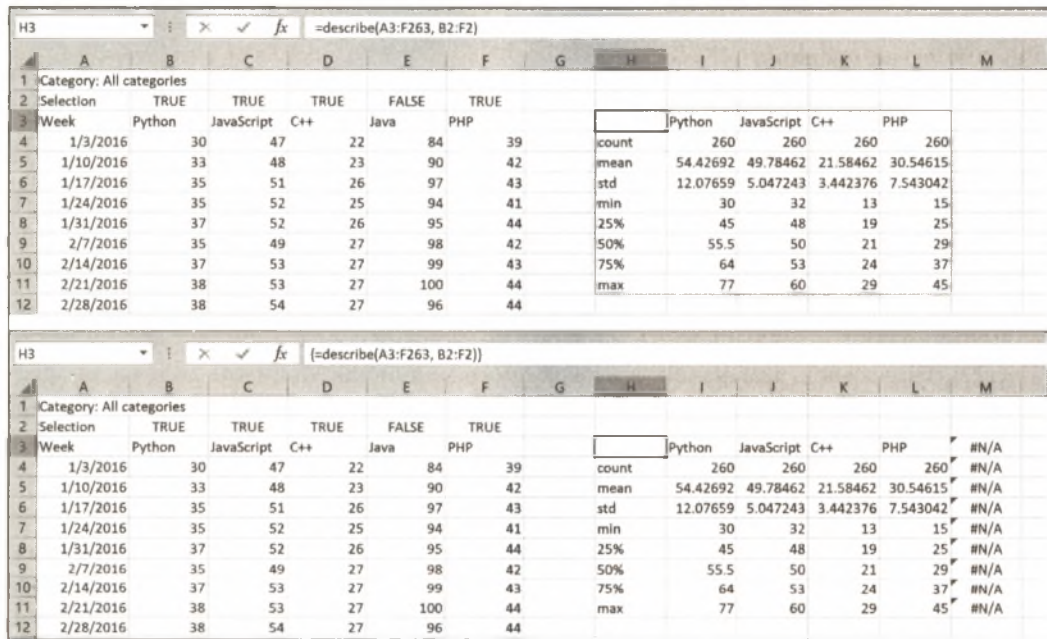


Рис. 12.6. Динамические массивы (вверху) в сравнении с массивами CSE (внизу) после исключения столбца

После добавления декоратора `return` сохраните исходный файл Python, перейдите в Excel и нажмите комбинацию клавиш `<Ctrl>+<Alt>+<F9>` для пересчета: это изменит размер массива CSE и удалит столбец `#N/A`. Поскольку это обходной путь, я настоятельно рекомендую вам сделать все возможное, чтобы получить в свои руки версию Excel, которая поддерживает динамические массивы.



Порядок работы декораторов функций

Убедитесь, что декоратор `xw.func` находится над декораторами `xw.arg` и `xw.ret`; обратите внимание, что расположение декораторов `xw.arg` и `xw.ret` в определенном порядке не имеет значения.

Декоратор `return` работает концептуально так же, как и декоратор `argument`, с той лишь разницей, что вам не нужно указывать имя аргумента. Формально его синтаксис выглядит следующим образом:

```
@xw.ret(convert=None, option1=value1, option2=value2, ...)
```

Обычно вам не нужно предоставлять явный аргумент `convert`, поскольку `xlwings` распознает тип возвращаемого значения автоматически — такое же поведение мы видели в главе 9 в отношении метода `options` при записи значений в Excel.

В качестве примера, если вы хотите скрыть индекс возвращаемого `DataFrame`, используйте этот декоратор:

```
@xw.ret(index=False)
```

Динамические массивы

Познакомившись с работой динамических массивов в контексте функции `describe`, я уверен, вы согласитесь, что они являются одним из самых фундаментальных и интересных дополнений к Excel, которые Microsoft придумала за долгое время. Динамические массивы были официально представлены в 2020 году для подписчиков Microsoft 365, которые используют последнюю версию Excel. Чтобы убедиться, что ваша версия достаточно свежая, убедитесь в наличии новой функции `UNIQUE`: для этого начните вводить в ячейку `=UNIQUE`, и если Excel выдаст имя функции, значит, динамические массивы поддерживаются. Если вы используете Excel с постоянной лицензией, а не как часть подписки Microsoft 365, то, скорее всего, вы получите динамические массивы в версии, которая была анонсирована для выпуска в 2021 году и предположительно будет называться Office 2021. Вот несколько технических замечаний о поведении динамических массивов:

- ◆ если динамические массивы перезаписывают ячейку со значением, вы получите ошибку `#SPILL!` После освобождения места для динамического массива путем удаления или перемещения мешающей ячейки, массив будет записан. Обратите внимание, что декоратор `xlwings return expand="table"` не настолько сообразителен и перезапишет существующие значения ячеек без предупреждения!
- ◆ вы можете ссылаться на диапазон динамического массива, используя левую верхнюю ячейку, за которой идет знак `#`. Например, если ваш динамический массив находится в диапазоне `A1:B2` и вы хотите просуммировать все ячейки, напишите `=SUM(A1#)`;
- ◆ если вы хотите, чтобы ваши массивы снова вели себя как старые массивы CSE, начинайте формулу со знака `@`. Например, чтобы матричное умножение возвращало старый массив CSE, используйте `@MMULT()`.

Загрузка файла CSV и копирование/вставка значений в файл Excel отлично работали для этого вводного примера DataFrame, но копирование/вставка — это настолько подверженный ошибкам процесс, что вы захотите избавиться от него при первой возможности. С помощью Google Trends это действительно можно сделать, и в следующем разделе мы покажем вам, как это делается!

Получение данных из Google Trends

Все предыдущие примеры были очень простыми, практически только обертывание одной функции pandas. Чтобы разобраться с примером, приближенным к реальности, давайте создадим UDF, который загружает данные непосредственно из Google Trends, и благодаря этому UDF вам больше не придется выходить в интернет и загружать CSV-файл вручную. У Google Trends нет официального API (программного интерфейса приложения), но есть пакет Python под названием `pytrends`⁵, который восполняет этот пробел. Отсутствие официального API означает, что Google может изменить его в любой момент, поэтому есть риск, что примеры в этом разделе в какой-то момент перестанут работать. Однако, учитывая, что на момент написания этой статьи `pytrends` существует уже более пяти лет, есть также реальный шанс, что он будет обновлен, чтобы отразить изменения и заставить работать снова. В любом случае это хороший пример, показывающий, что в Python есть пакет практически для всего — утверждение, которое я сделал в главе 1. Если бы вы были ограничены использованием Power Query, вам, вероятно, потребовалось бы потратить гораздо больше времени, чтобы заставить что-либо работать — по крайней мере, я не смог найти бесплатное решение, которое можно было бы подключить и использовать. Поскольку `pytrends` не является частью Anaconda, а также не имеет официального пакета Conda, давайте установим его с помощью `pip`, если вы еще не сделали этого ранее:

```
(base)> pip install pytrends
```

Чтобы воспроизвести конкретный пример из онлайн-версии Google Trends, показанный на рис. 12.4, нам потребуется найти правильные идентификаторы для поисковых запросов с контекстом «Язык программирования». Для облегчения этой задачи `pytrends` умеет выводить в открывающемся списке различные поисковые запросы или типы, которые предлагает Google Trends. В следующем примере кода поисковый запрос `mid` означает *Machine ID*, который и является нужным нам идентификатором:

```
In [4]: from pytrends.request import TrendReq
In [5]: # Во-первых, давайте создадим объект TrendRequest
        trend = TrendReq()
In [6]: # Теперь мы можем отобразить предложения в том виде, в котором
        # они будут выглядеть
        # онлайн в открывающемся списке Google Trends после ввода слова
        # "Python"
        trend.suggestions("Python")
```

⁵ <https://oriel.ly/SvnLI>. — Примеч. пер.


```
Out[6]: [{'mid': '/m/05z1_', 'title': 'Python', 'type': 'Programming language'},
        {'mid': '/m/05tb5', 'title': 'Python family', 'type': 'Snake'},
        {'mid': '/m/0cv6_m', 'title': 'Pythons', 'type': 'Snake'},
        {'mid': '/m/06bxxb', 'title': 'CPython', 'type': 'Topic'},
        {'mid': '/g/lq6j3gsvm', 'title': 'python', 'type': 'Topic'}]
```

Повторив данную операцию для других языков программирования, мы получим корректный запрос mid для всех языков и сможем написать UDF. Этот UDF показан в примере 12.3. Вы найдете исходный код в каталоге `google_trends` в папке `udfs` партнерского репозитория.

Пример 12.3. Функция `get_interest_over_time` в файле `google_trends.py` (выдержка с соответствующими операторами импорта)

```
import pandas as pd
from pytrends.request import TrendReq
import xlwings as xw

@xw.func(call_in_wizard=False) 1
@xw.arg("mids", doc="Machine IDs: A range of max 5 cells") 2
@xw.arg("start_date", doc="A date-formatted cell")
@xw.arg("end_date", doc="A date-formatted cell")
def get_interest_over_time(mids, start_date, end_date):
    """Query Google Trends - replaces the Machine ID (mid) of
    common programming languages with their human-readable
    equivalent in the return value, e.g., instead of "/m/05z1_"
    it returns "Python" (Запрос Google Trends - заменяет машинный
    идентификатор (mid) распространенных языков программирования
    на их понятный человеку эквивалент в возвращаемом значении,
    например, вместо "/m/05z1_" возвращает "Python").
    """ 3

    # Проверка и преобразование параметров
    assert len(mids) <= 5, "Too many mids (max: 5)" 4
    start_date = start_date.date().isoformat() 5
    end_date = end_date.date().isoformat()

    # Выполните запрос Google Trends и верните DataFrame
    trend = TrendReq(timeout=10) 6
    trend.build_payload(kw_list=mids,
                       timeframe=f"{start_date} {end_date}") 7
    df = trend.interest_over_time() 8

    # Замените Google "mid" на слово, понятное человеку
    mids = {"/m/05z1_": "Python", "/m/02p97": "JavaScript",
           "/m/0jgqg": "C++", "/m/07sbkfb": "Java", "/m/060kv": "PHP"}
    df = df.rename(columns=mids) 9

    # Убрать колонку isPartial
    return df.drop(columns="isPartial") 10
```

- 1 По умолчанию Excel выполняет вызов функции при ее открытии в мастере функций. Поскольку эта операция может замедлить работу, особенно при наличии API-запросов, мы данную функцию отключаем.
- 2 Как вариант добавьте для аргумента функции строку docstring. Строка будет отображаться в мастере функций при редактировании соответствующего аргумента, как показано на рис. 12.8.
- 3 Строка docstring отображается в мастере функций, см. рис. 12.8.
- 4 Оператор `assert` — это простой способ предупредить об ошибке в случае, если пользователь указывает чрезмерное количество `mids`. Google Trends допускает не более пяти `mids` на один запрос.
- 5 `pytrends` ожидает, что даты начала и окончания будут представлены в виде одной строки в форме `YYYY-MM-DD YYYY-MM-DD`. Поскольку мы предоставляем даты начала и завершения как ячейки, отформатированные по дате, они будут получены как объекты `datetime`. Вызов методов `date` и `isoformat` отформатирует их должным образом.
- 6 Мы создаем объект `request` в `pytrends`. Установив `timeout` на десять секунд, мы уменьшаем риск появления `requests.exceptions`. Ошибка `ReadTimeout`, появляется в тех случаях, если Google Trends требуется немного больше времени для ответа. Если ошибка по-прежнему проявляется, просто запустите функцию еще раз или увеличьте тайм-аут.
- 7 Мы объекту запроса предоставляем аргументы `kw_list` и `timeframe`.
- 8 Здесь выполняется фактический запрос, и для этого вызывается `interest_over_time`, который возвращает `pandas DataFrame`.
- 9 Мы переименовываем `mids` в понятный человеку и хорошо читаемый эквивалент.
- 10 Последний столбец именуется `isPartial`. `True` означает, что текущий интервал, например неделя, все еще продолжается и, следовательно, еще не содержит всех данных. Для упрощения и соответствия онлайн-версии мы при возврате `DataFrame` этот столбец опускаем.

Теперь откройте файл `google_trends.xlsx` из репозитория-компаньона, в дополнении `xlwings` нажмите кнопку **Import Functions**, а затем из ячейки A4 вызовите функцию `get_interest_over_time`, как показано на рис. 12.7.

Чтобы получить помощь в определении аргументов функции, нажмите кнопку **Вставить функцию**, расположенную слева от строки формул. При этом ячейка A4 должна быть выделена: откроется мастер функций, где вы найдете UDF в категории `xlwings`. После выбора `get_interest_over_time` вы увидите название аргументов функции, а также строку `docstring` как описание функции (ограниченное первыми 256 символами), как показано на рис. 12.8. Или же, прежде чем нажать кнопку **Insert Function**, начните вводить `=get_interest_over_time(` в ячейку A4 (включая открывающую скобку) — экран примет вид, как показано на рис. 12.8. Обратите вни-

мание, что UDF возвращают даты в неформатированном виде. Чтобы исправить это, щелкните правой кнопкой мыши на столбце с датами, выберите **Format Cells**, а затем в категории **Date** выберите нужный формат.

Кнопка Вставить функцию

	A	B	C	D	E	F
1	start date	1/1/2016	end date	12/26/2020		
2						
3	mid	/m/05z1	/m/02p97	/m/0jgqg	/m/07sbkfb	/m/060kv
4	date	Python	JavaScript	C++	Java	PHP
5	1/3/2016	30	47	22	84	39
6	1/10/2016	33	48	23	90	42
7	1/17/2016	35	51	26	97	43
8	1/24/2016	35	52	25	94	41
9	1/31/2016	37	52	26	95	44

Рис. 12.7. google_trends.xlsm

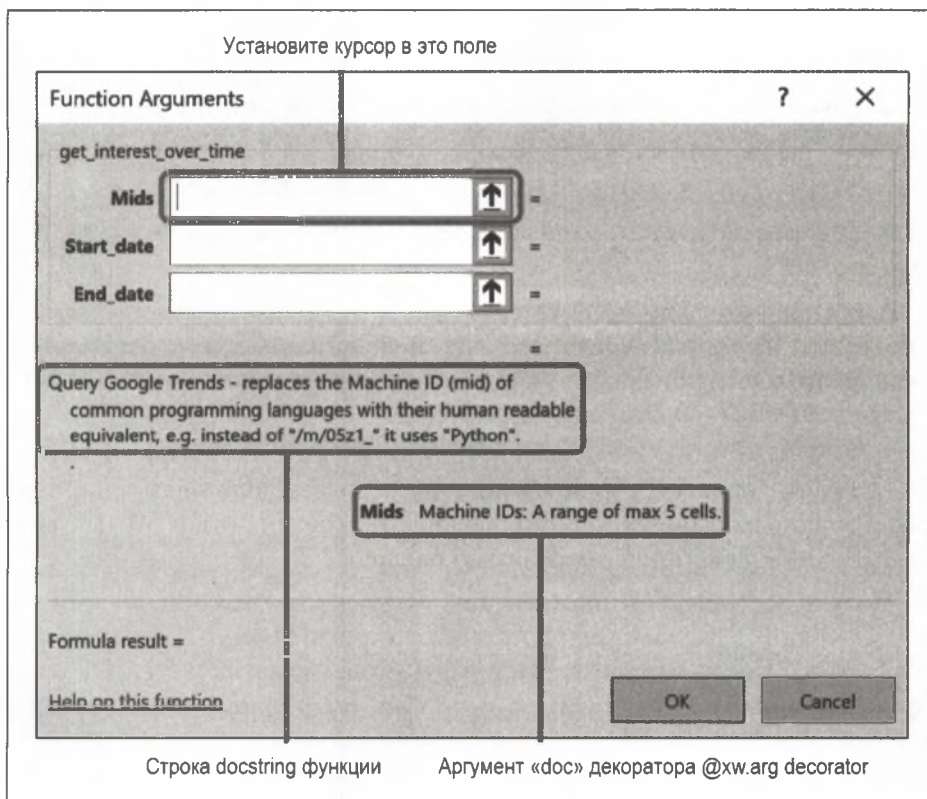


Рис. 12.8. Мастер функций

Если вы внимательно посмотрите на рис. 12.7, то по синей рамке вокруг массива результатов станет ясно, что в данном случае я снова использую динамические массивы. Поскольку снимок экрана обрезан в нижней части, а массив начинается с самого левого края, вы на изображении увидите только верхнюю и правую границы, начиная с ячейки A4, и даже они могут быть трудно различимы на снимке экрана. Если ваша версия Excel не поддерживает динамические массивы, используйте обходной путь, добавив следующий декоратор возврата в функцию `et_interest_over_time` (ниже имеющихся декораторов):

```
@xw.ret(expand="table")
```

Теперь, когда вы знаете, как работать UDF повышенной сложности, давайте посмотрим, как мы можем совместно с UDF использовать графики!

Построение графиков с помощью UDF

Как вы, возможно, помните из главы 5, вызов метода `plot` в `DataFrame` по умолчанию возвращает график Matplotlib. В главах 9 и 11 мы уже рассматривали, как в Excel добавить такой график в виде рисунка. При работе с UDF существует простой способ построения графиков: обратите внимание на вторую функцию в `google_trends.py`, показанную в примере 12.4.

**Пример 12.4. Функция `plot` в файле `google_trends.py`
(фрагмент с соответствующими операторами импорта)**

```
import xlwings as xw
import pandas as pd
import matplotlib.pyplot as plt

@xw.func
@xw.arg("df", pd.DataFrame)
def plot(df, name, caller):
    plt.style.use("seaborn")
    if not df.empty:
        caller.sheet.pictures.add(df.plot().get_figure(),
                                   top=caller.offset(row_offset=1).top,
                                   left=caller.left,
                                   name=name, update=True)
    return f"<Plot: {name}>"
```

- 1 Аргумент `caller` — это специальный аргумент, зарезервированный `xlwings`: этот аргумент не отображается при вызове функции из ячейки Excel. Вместо этого аргумент `caller` `xlwings` получит скрытно от нас, и этот аргумент будет соответствовать ячейке, из которой вы вызываете функцию (в виде объекта `xlwings range`). Наличие объекта `range` вызывающей ячейки упрощает размещение графика, и для этого в `pictures.add` используются аргументы `top` и `left`. Аргумент `name` будет определять имя рисунка в Excel.

- 2 Мы установили стиль `seaborn`, чтобы сделать график визуально более привлекательным.
- 3 Вызывайте метод `plot` только в том случае, если `DataFrame` заполнен. Вызов метода `plot` с незаполненным `DataFrame` приведет к ошибке.
- 4 `get_figure()` возвращает объект `Matplotlib figure` из графика `DataFrame`, чего и ожидает `pictures.add`.
- 5 Аргументы `top` и `left` используются только при первой вставке графика. Предоставленные аргументы помещают график в удобное место — на одну ячейку ниже той, откуда вы вызываете эту функцию.
- 6 Аргумент `update=True` гарантирует, что повторные вызовы функции будут обновлять в Excel существующий рисунок с указанным именем, не изменяя его положение или размер. Без этого аргумента `xlwings` будет утверждать, что в Excel картинка с таким именем уже есть.
- 7 Хотя вам и не требуется что-либо возвращать, данная команда значительно облегчит вам жизнь, если вы строку все же вернете: такое действие позволит вам узнать, в каком месте вашего листа находится функция построения графика.

В файле `google_trends.xlsm` в ячейке H3 вызовите функцию `plot` как показано ниже:

```
=plot(A4:F263, "History")
```

Как показано на рис. 12.9, если ваша версия Excel поддерживает динамические массивы, чтобы преобразовать источник в динамический, вместо `A4:F263` следует использовать `A4#`. Предположим, что вы немного запутались в том, как работает функция `get_interest_over_time`. Одним из вариантов лучшего понимания является отладка кода — в следующем разделе показано, как это делается в отношении UDF.

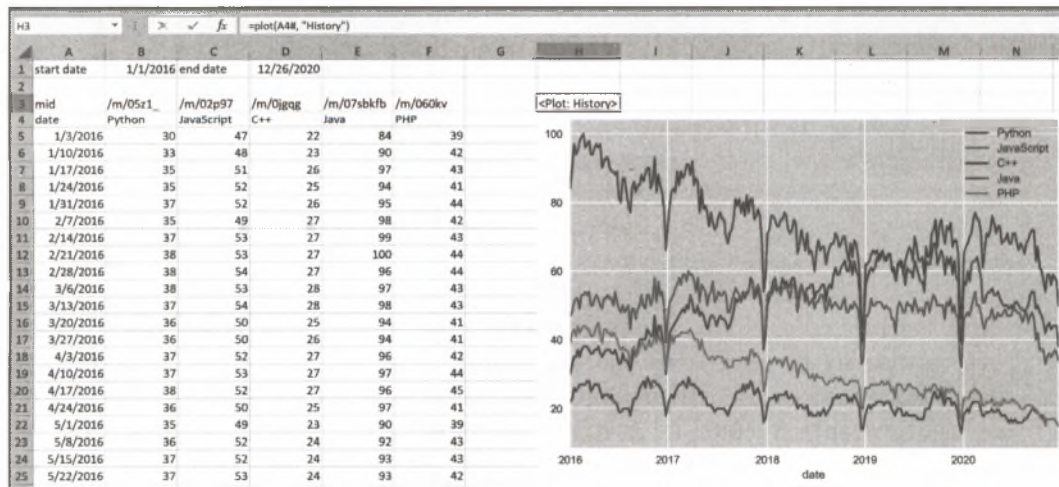


Рис. 12.9. Функция `plot` в действии

Отладка UDFs

Простым способом отладки UDF является использование функции `print`. Если у вас в надстройке `xlwings` включен параметр **Show Console**, вы сможете вывести значение переменной в командной строке, которая появляется при вызове UDF. Более удобным вариантом является использование отладчика VS Code, который позволит вам останавливаться в точках останова и проходить по коду строка за строкой. Чтобы использовать отладчик VS Code (или любой другой отладчик IDE), вам нужно выполнить две операции:

1. В надстройке Excel установите флажок **Debug UDFs**. Это не позволит Excel автоматически запускать Python, поэтому вам придется делать это вручную, как описано в следующем пункте.
2. Самый простой способ запустить сервер Python UDF вручную — это добавить ниже перечисленные строки в самый низ файла, который вы пытаетесь отладить. Я уже добавил эти строки в самый низ файла `google_trends.py` в партнерском репозитории:

```
if __name__ == "__main__":
    xw.serve()
```

3. Как вы помните из *главы 11*, оператор `if` гарантирует, что код будет выполняться только при запуске файла в виде сценария, и он не будет выполняться при импорте кода в виде модуля. Добавив команду `serve`, запустите файл `google_trends.py` в VS Code в режиме отладки, нажав клавишу <F5> и выбрав **Python File**. Обязательно убедитесь, что вы не запускаете файл, нажимая кнопку **Run File**, так как в этом случае точки останова будут игнорироваться.

Давайте установим точку останова на строке 29, щелкнув слева от номера строки. Если вы не знакомы с использованием отладчика VS Code, пожалуйста, загляните в *приложение B*, где я рассказываю о нем более подробно. Теперь когда вы пересчитаете ячейку A4, вызов функции остановится на точке останова, и вы сможете просмотреть переменные. И обратите внимание: запуск `df.info()` всегда помогает во время отладки. Выберите вкладку **Debug Console**, введите в подсказку внизу `df.info()` и подтвердите ввод нажатием клавиши <Enter>, как показано на рис. 12.10.



Отладка в VS Code и Anaconda

Это то же самое предупреждение, что и в *главе 11*: в Windows при первом запуске отладчика VS Code с кодом, использующим `pandas`, вы можете столкнуться с ошибкой: «Exception has occurred: Import Error, Unable to import required dependencies: numpy» (Произошло исключение: Ошибка импорта, невозможно импортировать необходимые зависимости: numpy.) Это происходит потому, что отладчик был запущен до того, как среда Conda активировалась должным образом. В качестве обходного пути остановите отладчик, нажав на значок «стоп», и снова нажмите клавишу <F5> — во второй раз отладка сработает.

Если вы в точке останова удерживаете программу в режиме паузы более 90 сек., Excel отобразит всплывающее окно с сообщением «Microsoft Excel is waiting for another application to complete an OLE action» (Microsoft Excel ждет, пока другое

приложение выполнит действие OLE). Это никак не повлияет на отладку, за исключением необходимости выполнить подтверждение всплывающего окна, чтобы оно исчезло после завершения отладки. Чтобы завершить сеанс отладки, нажмите в VS Code на кнопку Stop (см. рис. 12.10) и убедитесь, что в ленточной надстройке напротив параметра Debug UDFs xlwings флажок снят. Если флажок напротив параметра **Debug UDFs** не снять, ваши функции при следующем пересчете будут возвращать ошибку.

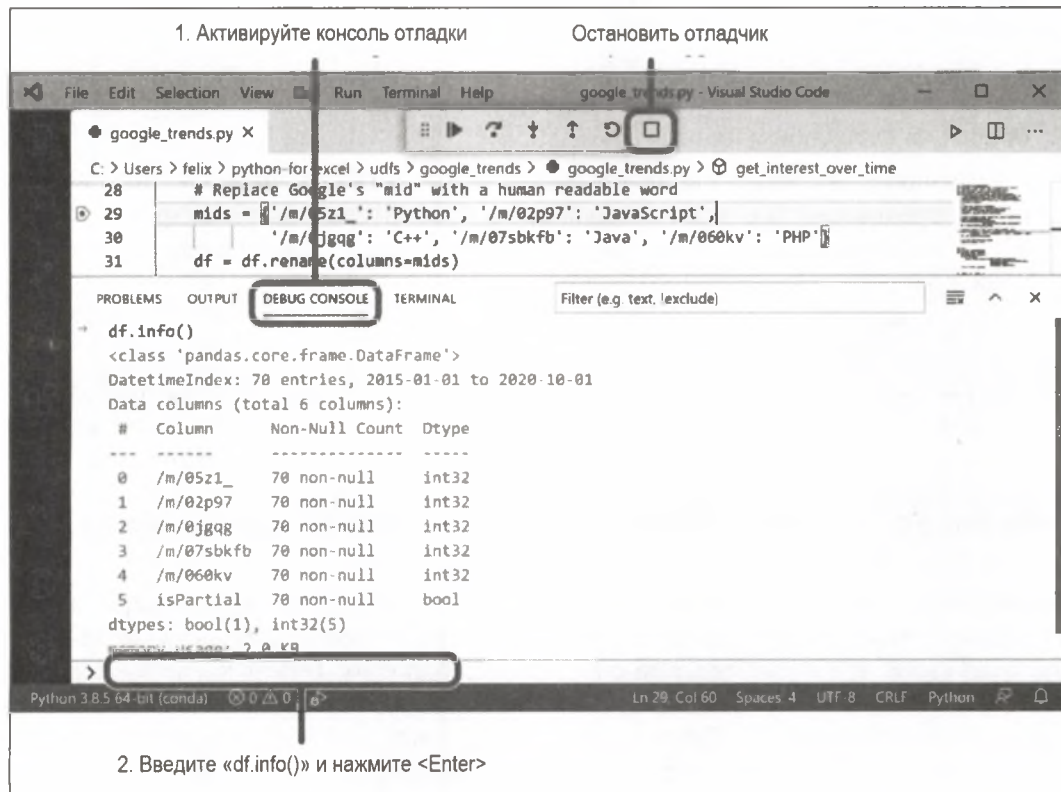


Рис. 12.10. Использование консоли отладки во время приостановки кода в точке останова

В этом разделе на примере исследования Google Trends вы познакомились с наиболее часто используемыми функциями UDF. В следующем разделе мы затронем несколько более сложных вопросов, включая производительность UDF и декоратор `xw.sub`.

Дополнительные вопросы по UDF

Если в рабочей книге применяется большое количество UDF, производительность может понизиться. Этот раздел начинается с демонстрации базовых средств оптимизации производительности, которые мы рассматривали в главе 9, но примени-

тельно к UDF. Вторая часть посвящена кэшированию, технике дополнительной оптимизации производительности, которую мы можем использовать с UDF. Попутно мы узнаем, как обеспечить поступление аргументов функции в виде объектов диапазона `xlwings`, а не в виде значений. В конце этого раздела я познакомлю вас с декоратором `xw.sub`, который можно использовать в качестве альтернативы вызову `RunPython`, если вы работаете исключительно под Windows.

Базовая оптимизация производительности

В этой части рассматриваются две техники оптимизации производительности: как минимизировать перекрестные вызовы приложений и как использовать конвертер необработанных значений.

Минимизация перекрестных вызовов приложений

Как вы, вероятно, помните из главы 9, перекрестные вызовы приложений, то есть вызовы между Excel и Python, происходят довольно медленно, и, чем меньше у вас UDF, тем лучше. Поэтому всегда, при любой возможности старайтесь работать с массивами — наличие версии Excel, поддерживающей динамические массивы, определенно облегчит эту часть работы. Когда вы работаете с pandas DataFrames, вряд ли что-то может пойти не так. Но есть некоторые формулы, в отношении которых вы можете не подумать об автоматическом использовании массивов. Рассмотрим пример, показанный на рис. 12.11, где общая выручка рассчитывается как сумма заданного базового сбора плюс переменный сбор, определяемый умножением цены на количество пользователей.

=revenue(\$B\$5, \$A9, B\$8)						=revenue2(H5, G9:G13, H8:K8)					
A	B	C	D	E	F	G	H	I	J	K	
1	Total Revenue										
2											
3	Single-cell formulas					Array-based formulas					
4											
5	Base Fee	100				Base Fee	100				
6											
7	Price					Price					
8	Users	30	28	26	24	Users	30	28	26	24	
9	1	130	128	126	124	1	130	128	126	124	
10	2	160	156	152	148	2	160	156	152	148	
11	5	250	240	230	220	5	250	240	230	220	
12	10	400	380	360	340	10	400	380	360	340	
13	20	700	660	620	580	20	700	660	620	580	

Рис. 12.11. Формулы для одной ячейки (слева) в сравнении с формулами на основе массива (справа)

Формулы для одной ячейки

В левой таблице на рис. 12.11 в ячейке B9 используется формула `=revenue($B$5, $A9, B$8)`. Далее эта формула применяется ко всему диапазону B9:E13. Это означает, что у вас имеется 20 формул в одной ячейке, которые обращаются к функции `revenue`.

Формулы, основанные на массиве

В правой таблице на рис. 12.11 используется формула `=revenue2(H5, G9:G13, H8:K8)`. Если в вашей версии Excel динамические массивы отсутствуют, в функцию `revenue2` следует добавить декоратор `xw.ret(expand="table")` или превратить массив в унаследованный массив CSE, выделив диапазон ячеек H9:K13, нажав клавишу <F2> для редактирования формулы и подтвердив выбор нажатием комбинации клавиш <Ctrl>+<Shift>+<Enter>. В отличие от формулы для отдельной ячейки в этой версии функция `revenue2` вызывается только один раз.

Вы можете ознакомиться с кодом Python для двух UDF, рассмотрев пример 12.5, а исходный файл вы найдете в папке `revenues` в каталоге `udfs` партнерского репозитория.

Пример 12.5. `revenues.py`

```
import numpy as np
import xlwings as xw

@xw.func
def revenue(base_fee, users, price):
    return base_fee + users * price

@xw.func
@xw.arg("users", np.array, ndim=2)
@xw.arg("price", np.array)
def revenue2(base_fee, users, price):
    return base_fee + users * price
```

Когда вы в ячейке B5 или H5 соответственно поменяете значение базовой ставки, вы увидите, что в правом примере, показанном на рис. 12.11, пересчет значений произойдет гораздо быстрее, чем в левом. Различия в функциях Python минимальны и заключаются только в декораторах аргументов: версия на основе массивов считывает данные `users` и `prices` как массивы NumPy. Единственная особенность здесь — данные `users` с помощью установки `ndim=2` в декораторе аргументов считываются как двумерный вектор-столбец. Вы, вероятно, помните, что массивы NumPy похожи на DataFrames, но без индекса или заголовка и только с одним типом данных, но если вы желаете получить более подробную справку, загляните в главу 4.

Использование необработанных значений

Использование необработанных значений означает, что вы не учитываете этапы подготовки и очистки данных, которые xlwings выполняет поверх `pywin32`. Это зависимости xlwings от Windows. Что, например, означает, что вы больше не можете работать с DataFrames напрямую, поскольку `pywin32` их не распознает, но это может не стать проблемой, если вы работаете со списками или массивами NumPy. Чтобы использовать UDF с необработанными значениями, используйте строку `raw`

в качестве аргумента `convert` в декораторе `argument` или `return`. Это эквивалентно использованию конвертера `raw` в методе `options` объекта `xlwings range`, как мы делали в *главе 9*. И поэтому, учитывая то, что мы видели ранее, наибольшую производительность вы получите во время операций записи. Например, вызов следующей функции без декоратора `return` на моем ноутбуке работал бы примерно в три раза медленнее:

```
import numpy as np
import xlwings as xw
@xw.func
@xw.ret("raw")
def randn(i=1000, j=1000):
    """Returns an array with dimensions (i, j) with normally distributed
    pseudorandom numbers provided by NumPy's random.randn (Возвращает
    массив с размерностью (i, j) с нормально распределенными
    псевдослучайными числами, предоставленными NumPy's random.randn)
    """
    return np.random.randn(i, j)
```

Вы найдете соответствующий пример в партнерском репозитории в папке `raw_values` в каталоге `udfs`. При работе с UDFs у вас есть еще один простой способ повысить производительность: предотвратить повторные вычисления медленных функций путем кэширования их результатов.

Кэширование

Когда вы вызываете детерминированную функцию, то есть функцию, которая при одних и тех же входных данных всегда возвращает один и тот же выход, вы можете хранить результат в кэше: при повторных вызовах функции больше не нужно ждать медленного вычисления, а можно взять результат из кэша, где он уже предварительно вычислен. Это лучше всего объяснить на коротком примере. Очень простой механизм кэширования можно запрограммировать с помощью словаря:

```
In [7]: import time
```

```
In [8]: cache = {}
```

```
def slow_sum(a, b):
    key = (a, b)
    if key in cache:
        return cache[key]
    else:
        time.sleep(2) # сон в течение 2 секунд
        result = a + b
        cache[key] = result
        return result
```

Когда вы в первый раз вызываете эту функцию, кэш пуст. Поэтому код выполнит условие `else` с искусственной двухсекундной паузой, имитирующей медленное вычисление. После выполнения вычисления код, прежде чем вернуть результат, доба-

вит полученный результат в словарь кэша. Когда вы вызовете эту функцию во второй раз с теми же аргументами, но во время этого же сеанса Python, код найдет результат в кэше и вернет сразу же, без повторного медленного вычисления. Кэширование результата на основе его аргументов также называется *запоминанием*. Соответственно, вы увидите разницу во времени между первым и вторым вызовом функции:

```
In [9]: %%time
        slow_sum(1, 2)
Wall time: 2.01 s
Out[9]: 3
In [10]: %%time
        slow_sum(1, 2)
Wall time: 0 ns
Out[10]: 3
```

В Python есть встроенный декоратор `lru_cache`, способный значительно облегчить вам жизнь. Он импортируется из модуля `functools`, являющегося частью стандартной библиотеки. `lru` соответствует кэшу, который использовался последним. Это означает, что кэш хранит максимальное количество результатов (по умолчанию 128) и самые старые результаты будут удалены только после заполнения кэша. Мы можем использовать этот декоратор в нашем примере Google Trends из предыдущего раздела. Пока мы запрашиваем только значения за прошлые периоды, мы можем смело кэшировать результат. Это не только ускорит выполнение нескольких вызовов, но и уменьшит количество запросов, которые мы отправляем в Google, снижая вероятность того, что Google заблокирует нас — а это может произойти, если вы отправите слишком много запросов за короткое время. Вот первые несколько строк функции `get_interest_over_time` с необходимыми изменениями, позволяющими использовать кэш:

```
from functools import lru_cache 1

import pandas as pd
from pytrends.request import TrendReq
import matplotlib.pyplot as plt
import xlwings as xw

@lru_cache 2
@xw.func(call_in_wizard=False)
@xw.arg("mids", xw.Range, doc="Machine IDs: A range of max 5 cells") 3
@xw.arg("start_date", doc="A date-formatted cell")
@xw.arg("end_date", doc="A date-formatted cell")
def get_interest_over_time(mids, start_date, end_date):
    """Query Google Trends - replaces the Machine ID (mid) of
    common programming languages with their human-readable
    equivalent in the return value, e.g., instead of "/m/05z1_"
    it returns "Python" (Запрос Google Trends - заменяет машинный
    идентификатор (mid) распространенных языков программирования на их
```


эквивалент, понятный человеку, в возвращаемом значении, например, вместо `"/m/05z1_"` возвращает `"Python"`).

```
"""
```

```
mids = mids.value
```

4

- 1 Импорт декоратора `lru_cache`.
- 2 Использование декоратора. Декоратор должен находиться выше декоратора `xw.func`.
- 3 По умолчанию `mids` — это список. Но в данном случае, это создает проблему, потому что функции со списками в качестве аргументов не могут быть кэшированы. Основная проблема заключается в том, что списки — это изменяемые объекты, которые нельзя использовать в качестве ключей в словарях; более подробную информацию о изменяемых и неизменяемых объектах см. в *приложении С*. Использование конвертера `xw.Range` позволяет нам получить `mids` в виде объекта `xlwings range`, а не в виде списка, что решает нашу проблему.
- 4 Чтобы остальная часть кода снова заработала, нам теперь нужно получить значения через значение `value` объекта `range xlwings`.



Кэширование с помощью различных версий Python

Если вы используете Python версии ниже 3.8, вам придется использовать декоратор со скобками, как показано ниже: `@lru_cache()`. При использовании Python 3.9 или более поздней версии замените `@lru_cache` на `@cache`, что аналогично `@lru_cache(maxsize=None)`, то есть кэш никогда не избавляется от старых значений. Вам также необходимо импортировать декоратор `cache` из раздела `func-tools`.

Конвертер `xw.Range` может быть полезен и в других обстоятельствах. Например, если вам нужно получить доступ к формулам в ячейках, а не к значениям в UDF. В предыдущем примере для доступа к формулам ячеек можно было написать `mids.formula`. Полностью пример вы найдете в папке `google_trends_cache` в каталоге `udfs` партнерского репозитория.

Теперь, когда вы знаете, как регулировать производительность UDF, давайте завершим этот раздел знакомством с декоратором `xw.sub`.

Декоратор Sub

В *главе 10* я показал вам, как ускорить вызов `RunPython`, задействовав параметр `Use UDF Server`. Если вы живете только в мире Windows, существует альтернатива комбинации `RunPython/Use UDF Server` в виде декоратора `xw.sub`. Это позволит вам импортировать в Excel функции Python как Sub-процедуры без необходимости вручную писать вызовы `RunPython`. В Excel вам понадобится процедура `Sub`, которую

можно будет прикрепить к кнопке — функция Excel, полученная при использовании декоратора `xw.func`, не подойдет. Для проверки работоспособности создайте новый проект `quickstart` под именем `importsub`. Как обычно, сначала убедитесь, что вы вошли в каталог, в котором будет создан проект:

```
(base)> xlwings quickstart importsub
```

В File Explorer перейдите в созданную папку `importsub` и откройте `importsub.xlsm` в Excel и `importsub.py` в VS Code. Затем к главной функции добавьте декоратор `@xw.sub`, как показано в примере 12.6.

Пример 12.6. `importsub.py` (фрагмент)

```
import xlwings as xw
```

```
@xw.sub
```

```
def main():
```

```
    wb = xw.Book.caller()
```

```
    sheet = wb.sheets[0]
```

```
    if sheet["A1"].value == "Hello xlwings!":
```

```
        sheet["A1"].value = "Bye xlwings!"
```

```
    else:
```

```
        sheet["A1"].value = "Hello xlwings!"
```

В надстройке `xlwings`, чтобы увидеть доступные макросы, нажмите кнопку **Import Functions**, а затем комбинацию клавиш `<Alt>+<F8>`: в дополнение к `SampleCall`, который использует `RunPython`, теперь вы также увидите макрос под названием `main`. Выберите этот макрос и нажмите кнопку **Run** — в ячейке `A1` появится знакомое приветствие. Теперь, как это было сделано в *главе 10*, вы можете назначить макрос `main` кнопке. Хотя декоратор `xw.sub` может облегчить вам жизнь в Windows, имейте в виду, что, используя его, вы теряете межплатформенную совместимость. Благодаря `xw.sub` мы познакомились со всеми декораторами `xlwings` — и я снова обобщил их в табл. 12.1.

Таблица 12.1. Декораторы `xlwings`

Декоратор	Описание
<code>xw.func</code>	Поместите этот декоратор поверх всех функций, которые вы хотите импортировать в Excel как функцию Excel
<code>xw.sub</code>	Поместите этот декоратор поверх всех функций, которые вы хотите импортировать в Excel как процедуру Excel Sub
<code>xw.arg</code>	Применяйте конвертеры и опции к аргументам, например добавьте строку <code>docstring</code> через аргумент <code>doc</code> , или вы можете получить диапазон в виде <code>DataFrame</code> , указав <code>pd.DataFrame</code> в качестве первого аргумента (это предполагает, что вы импортировали <code>pandas</code> как <code>pd</code>)
<code>xw.ret</code>	Применяйте конвертеры и опции к возвращаемым значениям, например подавить индекс <code>DataFrame</code> , указав <code>index=False</code>

Для получения более подробной информации об этих декораторах ознакомьтесь с документацией `xlwings`⁶.

Заключение

Эта глава была посвящена написанию функций Python и их импорту в Excel в качестве UDF, что позволяет вызывать их через формулы ячеек. Изучив пример Google Trends, вы узнали, как влиять на поведение аргументов и возвращаемых значений функции с помощью декораторов `arg` и `ret` соответственно. В прошлой части мы показали вам несколько приемов повышения производительности и представили декоратор `xw.sub`, который можно использовать в качестве замены `RunPython`, если вы работаете исключительно с Windows. Преимущество написания UDF на Python заключается в том, что это позволяет вам заменить длинные и сложные формулы ячеек кодом на Python, который проще понимать и поддерживать. Я предпочитаю работать с UDF-файлами, используя `pandas DataFrames` с новыми динамическими массивами Excel. Эта комбинация позволяет легко обрабатывать данные, которые мы получаем из Google Trends, т.е. `DataFrames` с динамическим значением количества строк.

Вот и все — мы подошли к концу книги! Большое спасибо за ваш интерес к моей интерпретации современной среды автоматизации и анализа данных для Excel! Я хотел познакомить вас с миром Python и его мощными пакетами с открытым исходным кодом, чтобы вы самостоятельно могли написать код на Python для своего следующего проекта, а не разбираться с собственными решениями Excel, такими как VBA или Power Query, сохраняя возможность легко отказаться от Excel в случае необходимости. Хочу надеяться, что смог дать вам несколько практических советов, чтобы облегчить начало работы. Прочитав эту книгу, вы теперь знаете, как:

- ◆ заменить рабочую книгу Excel блокнотом Jupyter и кодом `pandas`;
- ◆ применить пакетную обработку рабочих книг Excel путем их чтения с помощью `OpenPyXL`, `xlrd`, `pyxlsb` или `xlwings`, а затем их объединения с помощью `pandas`;
- ◆ создать отчеты Excel с помощью `OpenPyXL`, `XlsxWriter`, `xlwt` или `xlwings`;
- ◆ использовать Excel как внешний модуль и подключить его к практически любому инструменту через `xlwings`, просто нажав на кнопку или написав UDF.

Однако вскоре вы захотите выйти за рамки этой книги. Я приглашаю вас время от времени заглядывать на главную страницу книги⁷, чтобы вовремя узнавать об обновлениях и дополнительных материалах. В связи с этим я предлагаю вам несколько идей, которые вы можете исследовать самостоятельно:

- ◆ запланируйте периодическое выполнение сценария Python с помощью планировщика задач в Windows или задания `cron` в macOS или Linux. Например, вы

⁶ <https://oriel.ly/h-sT>. — Примеч. пер.

⁷ <https://xlwings.org/book>. — Примеч. пер.

можете создавать отчет Excel каждую пятницу на основе данных, получаемых из REST API или базы данных;

- ◆ напишите сценарий Python, который отправляет оповещения по электронной почте всякий раз, когда значения в ваших файлах Excel соответствуют определенному условию. Это может произойти, когда, например, баланс вашего счета, объединенный из нескольких рабочих книг, падает ниже определенного значения, или когда он показывает значение, отличное от того, которое вы ожидаете, основываясь на вашей внутренней базе данных;
- ◆ напишите код, который находит ошибки в рабочих книгах Excel: проверьте ошибки в ячейках, такие как #REF! или #VALUE! или логические ошибки. Например, убедитесь, что формула включает все ячейки, которые она должна включать;
- ◆ если вы начнете отслеживать свои критически важные рабочие книги с помощью профессиональной системы контроля версий, такой как Git, вы сможете даже запускать эти тесты автоматически при регистрации новой версии.

Если эта книга подтолкнет вас к автоматизации ежедневной или еженедельной работы по загрузке данных и копированию/вставке их в Excel, я буду лишь рад. Автоматизация не только сохраняет время, но и значительно снижает вероятность совершения ошибок. Если у вас есть какие-либо замечания, пожалуйста, дайте мне знать об этом! Вы можете связаться со мной через O'Reilly, задав вопрос в сопроводительном репозитории⁸ или в Twitter по адресу **@felixzumstein**⁹.

⁸ <https://oriel.ly/vVHnR>. — Примеч. пер.

⁹ <https://twitter.com/felixzumstein>. — Примеч. пер.

ПРИЛОЖЕНИЯ

В *главе 2* я представил среду Conda и объяснил, что параметр `(base)` при запуске Anaconda Prompt обозначает активную в данный момент среду Conda, которой присвоено имя `base`. Для Anaconda требуется, чтобы вы всегда работали в активированной среде, но для среды `base` активация выполняется автоматически, при запуске Anaconda Prompt в Windows или терминала (Terminal) в macOS. Использование сред Conda позволяет правильно разделить зависимости ваших проектов: если вы хотите опробовать новую версию пакета, например `pandas`, не меняя базовой среды, вы можете сделать это в отдельной среде Conda. В первой части этого приложения я покажу вам процесс создания среды Conda, которая называется `x138`, где мы установим все пакеты в том варианте, в котором они были использованы для написания этой книги. Благодаря этому вы сможете запускать примеры из этой книги, не внося изменения, даже если некоторые пакеты за это время вышли в новых версиях, содержащих серьезные изменения. Во второй части приложения я покажу вам, как отключить автоматическую активацию базовой среды, если вас не устраивает стандартная конфигурация.

Создание новой среды Conda

Чтобы создать новую среду с именем `x138`, в которой используется Python 3.8, выполните в Anaconda Prompt следующую команду:

```
(base)> conda create --name x138 python=3.8
```

После нажатия клавиши `<Enter>` Conda выведет на экран информацию о том, что будет установлено новое окружение, и попросит вас подтвердить продолжение установки:

```
Proceed ([y]/n)? (Продолжить ([y]/n)?)
```

Нажмите клавишу `<Enter>` для подтверждения продолжения установки или нажмите клавишу `n`, если установку желаете отменить. После завершения установки активируйте новую среду следующим образом:

```
(base)> conda activate x138
(x138)>
```

Теперь имя среды изменилось с `base` на `x138`, и вы можете использовать Conda или `pip` для установки пакетов в эту новую среду, не затрагивая другие среды (напоми-

наю: используйте `pip`, только если пакет недоступен через Conda). Давайте продолжим и установим все пакеты из этой книги в той версии, в которой я их использовал. Сначала убедитесь, что вы работаете в среде `x138`, то есть в Anaconda Prompt отображается приглашение (`x138`), затем установите пакеты Conda, выполнив команду, показанную ниже (обратите внимание: данная команда должна быть набрана в одну строку; ниже разрывы строк предназначены только чтобы показать ее на странице книги):

```
(x138)> conda install lxml=4.6.1 matplotlib=3.3.2 notebook=6.1.4
                openpyxl=3.0.5 pandas=1.1.3 pillow=8.0.1
                plotly=4.14.1 flake8=3.8.4 python-dateutil=2.8.1
                requests=2.24.0 sqlalchemy=1.3.20 xlrd=1.2.0
                xlswriter=1.3.7 xlutils=2.0.0 xlwings=0.20.8
                lwt=1.3.0
```

Убедитесь, что в плане установки нет ошибок, и завершите создание среды, установив с помощью `pip` две оставшиеся зависимости:

```
(x138)> pip install pyxlsb==1.0.7 pytrends==4.7.3
(x138)>
```



Использование среды `x138`

Если вы хотите использовать среду `x138`, а не базовую среду для работы над примерами в этой книге, убедитесь, что ваша среда `x138` при запуске была активирована:

```
(base)> conda activate x138
```

То есть, там, где ранее вы видели приглашение Anaconda Prompt в виде строки `(base)>`, должна отображаться строка `(x138)>`.

Чтобы отключить среду `x138` и вернуться к среде `base`, введите:

```
(x138)> conda deactivate
(base)>
```

Если вы хотите полностью удалить среду `x138`, выполните следующую команду:

```
(base)> conda env remove --name x138
```

Чтобы не выполнять шаги по созданию среды `x138` вручную, вы можете воспользоваться файлом среды `x138.yml`, который я включил в папку `conda` партнерского репозитория. Для автоматической установки среды `x138` выполните следующие команды:

```
(base)> cd C:\Users\username\python-for-excel\conda
(base)> conda env create -f x138.yml
(base)> conda activate x138
(x138)>
```

По умолчанию при запуске терминала на компьютере, где установлена операционная система macOS, или Anaconda Prompt на компьютере под управлением операционной системы Windows, Anaconda всегда активирует среду `base`. Если вас это не устраивает, вы можете отключить автоматическую активацию, о чем я расскажу далее.

Отключение автоматической активации

Если вы не хотите, чтобы окружение `base` при каждом запуске Anaconda Prompt активировалось автоматически, его можно отключить: для этого, прежде чем вы сможете использовать Python, вам придется в командной строке (Windows) или терминале (macOS) ввести вручную команду `conda activate base`.

Windows

В Windows вместо Anaconda Prompt вам придется использовать обычную командную строку. Следующие шаги позволят выполнить команду `conda` в командной строке. Не забудьте заменить путь в первой строке на путь, по которому Anaconda установлена в вашей системе:

```
> cd C:\Users\username\Anaconda3\condabin  
> conda init cmd.exe
```

Теперь ваша командная строка настроена на Conda и в дальнейшем вы можете активировать базовое окружение следующим образом:

```
> conda activate base  
(base) >
```

macOS

На macOS, чтобы отключить автоматическую активацию, выполните в терминале следующую команду:

```
(base) > conda config --set auto_activate_base false
```

Если вы захотите вернуться назад, выполните ту же команду снова, но вместо `false` введите `true`. Изменения вступят в силу после перезапуска терминала. В дальнейшем, прежде чем снова использовать команду `python`, вам нужно будет активировать среду `base` следующим образом:

```
> conda activate base  
(base) >
```

Расширенные функциональные возможности VS Code

В этом приложении вы узнаете, как работает отладчик в VS Code и как можно запускать блокноты Jupyter непосредственно из VS Code. Рассматриваемые разделы не зависят друг от друга, поэтому вы можете читать их в любом порядке.

Отладчик

Если вы когда-нибудь пользовались отладчиком VBA в Excel, у меня для вас хорошие новости: отладка в VS Code осуществляется примерно таким же образом. Для начала откроем в VS Code файл `debugging.py` из партнерского репозитория. Щелкните в поле слева от строки номер 4 так, чтобы там появилась красная точка, — это будет точка останова, в которой выполнение кода будет приостановлено. Теперь, чтобы начать отладку, нажмите клавишу `<F5>`: появится командная панель для выбора конфигурации отладки. Для отладки активного файла выберите **Python File** и выполняйте код до тех пор, пока он не достигнет точки останова. Строка будет выделена, а выполнение кода приостановлено (рис. В.1). Во время отладки строка состояния окрасится в оранжевый цвет.

Если раздел **Variables** (Переменные) автоматически не отобразится слева, то чтобы увидеть значения переменных, щелкните мышью на меню **Run** (Выполнить). Или наведите курсор на переменную, находящуюся в исходном коде, и вы получите всплывающую подсказку, в которой будет указано ее значение. В верхней части вы увидите панель инструментов отладки, которая предоставит вам доступ к следующим кнопкам (слева направо): **Continue** (Продолжить), **Step Over** (Шаг с обходом), **Step Into** (Шаг с заходом), **Step Out** (Шаг с выходом), **Restart** (Перезапустить) и **Stop** (Стоп). При наведении на эти кнопки указателя мыши вы также увидите соответствующие данным кнопкам комбинации клавиш.

Рассмотрим, для чего предназначена каждая из этих кнопок:

Continue (Продолжить)

Программа продолжает выполняться до тех пор, пока не достигнет следующей точки останова или конца программы. Когда программа достигает конца программы, процесс отладки останавливается.

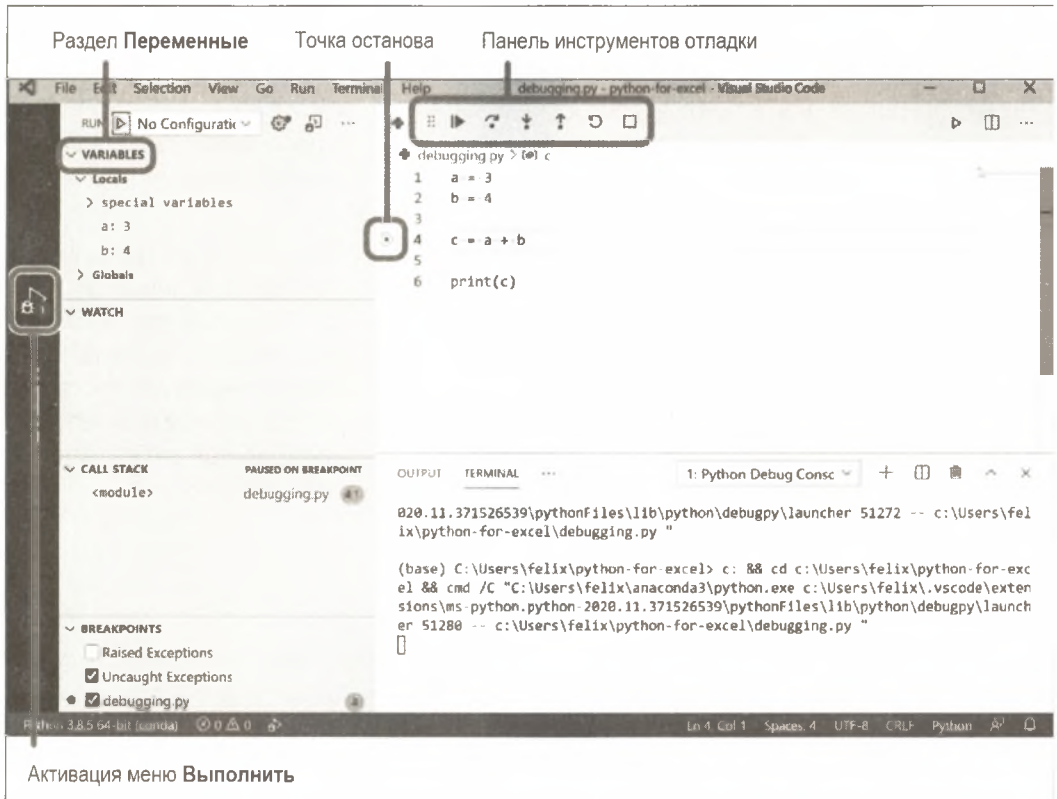


Рис. В.1. VS Code с остановленным отладчиком в точке останова

Step Over (Шаг с обходом)

Отладчик перейдет на одну строку вперед. **Step Over** означает, что отладчик не будет показывать строки кода, которые в данный момент скрыты. Например, он не будет заходить в код функции, которую вы вызываете построчно, но функция все равно будет вызвана.

Step Into (Шаг с заходом)

Если у вас есть код, который вызывает функцию, класс и т. д., **Step Into** заставит отладчик войти в эту функцию или класс. Если функция или класс находятся в другом файле, отладчик этот файл откроет.

Step Out (Шаг с выходом)

Если вы вошли в функцию с помощью **Step Into**, **Step Out** заставит отладчик вернуться на следующий, более высокий уровень, пока в конечном счете вы не вернетесь на самый высокий уровень, с которого вы первоначально вызвали **Step Into**.

Restart (Перезапустить)

Данная кнопка остановит текущий процесс отладки и запустит новый процесс с самого начала.

Stop (Cmon)

Остановка текущего процесса отладки.

Теперь, когда вы знаете, что делает каждая кнопка, нажмите на кнопку **Step Over**, чтобы перейти на одну строку вперед и посмотреть, как переменная `c` появляется в разделе **Variables**, а затем завершите это отладочное упражнение, нажав на кнопку **Continue**.

Если вы сохраните конфигурацию отладки, панель команд не будет появляться и спрашивать вас о конфигурации каждый раз, когда вы нажимаете клавишу `<F5>`; щелкните мышью на значке **Выполнить** на панели действий, затем нажмите «создать файл `launch.json`». В результате вновь появится панель команд, и когда вы выберете **Python File**, файл `launch.json` будет создан в каталоге `.vscode`. Когда вы нажмете клавишу `<F5>`, отладчик запустится немедленно. Если вам нужно изменить конфигурацию или вы хотите снова отобразить всплывающую командную панель, отредактируйте или удалите файл `launch.json` в каталоге `.vscode`.

Блокноты Jupyter в VS Code

Вместо того чтобы запускать блокноты Jupyter в веб-браузере, вы можете запускать их из VS Code. Кроме того, VS Code предлагает удобный проводник переменных, а также опции для преобразования блокнота в стандартные файлы Python без потери функциональности ячеек. Это облегчает использование отладчика или копирование/вставку ячеек между разными блокнотами. Давайте начнем с запуска блокнота в VS Code!

Запуск блокнотов Jupyter

Нажмите на значок **Explorer** (Проводник) на панели активности и откройте файл `ch05.ipynb` выбрав его из партнерского репозитория. Чтобы продолжить, вам нужно сделать блокнот доверенным, нажав в появившемся уведомлении на **Trust** (Доверять). Разметка блокнота выглядит немного иначе, чем в браузере, и соответствует остальной части VS Code, однако в остальном это тот же интерфейс, включая все сочетания клавиш. Давайте запустим первые три ячейки с помощью сочетания клавиш `<Shift>+<Enter>`. В результате запустится сервер блокнота Jupyter, если он еще не был запущен (вы увидите статус в правом верхнем углу блокнота). После запуска ячеек нажмите на кнопку калькулятора в меню в верхней части блокнота: откроется проводник переменных, в котором вы сможете увидеть значения всех существующих на данный момент переменных, как показано на рис. В.2. То есть вы обнаружите только переменные из ячеек, которые были запущены.



Сохранение блокнотов Jupyter в VS Code

Чтобы сохранить блокноты в VS Code, нужно воспользоваться кнопкой **Save** в верхней части блокнота или нажать сочетание клавиш `<Ctrl>+<S>` в Windows или `<Command>-<S>` в macOS. Команда меню **Файл > Сохранить** не работает!

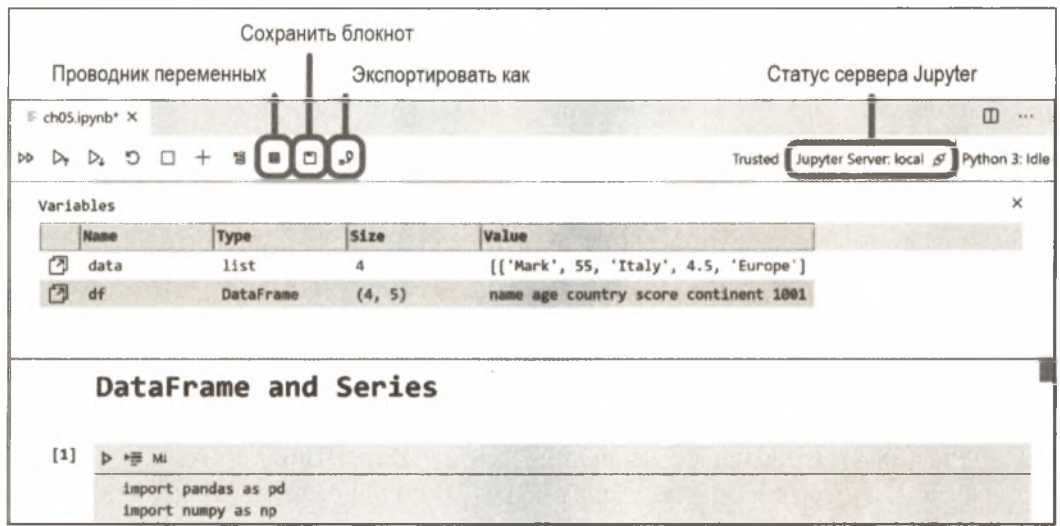


Рис. В.2. Jupyter notebook Проводник переменных

Если вы используете структуры данных, такие как вложенные списки, массивы NumPy или DataFrames, дважды щелкните по переменной: это откроет Data Viewer и предоставит вам привычный вид, похожий на электронную таблицу. На рис. В.3 показан просмотрщик данных после двойного щелчка на переменной `df`.

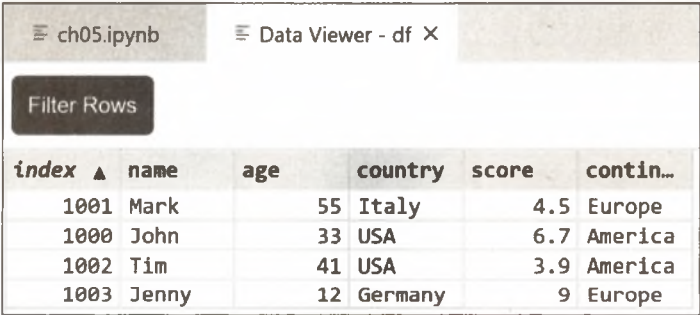


Рис. В.3. блокнот Jupyter Просмотрщик данных

Хотя VS Code позволяет запускать стандартные файлы блокнотов Jupyter, с помощью VS Code также можно преобразовывать блокноты в обычные файлы Python без потери ячеек. Давайте посмотрим, как это делается!

Сценарии Python с ячейками кода

Чтобы использовать ячейки блокнота Jupyter в стандартных файлах Python, в VS Code используется специальное сочетание для обозначения ячеек: `# %%`. Чтобы пре-

образовать текущий блокнот Jupyter, откройте его и нажмите кнопку **Export As** в меню в верхней части блокнота (см. рис. В.2). Это позволит вам выбрать команду **Python File** из палитры команд. Однако вместо того, чтобы преобразовывать существующий файл, давайте создадим новый файл, который назовем `cells.py` со следующим содержимым:

```
# %%
3 + 4
# %% [уценка]
# # Это название
#
# Некоторые материалы для разметки
```

Ячейки уценки должны начинаться с `# %% [markdown]` (уценка), и необходимо, чтобы вся ячейка была помечена как комментарий. Если вы хотите запустить такой файл, как блокнот, щелкните мышью на ссылке **Run Below**, которая появляется, если навести курсор на первую ячейку. В результате справа откроется интерактивное окно Python, как показано на рис. В.4.

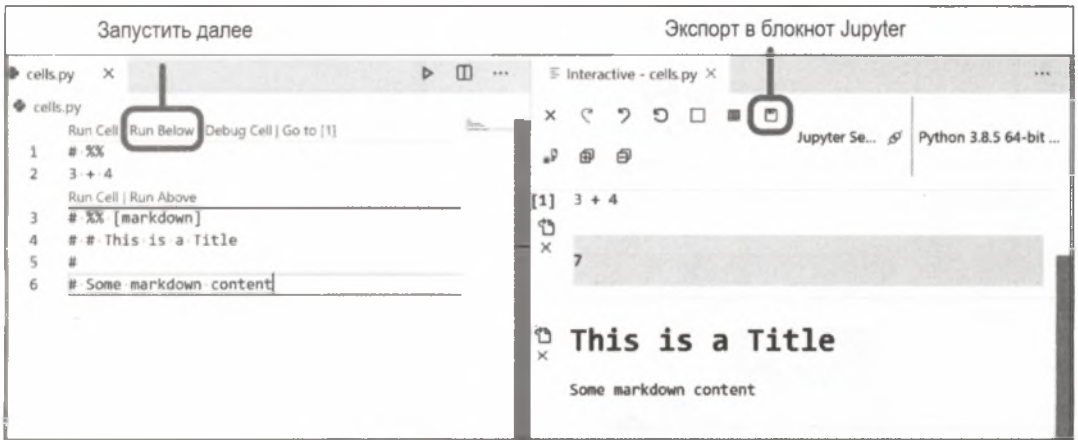


Рис. В.4. Интерактивное окно Python

Интерактивное окно Python вновь представлено в виде блокнота. Чтобы экспортировать файл в формате `ipynb`, щелкните мышью на значке **Save (Export as Jupyter notebook)** в верхней части интерактивного окна Python. Интерактивное окно Python также предоставляет вам ячейку в нижней части, откуда вы можете выполнять код в интерактивном режиме. Использование обычных файлов Python в отличие от блокнотов Jupyter позволяет задействовать отладчик VS Code и облегчает работу с контролем версий, поскольку ячейки вывода, которые обычно приносят большое количество ошибок при переходе от одной версии к другой, игнорируются.

Дополнительные концепции Python

В этом приложении мы подробно рассмотрим следующие три раздела: классы и объекты; объекты с привязкой к часовому поясу; изменяемые и неизменяемые объекты. Разделы не зависят друг от друга, поэтому вы можете читать их в любом порядке.

Классы и объекты

В этом разделе мы напишем свой собственный класс, чтобы лучше понять, как между собой связаны классы и объекты. Классы определяют новые типы объектов: класс подобен форме, которую вы используете для выпечки торта. В зависимости от используемых ингредиентов получается различный торт. Например, шоколадный или творожный. Процесс извлечения торта (объекта) из формы (класса) называется *созданием экземпляра*, поэтому объекты также называют *экземплярами класса*. Будь то шоколадный или творожный торт, они оба являются разновидностью торта: классы позволяют определить новые типы данных, которые хранят связанные данные (*атрибуты*) и функции (*методы*) вместе и, следовательно, помогают структурировать и упорядочить код. Позвольте мне теперь вернуться к примеру с автомобильной гоночной игрой из *главы 3*, чтобы определить наш собственный класс:

```
In [1]: class Car:
        def __init__(self, color, speed=0):
            self.color = color
            self.speed = speed

        def accelerate(self, mph):
            self.speed += mph
```

Это простой класс `Car` (автомобиль), с двумя методами. Методы — это функции, которые являются частью определения класса. У этого класса есть один обычный метод под названием `accelerate`. Этот метод изменяет данные (`speed`) экземпляра данного класса. Кроме того, здесь вы обнаружите специальный метод, который начинается и заканчивается двойным подчеркиванием, и называется `__init__`. Он будет автоматически вызываться Python при инициализации объекта, чтобы прикре-

пить к нему некоторые начальные данные. Первый аргумент каждого метода представляет собой экземпляр класса и по традиции называется `self`. Понятнее это станет, когда вы увидите, как используется класс `Car`. Сначала давайте создадим два автомобиля. Это делается таким же образом, как и вызов функции: вызываете класс, добавляя круглые скобки и указывая аргументы метода `__init__`. Для `self` никогда ничего не надо указывать, так как Python сам позаботится об этом. В этом примере `self` будет `car1` или `car2` соответственно:

```
In [2]: # Давайте создадим два объекта car
        car1 = Car("red")
        car2 = Car(color="blue")
```

Когда вы вызываете класс, вы на самом деле вызываете функцию `__init__`, поэтому все, что касается аргументов функций, применимо и здесь: для `car1` мы предоставляем аргумент в виде позиционного аргумента, а для `car2` используем аргументы в виде ключевых слов. После создания двух объектов `car` из класса `Car` мы рассмотрим их атрибуты и вызовем их методы. Как мы увидим, после ускорения `car1` скорость `car1` изменится, но останется неизменной для `car2`, так как эти два объекта независимы друг от друга:

```
In [3]: # По умолчанию объект передает свое местоположение в память
        car1
Out[3]: <__main__.Car at 0x7fea812e3890>
In [4]: # Атрибуты дают вам доступ к данным объекта
        car1.color
Out[4]: 'red'
In [5]: car1.speed
Out[5]: 0
In [6]: # Вызов метода ускорения для car1
        car1.accelerate(20)
In [7]: # Атрибут скорости car1 изменился
        car1.speed
Out[7]: 20
In [8]: # Атрибут скорости car2 остался неизменным
        car2.speed
Out[8]: 0
```

Python также позволяет изменять атрибуты напрямую, без использования методов:

```
In [9]: car1.color = "green"
In [10]: car1.color
Out[10]: 'green'
In [11]: car2.color # без изменений
Out[11]: 'blue'
```

Вкратце: классы определяют атрибуты и методы объектов. Классы позволяют группировать связанные функции («методы») и данные («атрибуты») вместе, чтобы к ним можно было удобно обращаться с помощью точечной нотации: `myobject.attribute` или `myobject.method()`.

Работа с объектами `datetime` с учетом временной зоны

В *главе 3* мы кратко рассмотрели объекты `datetime`, не зависящие от часового пояса (time-zone-naive). Если часовой пояс имеет большое значение, то обычно вы работаете в часовом поясе UTC и преобразуете его в локальные часовые пояса только для удобства просмотра. UTC означает *всемирное координированное время* и является преемником среднего времени по Гринвичу (GMT). При работе с Excel и Python вам может понадобиться превратить обычные временные метки, предоставляемые Excel, в объекты `datetime` с учетом временных зон. Для поддержки часовых поясов в Python можно использовать пакет `dateutil`, который не является частью стандартной библиотеки, но поставляется в комплекте с Anaconda. В следующих примерах показаны несколько распространенных операций при работе с объектами `datetime` и часовыми поясами:

```
In [12]: import datetime as dt
         from dateutil import tz

In [13]: # Объект времени с привязкой к часовому поясу
         timestamp = dt.datetime(2020, 1, 31, 14, 30)
         timestamp.isoformat()

Out[13]: '2020-01-31T14:30:00'

In [14]: # Объект времени с учетом часового пояса
         timestamp_eastern = dt.datetime(2020, 1, 31, 14, 30,
                                         tzinfo=tz.gettz("US/Eastern"))

         # Печать в isoformat позволяет легко увидеть смещение от UTC
         timestamp_eastern.isoformat()

Out[14]: '2020-01-31T14:30:00-05:00'

In [15]: # Присвоение часового пояса для наивного объекта datetime
         timestamp_eastern = timestamp.replace(tzinfo=tz.gettz("US/Eastern"))
         timestamp_eastern.isoformat()

Out[15]: '2020-01-31T14:30:00-05:00'

In [16]: # Перевести из одного часового пояса в другой.
         # Поскольку часовой пояс UTC очень распространен, для него
         # используется сокращение: tz.UTC
         timestamp_utc = timestamp_eastern.astimezone(tz.UTC)
         timestamp_utc.isoformat()

Out[16]: '2020-01-31T19:30:00+00:00'

In [17]: # Переход к наивной временной зоне
         timestamp_eastern.replace(tzinfo=None)

Out[17]: datetime.datetime(2020, 1, 31, 14, 30)

In [18]: # Текущее время без учета часового пояса
         dt.datetime.now()

Out[18]: datetime.datetime(2021, 1, 3, 11, 18, 37, 172170)

In [19]: # Текущее время в часовом поясе UTC
         dt.datetime.now(tz.UTC)

Out[19]: datetime.datetime(2021, 1, 3, 10, 18, 37, 176299, tzinfo=tzutc())
```

Часовые пояса в Python 3.9

В Python 3.9 в стандартную библиотеку добавлена поддержка часовых поясов в виде модуля `timezone`. Используйте его для замены вызовов `tz.gettz` из `dateutil`:

```
from zoneinfo import ZoneInfo
timestamp_eastern = dt.datetime(2020, 1, 31, 14, 30,
                                tzinfo=ZoneInfo("US/Eastern"))
```

Изменяемые и неизменяемые объекты Python

В Python объекты, которые могут изменять свои значения, называются *изменяемыми*, а те, которые не могут, — *неизменяемыми*. В табл. С.1 показано, как квалифицируются различные типы данных.

Таблица С.1. Изменяемые и неизменяемые типы данных

Изменяемость	Типы данных
Изменяемый	Списки, словари, множества
Неизменяемый	Целые числа, числа с плавающей запятой, булевы типы данных, строки, тип данных <code>datetime</code> , кортежи

Знать об этих различиях необходимо, поскольку поведение изменяемых объектов может отличаться от поведения, к которому вы привыкли в других языках, включая VBA. Взгляните на следующий фрагмент кода VBA:

```
Dim a As Variant, b As Variant
a = Array(1, 2, 3)
b = a
a(1) = 22
Debug.Print a(0) & ", " & a(1) & ", " & a(2)
Debug.Print b(0) & ", " & b(1) & ", " & b(2)
```

В результате выводится следующее:

```
1, 22, 3
1, 2, 3
```

Теперь выполним этот же пример в Python применительно к списку:

```
In [20]: a = [1, 2, 3]
        b = a
        a[1] = 22
        print(a)
        print(b)

[1, 22, 3]
[1, 22, 3]
```

Что в этом случае произошло? В Python переменные — это имена, которые вы «прикрепляете» к объекту. В выражении `b = a` вы присоединяете оба имени к од-

ному и тому же объекту — списку [1, 2, 3]. Поэтому все переменные, присоединенные к этому объекту, будут отображать изменения в списке. Однако это происходит только с изменяемыми объектами: если бы вы подменили список неизменяемым объектом, например кортежем, при изменении объекта `a` объект `b` остался бы неизменным. Если вы хотите, чтобы такой изменяемый объект, как `b`, не зависел от изменений объекта `a`, вы должны явно выполнить копирование списка:

```
In [21]: a = [1, 2, 3]
        b = a.copy()

In [22]: a
Out[22]: [1, 2, 3]
In [23]: b
Out[23]: [1, 2, 3]
In [24]: a[1] = 22 # Изменение "a"...
In [25]: a
Out[25]: [1, 22, 3]
In [26]: b # ... не влияет на "b"
Out[26]: [1, 2, 3]
```

Используя метод `copy` списка, вы создаете *поверхностную копию*: т. е. получите копию списка, но если список содержит изменяемые элементы, эти элементы все равно будут разделены. Если же вы хотите рекурсивно¹ скопировать все элементы, вам необходимо создать *глубокую копию*, используя модуль `copy` из стандартной библиотеки:

```
In [27]: import copy
        b = copy.deepcopy(a)
```

Давайте теперь посмотрим, что происходит, когда вы используете изменяемые объекты в качестве аргументов функций.

Вызов функций с изменяемыми объектами в качестве аргументов

Если вы знакомы с VBA, то вероятно, привыкли помечать аргументы функций как `pass-by-reference` (`ByRef`) или `pass-by-value` (`ByVal`): когда вы передаете переменную функции в качестве аргумента, функция получит возможность изменить ее (`ByRef`) или будет работать над копией значений (`ByVal`), таким образом, оставляя исходную переменную в неприкосновенности. В VBA по умолчанию используется `ByRef`. Рассмотрим следующую функцию в VBA:

```
Function increment(ByRef x As Integer) As Integer
    x = x + 1
    increment = x
End Function
```

¹ Рекурсия — определение, описание, изображение какого-либо объекта или процесса внутри самого этого объекта или процесса, то есть ситуация, когда объект является частью самого себя.

Затем вызовите функцию следующим образом:

```
Sub call_increment()
    Dim a As Integer
    a = 1
    Debug.Print increment(a)
    Debug.Print a
End Sub
```

В результате будет выведено следующее:

```
2
2
```

Однако, если вы в функции инкремента измените ByRef на ByVal, будет выведено:

```
2
1
```

Как это работает в Python? Когда вы передаете переменные, вы передаете имена, которые указывают на объекты. Это означает, что поведение зависит от того, является ли объект изменяемым или нет. Давайте сначала рассмотрим неизменяемый объект:

```
In [28]: def increment(x):
          x = x + 1
          return x

In [29]: a = 1
          print(increment(a))
          print(a)
```

```
2
1
```

Теперь повторим пример с изменяемым объектом:

```
In [30]: def increment(x):
          x[0] = x[0] + 1
          return x

In [31]: a = [1]
          print(increment(a))
          print(a)
```

```
[2]
[2]
```

Если объект является изменяемым, а вы хотите оставить исходный объект неизменным, вам необходимо создать копию объекта:

```
In [32]: a = [1]
          print(increment(a.copy()))
          print(a)
```

```
[2]
[1]
```

Еще один случай, за которым следует следить, это использование изменяемых объектов в качестве аргументов по умолчанию в определениях функций — давайте посмотрим, почему!

Функции с изменяемыми объектами в качестве аргументов по умолчанию

Когда вы пишете функции, как правило, не следует использовать изменяемые объекты в качестве стандартных аргументов. Причина в том, что значение аргументов по умолчанию оценивается только один раз как часть определения функции, а не при каждом вызове функции. Поэтому использование изменяемых объектов в качестве аргументов по умолчанию может привести к неожиданному результату:

```
In [33]: # Не делайте так:
```

```
def add_one(x=[]):  
    x.append(1)  
    return x
```

```
In [34]: add_one()
```

```
Out[34]: [1]
```

```
In [35]: add_one()
```

```
Out[35]: [1, 1]
```

Если вы хотите использовать пустой список в качестве аргумента по умолчанию, сделайте следующее:

```
In [36]: def add_one(x=None):
```

```
    if x is None:  
        x = []  
    x.append(1)  
    return x
```

```
In [37]: add_one()
```

```
Out[37]: [1]
```

```
In [38]: add_one()
```

```
Out[38]: [1]
```

Феликс Зумштейн (Felix Zumstein) — создатель xlwings, популярного пакета с открытым исходным кодом, который позволяет автоматизировать Excel с помощью Python на Windows и macOS. Автор является организатором встреч в Лондоне и Нью-Йорке по темам xlwings и распространения широкого спектра инновационных решений для Excel.

Будучи генеральным директором xltrail — системы контроля версий файлов Excel, автор общался с сотнями пользователей, использующих Excel для решения важных бизнес-задач, и поэтому получил глубокое представление о типичных случаях использования и проблемах Excel в различных отраслях.

На обложке книги Python для Excel изображена коралловая сверташка (*Anilius scytale*), известная также как коралловая вальковатая змея.. Обитает в Гвианасском регионе Южной Америки, в тропических лесах Амазонки, а также в Тринидаде и Тобаго.

Коралловая сверташка вырастает примерно до 70 см в длину. Основная окраска яркая с преобладанием красного цвета. Имеет поперечные черные кольца разной ширины. Ее полосатый вид похож на коралловую змею, от которой и происходит одно из ее общих названий; однако у коралловой сверташки отсутствуют характерные желтые полосы «настоящей» коралловой змеи. На протяжении большей части длины ее тело имеет одинаковый диаметр, а хвост очень короткий, что придает ей трубкообразный вид. Маленькие глаза закрыты крупными чешуйками на голове.

Эта норовистая змея является яйцеживородящей. Она питается жуками, амфибиями, ящерицами, рыбами и мелкими змеями. Коралловая сверташка имеет характерные шпоры (небольшие выступающие кусочки кости возле брюшка), которые являются рудиментарными следами бедер, а иногда и кости верхней части ноги. Эта особенность, наряду с толстыми костями и характерной формой черепа, делает коралловую сверташку очень похожей на предков змей, напоминающих ящериц.

Статус сохранения коралловой сверташки — «недостаток данных», что означает, что пока нет достаточной информации, чтобы судить об угрозе ее исчезновения. Многие из животных, изображенных на обложках книг О'Рейли, находятся под угрозой исчезновения и все они имеют важное значение для окружающего мира.

Иллюстрация на обложке выполнена Карен Монтгомери (Karen Montgomery) на основе черно-белой гравюры из английской «*Циклопедии естественной истории*». Шрифты для обложки — Gilroy Semibold и Guardian Sans. Шрифт текста — Adobe Minion Pro; шрифт заголовка — Adobe Myriad Condensed; шрифт кода — Dalton Maag's Ubuntu Mono.

Предметный указатель

A

Anaconda, 39, 63, 188, 232, 240
Anaconda Prompt, 39, 40, 42, 43, 58, 61, 275
AppleScript, 226

B

big data, 11
Binder, 53

C

Cascading Style Sheets, 198
Code, 49
COM, Component Object Model, 225
Command palette (Палитра команд), 57
Conda, 46, 187, 194, 287, 305
Continue, 308

D

DataFrame, 105, 108, 110, 114, 123, 174,
197, 216, 282, 311
datetime, 315
DatetimeIndex, 146
◊ фильтрация, 148

E

Emacs, 60
Excel
◊ модернизированный, 29
ExcelWriter, 173
Explorer (Проводник), 56
Extensions (Расширения), 56

F

Format Specification Mini-Language, 126
Full outer join, 129

G

Git, 28
Google Colab, 53
Google Trends, 281, 287

I

inner join, 129

J

JavaScript, 139
JavaScript Object Notation, 248
Join, 129
JSON, 248
Jupyter, 60, 63, 107, 135, 165, 206, 301, 310
◊ блокнот, 310
Jupyter Notebook, 13, 36, 38, 47, 50

K

Kaggle, 53
Komodo IDE, 60

L

left join, 129

M

Matplotlib, 135, 220
Modern Portfolio Theory, 34

Modin, 194
MultiIndex, 116

N

Notepad++, 60
NumPy, 21, 35, 97, 99, 103, 123, 152, 201,
212, 311
◊ массивы, 103, 105

O

OpenPyXL, 177, 181, 184, 191, 193, 195
outer join, 129

P

pandas, 13, 22, 151, 158, 163, 174, 230
Pip, 33
Plotly, 137, 155
PostgreSQL, 271
Power BI, 30
Power Pivot, 29, 30
Power Query, 11, 29, 30
Pull request, 28
PyCharm, 60
pylightxl, 199
PyPI, 33
Python, 11, 21, 29, 30, 39, 43, 58, 78, 90,
106, 163, 222, 226
Python Enhancement Proposals (PEP), 90
Python Package Tracker, 245
PYTHONPATH, 243
pyxlsb, 177, 186

Q

Quickstart, 233, 238

R

read_excel, 167
Reader, 199, 201
REST, 249
Restart, 309

Run (Выполнить), 56
RunPython, 235, 238, 243

S

Search (Поиск), 56
Series, 105, 123
Source Control (Контроль источников), 56
Spyder, 60
SQL (Structured Query Language), 252, 256
Status Bar (Строка состояния), 57
Step Into, 309
Step Out, 309
Step Over, 309
Stop, 310
Sublime, 60

U

UDF, 274, 293
UDF Quickstart, 275
UserVoice, 11

V

VBA (Visual Basic for Applications), 11
Vim, 60
Visual Studio, 38
VS Code, 308

W

Wall time, 193
WinPython, 240
with, 170, 180

X

xlrd, 177, 187, 190, 192
xls, 192
XlsxWriter, 177, 184, 189, 195, 197
xltrail, 29
xlutils, 177, 187, 190
xlwings, 205, 206, 211, 225, 229, 230, 239,
285, 300
xlwt, 177, 187, 190

А

Автоматическая активация, 307
 Автономные рабочие книги, 241
 Агрегация, 132
 Анализ временных рядов, 159
 Анализ списков, словарей и множеств, 83
 Аннотация, 93
 Антивирусное программное обеспечение, 232
 Арифметические операции, 123
 Атрибуты, 65, 211, 313, 314

Б

База данных трекера пакетов, 253
 Базовая оптимизация производительности, 295
 Базы данных, 251
 Библиотека анализа данных, 105
 Блок кода, 78, 91
 Блокнот Jupyter, 38, 47, 301
 Большие массивы данных, 11, 145
 Бэкэнд, 137

В

Валидация данных, 265
 Веб-интерфейсы API, 248
 Векторизация, 97, 99
 Виртуальное окружение, 46
 Включение макросов, 235
 Внешний интерфейс, 264
 Внешний ключ, 254
 Внутренний интерфейс, 264, 268
 Временной ряд, 145
 Временные ряды, 151
 Встроенная константа None, 68
 Встроенные инструменты, 55
 Вход, 25
 Выбор данных, 111

- ◊ по метке, 112
- ◊ по позиции, 113
- ◊ с помощью MultiIndex, 116
- ◊ с помощью булевой индексации, 114

Выполнение кода, 54
 Выходы, 25

Г

Графики, 135, 220
 Графический пользовательский интерфейс, 231
 Группировка, 133

Д

Дата и время, 145
 Декларативный язык, 256
 Декоратор, 286, 291, 299

- ◊ функций, 277

 Диаграммы, 135
 Диаграммы, 220

- ◊ Excel, 220

 Диапазон массива, 103
 Динамическая типизация, 64
 Динамические массивы, 282, 283, 284
 Диск общего доступа, 240
 Дискретизация:

- ◊ повышающая, 156
- ◊ понижающая, 156

 Дистрибутив, 137
 Доверительный доступ, 275
 Дубликаты данных, 122

Е

Единичная матрица, 103

З

Заморозка исполняемого файла, 241
 Запуск кода VBA, 215
 Зацикливание ячеек, 179
 Значимое белое пространство, 78
 Зомби-процессы, 226

И

Иерархия конфигурации, 242
 Изменение данных, 117

- ◊ по метке или позиции, 118
- ◊ с помощью булевой индексации, 118

- ◊ с помощью добавления нового столбца, 120
- ◊ с помощью замены значений, 119

Изменение конфигурации, 127

Именованные аргументы, 85

Именованный диапазон, 222, 265

Импорт:

- ◊ DataFrames, 140

Импорт функций, 279

Индекс:

- ◊ конечный, 102
- ◊ начальный, 102
- ◊ отсортированный, 148

Индекс, 108, 123

Индексирование, 70

Инстанцирование, 65

Интерактивная сессия Python, 43

Интерпретатор Python, 243

Интерфейс, 231

Исключения, 260

К

Каскадные таблицы стилей, 198

Классы, 314

Ключи, 76

Кнопка запуска, 59

Код Python, 244

Коллекции, 208, 216

Командный режим, 51

Комментарий, 67

Коммит, 28

Конвертеры, 216, 217

Контекстные менеджеры, 170

Контроллер, 263

Контроль версий (управление версиями), 27

Конфигурация, 233

- ◊ каталогов, 242

Копирование, 103, 127

Корреляция, 153

Кортежи, 76

Красные цифры, 201

Кросс-платформенная, 55

- ◊ совместимость, 36

Кэширование, 297, 299

Л

Легкое масштабирование, 252

Линтер, 92

Литералы, 77

Логарифмическая доходность, 151

Логарифмы, 151

Логический тип данных, 67

Лямбда-выражения, 126

Лямбда-функций, 22

М

Магические команды, 137

Манипуляции с данными, 111

Массив, 103

Математические операторы, 66

Медлительность, 191

Менеджер пакетов, 32

Метод:

- ◊ Монте-Карло, 102
- ◊ расширения, 216

Методы, 65, 313

- ◊ объектов, 314

Методы программирования, 24

Мини-язык форматирования, 126

Множества, 77

Модули, 86, 87

Модульность, 24

Модульные тесты, 27

Н

Надстройка, 275

Нарезка (Slicing), 70

Научные вычисления, 34

Недостаточная функциональность, 145

Неявные переносы строк, 73

О

Обработка значений, 169

Объединение, 128

Объектная модель Excel, 205, 208

Объектная модель проекта VBA, 275
 Одномерный вариант, 218
 Описательная статистика, 132
 Определение функции, 84
 Определённые имена, 222
 Опции, 216, 217
 Отдельная среда, 46
 Открытый исходный код, 33
 Отладка, 271, 293
 Отладчик, 54, 308
 Отсутствие:
 ♦ миграций баз данных, 252
 ♦ объединений таблиц, 252
 Отсутствующие данные, 120
 Отчетность в Excel, 163

П

Пакет отчетов, 244
 Партнерский репозиторий, 87, 106, 143, 167, 194, 235
 Первичный ключ, 254
 Передача состояния представления, 249
 Перезапустить, 309
 Переключение между пакетами, 178
 Переменная, 64
 Пересчет, 153
 Повторная выборка, 156
 Позиционные аргументы, 85
 Пользовательская конфигурация, 242
 Пользовательские функции (UDFs), 14
 Последнее известное значение, 157
 Построение графиков, 291
 Представление, 103, 263
 Преобразование типов данных, 178
 Приглашение Anaconda, 40
 Приложения Excel, 205
 Принцип DRY, 26
 Программный интерфейс приложения, 248, 287
 Программное обеспечение, 33
 Просмотр, 127
 ♦ команд, 42

Р

Работа с большими файлами, 191
 Развертывание, 240
 Разделение задач, 24
 Размерность массива, 98

Расчет, 25
 Расширения, 55
 ♦ файлов, 43
 ♦ нотация, 83
 Расширенные темы xlwings, 225
 Расширенный круг задач, 190
 Режим редактирования, 51
 Рекурсия, 317
 Реляционная база данных, 252
 Репозитории, 302
 Ресурсы без статических данных, 249
 Рисунки, 220
 Родительский каталог, 42

С

Сетка, 267
 Система контроля версий (VCS), 28
 Скаляр, 99
 Скрипт, 54, 57, 84, 86, 88, 205
 Слияние, 129
 Словари, 75
 Слой бизнес-логики (Domain Layer), 24
 Слой данных (Data Layer), 24
 Слой представления (Presentation Layer), 24, 25, 263
 Случайные выборки, 281
 Сновы, 225
 Списки, 72
 Стандартная библиотека, 32
 Столбцы, 123
 Стоп, 310
 Строка, 99
 Структуры данных, 72

Т

Таблицы, 265
 Текстовая колонка, 125
 Тематическое исследование, 200
 Тернарные операторы, 80
 Тестирование, 27
 Типы данных, 63
 Транслирование, 97, 100, 104

У

Удобство чтения и обслуживания, 79
 Улучшение производительности, 227

Универсальная функция, 97, 101

Управление:

- ◊ версиями, 54
- ◊ потоком, 78

Условное форматирование, 266

Условные выражения, 80

Установка, 232

Ф

Файлы Excel, 205

Фиксированная запятая, 126

Форматированный строковый литерал, 69

Формулы:

- ◊ для одной ячейки, 295
- ◊ основанные на массиве, 296

Функция 26

- ◊ кубического сплайна, 26
- ◊ с изменяемыми объектами, 319

Х

Хэш-символ, 67

Ц

Цепная индексация, 102

Ч

Часовые пояса, 150

Число с плавающей запятой, 99

Числовые типы, 65

Читабельность, 31, 79

Чтение и запись файлов, 13

Ш

Шаг с выходом, 309

Шестнадцатеричная система, 183

Э

Экземпляры класса, 89, 313

Экспорт:

- ◊ DataFrames, 140
- ◊ CSV файлов, 141

Элементы управления ActiveX, 36

Я

Ядро Jupyter, 52

Язык структурированных запросов, 252

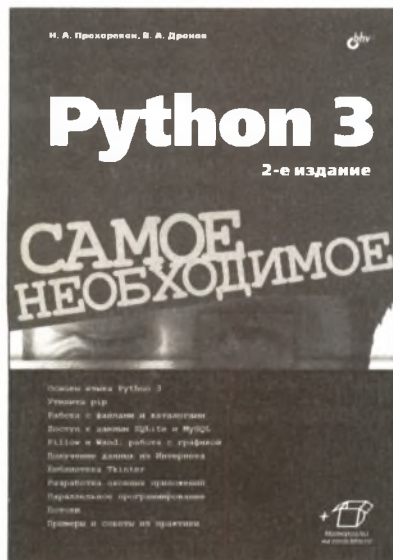
Ячейки блокнота, 311

Веб-дизайн для начинающих. HTML, CSS, JavaScript и веб-графика, 5-е изд.

Отдел оптовых поставок:

e-mail: opt@bhv.ru

Быстро и легко осваиваем Python — самый стильный язык программирования



- Основы языка Python 3
- Утилита pip
- Работа с файлами и каталогами
- Доступ к данным SQLite и MySQL
- Pillow и Wand: работа с графикой
- Получение данных из Интернета
- Библиотека Tkinter
- Разработка оконных приложений
- Параллельное программирование
- Потоки
- Примеры и советы из практики

Описан базовый синтаксис языка Python 3: типы данных, операторы, условия, циклы, регулярные

выражения, встроенные функции, объектно-ориентированное программирование, обработка исключений, часто используемые модули стандартной библиотеки и установка дополнительных модулей. Даны основы SQLite, описан интерфейс доступа к базам данных SQLite и MySQL, в том числе посредством ODBC. Рассмотрена работа с изображениями с помощью библиотек Pillow и Wand, получение данных из Интернета и использование архивов различных форматов.

Во втором издании описана актуальная версия Python — 3.6.4, добавлены описания утилиты pip, работы с данными в формате JSON, библиотеки Tkinter и разработки оконных приложений с ее помощью, реализации параллельного программирования и использования потоков для выполнения программного кода.

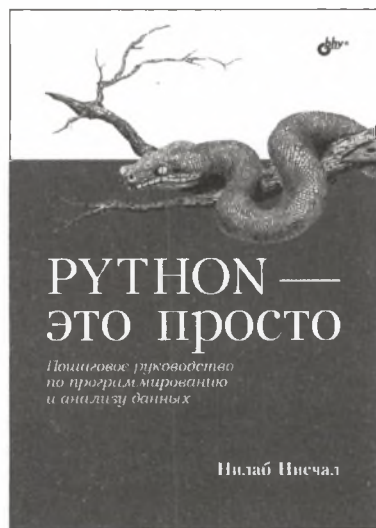
Прохоренок Николай Анатольевич, профессиональный программист, имеющий большой практический опыт создания и продвижения динамических сайтов с использованием HTML, JavaScript, PHP, Perl и MySQL. Автор книг «HTML, JavaScript, PHP и MySQL. Джентльменский набор Web-мастера» и др.

Дронов Владимир Александрович, профессиональный программист, писатель и журналист, работает с компьютерами с 1987 года. Автор более 20 популярных компьютерных книг, в том числе «HTML, JavaScript, PHP и MySQL. Джентльменский набор Web-мастера», «Python 3 и PyQt 5. Разработка приложений» и др. Его статьи публикуются в журналах «Мир ПК» и «ИнтерФейс» (Израиль) и на интернет-порталах IZ City и TheVista.ru.

Python — это просто. Пошаговое руководство по программированию и анализу данных

Отдел оптовых поставок:

e-mail: opt@bhv.ru



- Исследуйте возможности Python с использованием дистрибутива Anaconda
- Узнайте, как установить и использовать Python на своем компьютере
- Создавайте свои переменные, объекты и изучите их синтаксис
- Изучите встроенные типы объектов Python, такие как строки, списки, кортежи, множества и словари
- Научитесь вызывать встроенные функции, а также писать свои собственные
- Организуйте свой код и другие объекты в более крупные компоненты с помощью модулей
- Исследуйте классы — инструмент объектно-ориентированного программирования

- Пишите сложный код, научитесь обрабатывать ошибки и исключения
- Узнайте о массивах NumPy и операциях с ними
- Изучите анализ данных с помощью Pandas
- Погрузитесь в захватывающий мир визуализации с использованием Matplotlib
- Научитесь создавать приложения Python с графическим интерфейсом

Рассмотрены основы синтаксиса языка Python на примере дистрибутива Anaconda. Показаны приложения IPython, Spyder IDE, Jupyter Notebook. Описан синтаксис переменных, функций, циклов. Подробно изучаются структуры данных в Python: строки, списки, кортежи, множества и словари. Объясняется понятие классов и их применение в объектно-ориентированном программировании. Описаны возможности библиотеки обработки изображений Pillow, библиотеки Tkinter для создания приложений с графическим интерфейсом. Отдельный раздел посвящен обработке ошибок и исключений в программах. Рассматриваются библиотеки NumPy и Pandas, приводятся практические примеры их использования для анализа и обработки данных. Описана библиотека Matplotlib и ее возможности в сфере визуализации данных.

Нисчал Нилаб, имеет степень магистра менеджмента, работает штатным специалистом по маркетингу и ведущим аналитиком данных на протяжении более 14 лет. Он обучает студентов колледжей как в инженерной области, так и в области управления. Страсть к принятию осмысленных бизнес-решений на основе анализа данных привела его к глубокому изучению языков R и Python. Результатом данных изысканий и стала эта книга.

Отдел оптовых поставок:

e-mail: opt@bhv.ru



Вы научитесь:

- Писать код для контроллера привода двигателя
- Строить карты на основе данных лидара
- Создавать собственные алгоритмы автономного планирования траектории движения
- Писать код для автоматической отправки путевых точек контроллеру привода
- Создавать программы картографии и навигации для автономных роботов

Рассказано о технологии создания автономных роботов на базе одноплатного компьютера Raspberry Pi и о разработке программ для них на языке C++. Показаны принципы написания и даны примеры кода для контроллера привода двигателя, продемонстрированы способы использования датчиков для обнаружения препятствий и построения карт на основе данных лидара. Описаны методы разработки собственных алгоритмов автономного планирования траектории движения, приведен код для автоматической отправки путевых точек контроллеру привода. Рассмотрены библиотеки C++ для написания программ картографии и навигации автономных роботов, даны сведения об использовании контактов аппаратного интерфейса Raspberry Pi GPIO.

Электронный архив на сайте издательства содержит код описанных в книге программ.

Бромбах Ллойд, инженер, программист и энтузиаст электроники и робототехники. Участвовал в соревнованиях по робототехнике, таких как финансируемый НАСА конкурс Lunar Regolith Excavation Challenge 2007 и 27-й конкурс Intelligent Ground Vehicle Challenge.

Python 3 и PyQt 6. Разработка приложений

Отдел оптовых поставок:

e-mail: opt@bhv.ru

Быстрое создание программ с графическим интерфейсом



- Типы данных Python
- Объектно-ориентированное программирование
- Работа с файлами и каталогами
- Взаимодействие с Windows
- Создание оконных программ
- Работа с базами данных
- Мультимедиа
- Запись звука, видео и фото
- Печать и экспорт в формат PDF
- Работающий пример: приложение «Судoku»

Описан язык Python 3: типы данных, операторы, условия ветвления и выбора, циклы, регулярные выражения, функции, классы, работа с файлами и каталогами, взаимодействие с механизмами Windows, часто используемые модули стандартной

библиотеки. Особое внимание уделено библиотеке PyQt, позволяющей создавать приложения с графическим интерфейсом. Описаны средства для создания и вывода окон, основных компонентов (кнопок, полей, списков, таблиц, меню, панелей инструментов и др.). Рассмотрены обработка событий и сигналов, разработка многопоточных программ, работа с базами данных, вывод графики, воспроизведение мультимедиа, запись аудио, видео и фото, печать документов, экспорт их в формат Adobe PDF и сохранение настроек программ. Дан пример полнофункционального приложения для создания и решения головоломок судoku. На сайте издательства размещен электронный архив со всеми примерами из книги.

Прохоренок Николай Анатольевич, профессиональный программист, имеющий большой практический опыт создания приложений с использованием C, Java, Python, HTML, JavaScript, PHP и MySQL. Автор более 20 книг, в том числе «HTML, JavaScript, PHP и MySQL. Джентльменский набор Web-мастера», «JavaScript и Node.js для веб-разработчиков», «Bootstrap и CSS-препроцессор Sass. Самое необходимое», «Python 3. Самое необходимое», «Qt 6. Разработка оконных приложений на C++» и др.

Дронов Владимир Александрович, профессиональный программист, писатель и журналист, работает с компьютерами с 1987 года. Автор около 50 популярных книг по информационным технологиям, в том числе «Django 3.0. Практика создания веб-сайтов на Python», «Laravel 8. Быстрая разработка веб-сайтов на PHP», «HTML и CSS. 25 уроков для начинающих», «JavaScript. 20 уроков для начинающих», «PHP и MySQL. 25 уроков для начинающих», «JavaScript. Дополнительные уроки для начинающих» и «React 17. Разработка веб-приложений на JavaScript».



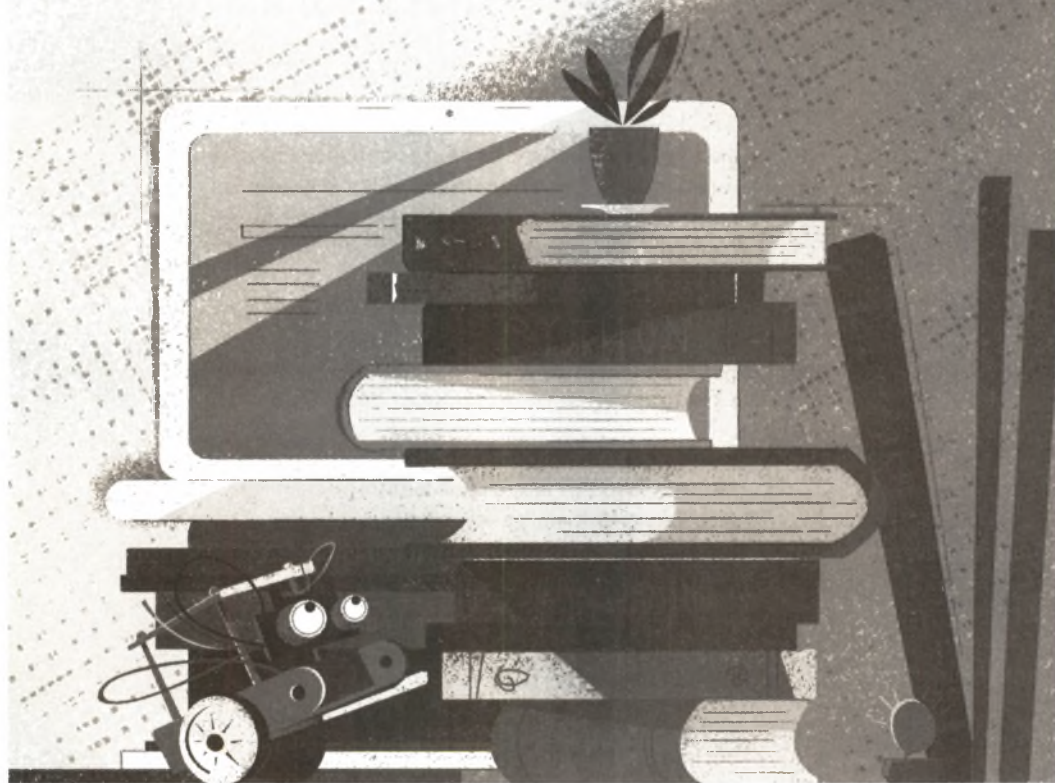
ИНТЕРНЕТ-МАГАЗИН

BHV.RU

КНИГИ, РОБОТЫ,
ЭЛЕКТРОНИКА

Интернет-магазин издательства «БХВ»

- Более 25 лет на российском рынке
- Книги и наборы по электронике и робототехнике по издательским ценам
- Электронные архивы книг и компакт-дисков
- Ответы на вопросы читателей



Скидка 15% на бумажные книги
по промокоду: JANE22

КРОК

СОЗДАЁМ НАСТОЯЩЕЕ,
ИНТЕГРИРУЕМ БУДУЩЕЕ



croc.ru

КРОК — технологический партнер с комплексной экспертизой в области построения и развития инфраструктуры, внедрения информационных систем, разработки программных решений и сервисной поддержки.

Центры компетенций КРОК фокусируются на ключевых отраслевых кластерах — промышленность, финансовый сектор, розничные продажи, муниципальное управление, спорт и культура.

Ежегодно сотни проектов КРОК становятся системообразующими для экономики и социально-культурной сферы.

